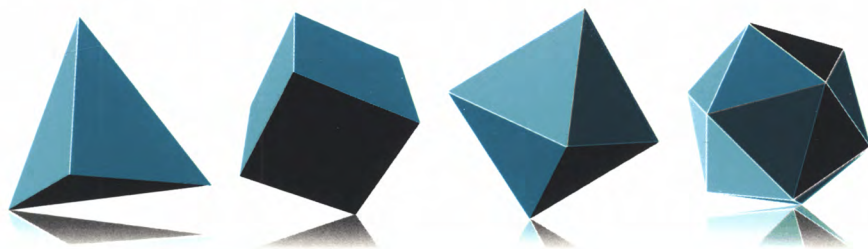


Григорий Макеев

Объектно-ориентированное программирование: с нуля к **SOLID** и **MVC**



Вы узнаете, как:

- инкапсулировать данные и код в объекты
- работать с объектами в C++ и других востребованных языках
- управлять композицией и взаимодействием объектов
- управлять временем жизни объектов
- использовать паттерны проектирования
- применять принципы SOLID
- создавать событийно-ориентированные приложения
- решать задачи с помощью архитектуры MVC

Григорий Макеев

**Объектно-
ориентированное
программирование:
с нуля
к SOLID и MVC**

Санкт-Петербург

«БХВ-Петербург»

2024

УДК 004.4'236
ББК 32.973.26-018
М15

Макеев Г. А.

М15 **Объектно-ориентированное программирование: с нуля к SOLID и MVC. —**
СПб.: БХВ-Петербург, 2024. — 272 с.: ил. — (С нуля)

ISBN 978-5-9775-1913-7

Базовая книга по приёмам объектно-ориентированного программирования (ООП) с примерами на языке C++. Разобраны основные принципы и техники разработки объектно-ориентированных программ, объяснены шаблоны проектирования (паттерны), особое внимание уделено принципам SOLID, архитектуре MVC (модель-представление-контроллер) и особенностям работы с ООП в C++.

Для начинающих разработчиков, специалистов по C++, студентов вузов

УДК 004.4'236
ББК 32.973.26-018

Группа подготовки издания:

Руководитель проекта	<i>Олег Сивченко</i>
Зав. редакцией	<i>Людмила Гауль</i>
Компьютерная верстка	<i>Ольги Сергиенко</i>
Дизайн обложки	<i>Зои Канторович</i>

Подписано в печать 08.11.23
Формат 70×100^{1/16}. Печать офсетная. Усл. печ. л. 21,93
Тираж 1000 экз. Заказ № 7200
"БХВ-Петербург", 191036, Санкт-Петербург, Гончарная ул., 20

Отпечатано в АО "Первая Образцовая типография"
Филиал "Чеховский Печатный Двор"
142300, Московская область, г. Чехов, ул. Полиграфистов, д. 1
Сайт: www.chpd.ru, E-mail: sales@chpd.ru, тел. 8(499)270-73-59

ISBN 978-5-9775-1913-7

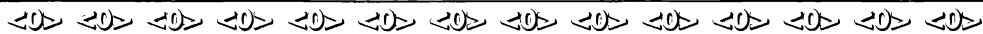
© Макеев Г. А., 2024
© Оформление: ООО "БХВ-Петербург", ООО "БХВ", 2024



ОГЛАВЛЕНИЕ

Посвящения и благодарности	5
Введение.....	7
Некоторые обозначения и сокращения.....	7
Что это и для кого	7
Зачем это всё	8
Методическая проблема.....	9
 Глава 1. Основы ООП.....	 11
1.1. Основные понятия	11
1.2. Хватит бла-бла-бла, давайте уже код!	16
1.3. Определение классов и создание объектов	17
1.4. Работа с объектами	20
1.5. Методы	23
1.6. Скрытие свойств и методов	26
1.7. Конструкторы и деструкторы, жизненный цикл объектов	29
1.8. Наследование	38
1.9. Расширение и изменение объектов при наследовании.....	45
1.10. Композиция	46
1.11. Глубокое и поверхностное копирование, value- и reference-семантика.....	51
1.12. Наследование, композиция, агрегация, ассоциация, зависимость	52
1.13. Диаграммы классов, последовательностей, состояний и прочих UML.....	55
1.14. Композиция или наследование?	60
1.15. Помещение объектов в переменные различных типов	63
1.16. Полиморфизм	66
1.17. Приведение типов	74
1.18. Передача и возврат объектов из функций в C++	83
1.19. Проблемы работы с памятью и не только, в C++ и не только.....	87
1.20. Технологии и практики управления долгоживущими объектами в C++.....	90
 Глава 2. Приемы ООП.....	 99
2.1. Демистификация паттернов ООП	99
2.2. Обычные приемы и антиприемы ООП, паттерны и антипаттерны.....	100
2.3. Template Method.....	103
2.4. Prototype.....	106
2.5. Бесклассовое (прототипно-ориентированное) ООП.....	109
2.6. Интерфейсы.....	110

2.7. Множественное наследование	113
2.8. Примеси (миксины, mixins)	115
2.9. Singleton	116
2.10. Chain of Responsibility	119
2.11. Делегирование, объектная шизофрения	126
2.12. Strategy	130
2.13. Adapter	135
2.14. Iterator	142
2.15. Шаблоны C++. Осторожно по минному полю	150
2.16. Стандартные контейнеры STL	152
2.17. Decorator	154
2.18. Proxy	160
2.19. Composite	163
2.20. Observer сам по себе и как пример mixin	167
2.21. Command	171
2.22. Factory Method, Abstract Factory	179
2.23. Abstract Factory еще раз	187
2.24. Memento	190
2.25. Bridge	193
2.26. Visitor	201
2.27. Одинарная и двойная диспетчеризация	209
2.28. SOLID-принципы	211
2.29. Объектная интроспекция и рефлексия	221
2.30. Событийно-управляемое приложение: событие, типы событий, сообщение, обработчик, механизм обмена сообщениями, очередь сообщений	225
2.31. Архитектура Model-View-Controller (MVC) и ее варианты. Самый простой способ ее нарушить и исправить	237
Приложение. Ликбез по переменным и указателям в C++	249
П.1. Обозначения	249
П.2. Краткое описание объявления и примеры использования	249
П.3. Полное описание	250
П.3.1. Переменная	250
П.3.2. Память	251
П.3.3. Адрес переменной	252
П.3.4. Переменные-указатели	253
П.3.5. Операции с указателями	256
П.3.6. Массивы	257
П.3.7. Ссылки	259
П.4. Вопросы для самоконтроля	262
Список литературы	265
Предметный указатель	267



ПОСВЯЩЕНИЯ И БЛАГОДАРНОСТИ

Эта книга посвящается моему дипломному руководителю и руководителю моей кандидатской диссертации Юсуповой Нафисе Исламовне, которая ушла от нас в марте 2023 года. Нафиса Исламовна руководила факультетом информатики и робототехники Уфимского государственного авиационного технического университета более 30 лет, руководила кафедрой вычислительной математики и кибернетики, и мы, ее ученики, не забудем, как много она сделала для нас.

Эта книга была бы невозможна без неоценимых советов коллег и более опытных специалистов (две эти категории во многом пересекаются). Глубокая благодарность за помощь объявляется Бабичеву Эмилию, Соловьеву Денису, Рыжикову Владимиру, Мирянову Сергею, Борщуку Олегу, а также всем, кого автор забыл упомянуть (если вы ненавязчиво напомните о себе хотя бы еще одним советом, то, честно-честно, попадете в этот список и прославитесь), а также всем, кто хотел помочь, но не успел или забыл, а также студентам, читавшим книгу достаточно внимательно, чтобы замечать опечатки.

С одной стороны, я буду стараться рассказывать об ООП так, чтобы знания и навыки были применимы к любому ООП-языку. Большинство идей важны сами по себе и реализуемы в любом ЯП, поэтому именно идеи знать важнее всего. Вы всегда сможете нагуглить «как сделать X в языке Y», если вы знаете, что вам нужен именно X, а это требует как раз знания идей. С другой стороны, здесь будет много примеров и низкоуровневых деталей из C++ просто потому, что, во-первых, опыт в C++ наверняка окажется вам полезен; во-вторых, даже если и не окажется, то вы, как программисты, должны уметь с такими сложностями справляться.

Объектно-ориентированное программирование (ООП), на мой взгляд, является одним из самых важных предметов, которые встретятся на пути у любого студента-программиста. Если браться за самостоятельное изучение программирования в попытке «войти в IT» (хотя, учитывая текущий заслуженный хайп вокруг ChatGPT, некоторые считают, что учиться на программистов уже поздно), первым делом стоит изучать собственно базовое программирование, потом взять курс структур и алгоритмов обработки данных, а потом браться именно за ООП. При этом совсем не обязательно вы будете ООП использовать в своей рабочей деятельности каждый день — это всего лишь один из инструментов наведения порядка и борьбы с программной сложностью. Программисты встраиваемых систем пишут на ассемблере или других низкоуровневых ЯП и не используют ООП. Программисты, которые пишут запросы к реляционным базам данных, тоже могут не встретить на своем пути ООП и при этом успешно решать свои задачи. Важность ООП для них я все равно вижу в том, что ООП — это про абстракции и абстрагирование, а это, на мой взгляд, и составляет суть программирования.

Я буду подразумевать, что, раз вы взялись за эту книгу, вы уже:

- ☐ обладаете общими навыками программирования и алгоритмизации: умеете программировать простые последовательные вычисления и простые алгоритмы обработки данных;
- ☐ владеете знаниями синтаксиса C/C++ и легко используете его основные конструкции — присваивания, циклы, ветвления, объявления и вызовы функций (по значению, по ссылке), а также используете стандартную библиотеку для выполнения типовых действий ввода-вывода при работе с консолью и файлами;
- ☐ понимаете, что такое косвенная адресация: указатели, ссылки и т. д.; а если не владеете уверенно, то тут есть специальное приложение с ликбезом;
- ☐ владеете навыками использования какой-нибудь среды разработки, в которой умеете создавать проекты на C/C++, компилировать и запускать их;
- ☐ умеете производить отладку в своей любимой среде разработки: выполнять программу по шагам, использовать точки останова и просматривать текущие значения переменных и выражений.

Зачем это всё

Зачем нужна эта книга, если про ООП и так написан весь Интернет, где все можно найти, а также нетленка от классиков, которых простым людям не превзойти? Я постараюсь максимально сжато рассказать все, что, по моему мнению, можно и нужно рассказать в таком кратком курсе, и способ подачи и объяснений материала считаю главной ценностью этой книги. Действительно, литературы, в которой описывается все, про что говорится в этой книге, очень много. И практически по всем обсуждаемым темам всегда можно нагуглить множество

материалов. Гораздо важнее — знать, что именно гуглить: что крайне важно, что менее важно, что совсем не важно. Такое знание появляется у опытного специалиста, который «задним числом» мог бы сформулировать, что, на его взгляд, самое важное, а изучение чего можно и отложить. Не могу назвать себя очень опытным специалистом, особенно в C/C++, но свое видение того, что важно студенту второго курса, у меня есть.

Начинающий программист стоит перед морем информации и лишен именно этого — понимания, что в этом море и в каком порядке стоит изучать. Я хотел бы думать, что даже если в этой книге нет чего-то важного, никакую часть этой книги нельзя заменить на что-то более важное для начинающих программистов (например, моих студентов второго курса). Если вы знаете что-то, что в эту книгу обязательно нужно добавить, то сообщите мне сразу, что именно при этом нужно выкинуть, потому что все в этот краткий курс поместить не получится.

Можно написать «из авторского в этой книге только ошибки автора», но есть и еще одно соображение. Самое важное, я считаю, это не знания, а дорожки из нейронов, которые волей преподавателя будут проложены в голове у изучающих этот курс, и мне кажется, что мой опыт преподавания поможет мне их проложить у вас правильным образом. По-моему, лучше всего запоминается то, что связано причинно-следственной цепочкой, где все рассказываемое логически вытекает из того, что было рассказано раньше. Я старался выстроить повествование, придерживаясь именно такого подхода, даже если для этого пришлось во второй части книги перемешивать паттерны с теорией и практикой ООП.

Методическая проблема

Мы знаем, что эта задача не имеет решения,
мы хотим знать, как ее решать.

Главная нерешаемая нерешенная проблема этой книги заключается в том, что она должна сопровождать студентов-программистов на протяжении одного семестра и должна начинаться с объяснений того, что такое указатели, а заканчиваться обсуждением принципов SOLID и паттерном Visitor.

Еще одна проблема, которую я так для себя пока и не решил, заключается в том, правильно ли для примеров в этой книге использовать C++, и если да, то какую версию. Взаимоисключающие и противоречивые соображения по этому поводу приведены здесь:

- ☐ нужно использовать C++ потому, что студенты именно его изучают на первых курсах;
- ☐ опыт использования C++ будет полезен студентам во множестве областей, с которыми они потенциально столкнутся;
- ☐ нужно использовать везде разные возможности C++ для обучения;
- ☐ нужно использовать везде один и тот же стиль, одни и те же самые правильные и самые современные возможности C++, потому что это правильно;

- ❑ автор не владеет C++ в степени достаточной, чтобы выдерживать правильный, единый, целостный и непротиворечивый стиль и подход к использованию разных приемов C++ из разных стандартов C++;
- ❑ конкретные примеры параллельно с ООП должны обучать C++ тех студентов, которые владеют им еще хуже автора;
- ❑ конкретные примеры должны быть легко переносимы на любой другой объектно-ориентированный язык;
- ❑ нужно использовать старые и низкоуровневые возможности C++, потому что студенты должны владеть низкоуровневыми деталями организации вычислений;
- ❑ нужно использовать новые и высокоуровневые возможности C++, потому что именно они позволяют проще всего писать переносимый на другие объектно-ориентированные языки код;
- ❑ вместо использования Java и C# можно ровно в той же самой манере писать эквивалентный код на C++, который будет почти работать в Java и C# после удаления символов «*» и «&»;
- ❑ лучше вообще не учить C++, чем учить использовать `new` и `delete`;
- ❑ лучше вообще не учить C++, если не использовать `new` и `delete`;
- ❑ надо учить не наступать на грабли;
- ❑ при обучении надо наступить на большинство типовых грабель.

ГЛАВА 1

ОСНОВЫ ООП

1.1. Основные понятия

Прежде чем начать обсуждать любую тему, нужно договориться об используемом языке. В математике обычно начинают с определений основных понятий, и мы должны тоже согласовать если не определения, то смысл, значения основных слов, которыми будем пользоваться.

Значение — «кусочек» данных, нечто, являющееся ответом на вопрос или результатом вычислений, который «окончательный» в том смысле, что его нельзя вычислять дальше.

Тип — множество значений, «похожих» друг на друга тем, что они все имеют одинаковую природу, одинаковые единицы измерений, могут являться результатами одних и тех же вычислений и к которым могут применяться одни и те же операции. Таким образом, любое значение принадлежит какому-то типу.

Выражение — построенная по правилам ЯП синтаксическая конструкция, которую можно вычислять, упрощать, «редуцировать» и в результате этого вычисления, возможно, получить значение.

Переменная — область памяти, имеющая имя и хранящая значение, которое, возможно, будет изменяться в процессе работы программы. Любое значение имеет тип, поэтому раз переменная хранит значение, а значение имеет тип, то можно сказать, что переменная тоже имеет тип. В некоторых ЯП переменные могут изменять свои значения, но только на значения того же типа, — тогда получается, что именно переменная имеет тип, — такие языки называются *статически типизируемыми*. Если в разные моменты времени тип хранимого в переменной значения может быть разным, язык будет *динамически типизируемым*.

Указатель — специальная переменная, хранящая адрес какой-то другой области памяти. *Ссылка* — альтернативное имя для уже существующей переменной или для какой-то безымянной области памяти. Указатели и ссылки — разные по смыслу и реализации понятия, необходимые для организации непрямого доступа, косвенной адресации. Грубо говоря, это способы с помощью разных имен получать доступ к одному и тому же объекту или с помощью одного и того же имени получать доступ к разным объектам. Разные ЯП по-разному реализуют

непрямой доступ, детали реализации отличаются в разных языках, но непрямой доступ есть, наверное, в любом достаточно развитом объектно-ориентированном языке. Детально переменные, указатели и ссылки в C++ описаны в *приложении*: убедитесь, что вы все там отлично понимаете, прежде чем продолжать.

Объектно-ориентированное программирование, как следует из названия, — про объекты и про объектно-ориентированный взгляд на реальность. Поэтому теперь немного отвлечемся от программирования и посмотрим вокруг: что мы готовы называть объектом реального мира? Речь не об определении, а о таком объяснении, против которого не будет протестовать наша интуиция и здравый смысл. На мой взгляд, объект реального мира — это нечто, что мы можем воспринимать нашими органами чувств (видеть, осязать) как единое целое, отделенное от всего остального. Окно — объект, рама окна — объект, но верхняя часть окна вряд ли объект, потому что сложно сказать, где начало того конца, которым оканчивается начало! Ограниченность, законченность (потом мы еще скажем и «инкапсулированность», но пока не заморачивайтесь) — физическая и смысловая — вот те свойства, которые наша интуиция требует от любого объекта.

Таких объектов вокруг нас много. Пытаясь наводить между ними порядок, мы неизбежно замечаем, что некоторые объекты ближе друг к другу, чем другие. Так, две книги, пусть даже одна красная, а другая зеленая, воспринимаются нами более «похожими», чем красная книга и красный, например, флаг. Если задуматься почему, то мы неизбежно придем к очевидному выводу: красная и зеленая книги похожи друг на друга тем, что они имеют одинаковую природу, могут являться результатами одного и того же производства и могут применяться для одних и тех же целей. Видите прямую аналогию со значениями и типами? Аналогично тому, как мы объединяли «похожие» значения в множества, т. е. типы, мы можем объединять «похожие» объекты в аналогичные множества, которые уже можно назвать *классами*.

Множество разных книг — это класс «книга», множество разных студентов — это класс «студент» и т. д. Как тип есть множество однотипных значений, объединенных общей природой, единицами измерения, применяемыми операциями, так и классом можно назвать множество «однотипных» объектов, объединенных общей природой, общим происхождением и предназначением. Слово «класс» в этом смысле выбрано довольно случайно. В биологии ближайшим аналогом, я думаю, будет понятие «вид»: так биологический вид «кролик» есть (довольно условное) обозначение для большого множества реальных кроликов, способных спариваться и давать жизнеспособное потомство (определение биологического понятия «вид»).

Таким образом, если класс — это множество объектов, то конкретный объект может входить в конкретное множество, а может и не входить. Математики тут говорят, что для объекта и класса определена операция «принадлежать». Попробуем посмотреть, а какие еще операции у нас могут применяться к объектам? Для двух объектов можно определить операцию «один и тот же» и, например, операцию «равен». Так, обсуждая два объекта «маршрут между городами», мы

с другим водителем можем обсуждать маршрут А и маршрут Б и в итоге выяснить, что мы говорим про один и тот же маршрут, проходящий через одни и те же населенные пункты. А может быть, мы выясним, что маршрут А и маршрут Б — это два разных маршрута, но сойдемся на том, что для нас, как водителей, эти оба маршрута «равны». Математики тут опять-таки скажут, что на множестве объектов определены в общем случае разные операции тождественности и равенства.

Про тождественность вроде бы все понятно в рамках обычного здравого смысла: один и тот же объект и есть один и тот же объект. А вот с равенством могут возникнуть непонятки. Когда мы готовы две разные книги, принадлежащие, очевидно, одному и тому же классу «книга», признать «равными»? Ведь очевидно, что они хоть чем-то обязательно отличаются: случайно поплавшей печатью 35-й буквы в 12-й строке на 42-й странице, легкой вмятиной на обложке, инвентарным номером на подкладке, датой продажи и т. д. Какие-то отличия могут быть нам важны, а какие-то — нет, причем это может зависеть от нашей цели в данный конкретный момент времени: какие-то отличия могут быть нам не важны «вообще», и мы готовы про них забыть, а какие-то — не важны «конкретно сейчас», но мы подозреваем, что могут стать важны «потом», поэтому мы про них забывать не готовы.

Если подумать, каким общим словом мы готовы назвать «инвентарный номер», «название», «дата продажи» — короче говоря, все то об объекте, что мы не готовы забывать и разницей в чем не готовы пренебрегать, мы придем к понятию «свойство», или «атрибут». У конкретного объекта «книга» есть атрибут «количество страниц», и этот атрибут имеет значение, например 42. А как быть с другими книгами?

Самый интуитивно приемлемый вариант состоит в том, что другие книги имеют те же самые свойства, но, возможно, другие значения (некоторых или всех) свойств. Так, у других книг могут быть те же или другие названия, то же или другое количество страниц, тот же или другой автор, но у всех книг так или иначе есть (какое-то) название, количество страниц и автор. Если спросить себя, что же тогда объединяет все книги, если у них и авторы разные, и количество страниц, и названия, то получается, что все они принадлежат одному классу «книга» именно потому, что у них у всех есть (пусть даже разные) название, количество страниц и автор.

Что такое в этом случае «класс»? Получается, что класс «книга» есть множество свойств, таких как «название», «автор», «издательство», при этом значения этих свойств в классе не задаются, а появляются только у объектов.

- Класс определяет набор свойств, которыми должны обладать все объекты, принадлежащие этому классу, но ничего не говорит о значениях этих свойств.
- Все объекты, принадлежащие одному классу, обязательно имеют все свойства, определенные для этого класса, но, возможно, разные значения этих свойств.

- Два объекта, принадлежащие одному классу, могут иметь разные значения свойств, но сам набор свойств у них обязательно одинаковый.
- Может ли быть два объекта одного класса, у одного из которых значение «цвет» равно «красный», а у другого — «зеленый»? Да, может.
- Может ли быть два объекта разных классов, у обоих из которых значение «цвет» равно «красный»? Да, может.
- Может ли быть два объекта одного класса, у одного из которых есть свойство «цвет», а у другого нет? Нет, не может.

Последнее может показаться в некоторых случаях контринтуитивным: мы как будто вполне можем себе представить два смартфона, у одного из которых есть инфракрасный порт и соответственно свойство «скорость инфракрасного порта», а у другого порта нет. Надо иметь в виду, что жизнь — бесконечно сложна, и пытаться точно описать ее бесконечное разнообразие каким-либо формализмом — изначально бесперспективное занятие. Поэтому — да, мы сознательно идем на определенное упрощение: мы договариваемся считать объекты принадлежащими одному классу только тогда, когда их набор свойств «полностью» совпадает.

Но ведь очевидно, что смартфон, описанный конечным набором свойств своего класса, есть огромное упрощение по сравнению с реальным смартфоном. Мы сознательно идем на это упрощение, оставляя только набор «важных» свойств и отбрасывая, игнорируя все остальные свойства смартфона, «неважные» для нас. Эта операция и называется *абстрагированием*, в результате чего появляется упрощенный, «идеализированный», «абстрагированный» объект, от которого остались только «важные» для нас свойства, при этом набор «важных» свойств в зависимости от задачи может быть, очевидно, разным. При этом степень абстрагирования может быть разной: чем меньше свойств у объекта остается и чем больше свойств мы соответственно «забываем», тем абстрактнее наш взгляд на этот объект, тем сильнее мы абстрагируемся от его бесконечного разнообразия.

Итак, в чем заключается объектно-ориентированный взгляд на реальность? Он заключается в том, что мы видим разнообразные объекты, каждый из которых отличается от всех остальных или набором свойств, или значениями этих свойств. При этом (что совсем не обязательно для ООП, но чаще всего имеет место) мы объединяем объекты в множества, которые называем классами так, что в каждом таком множестве-классе объекты обладают одинаковым набором свойств, но разными значениями этих свойств.

Распределяя объекты по классам как множествам «похожих» объектов, мы можем получить классы с разной степенью детализации в зависимости от того, насколько сильно мы абстрагируемся от деталей. Например, глядя на разнообразную коллекцию книг, мы можем отнести их все к классу «книги», а можем и отнести их к классу «учебники по высшей математике». Может быть, нам вообще придется сделать два класса: «учебники по высшей математике» и «книги по истории» и часть книг отнести к первому классу, а часть — ко второму. А мо-

жем ли мы отнести их все к одному классу? Как будет называться такой класс, к которому можно отнести их все? Похоже, это будет класс «книги»?

С другой стороны, глядя на реальный мир, классифицируя все встречающиеся объекты по классам, мы можем заметить, что и классы у нас получаются разные: некоторые классы, очевидно, больше «похожи» в том смысле, что имеют больше «общего», чем другие. Глядя в окно на проспект и отмечая, что среди всего разнообразия объектов очевидно выделяются такие классы объектов, как «автомобили», «автобусы», «троллейбусы», «трамваи», «дома», «деревья», «облака», «люди», мы не можем не отметить, что класс «автобусы» гораздо «ближе» к классу «троллейбусы», чем к классам «деревья» и «люди». В каком смысле «гораздо ближе»? В смысле общего набора свойств! А если при этом кто-то рядом абстрагировался от деталей гораздо сильнее нас и вместо классов «автобусы», «троллейбусы» и «трамваи» относил все объекты к классу «общественный транспорт», то мы должны спросить у тебя, а как относятся друг к другу классы «общественный транспорт» и «троллейбусы»?

Получается, что классы бывают более конкретными и менее конкретными. Условно говоря, класс «троллейбусы» включает гораздо меньше объектов, чем класс «общественный транспорт». Многие знают этот детский вопрос: «Кого больше, ворон или птиц?». Получается, что конкретная ворона принадлежит и множеству ворон, и множеству птиц, а значит, и к классу «вороны», и к классу «птицы»? А верно ли это для других ворон? Для всех?

Получается, что классы «вороны», «птицы», «троллейбусы» и «общественный транспорт» устроены так, что если объект принадлежит классу «вороны», то он обязательно принадлежит и к классу «птицы», а если другой объект принадлежит классу «троллейбусы», то он обязательно принадлежит и к классу «общественный транспорт». Считая классы множествами, мы можем сказать, что множество «вороны» является подмножеством множества «птицы», а множество «троллейбусы» является подмножеством множества «общественный транспорт». Математики тогда скажут, что на множестве всех классов определено отношение «является подмножеством» или «входит в», но мы будем проще и скажем, что в этом случае будем считать класс «вороны» подклассом класса «птицы», а класс «троллейбусы» подклассом класса «общественный транспорт».

Класс А является подклассом класса Б, если любой объект, принадлежащий классу А, обязательно принадлежит одновременно и классу Б. Класс А тогда называется дочерним, а класс Б — родительским, класс А называется потомком, а класс Б — предком. Класс Б при этом можно назвать более абстрактным («птицы»), а класс А можно назвать более конкретным («вороны»). Обратите внимание, что слова «предок» и «потомок» при этом употребляются совсем не в том смысле, которым эти слова обладают в обычной речи. Мы говорим «предок» и «потомок» в обычной жизни применительно к биологическому родству, но в ООП эти слова имеют, очевидно, другой смысл.

Очевидно, что если класс А («вороны») является подклассом класса Б («птицы»), а класс Б является подклассом класса В («животные»), то класс А является

одновременно и подклассом класса В. Математики в этом случае говорят о транзитивности отношения «предок-потомок» между классами, но мы будем проще и лишь отметим, что все классы можно тогда выстраивать в своеобразную иерархию, где у каждого класса есть не более одного предка.

Каждый конкретный объект в этом случае принадлежит одному и только одному «непосредственному», или «родному», классу, но при этом является не только его экземпляром, но и экземпляром всех предков «родного» класса. При этом «родной» класс можно еще называть «конкретным» классом, а всех его предков — абстрактными классами, при этом тем более абстрактными, чем выше по иерархии они находятся. Абстрактным при этом называется любой такой класс, объекты которого обязательно принадлежат одному из потомков этого класса. Например, класс «птицы» мы назовем абстрактным потому, что если какой-то объект принадлежит классу «птицы», он одновременно принадлежит одному из классов «вороны», «лебеди», «воробьи», «синицы» и т. д. Другими словами, в реальности не существует конкретной летающей «птицы», которая не была бы одновременно представителем конкретного подвида птиц!

1.2. Хватит бла-бла-бла, давайте уже код!

Теперь, когда мы разобрались с особенностями объектно-ориентированного подхода к анализу реальности, у нас есть два варианта. Желющие могут погрузиться в объектно-ориентированный анализ вместе с классикой жанра [4], а мы уже готовы вернуться обратно в область computer science и попробовать применить аналогичный подход там. На самом деле объектно-ориентированный подход вполне очевиден, учитывая возрастание сложности используемых структур данных, но я попробую показать, почему именно.

Предположим, нам нужно хранить координаты точек на оси X. Понятно, что при большом количестве точек для этого нецелесообразно использовать отдельные переменные, а лучше хранить точки в массиве:

```
int x[10];
```

Если вдруг нам понадобится хранить координаты не одномерных, а двумерных точек, то возможным решением будет добавление еще одного массива:

```
int x[10];  
int y[10];
```

В чем недостаток такого решения? Мы уже здесь видим, что координаты какой-то конкретной точки хранятся в двух разных массивах, которые потенциально имеют разную длину: если нам понадобится увеличить количество точек с 10 до 11, нам придется менять два массива. Если нам понадобится функция, которая, например, сдвигает какую-то точку по обеим координатам, то в такую функцию нам придется передать и оба массива, и индекс:

```
int x[10];  
int y[10];
```

```
void shift(int x_to_shift[], int y_to_shift[], int idx)
{
    ...
}
```

Если вдруг понадобится увеличить длину массива, то нам придется сделать это в двух разных местах, и это может стать причиной ошибки. Но первопричиной проблемы станет то, что информация, относящаяся к одному конкретному объекту (какой-то из наших 10 точек) «размазана» по разным местам программного кода, а именно по двум разным массивам.

```
int z[10];
```

Где-то появилась третья строка с `z` — что это? Может, это еще один массив одномерных точек? Или это мы вместо двумерных точек решили сделать трехмерные? Структуры данных, их логика и код размазываются тонким слоем по всему коду, тогда как здравый смысл, наоборот, диктует нам универсальное правило — взаимосвязанные и влияющие данные и код должны быть описаны как можно ближе друг к другу. Это правило универсальное: оно вообще не имеет отношения к ООП! В книге [5], например, показывается, как можно реализовать ООП в процедурном, а не объектно-ориентированном языке, что очень поучительно, а в [6] самым первым примером в секции про классы говорят, по сути, о том же самом, — хотя и не произносят слова «инкапсуляция».

```
// плохо: неявные взаимосвязи
void draw(int x, int y, int x2, int y2);
// явное лучше, чем неявное
void draw(Point from, Point to);
```

1.3. Определение классов и создание объектов

Слово «инкапсуляция» — довольно многозначное, имеющее несколько смыслов. Принципом «инкапсуляция-1» мы назовем описанное выше правило: все относящееся к описанию чего-то одного и того же целостного должно храниться вместе. Все данные (в широком смысле), описывающие один и тот же объект (в широком смысле), должны храниться в одной структуре.

Очевидно, что именно для этого и придумали такие составные типы данных, как массивы и структуры, а в нашем случае — классы:

```
class Point {
public:
    int x;
    int y;
};
```

Этот код определяет новый класс, задавая его как набор свойств и их типов. Значений, как мы помним, у свойств класса нет, — они появляются только при создании объектов этого класса.

Если в реальной жизни объекты уже существовали и про свои классы ничего не знали, пока наш разум не занялся их классификацией, то в той версии ООП, которую мы будем рассматривать (классовое ООП, в противовес бесклассовому, прототипно-ориентированному ООП), все наоборот: сначала создаются классы, а потом уже создаются объекты-экземпляры этих классов. Создание объектов и работа с ними напрямую или через указатели большей частью совершенно повторяет работу с обычными переменными. Это, кстати, дает нам возможность говорить, что значения обычных типов, например целые числа, — это тоже объекты такого упрощенного класса `int`, — аналогия позволяет нам так думать (хотя компилятор часто и считает иначе). Некоторые языки идут еще дальше и в явном виде постулируют, что все, в том числе и значения примитивных типов, например числа, есть объект. Примеры показаны в табл. 1.1.

Таблица 1.1

Действие	«Простые значения»	Объекты
Обычное создание	<code>int a;</code> <code>int b;</code>	<code>Point a;</code> <code>Point b;</code>
Создание через указатель	<code>int *pa = new int;</code> <code>int *pb = new int;</code>	<code>Point *pa = new Point;</code> <code>Point *pb = new Point;</code>
Копирование объектов	<code>b = a;</code> <code>*pb = *pa</code>	<code>b = a;</code> <code>*pb = *pa;</code>
Копирование указателей	<code>pb = pa;</code>	<code>pb = pa;</code>
Удаление через указатель	<code>delete pa;</code> <code>delete pb;</code>	<code>delete pa;</code> <code>delete pb;</code>

В первой строке слева создаются две переменные, содержащие два значения типа `int`, а справа аналогичным образом создаются две переменные, содержащие два объекта типа `Point`. Ну или можно, наоборот, сказать, что справа создаются два объекта класса `Point`, а слева создаются два объекта класса `int`, — это уж как вам привычнее смотреть на вещи. Обратите внимание, что мы как будто только объявляем переменные, но при этом объекты уже создаются (и объекты типа `int`, и объекты типа `Point`): под них выделяется место в памяти, и с этими объектами уже можно работать. Такой способ создания объектов есть в C++, но его нет в некоторых других языках. Кто-то называет такой способ создания объектов «статическим» (не путать со словом `static`!), другие называют его «создание на стеке», третьи — «автоматическое создание»: выберите любой из понравившихся вариантов.

Во второй строке таблицы создаются те же самые два объекта `int` и два объекта `Point`, но другим способом: кто-то назовет его «динамическое создание», кто-то — «создание на куче». Нам же важно, что такой способ (с точностью до син-

таксиса) используется чаще всего в объектно-ориентированных языках. Его особенность в том, что в этой строке происходит сразу несколько действий:

```
Point *pa = new Point;
```

Во-первых, создается переменная-указатель `pa`; во-вторых, `new` создает где-то в динамической памяти («на куче») новый объект класса `Point` и возвращает его адрес; в-третьих, присваиванием этот адрес заносится в переменную `pa`, которая — вот удача-то! — как раз и предназначена для хранения адресов. Как я уже говорил, этот способ чаще всего используется в объектно-ориентированных языках (в разном синтаксисе), и его главная особенность заключается именно в том, что создание указателя («ссылки» или других аналогов в других языках) неравнозначно созданию объекта, а представляет собой отдельное действие. Можно даже разнести эти два действия в разные строки:

```
Point *pa;  
pa = new Point;
```

В первой строке создается переменная-указатель (она получает какое-то «мусорное» значение, а значит, пока не указывает ни на какой объект), а во второй строке создается объект `Point` и его адрес складывается в переменную-указатель.

Возвращаясь к нашей таблице: третья и четвертая строки там не зря абсолютно совпадают: это означает, что синтаксически работать с переменными, содержащими объекты, можно точно так же, как с переменными, содержащими обычные, «простые» значения. При копировании одной целочисленной переменной в другую содержимое одной области памяти, где хранится битовое представление числа, побитово копируется в другую область памяти, и тем самым переменные не становятся одной и той же переменной (они лежат в разных местах памяти), но они после такого присваивания хранят одно и то же значение. Если в переменной лежит не `int`, а объект `Point`, то с ним произойдет то же самое: область памяти, которую занимает объект `Point` (а значит, и оба значения `int` координат внутри него), будет побитово скопирована в область памяти, где лежит другой объект `Point`. Отметим, что такое поведение еще называется *value semantics*, и так ведут себя объекты в C++ и в некоторых других (но не во всех) объектно-ориентированных языках.

В четвертой строке произойдет одно и то же некорректное (в нашем случае) действие, что с `int`, что с `Point`. Почему некорректное? Потому что `pa` и `pb` были двумя разными указателями, которые указывали (хранили адрес) на разные места в памяти, а после выполнения копирования указателей оба указателя хранят адрес одного и того же объекта (`pa`). А что же с объектом, на который указывал раньше `pb`? Он так и остается жить где-то в памяти, но его адрес больше не хранится ни в какой переменной-указателе, а значит, мы, скорее всего, никак больше не сможем получить к нему доступ. Такая ситуация приводит к так называемой утечке памяти: память занята, но ни использовать, ни даже освободить ее мы больше не можем.

Означает ли это, что указатели нельзя присваивать друг другу? Конечно, нет, это можно и нужно делать! Просто при этом нужно очень тщательно контролировать, какой из указателей указывает на какой объект, и не допускать незапланированного появления объектов, на которые не указывает ни один указатель.

В пятой строке мы удаляем два объекта, созданных через указатель, и выглядит это одинаково для простых значений и для объектов. Больше про удаление объектов говорится в *разд. 1.7*, посвященном их жизненному циклу.

1.4. Работа с объектами

Как теперь работать с объектом, не копируя его целиком? Переменным простых типов мы можем очевидным образом присваивать значения, а наши составные объекты, объекты-экземпляры созданных нами классов состоят из множества свойств, имеющих разные значения. Для доступа к значениям свойств у объектов мы можем использовать операции «точка» («.») и «стрелка» («->»), соответственно для переменных-объектов и переменных-указателей (в большинстве других языков используется только «.»):

```
Point a;  
a.x = 5;  
a.y = 6;  
Point *pa = new Point;  
pa->x = 5;  
pa->y = 6;
```

Обратите внимание, что, используя обычный способ разыменования указателей, без стрелочки можно было бы и обойтись, но выглядит это гораздо хуже (скобки нужны, потому что приоритет у операции «.» выше, чем у операции «*»):

```
Point *pa = new Point;  
(*pa).x = 5;  
(*pa).y = 6;
```

Теперь, когда мы научились разными способами создавать объекты и менять значения их свойств, надо научиться передавать их в функции. Например, нам нужна функция, которая возвращает квадрат расстояния между двумя точками. Очевидно, она будет принимать две точки и возвращать целое число:

```
int dist(Point p1, Point p2){  
    return (p1.x - p2.x)*(p1.x - p2.x) +  
           (p1.y - p2.y)*(p1.y - p2.y);  
}
```

Или функция, проверяющая, находится ли точка внутри области от 0 до 100 по обоим координатам:

```
bool bound100(Point p1){  
    return (p1.x > 0) && (p1.x < 100) &&  
           (p1.y > 0) && (p1.y < 100);  
}
```

Или функция, присваивающая заданные значения обеим координатам точки:

```
void set(Point p1, int x, int y){
    p1.x = x;
    p1.y = y;
}
```

Отлаживая эту последнюю функцию, вы можете наткнуться на интересный эффект:

```
Point p;
p.x = 0;
p.y = 0;
set(p, 5, 5);
// p.x => 0
// p.y => 0
}
```

Вызвав эту функцию для точки `p` в режиме отладки, вы можете убедиться, что после ее вызова свойства `x` и `y` как были у объекта нулевыми, так и остаются. Почему так? Да потому, что аргументы в функцию в C++ по умолчанию передаются «по значению», т. е. копированием значения, и когда мы пытаемся передать в функцию `set` наш объект `p`, неявно создается копия этого объекта и свойства меняются именно у копии! Это явно не то, что нам нужно, — мы хотим изменить именно тот объект, который мы в функцию передаем, а не его копию. Для этого, как вы знаете, и можно использовать механизм передачи аргументов в функцию «по ссылке», при этом копирования не происходит и функция работает именно с переданной в функцию переменной, а не с ее копией:

```
void set(Point &p1, int x, int y){
    p1.x = x;
    p1.y = y;
}
```

В вызывающем коде при этом ничего не изменится, но поведение будет именно то, которого мы ждем:

```
Point p;
p.x = 0;
p.y = 0;
set(p, 5, 5);
// p.x => 5
// p.y => 5
}
```

Другой вариант добиться того же, что нам нужно, — передавать в функцию не саму точку, а указатель на нее. Но и внутри функции тогда придется со значениями работать по-другому и вызывать ее для существующей точки тоже придется по-другому:

```
void set(Point *p1, int x, int y){
    p1->x = x;
    p1->y = y;
}
```

```
Point p;
p.x = 0;
p.y = 0;
set(&p, 5, 5);
// p.x => 5
// p.y => 5
}
```

А что с предыдущими двумя функциями? Там тоже происходит копирование? Именно так, но оно нам там не мешает, потому что мы не изменяем свойства переданных объектов, а возвращаем результат примитивного типа. Однако копирование есть копирование, оно занимает какое-то время, и если такие функции вызывать для объектов 100 500 раз, время на копирование может стать существенным (конечно, не с такими примитивными классами, как у нас, где два `int` будут размером как раз равны одному указателю). Поэтому и в случае таких функций при отсутствии прямой необходимости копировать передаваемый объект есть смысл передавать его «по ссылке»:

```
int dist(Point &p1, Point &p2){
    return (p1.x - p2.x)*(p1.x - p2.x) +
           (p1.y - p2.y)*(p1.y - p2.y);
}

bool bound100(Point &p1){
    return (p1.x > 0) && (p1.x < 100) &&
           (p1.y > 0) && (p1.y < 100);
    p1.x = 0;
}
```

Но теперь другая беда — внутри такой функции мы можем случайно, сами того не желая, изменить свойства переданной точки. Пока это была копия, большой беды в этом не было, а теперь нужно или внимательно следить, чтобы не «испортить» переданные аргументы, или синтаксически запретить их изменение (теперь этот код не скомпилируется):

```
int dist(const Point &p1, const Point &p2){
    return (p1.x - p2.x)*(p1.x - p2.x) +
           (p1.y - p2.y)*(p1.y - p2.y);
}

bool bound100(const Point &p1){
    return (p1.x > 0) && (p1.x < 100) &&
           (p1.y > 0) && (p1.y < 100);
    p1.x = 0;
}
```

1.5. Методы

Давайте теперь внимательно посмотрим на функции для работы с нашим объектом класса `Point`:

```
void set(Point &p1, int x, int y){
    p1.x = x;
    p1.y = y;
}

void shift(Point &p1, int dx, int dy){
    p1.x = p1.x + dx;
    p1.y = p1.y + dy;
}

void reset(Point &p1){
    p1.x = 0;
    p1.y = 0;
}
```

Что у них у всех общего? Во-первых, они все работают с нашей точкой `Point`, и это уже проблема, потому что класс `Point` объявлен в одном месте кода, а функции эти мы можем написать где угодно. Почему это проблема? Да потому что это все то же нарушение принципа «инкапсуляция-1» — все, относящееся к описанию точки `Point`, должно храниться в одном месте, а правило сдвига точки, описываемое функцией `shift`, безусловно относится или должно относиться к описанию точки `Point`. Во-вторых, у всех этих функций общим является то, что они работают с внутренним состоянием какой-то точки `Point`, и то, с какой точкой они должны работать, передается в них первым аргументом. Это как будто не проблема, но, согласитесь, хочется как-то подчеркнуть их отличие от всех остальных функций, принимающих другие аргументы.

Именно для этого и нужно такое понятие, как метод. *Метод* — это функция, объявленная внутри класса и имеющая доступ к полям этого класса. Объявляются методы вот так:

```
class Point {
public:
    int x;
    int y;
    void set(int x, int y){
        x = x;
        y = y;
    }
    void shift(int dx, int dy){
        x = x + dx;
        y = y + dy;
    }
}
```

```
void reset(){
    x = 0;
    y = 0;
}
};
```

Если мы посмотрим, например, в метод `reset`, то увидим, что из параметров пропал параметр с точкой, к которой следовало применять изменения. Чьи тогда `x` и `y` изменяются внутри этой функции? А это и есть главная особенность метода: он выполняется в контексте того объекта, у которого вызывается, т. е. имеет доступ к атрибутам того объекта, для которого вызывается, — т. е. это будут `x` и `y` той точки, у которой мы его будем вызывать!

```
Point a;
Point *pb = new Point;
a.reset();
pb->reset();
```

Если поставить брейкпоинт внутри метода `reset`, то мы туда попадем два раза, и в первом случае `x` и `y` будут принадлежать точке `a`, а во втором случае — точке `pb`. Как видим, обращение к методам выполняется точно так же, как к свойствам, — через точку для объектов и через стрелку к указателям на объекты.

Тут есть один нюанс, и он касается функции `set`. Внутри этой функции упоминаются `x` и `y`, но чьи это `x` и `y`? Может быть, это свойства самого объекта, как в функциях `reset` и `shift`? А может быть, это имена параметров? Чтобы не возникало таких неопределенностей (у нас, конечно, у компилятора никаких неопределенностей нет, для него `x` и `y` — это аргументы метода), можно переименовать параметры у этого метода, а можно воспользоваться специальным ключевым словом `this`, которое неявно доступно внутри любого метода и обозначает указатель на тот объект, для которого метод сейчас вызван:

```
class Point {
public:
    int x;
    int y;
    void set(int x, int y){
        this->x = x;
        this->y = y;
    }
};
```

Ключевое слово `this` доступно внутри каждого метода так, как будто у каждого метода есть невидимый первый параметр с таким именем:

```
class Point {
public:
    int x;
    int y;
```

```
void set(Point *this, int x, int y){
    this->x = x;
    this->y = y;
}
};
```

В других языках будет другое ключевое слово, и оно будет означать такой же неявный аргумент, а в некоторых языках не стали ничего прятать и так и оставляют первым параметром любого метода класса аргумент, обозначающий объект, для которого метод вызывается.

Как быть с теми функциями, которые принимали два объекта `Point`? Главное, что нужно сначала сделать, — это поместить себя внутрь объекта и начать смотреть на все изнутри объекта. Если раньше мы говорили, что мы хотим искать расстояние между двумя точками, то теперь, глядя изнутри одной точки, мы скажем, что мы хотим найти расстояние до другой точки:

```
class Point {
public:
    int x;
    int y;
    int distTo(const Point &p2){
        return (x - p2.x)*(x - p2.x) +
            (y - p2.y)*(y - p2.y);
    }
};
```

Видите, как пропал первый аргумент из этой функции? Он не нужен, потому что мы вызываем эту функцию для какой-то точки и передаем поэтому только вторую. Ключевое слово `const` при этом все равно осталось, потому что здравый смысл предлагает нам защищать от случайных изменений все, что не должно по задумке меняться. Ну, и название функции было изменено, чтобы лучше отражать ее изменившийся смысл.

Это так важно, что часто под инкапсуляцией понимают именно объединение в одной сущности (в классе) данных и функций, которые их обрабатывают. Мы говорили, что инкапсуляция — слово многозначное, так вот это — «инкапсуляция-2». И снова к ООП она имеет довольно опосредованное отношение, потому что принцип универсален: данные и код, который их обрабатывает, сильно взаимосвязаны и поэтому должны всегда жить вместе.

Еще одна фишка, которую стоит рассказать про методы, заключается в том, что их можно только объявлять внутри класса, а саму реализацию писать уже после него. В случае если методы содержат много кода, это позволяет само объявление класса сделать небольшим и легко читаемым. Выглядит это вот так:

```
class Point {
public:
    int x;
    int y;
```

```
int distTo(const Point &p2);  
};  
  
int Point::distTo(const Point &p2){  
    return (x - p2.x)*(x - p2.x) + (y - p2.y)*(y - p2.y);  
}
```

Итак, принцип «инкапсуляция-2» утверждает, что все функции, тесно работающие с какими-то данными, должны быть объявлены методами этих данных внутри какого-то класса.

Пропустите эту пару абзацев, если вы читаете книгу первый раз. Справедливо-сти ради следует сказать, что есть и другое мнение. В [6] предлагается функцию делать методом класса, только если ей требуется доступ к внутренним полям класса. А если функция работает с классом только через публичные свойства и методы, то предлагается ее так и оставить отдельной функцией (так называемой хелпер-функцией). Легко понять, что принцип инкапсуляции при этом трещит по швам, зато существенная часть объекта (эти самые хелпер-функции, пусть они и не будут находиться внутри класса) не будет зависеть от его внутренней реализации. Кто я такой, чтобы говорить что-то поперек [6], но, с другой стороны, не именно ли для этого как раз и придуман описанный в *главе 2* принцип Pimpl и паттерн Bridge? Вернитесь сюда, когда разберетесь с этим паттерном, и обдумайте эту проблему еще раз. Что касается инкапсуляции-1 как объединения информации, относящейся к одной и той же сущности или одной и той же стороне проблемы, то и этот принцип не является универсальной «серебряной пулей». Если у нас есть телевизоры и кронштейны для крепления телевизоров к стене, то с чем должна быть «инкапсулирована» инструкция об использовании конкретного кронштейна с конкретным телевизором? Смысловой частью чего она является: телевизора или кронштейна? Какой бы вы ни выбрали вариант, вам могут сказать, что вы нарушаете инкапсуляцию. Жизнь всегда сложнее любого принципа (кроме того, который заявлен в этом самом предложении).

1.6. Соккрытие свойств и методов

Мы создавали классы, чтобы хранить вместе данные и функции, их изменяющие. Но смысл инкапсуляции не только в этом. Так или иначе фактически, в конце концов, мы храним данные в переменных простейших типов — в числах, строках и логических значениях. Хранимые данные так или иначе часто связаны с реальной жизнью и призваны ее отражать и моделировать, а реальная жизнь подчиняется физическим законам и накладывает ограничения на возможные значения параметров и их комбинаций. Имея класс `Point`, который моделирует, например, пиксел на экране, мы вынуждены ограничивать значения `x` и `y` целыми положительными числами в пределах ширины и высоты экрана. Имея класс прямоугольника `Rect` с шириной и высотой, мы можем сами ввести ограничение о том, что ширина должна быть больше высоты, а если это не так, то

менять их местами. Другими словами, при создании сложных объектов данных мы можем иметь в виду некоторые жизненные ограничения, называемые бизнес-правилами или бизнес-логикой, которую мы хотим поддерживать и не допускать произвольных значений свойств.

Казалось бы, в чем проблема? Если значения свойств должны иметь определенный вид, то пусть они такой вид и имеют? Если мы не хотим, чтобы значения свойств точки x и y могли быть только целыми положительными в пределах FullHD, то просто не надо присваивать им значения вне этого диапазона? Проблема в том, что доброй воли, как правило, недостаточно для поддержания целостности данных, потому что над кодом работает каждый день новый человек (даже если это вы сами), который может забыть или случайно допустить внесение в данные некорректных изменений. Добрым словом и физическим запретом, перефразируя известную поговорку, можно добиться гораздо большего, чем просто добрым словом.

Для обеспечения целостности данных и выполнения бизнес-правил на уровне языка используется механизм сокрытия доступа к свойствам и методам с помощью специальных слов `private` и `public`. Выглядит это так:

```
class Point {
private:
    int x;
    int y;
public:
    void set(int x, int y){
        this->x = x;
        this->y = y;
    }
};
```

В классе `Point` теперь свойства x и y объявлены приватными с помощью слова `private`, и семантика, смысл этого действия в том, что все приватные свойства и методы доступны только внутри методов этого класса. Это значит, что попытка скомпилировать следующий код окажется неудачной:

```
Point p;
p.x = -42;
p.y = -42;
```

Свойства x и y недоступны нигде в коде, кроме как в методах класса. И ваша среда разработки даже, скорее всего, не будет показывать их в выпадающем списке, когда вы напишете «`p.`». Для вас все будет выглядеть так, будто этих свойств у `Point` вообще нет, а в списке доступных свойств и методов будут отображаться только публичные (`public`) свойства и методы. А значит, если мы хотим поддерживать какое-нибудь бизнес-правило на x и y , мы можем реализовать изменения этих свойств только через публичные методы, обеспечить в этих методах выполнение этих правил, и тогда будем уверены, что никто не сможет их нарушить:


```
class Point {  
private:  
    int x;  
    int y;  
public:  
    void set(int x, int y){  
        if ((x > 0) && (x <= 1920))  
            this->x = x;  
        if ((y > 0) && (y <= 1080))  
            this->y = y;  
    }  
};
```

Вообще говоря, любое бизнес-правило может поддерживаться разными способами, и в зависимости от ситуации вы будете по-разному реагировать на попытку его нарушить. В приведенном выше примере мы просто молча не изменяем значения *x* и *y*, если новые значения пытаются нарушить поддерживаемые нами ограничения, но это не единственная возможная реакция. В других ситуациях мы могли попытаться выставить в этом случае значения *x* и *y*, которые максимально близки к желаемым, а в каких-то ситуациях мы в этом случае подняли бы *exception*. Дальнейшее развитие этой темы идет в направлении MVC (см. разд. 2.31).

Если уж отвлекаться от темы, то любая модель, которая поддерживает ограничения на свое внутреннее состояние (а объект программный очень часто как раз и является моделью какого-то объекта реального), может реализовывать три описанных выше поведения: игнорирующее (действия, нарушающие правила, игнорируются), корректирующее (действия, нарушающие правила, минимально корректируются, чтобы правила не нарушать) и запрещающее (действия, нарушающие правила, вызывают ошибку или исключение).

Если вы заметили, то везде раньше, где мы объявляли классы, мы были вынуждены сразу писать ключевое слово `public`. Это потому, что по умолчанию все объявляемые свойства и методы класса в C++ являются `private` (это отличает их, кстати, от очень похожей конструкции `struct` того же самого C++, где члены класса по умолчанию, наоборот, `public`).

Таким образом, с помощью слов `private` и `public` мы как бы разделяем любой класс, любой объект на две части — открытую и скрытую. Открытой частью любого объекта являются его публичные свойства и методы, доступные для использования: эту часть объекта можно назвать его интерфейсом (еще одно очень многозначное слово), это то, как объект выглядит снаружи для наблюдателя, и то, как им будут пользоваться. Закрытой частью любого объекта являются все остальные его члены (приватные и некоторые другие непубличные). Разделение объекта на открытую и закрытую части — это третье значение, которым наделяется слово «инкапсуляция».

Будем называть «инкапсуляцией-3» сокрытие внутренней реализации объекта от непреднамеренных изменений. По сути, любой сложный технический прибор

обязательно реализует этот принцип. Взять хотя бы любую микроволновку: она обязательно реализует инкапсуляцию, скрывая внутри себя магнетрон с несколькими тысячами вольт от случайного доступа вашими пальцами и оставляя снаружи только «интерфейс»: доступные вам публичные свойства и методы микроволновки — всякие там кнопки и крутки. Очевидно, если только вы не Креосан, вы будете благодарны создателям микроволновки и ее инкапсуляцию нарушать не будете.

Общее правило использования приватных и публичных свойств — простое: все по умолчанию должно быть скрыто, а открывать с помощью `public` нужно только как можно более ограниченный набор свойств и методов, который к тому же должен быть тщательно продуман, чтобы его как можно реже пришлось менять. Почему последнее важно? Потому что если вы меняете интерфейс объекта (его публичные методы), то вам, скорее всего, придется менять тот код, который этими публичными свойствами и методами пользуется. А если вы меняете внутреннюю реализацию объекта, то на использующем коде это, скорее всего, не скажется.

Порядок объявления секций `private` и `public` языком чаще всего не определен, но любая группа программистов может придерживаться в этом определенной системы, как и с любыми другими самоограничениями, типа именования переменных. Например, часто используется подход, при котором обязательно сначала должны объявляться публичные свойства и методы, определяющие интерфейс класса, а только затем уже — приватные.

1.7. Конструкторы и деструкторы, жизненный цикл объектов

Вернемся к нашему примеру с точкой:

```
class Point {
private:
    int x;
    int y;
public:
    void set(int x, int y){
        if ((x > 0) && (x <= 1920))
            this->x = x;
        if ((y > 0) && (y <= 1080))
            this->y = y;
    }
};
```

Что мы делаем первым делом после того, как создаем новый объект? Похоже, всякий раз нам приходится начинать с того, чтобы вызывать у него функцию `set` для установки начальных значений свойств:

```
Point a;  
a.set(0, 0);
```

До вызова функции `set` (поставьте брейкпоинт и убедитесь!) значения свойств `x` и `y` не определены и содержат всякий разный мусор! Да, возможно, компилятор за вас постарается и занулит их значения. Попробуйте сделать вот так и убедитесь, что у `pb`, в отличие от `pa` и `a`, значения свойств занулены, — но будет ли это так в другом вашем любимом языке и другом компиляторе? А если там осталась память от чужого процесса и пароли в содержимом этой памяти?

```
Point a;  
Point *pa = new Point;  
Point *pb = new Point();
```

Кстати, такой же трюк с объектом `a` не пройдет (попробуйте!): компилятор будет `Point a();` воспринимать как объявление функции, а не объект.

Хочешь все сделать хорошо — сделай это сам, говорил популярный киногерой, и переносимым, хорошим, годным стилем будет инициализировать все значения свойств самостоятельно (а иногда еще и деинициализировать: ни байта значащей информации врагу!). Но что теперь, у каждого класса создавать функцию `set` или обеспечить какой-то общий порядок? Для этого (а также для много другого) в ООП-языках придумано понятие конструкторов.

Конструктор — это специфический метод, который вызывается автоматически при создании объекта и предназначен для инициализации начальных значений всех его свойств, выделения нужных ресурсов, а также выполнения всех прочих действий, которые должны быть выполнены при создании объекта. В разных языках конструкторы описываются по-разному, но, как и для всех остальных приемов ООП, главное — это идеи, и идея конструктора одна и та же везде. В C++ конструкторы, в отличие от обычных методов, имеют имя, совпадающее с именем класса, и не имеют возвращаемого значения:

```
class Point {  
private:  
    int x;  
    int y;  
public:  
    Point() {  
        x = 0;  
        y = 0;  
    }  
    Point(int x, int y) {  
        this->x = x;  
        this->y = y;  
    }  
    Point(const Point &t) {  
        x = t.x;
```

```
y = t.y;  
}  
};
```

Ух ты, да тут не один конструктор, а сразу три! Зачем нам может понадобиться три разных конструктора? Да потому что нам может понадобиться много разных удобных способов создавать объекты какого-то класса. Поэтому — больше конструкторов, хороших и разных, но вот эти три особенно важны.

Первый из них называется конструктором по умолчанию. Он будет использоваться, когда нам нужно просто создать объект, значения свойств которого мы пока заполнять не знаем как, но мусора там допускать не хотим (например, потому, что мусор может сразу нарушать наши бизнес-правила для объекта).

Второй конструктор называется конструктором с параметрами. Он не такой особенный, но мы будем такие конструкторы использовать, когда хотим создать объект, полностью задавая у него сразу все значения свойств.

А вот третий конструктор, как и первый, тоже имеет для компилятора особенное значение. Он называется конструктором копирования и предназначен для такого создания объекта, при котором значения всех его свойств должны быть взяты из объекта-прототипа, который передается как раз для этого в качестве параметра. Обратите внимание, что объект-прототип обязательно передается по ссылке, а не по значению или указателю. В других, более высокоуровневых языках, где нет ни указателей, ни ссылок, ни объектных значений (а точнее, все есть ссылка), конструктор копирования тоже будет, просто оформляться будет по-другому. Если вы знаете, чего хотите, то всегда загуглите, как этого добиться.

Когда будут вызываться эти конструкторы? Если не рассматривать передачу и возвращение объектов в функции (в отдельной главе!), то вот практически все примеры:

```
Point a;  
Point b(5,6);  
Point c(a);  
Point d = a;
```

В первой строке тут вызывается конструктор по умолчанию, во второй — конструктор с параметрами, в третьей — конструктор копирования, в четвертой — тоже конструктор копирования. Может показаться, что в четвертой строке на самом деле должен вызываться конструктор по умолчанию, после чего выполняться побитовое копирование одного объекта в другой. Но обвешайтесь брейк-поинтами и выполните для сравнения вот такой код и убедитесь, что он не эквивалентен:

```
Point a;  
Point d;  
d = a;
```

Стандарты разных версий C++ описывают много особенностей и разных правил вызовов конструкторов, включающих избегание копирования (copy elision), выбор наиболее подходящего по фактическим параметрам конструктора (overload resolution применительно к конструкторам). Есть еще один способ вызвать конструктор, но он тоже специфичный для C++, поэтому я рассказываю вам про него только потому, что начинающие часто такой косяк допускают:

```
Point a = Point(5,6);
```

Если у вас есть все показанные выше конструкторы с отладочным выводом и отключена оптимизация, то вы с удивлением для себя выясните, что в этой строке происходит очень многое. Сначала по вызову `Point(5,6)` создается точка (безымянная) с координатами (5, 6), потом создается объект `Point a` с помощью конструктора копирования этой безымянной точки, а потом, как только заканчивается текущая команда и мы выходим из области видимости безымянной точки, она уничтожается. И зачем, спрашивается, нам была нужна эта безымянная точка, если мы просто могли нужную нам `Point a` создать с помощью конструктора с параметрами?

Для динамически создаваемых объектов все очень похоже:

```
Point *pa = new Point;  
Point *pb = new Point();  
Point *pc = new Point(5,6);  
Point *pd = new Point(*pa);  
Point *pe = new Point(pa);
```

В первых двух строках вызывается конструктор по умолчанию, в третьей строке — конструктор с параметрами, а в четвертой строке — конструктор копирования. Нам приходится в четвертой строке разыменовывать указатель `pa`, потому что конструктор копирования принимает сам объект, а не указатель. По этой же причине пятая строка не компилируется вообще — с указанием, что нужный конструктор отсутствует (но, кстати, вам ничего не мешает его тоже добавить).

Надо понимать, что вызов конструктора — это часть процесса создания объекта, и вызывается он после того, как память под сам объект (под все его свойства) уже была выделена, поэтому внутри конструктора мы можем так смело обращаться к свойствам объекта, хотя инициализация объекта еще не завершена. Кроме этого, раз конструктор не возвращает значения, то показать вызывающему коду, что в конструкторе произошел какой-то косяк и объект не должен был создаваться, можно только подняв исключение. В этом случае, и в случае если исключение (ошибка) произошло по независящим от нас причинам внутри конструктора, создание объекта будет отменено и вся память освобождена (про освобождение памяти еще будет дальше).

Еще одна фишка конструкторов, специфическая для C++, заключается в возможности использовать список инициализации, чтобы заполнять значениями свойства не в теле конструктора, а как бы до вызова тела самого конструктора.

Выглядит это так:

```
class Point {
private:
    int x;
    int y;
public:
    Point(): x(0), y(0) {
    }
    Point(int x, int y): x(x), y(y) {
    }
    Point(const Point &t): x(t.x), y(t.y) {
    }
};
```

Можно еще и собственный (другой) конструктор вызвать в том же списке инициализации:

```
//...
    Point(int x): Point(x, x) {
    }
//...
```

В новых версиях стандарта C++ добавляется еще один тип конструктора: конструктор ~~цыганский~~ перемещения, похожий на конструктор копирования, но предназначенный для того, чтобы отказаться от лишнего перевыделения и копирования ресурсов в процессе создания копии. Если копирующий конструктор обязан сохранить исходный объект «живым», то конструктор перемещения имеет право (и пользуется им) просто «забрать» свойства образца себе, а не создавать их копию (класс неполный, тут еще согласно «правилу пяти» должны быть оператор присваивания и оператор перемещения):

```
class Vector {
private:
    int size = 0;
    float* arr = nullptr;
public:
    Vector(){
    }
    Vector(const Vector &v) :
        size(v.size), arr(new float[v.size]) {
        // копируем значения из v.arr в arr
    }
    Vector(Vector &&v) : size(v.size), arr(v.arr) {
        v.size = 0;
        v.arr = nullptr;
    }
};
```

```
~Vector() {  
    delete[] arr;  
}  
};
```

Если мы говорим об использовании констант для инициализации значений свойств, то в C++ есть еще целых два варианта для этого, например так:

```
class Point {  
private:  
    int x = 0;  
    int y = 0;  
public:  
    Point() {  
    }  
    Point(int x, int y): x(x), y(y) {  
    }  
    Point(const Point &t): x(t.x), y(t.y) {  
    }  
};
```

...или вот так (рекомендуемый вариант по [6]):

```
class Point {  
private:  
    int x{0};  
    int y{0};  
public:  
    Point() {  
    }  
    Point(int x, int y): x(x), y(y) {  
    }  
    Point(const Point &t): x(t.x), y(t.y) {  
    }  
};
```

Чем эти три варианта хороши по сравнению с инициализацией в теле конструктора, особенно последние два? Во-первых, тем, что если (как это обычно делается) объявление класса помещают в h-файл, а реализацию — в cpp-файл, то инициализация свойств значениями по умолчанию будет прописана в заголовочном h-файле, а значит, будет на виду, сложнее будет забыть инициализировать новое добавленное свойство. Во-вторых, в двух последних случаях инициализация будет выполняться именно там, где сами свойства объявляются: «связанное храни вместе». Правда, говорят, раз эти константы попадают в h-файл, то, если понадобится их изменить, перекомпилировать по цепочке придется гораздо больше кода.

Во всех трех конструкторах после объявления и двоеточия через запятую перечисляются имена свойств и задаваемые им значения. Во многих случаях само

тело конструктора при этом может остаться вообще пустым, — но не оставляйте его пустым, пока изучаете ООП! — добавьте во все конструкторы отладочный вывод, чтобы понять, какой конструктор когда вызывается!

Если вы не создадите свой собственный конструктор по умолчанию или конструктор копирования, компилятор создаст их за вас, но, т. к. там не будет отладочного вывода, вы можете и не узнать, что у вас создан лишний объект или копия объекта. Нужно еще иметь в виду, что конструктор копирования по умолчанию будет запускать конструкторы копирования для всех объектных атрибутов класса и побитово копировать атрибуты обычных типов.

Как создаются объекты и какой код при этом вызывается, мы уже увидели. Но общее правило заключается в том, чтобы, думая о создании объектов, всегда сразу думать и об их уничтожении.

Мы видели, что создаваемые «статически», «на стеке», «автоматически» объекты создаются прямо там, где объявляются соответствующие переменные. Для таких объектов выполняется правило: они уничтожаются точно так же автоматически, как только поток исполнения программы выходит из их области видимости (выходит обычным порядком или «выпрыгивая» по исключению — не важно).

```
{  
    // ... 1  
    {  
        Point a;  
        // ... 2  
        {  
            // ... 3  
        }  
        // ... 4  
    }  
    // ... 5  
}
```

Если говорить про объект `Point a` в данном примере, то в точке 1 он еще не создан и не доступен, в точке 2 он уже создан и доступен, в точке 3 он еще доступен, в точке 4 он все еще доступен, несмотря на то, что закрывающая скобка уже была (это была не та закрывающая скобка), и наконец, в точке 5 он уже не доступен, потому что прямо перед точкой 5 он был автоматически уничтожен.

Как понять, был объект уничтожен или нет? Можно ли привязать какие-то действия к тому моменту, когда уничтожается объект? Можно. Для этого и существуют другие специальные методы у классов, называемые деструкторами. *Деструктор* — это специфический метод без параметров, который вызывается автоматически при уничтожении объекта и предназначен для освобождения ресурсов, если таковые были выделены в конструкторе, а также выполнения всех прочих действий, которые должны быть выполнены при уничтожении объ-

екта. В разных языках деструкторы описываются по-разному, но, как и для всех остальных приемов ООП, главное — это идеи, и идея деструктора одна и та же везде. В C++ деструкторы, в отличие от обычных методов, имеют имя, совпадающее с именем класса, перед которым поставили тильду «~», и не имеют возвращаемого значения:

```
class Point {
private:
    int x;
    int y;
public:
    Point() {
        x = 0;
        y = 0;
    }
    ~Point() {
    }
};
```

С помощью деструктора, добавив в него отладочный вывод или поставив брейк-поинт, вы можете выяснить, когда какой объект у вас уничтожился, но, конечно, главное его предназначение в другом. Если вы в конструкторе выделили какую-нибудь память или захватили какой-нибудь ресурс, то именно в деструкторе вы обязательно должны его освободить.

Если при создании объекта сначала выделяется под него память, а потом уже вызывается подходящий конструктор, для того чтобы значения в этой памяти инициализировать, то при уничтожении объекта все происходит наоборот: сначала вызывается деструктор, а только потом память, занятая объектом, освобождается. Таким образом, когда мы находимся внутри конструктора, память уже выделена и мы имеем доступ к свойствам объекта, а когда мы находимся внутри деструктора, память еще не освобождена и мы опять-таки имеем доступ к свойствам объекта.

Итак, с автоматически создаваемыми «на стеке» объектами все просто, а вот с создаваемыми «динамически», «на куче» все сложнее:

```
{
// ... 1
{
    Point *pa = new Point();
// ... 2
{
    // ... 3
}
// ... 4
}
// ... 5
}
```

В какой момент тут уничтожится создаваемый объект `Point`? На самом деле, ни в какой! Созданный здесь объект `Point`, как и остальные создаваемые с помощью `new` объекты, не уничтожается автоматически и требует ручного уничтожения. Непосредственно перед точкой 5 будет уничтожена переменная-указатель `pa` и освобождена занимаемая ею (переменной) память, но сам объект `Point` как был где-то в другом месте в памяти, так и останется там. Если на него не осталось ни одного указателя, то у нас проблема: память есть, а никакого указателя на нее нет, и мы даже освободить ее не можем. Эти проблемы детально описаны в разд. 1.19, а мы же пока должны научиться не допускать такие ситуации и вовремя уничтожать объекты, которые больше не нужны. Для уничтожения объекта, созданного с помощью `new`, мы должны использовать `delete`:

```
{
    // ... 1
    {
        Point *pa = new Point();
        // ... 2
        {
            // ... 3
        }
        delete pa;
        // ... 4
    }
    // ... 5
}
```

Непосредственно перед точкой 4 по команде `delete pa` объект `Point`, созданный ранее, будет уничтожен. При этом сама переменная-указатель `pa` в точке 4 все еще доступна, и более того, она, скорее всего, все еще хранит адрес только что уничтоженного объекта (зависит от компилятора)! Сам объект уже уничтожен, а его адрес все еще хранится в переменной-указателе, поэтому, по-хорошему, просто глядя на значение переменной-указателя в C++, нельзя сказать, жив по этому адресу объект или уже нет (и вообще, можно ли даже обратиться по этому адресу, чтобы как-то у самого объекта спросить, жив он или нет). В точке 5 и сам объект `Point`, и переменная-указатель `pa` уже уничтожены.

Таким образом, всякий раз, когда вы где-то создаете объект с помощью `new`, вы должны обязательно следить, чтобы не существовало таких путей исполнения программы, когда объект вам больше не нужен, но он в памяти все еще существует.

Здесь я рекомендую вам прерваться и тщательно поэкспериментировать с созданием и уничтожением объектов, чтобы убедиться, что вы понимаете, когда они создаются и уничтожаются. Лучше всего для этого использовать объект со всеми нужными конструкторами и деструктором, с отладочным выводом и возможностью всегда идентифицировать, с каким объектом вы имеете дело. Например, это может быть вот такой объект:

```

class Object {
private:
    string data;
public:
    Object(){
        data = "";
        printf("#%d Object() data=\"%s\"\n",
            (int)this % 1000, data.c_str());
    }
    Object(string _data){
        data = _data;
        printf("#%d Object(string _data) data=\"%s\"\n",
            (int)this % 1000, data.c_str());
    }
    Object(const Object &o){
        data = o.data;
        printf("#%d Object(Object &o) data=\"%s\" from #%d\n",
            (int)this % 1000, data.c_str(), (int)&o % 1000);
    }
    /*Object(Object *o){
        data = o->data;
        printf("#%d Object(Object *o) data=\"%s\" from #%d\n",
            (int)this % 1000, data.c_str(), (int)o % 1000);
    }*/
    ~Object(){
        printf("#%d ~Object() data=\"%s\"\n",
            (int)this % 1000, data.c_str());
        data = "";
    }
};

```

Здесь есть и куча конструкторов, и деструктор, и строковое поле, чтобы хоть что-то в конструкторе было, и каждый объект в отладочном выводе выводит и свое строковое поле, и свой собственный адрес (его последние три цифры). Адрес нужен для того, чтобы мы могли отличать две очень разные ситуации: в одной у нас два разных объекта с одинаковым полем data, а в другой — два разных указателя на один и тот же объект.

1.8. Наследование

В *разд. 1.1* мы обнаружили, что на множестве классов определено отношение «предок-потомок», имеющее смысл отношения вхождения одного множества в другое, если класс рассматривать как множество объектов. Короче говоря, выяснилось, что некоторые классы являются более общими, а другие — более конкретными, и одни классы являются подтипами других, как класс «жилой

дом» может являться подтипом, подклассом класса «строение», а класс «квадрат» является подклассом класса «фигура».

С одной стороны, это означало, что любой квадрат одновременно является фигурой (но не наоборот), любой жилой дом является строением (но не наоборот). Пессложные размышления о свойствах классов и подклассов, с другой стороны, приводят нас к выводу, что у квадрата есть все те же свойства, которые есть у класса фигуры, плюс, возможно, еще какие-то (обратное приводило бы к противоречию: если какое-то свойство *X* есть только у фигуры, но не у квадрата, то взяв квадрат, мы обнаружим, что у него нет свойства *X* фигуры, хотя фигурой он, как подкласс, тоже является).

Класс *A* является подклассом класса *B*, если любой объект, принадлежащий классу *A*, обязательно принадлежит одновременно и классу *B*. Класс *A* тогда называется *дочерним*, а класс *B* — *родительским*, класс *A* называется *потомком*, а класс *B* — *предком*. Класс *B* при этом можно назвать более абстрактным («птицы»), а класс *A* можно назвать более конкретным («вороны»).

Если класс *A* является подклассом класса *B*, то у класса-потомка *A* есть все те же свойства, что и у класса-предка *B*, плюс, возможно, какие-то еще. Если мы захотим описать классы *A* и *B*, используя минимальное количество текста, то нам проще всего сначала описать класс *B* и его свойства, а потом, не дублируя все эти свойства в классе *A*, просто сказать, что *A* является потомком *B*, и добавить к его набору свойств новые, не переписывая заново все уже существующие.

У круга и прямоугольника, безусловно, разные наборы свойств, потому что это разные классы. Однако, посмотрев внимательно, мы можем выделить для них такой набор свойств, который принадлежит и кругу, и прямоугольнику (например, координаты центра, цвет, толщину линии и т. д. — но не радиус и не длину-ширину). Можем ли мы в этом случае сделать круг потомком прямоугольника? Или прямоугольник потомком круга? Нет и нет — потому что в одном случае один набор свойств будет подмножеством другого, а кто-то из них будет являться подтипом другого, что нашему описанию реальности не соответствует. Как же тогда быть? Неужели придется их общие свойства дублировать в обоих классах? К счастью, не придется. В нашем объектно-ориентированном взгляде на реальность и круг, и прямоугольник имеют какой-то общий набор свойств не потому, что один из них является подтипом другого, а именно потому, что они «оба» являются подтипом какого-то третьего класса, который и содержит все эти их общие свойства. Этот третий класс можно будет назвать «фигура», потому что он по смыслу как раз и будет тем общим, чем одновременно являются и круг, и прямоугольник.

Теперь — к коду. Любой класс можно объявить потомком какого-то другого класса, и он при этом получит, «унаследует» от предка все его свойства и методы:

```
class Point {  
public:  
    int x;
```

```
int y;
Point() {
    x = 0;
    y = 0;
}
Point(int x, int y) {
    this->x = x;
    this->y = y;
}
int distTo(const Point &p2){
    return (x - p2.x)*(x - p2.x) + (y - p2.y)*(y - p2.y);
}
};

class ColoredPoint: public Point {
public:
    int color;
    ColoredPoint(){
        x = 0;
        y = 0;
        color = 0;
    }
    ColoredPoint(int x, int y, int color){
        this->x = x;
        this->y = y;
        this->color = color;
    }
    bool sameColorAs(const ColoredPoint &p2){
        return p2.color == this->color;
    }
};
```

Класс ColoredPoint объявлен публичным (об этом позже) наследником класса Point, это значит, что у ColoredPoint будут и свойства `x` и `y`, и метод `distTo`, — но, кроме этого, добавляются и свойство `color`, и метод `sameColorAs`, а значит, эти все свойства и методы есть и доступны у любого объекта ColoredPoint, если его создать:

```
ColoredPoint cp;
cp.x = 5;
cp.y = 6;
cp.color = 255;
```

«Наследование» хотя и называется одним из принципов ООП, имеет, на мой взгляд, неудачное название. В реальной жизни слова «наследование», «наследник» имеют совсем другой смысл. Правильнее было бы говорить о «создании подтипов», но уж что есть, то есть.

Еще нужно всегда держать в уме, что есть «наследование» как конкретная технология, способ добиться конкретного результата — перенести свойства из одного класса в другой. И есть «наследование» именно как отношение между классами, «создание подтипов», моделирующее жизненную ситуацию, в которой объект класса-подтипа обязательно является одновременно и объектом класса-супертипа.

```
class Circle: public Point {
public:
    int radius;
    Circle(){
        x = 0;
        y = 0;
        radius = 0;
    }
};
```

В приведенном примере технически действительно в классе `Circle` должны быть все те же свойства, что в классе `Point`, плюс еще свой собственный радиус, поэтому, казалось бы, наследование оправдано. Но наша интуиция протестует: круг это вряд ли частный случай точки! С `Point` и `ColoredPoint` все вроде проще — цветная точка все-таки понимается как частный случай точки. К этой проблеме мы еще вернемся, когда будем рассматривать SOLID-принципы, а вы пока можете подумать, кто должен быть подклассом кого в случае таких классов, как «эллипс» и «круг».

При наследовании классов обязательно нужно учитывать модификаторы видимости атрибутов и методов. Если в предке какие-то свойства и методы у класса были приватными, то потомок просто не сможет использовать их в своих методах! Например, вот такой код не скомпилируется, потому что `x` и `y` объявлены в базовом классе приватными, а значит, в методах других классов, даже собственных потомков, эти свойства недоступны:

```
class Point {
private:
    int x;
    int y;
public:
    Point() {
        x = 0;
        y = 0;
    }
};

class ColoredPoint: public Point {
private:
    int color;
```

```
public:
    ColoredPoint() {
        x = 0;
        y = 0;
        color = 0;
    }
};
```

Но не делать же их публичными?! Именно для такой ситуации и существует своеобразный промежуточный вариант с использованием свойства `protected`: обозначенные этим модификатором свойства и методы доступны не только в методах самого класса, но и в методах классов-потомков:

```
class Point {
protected:
    int x;
    int y;
...
}
```

Но и само наследование может быть трех типов: `public` (чаще всего), `protected` и `private`, что показывает использование ключевого слова при указании класса-предка:

```
class ColoredPoint: public Point {
...
}
```

Технически тип наследования указывает, что будет происходить с видимостью (доступностью) атрибутов класса-предка, а именно, будет ли их видимость понижена (табл. 1.2).

Таблица 1.2

Бывший тип атрибута	Каким станет атрибут при типе наследования...		
	...public	...protected	...private
public	public	protected	private
protected	protected	protected	private
private	private	private	private

То есть при `public`-наследовании видимость атрибутов не изменяется: какие они были у предка, такими и остаются у потомка. При `protected`-наследовании `public`-члены класса-предка «понижаются» до `protected`, а все остальные останутся такими, какими были. При `private`-наследовании все члены класса-предка «понижаются» и станут `private`.

Возвращаясь к нашим двум смыслам наследования при приватном наследовании, мы можем спросить сами себя: в приведенном ниже примере является ли объект `ColoredPoint` одновременно объектом `Point`?

```
class ColoredPoint: private Point {  
private:  
    int color;  
public:  
    ...  
};
```

Это не такой простой вопрос, как может показаться. С одной стороны, ColoredPoint внутри себя содержит все свойства и методы Point, а с другой, если посмотреть на ColoredPoint со стороны, — мы не увидим в нем ничего, что говорило бы о том, что он является Point.

```
ColoredPoint cp;  
cp.distTo(...); // ???
```

Этот вопрос важен тем, что задающий его может неявно спрашивать о двух очень разных вещах, и нужно понимать между ними разницу. Например: «Является ли объект X уткой?». Для кого-то этот вопрос может означать: «Является ли объект X биологическим объектом, несущим генотип семейства Anatidae?», а для кого-то этот вопрос может означать совсем другое: «Умеет ли объект X летать, плавать и крякать?». Другими словами, для кого-то в этом вопросе важно внутреннее содержание X, его реализация, а для кого-то — внешнее поведение, интерфейс. Чтобы не путаться, в каждом конкретном случае лучше уточнить, что имеется в виду — ну, или дать максимально развернутый ответ.

В случае непубличного наследования ColoredPoint от Point мы можем сказать, что ColoredPoint безусловно «внутри» является Point, потому что содержит все свойства и методы Point, но при этом снаружи ColoredPoint «не выглядит» как Point, потому что интерфейс (понимаемый как набор его публичных методов) ColoredPoint не включает интерфейс Point.

В случае публичного наследования ColoredPoint является Point, и выглядит как Point, поэтому в этом смысле разницы при публичном наследовании нет. Публичное же наследование и используется чаще всего.

И предок, и потомок могут иметь, а могут и не иметь некоторые (или все) конструкторы и деструкторы. При этом раз потомок одновременно является внутри себя и предком тоже (ColoredPoint содержит внутри себя Point и является им), то нельзя создать объект ColoredPoint, не создав внутри себя Point. Поэтому если даже создать два вот таких простых класса и добавить во все методы отладочный вывод, то можно убедиться, что при создании объекта класса ColoredPoint всегда сначала вызывается конструктор класса Point, а затем уже конструктор класса ColoredPoint, а при уничтожении этого объекта сначала вызывается деструктор ColoredPoint, а затем деструктор Point.

```
class Point {  
public:  
    Point() {  
        }
```



```
~Point() {  
    }  
};  
  
class ColoredPoint: public Point {  
public:  
    ColoredPoint() {  
    }  
    ~ColoredPoint() {  
    }  
};
```

Всегда при создании объекта, входящего в какую-то иерархию классов, перед вызовом конструктора класса вызываются все конструкторы всех его предков, начиная с самого «дальнего». И аналогичным образом всегда при удалении объекта, входящего в какую-то иерархию, после вызова деструктора класса вызываются все деструкторы всех его предков, заканчивая самым «дальним». В некоторых других языках вызов конструктора и деструктора нужно делать самостоятельно и явно.

Но даже с учетом того, что в C++ конструкторы вызываются явно, у нас есть возможность контролировать, какой именно конструктор предка будет вызван (если не указать, какой именно, то будет вызван конструктор по умолчанию) в секции инициализации, и это очень удобно:

```
class Point {  
private:  
    int x;  
    int y;  
public:  
    Point(): x(0), y(0) {  
    }  
    Point(int x, int y): x(x), y(y) {  
    }  
    Point(const Point &t): x(t.x), y(t.y) {  
    }  
    ~Point() {  
    }  
};  
  
class ColoredPoint: public Point {  
private:  
    int color;  
public:  
    ColoredPoint(): Point(), color(0) {  
    }  
    ColoredPoint(int x, int y, int color):  
        Point(x, y), color(color) {  
    }  
};
```

```

ColoredPoint(const ColoredPoint &t) :
    Point(t), color(t.color) {
}
~ColoredPoint(){
}
};

```

Конструктор по умолчанию `ColoredPoint` вызывает конструктор по умолчанию `Point`. Конструктор с параметрами `ColoredPoint` вызывает конструктор с параметрами `Point`, конструктор копирования `ColoredPoint` — конструктор `Point`, и во всех трех случаях конструкторы затем инициализируют оставшееся свойство.

1.9. Расширение и изменение объектов при наследовании

До сих пор мы делали акцент на новых данных, которые появляются в классе-наследнике. При наследовании класс-потомок объявляет новые свойства, но неизбежно наследует старые, он не может их «удалить» или «отменить». С методами все иначе: при наследовании потомка от предка первый получает все методы второго, но при этом может их (часть или все) переопределить. На самом деле наследование часто используют для того, чтобы изменить поведение существующего класса, не внося изменения в сам существующий класс (правильный ли это способ — вопрос открытый). Итак, класс-наследник может:

- ☐ добавлять новые свойства;
- ☐ добавлять новые методы;
- ☐ перекрывать (т. е., по сути, изменять) уже существующие методы.

Понятное дело, что наследование на самом деле никак не может изменить уже существующий класс, но может создать новый класс, который в целом дублирует поведение существующего, но изменяет, скажем, один какой-то метод. Для того чтобы переопределить существующий метод, нужно объявить в потомке метод с точно такой же сигнатурой, но с другой реализацией:

```

class Point {
private:
    int x;
    int y;
public:
    Point(int x, int y): x(x), y(y) {
    }
    float area () {
        return 0;
    }
    void dump() {
        printf("x = %d, y = %d\n", x, y);
    }
};

```

```
class Circle : public Point {
private:
    int radius;
public:
    Circle(int x, int y, int radius) :
        Point(x, y), radius(radius) {
    }
    float area() {
        return 3.14 * radius * radius ;
    }
    void dump() {
        printf("radius = %d, ", radius);
        Point::dump();
    }
};
```

Как видим, тут в классе-потомке `Circle` переопределяются сразу два метода: `area` и `dump`. При этом метод `area` полностью «отменяет» функциональность метода-предка, а метод `dump` сначала выполняет какую-то дополнительную функциональность, а потом вызывает с помощью `Point::dump()` метод предка. В других языках синтаксис, как обычно, может выглядеть по-другому, но идея остается одной и той же: метод потомка не «удаляет» метод предка, а заменяет его, при этом может его вызывать. По сути, это один из стандартных приемов: не нравится, как ведет себя какой-то метод какого-то класса? Создай потомка, переопредели там этот метод и затем реализуй в нем желаемое поведение, вызывая или не вызывая унаследованную реализацию метода. В примере выше унаследованная реализация вызывается после измененной, но часто она вызывается «до», а еще чаще — «вместо».

Лучшей иллюстрацией того, что старый метод в объекте все равно остается, служит вот такой синтаксис, с помощью которого, даже имея только объект класса-потомка, можно все равно вызвать метод предка (при условии, конечно, что он не приватный):

```
Circle c(5, 6, 7);
c.dump();
c.Point::dump();
```

В первом вызове, `c.dump()`, вызывается метод `dump` класса `Circle`, а во втором вызове, `c.Point::dump()`, мы явно вызываем унаследованную, а не перекрытую версию метода.

1.10. Композиция

До сих пор мы создавали только объекты, свойствами которых были значения примитивных типов данных, но это вовсе не обязательно! Свойствами одних, более крупных объектов, могут быть другие, более мелкие объекты, например наш объект `Point`:

```
class Section {
public:
    Point pbeg;
    Point pend;
    Section() {
    }
    Section(int x1, int y1, int x2, int y2) :
        pbeg(x1, y1), pend(x2, y2) {
    }
    Section(const Section &s) : pbeg(s.pbeg), pend(s.pend) {
    }
    ~Section() {
    }
};
```

Здесь мы объявляем новый класс `Section`, который имеет два свойства `pbeg` и `pend` типа `Point`. С помощью этого класса мы будем моделировать отрезок, состоящий из двух точек, соответственно начала и конца отрезка.

В конструкторе по умолчанию у `Section` вроде бы ничего не происходит, но если обвешаться отладочным выводом, то становится ясно, что к моменту вызова тела конструктора `Section` объекты `pbeg` и `pend` уже созданы — их конструкторы уже сработали. Мы раньше говорили, что конструктор у класса вызывается в тот момент, когда память под свойства класса уже выделена, — теперь можно сделать более сильное утверждение: к моменту запуска конструктора поля класса уже инициализированы (это означает, что если они являются объектами, то для них уже вызваны их конструкторы в свою очередь).

В то же время в конструкторе `Section` с параметрами мы видим, что можно вмешаться в процесс инициализации свойств `Section`, а именно управлять тем, какими конструкторами они будут создаваться: в данном случае нам удобно сразу вызвать у `pbeg` и `pend` конструкторы с параметрами, чтобы заполнить их координаты. Аналогично в секции инициализации конструктора копирования `Section` мы делаем так, чтобы `pbeg` и `pend` создавались с помощью конструкторов копирования: они должны создаваться как копии соответственно точек `s.pbeg` и `s.pend`, где `s` — это тот отрезок, который нам надо скопировать.

В деструкторе `Section` вроде бы ничего не происходит, но, опять-таки, отладочный вывод поможет вам выяснить, что сразу после выхода из тела деструктора `Section` произойдет уничтожение объектов `pbeg` и `pend`, а значит, выполнятся их деструкторы и освободится занимаемая ими память.

Такая зависимость между классами, когда объекты класса `Б` являются атрибутами класса `А` и при этом объекты класса `Б` создаются при создании объектов класса `А` и уничтожаются при уничтожении, называется композицией объектов. *Композиция* — это один из вариантов связи между классами. Композицию всегда можно читать с помощью фразы «состоит из»: отрезок состоит из двух точек, человек состоит из органов, квартира состоит из комнат и т. д. Основной

признак композиции как сильной связи между объектами — это вопрос владения и управления временем жизни. Композируемые объекты создаются композирующим при его создании и удаляются при удалении.

В других языках композиция тоже, разумеется, есть, но в других языках она будет ближе вот к такому варианту, который в C++ тоже, как мы видим, реализуем:

```
class Section {
public:
    Point *pbeg;
    Point *pend;
    Section() : pbeg(new Point(0, 0)), pend(new Point(0, 0)) {
    }
    Section(int x1, int y1, int x2, int y2) :
        pbeg(new Point(x1, y1)), pend(new Point(x2, y2)) {
    }
    Section(const Section &s) :
        pbeg(new Point(*s.pbeg)), pend(new Point(*s.pend)) {
    }
    ~Section(){
        delete pbeg;
        delete pend;
    }
};
```

Ну или если отказаться от использования строк инициализации в конструкторе, чтобы сделать код больше похожим на многие другие языки:

```
class Section {
public:
    Point *pbeg;
    Point *pend;
    Section(){
        pbeg = new Point(0, 0);
        pend = new Point(0, 0);
    }
    Section(int x1, int y1, int x2, int y2){
        pbeg = new Point(x1, y1);
        pend = new Point(x2, y2);
    }
    Section(const Section &s){
        pbeg = new Point(*s.pbeg);
        pend = new Point(*s.pend);
    }
    ~Section(){
        delete pbeg;
```

```

delete pend;
}
};

```

В этих двух примерах, как мы видим, атрибутами класса являются не сами объекты `Point`, а указатели на них. Учитывая, что в большинстве объектно-ориентированных языков объектных переменных в стиле C++ нет, а есть «ссылки», которые ближе всего именно к указателям, именно этот код потребует минимальных косметических изменений для работы в каком-нибудь Java или C#.

Раз атрибутами являются указатели, то в конструкторах мы с помощью `new` создаем сами объекты, адреса которых и складываем в эти указатели. В деструкторе соответственно две созданные точки удаляются. Здесь заслуживает особого внимания конструктор копирования, потому что он может быть реализован в двух вариантах. Мы могли реализовать его вот так:

```

Section(const Section &s){
    pbeg = s.pbeg;
    pend = s.pend;
}

```

Чтобы понять, в чем особенность такого решения, нужно нарисовать схему того, что будет происходить при создании и последующем удалении двух объектов (рис. 1.1):

```

{
    Section s1(1,2,3,4);
    Section s2(s1);
}

```

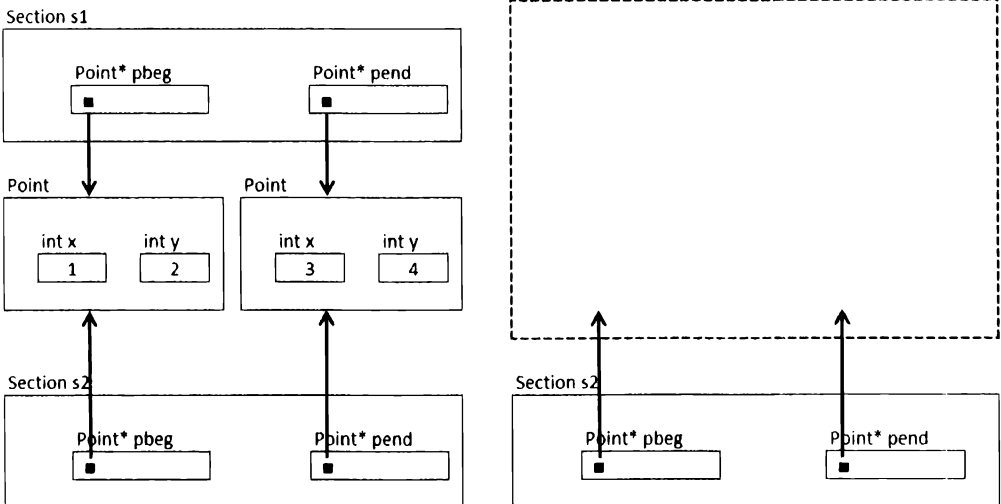


Рис. 1.1. Проблемы поверхностного копирования

При создании второго объекта конструктор копирования в `pbeg` и `pend` создаваемого объекта просто запишет адреса существующих точек `Point` из первого объекта. В результате мы получим два объекта `Section`, хранящие ссылки на те же самые два объекта `Point`. Но ведь деструктор `Section` удаляет созданные им точки! В итоге после удаления объекта `s1` две созданные для него точки удалятся, и указатели на них во втором объекте будут хранить адреса освобожденных участков памяти. Объект `s2` при удалении попытается выполнить свои две операции `delete` в деструкторе и освободить память, которая уже ему не принадлежит. В результате программа скрашится (если повезет, то сразу, потому что вообще-то это хуже, чем краш, это UB).

Что стало причиной этой проблемы? А ею стало поведение конструктора копирования `Section`, который скопировал указатели на объекты, при этом не создав копии тех структур, на которые они указывали. Такое поведение имеет специальное название: *поверхностное копирование*, копирование без копирования глубокой структуры. Но, конечно, нельзя считать, что поверхностное копирование плохое, — в других ситуациях бывает нужно именно оно. Но в этой ситуации, когда `Section` композитрует объекты `Point`, отвечает за время их жизни, конечно же, нельзя допускать такого пути выполнения программы, когда есть `new`, но нет соответствующего `delete`, или наоборот (наш вариант), нет `new`, но есть `delete`. Именно такой вариант, когда есть конструктор без `new` и деструктор с `delete`, в нашем случае и реализовался. Вернем обратно прошлый вариант конструктора копирования:

```
Section(const Section &s){
    pbeg = new Point(*s.pbeg);
    pend = new Point(*s.pend);
}
```

Как видим, здесь вместо того, чтобы напрямую скопировать указатель `pbeg = *s.pbeg` на уже существующий объект `Point`, с помощью конструктора копирования создается новый объект `Point`: `pbeg = new Point(*s.pbeg)` (рис. 1.2).

Такое создание копии называется *глубоким копированием*, когда вся вложенная структура данных с подструктурами копируется, а не переиспользуется. Тут мы видим, что каждый объект `Section` в полном соответствии с идеей композиции хранит свои два объекта `Point` и отвечает за их создание и уничтожение.

Если бы мы вообще не определили конструктор копирования для `Section`, то многое зависело бы от того, хранит `Section` объекты `Point` или указатели `Point *`. Дело в том, что конструктор копирования `Section`, создаваемый компилятором, в случае `Point` выполнил бы создание `Point` с помощью конструктора копирования (и таким образом получилось бы глубокое копирование), а в случае `Point *` просто скопировал бы адрес (и получилось бы поверхностное копирование).

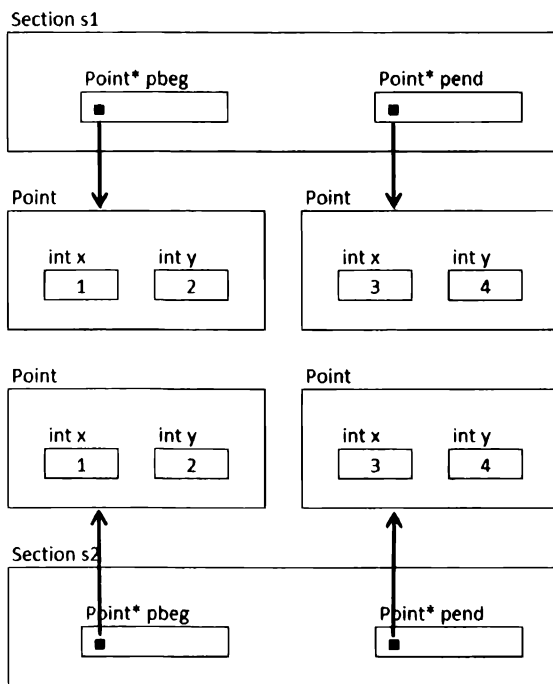


Рис. 1.2. После глубокого копирования

1.11. Глубокое и поверхностное копирование, value- и reference-семантика

Проблема глубокого и поверхностного копирования важна не только при создании копии объекта конструктором копирования, но и при присваивании объектов друг другу. В большинстве объектно-ориентированных языков используется так называемая value-семантика для примитивных типов данных и reference-семантика для сложных, и/или составных, и/или пользовательских типов данных. Легче всего разницу между ними можно понять на примере самого простого присваивания, но то же самое верно и, например, для передачи аргумента внутрь функции (потому что по факту это есть присваивание фактического значения аргумента формальному):

```
x = y;
```

Что здесь происходит? Вроде бы просто что-то из `y` переносится в `x`, но что именно? Если после такого присваивания что-то сделать с `y`, то изменится ли автоматически значение `x`? В случае value-семантики копируется «значение», но `x` и `y` продолжают «быть» разными, при работе с `x`, `y` не поменяется, и наоборот. В случае reference-семантики `x` и `y` представляют собой не значения, а «ссылки», «адреса», поэтому присваивание одного другому приводит к тому, что `x` и `y` те-

перь будут «ссылаться», «указывать» на одно и то же значение (а значит, работа с x будет автоматически видна через y). Если x и y — это, например, `int`, то и в C++, и в других языках будет использована *value*-семантика. Но вот если x и y — это объекты созданного пользователем класса, то в большинстве языков будет использована *reference*-семантика и, присвоив $x = y$, вы получите два имени для одного и того же объекта. Однако в C++ при этом все равно в соответствии с *value*-семантикой содержимое одного объекта будет скопировано в другой (речь идет о работе с объектами в C++, а не с указателями). По умолчанию содержимое одного объекта в C++ копируется в другой просто побитовым копированием всех атрибутов (аналогично тому, как побитово копируется объект при использовании конструктора копирования). Если вы хотите это изменить, то C++ позволяет переопределить работу оператора присваивания и повлиять на то, каким образом будет производиться «перенос» содержимого из одного объекта в другой.

Более того, в соответствии с так называемым эвристическим «правилом трех», если вам понадобилось создать хотя бы что-то одно из данного списка, то скорее всего вы должны создать сразу все в этом списке:

- ☐ конструктор копирования;
- ☐ оператор присваивания;
- ☐ деструктор.

Логика тут очень простая: зачем вам, например, мог понадобиться конструктор копирования, чем вас мог не устраивать конструктор копирования (побитового) по умолчанию? Наверное, потому, что вам приходится вручную выделять память атрибутам, чтобы у каждого объекта эта память была своя. Но это означает, что и деструктор вам обязательно понадобится, чтобы выделенную память освободить и ее же выделять в операторе присваивания, иначе проблема, который вы хотели избежать в конструкторе копирования, проявится при присваивании объектов друг другу. С появлением конструкторов перемещения и операторов перемещения это правило превращается в «правило пяти». Справедливости ради, нельзя не упомянуть и так называемое правило нуля: если есть вариант реализации, в котором можно никого из этого списка не реализовывать, то он лучше всего.

1.12. Наследование, композиция, агрегация, ассоциация, зависимость

Наследование, композиция, агрегация, ассоциация, зависимость — это все примеры разных отношений между классами, упорядоченные по убыванию «силы» связи. Но если с наследованием чаще всего всем все сразу ясно, то разница между остальными четырьмя отношениями часто не такая явная и поэтому вызывает непонятки и разные варианты толкования.

Композиция, как уже было сказано выше, есть такая форма отношения между объектами, когда один объект не просто находится внутри другого, но создается и уничтожается вместе с другим и просто не может существовать без него. И если современная медицина, кажется, скоро превратит композицию в агрегацию, потому что сердце и другие органы, «из которых состоит» человек, все чаще пересаживают, то, похоже, нужен другой пример, и он у меня есть. Коттедж не просто «состоит из» комнат, в каком-то смысле «он и есть» комнаты, а комнаты и есть коттедж. Если кто-то будет его сносить, чтобы проложить дорогу, он неизбежно снесет и комнаты тоже (без паники!).

Агрегация очень похожа на композицию тем, что объект-агрегатор достаточно долгое время хранит агрегируемые объекты и в определенном смысле тоже «состоит из» них. Но самое главное — агрегатор не отвечает за время жизни агрегируемых объектов: он сам, скорее всего, создавался без них, он не создавал их, он не будет их уничтожать.

Слева — композиция: каждый кусочек мозаики имеет свой смысл только в составе общего, а общее не имеет смысла само по себе без композитируемых кусочков. Справа — агрегация, как тасуемые кадры в фильме или картины, собранные на тематическую выставку: после выставки их переместят в другие помещения для участия в других выставках (рис. 1.3).

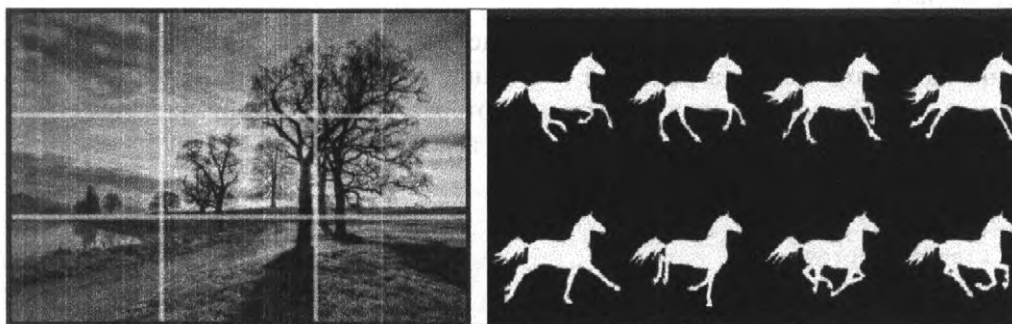


Рис. 1.3. <https://stackoverflow.com/users/1697099/premrj>

Если рассуждать о нашем отрезке `Section` и точках `Point`, то их особенность в том виде, в котором они были реализованы, заключается в том, что ими нельзя было, например, моделировать ломаную линию, состоящую из отрезков, конец каждого из которых является началом другого. У каждого из наших отрезков своя собственная точка на обоих концах, и даже если точка на конце одного отрезка будет равна точке на начале другого, они семантически не будут образовывать ломаную, потому что у точки в принципе нельзя будет спросить, кто к ней «прицеплен» с одной стороны, а кто — с другой. Чтобы можно было моделировать ломаную линию, а также все другие варианты, когда отрезки могут использовать одни и те же точки, придется отказаться от композиции, примерно так:

```
class Section {
private:
    Point *pbeg;
    Point *pend;
public:
    Section(Point* pb, Point* pe){
        pbeg = pb;
        pend = pe;
    }
    ~Section(){
    }
};
```

Как видим, теперь вроде бы все так же отрезок `Section` «состоит из» точек начала и конца, но теперь это именно агрегация, а не композиция: отрезок точки не создает и не удаляет, он их некоторое время хранит. Но при этом смысловая связь с ними все еще сильная — отрезок все еще по смыслу «и есть» две точки.

Часто под агрегатором понимают какой-нибудь контейнер, хранящий однотипные элементы. Но это не потому, что отношение агрегирования обязательно заключается в отношении один-ко-многим, а скорее, наоборот. Контейнер является только одним из примеров агрегирования, совсем при этом не обязательным свойством.

Следующая нечеткая граница, и, как следствие, проблемы с разделением понятий, находится между агрегацией и ассоциацией. Ассоциация — это еще чуть менее сильная связь между объектами, которая, однако, технически может выглядеть на первый взгляд очень похоже на агрегацию:

```
class ListNode {
private:
    ListNode *prev;
    ListNode *next;
    ListData *data;
public:
    ListNode(ListNode* p, ListNode* n, ListData d) :
        prev(p), next(n), data(d) {
    }
    ...
};
```

Если про группу в университете еще можно сказать, что она «состоит из студентов», хотя студенты не исчезают в случае, если исчезает группа (а значит это типичный пример агрегации, пусть даже он и льет воду на мельницу мнения, что агрегация — это обязательно контейнер), то про элемент двусвязного списка точно нельзя сказать, что он состоит из предыдущего и последующего элементов. Просто визуализируя семантику, картинку двусвязного списка в голове, мы не рисуем внутри элемента двусвязного списка другие элементы — мы рисуем

их рядом и соединяем стрелочками. Элемент списка не состоит из предыдущего и последующего, он «связан», «ассоциирован» с ними, но не составляет с ними по смыслу единое целое и уж тем более не отвечает за время их жизни.

И наконец, самое слабое отношение между классами — зависимость. По определению зависимость класса А от класса Б возникает в том случае, если какие-то потенциальные изменения в классе Б могут повлиять на класс А. Слабее могут быть связаны только полностью независимые классы, никак друг на друга не влияющие.

Откуда в классе А может возникнуть зависимость от класса Б? Например, класс А может получить экземпляр класса Б в аргументе какого-то своего метода или как результат вызова метода какого-нибудь другого объекта и т. д. Другими словами, как только в классе А где-то появляется переменная (постоянная или временная — не важно; ссылка, указатель или объектная переменная — не важно) класса Б, мы точно можем сказать, что класс А зависит от класса Б.

```
class Circle {  
    ...  
public:  
    int distTo(const Point &p){  
        ...  
        Ruler *r = Single::instance()->  
            getRulerFactory()->getCurvedRuler();  
        ...  
    }  
};
```

В данном примере мы можем смело утверждать, что ~~кто-то заглянул~~ класс Circle имеет зависимости от классов Point, Ruler, Single и RulerFactory, хотя последний класс даже не присутствует в явном виде в этом коде!

1.13. Диаграммы классов, последовательностей, состояний и прочих UML

Описанные выше отношения между классами вместе с самими классами встречаются на специфических картинках, называемых *диаграммами классов* языка UML. Вообще, если вы думали, что язык — это обязательно про буквы, то это не так. Язык — он про символы, а символы — это совсем не обязательно буквы: спросите у китайцев, корейцев или японцев. Язык UML — про картинки, но как буквы должны выглядеть не абы как, а понятным всем разделяющим их знание образом, так и картинки языка UML должны следовать определенным правилам, чтобы смысл, вложенный в них одним разработчиком, дошел без искажений до другого разработчика, который их будет «читать».

Чаще всего вам здесь будут встречаться именно диаграммы классов UML (или его предшественника и аналога ОМТ, потому что именно модифицированный

ОМТ используется в [7], картинки откуда и будут использованы). Классы на этих диаграммах показываются прямоугольниками, внутри которых отображается название класса, его атрибуты и методы, а отношения между классами отображаются в виде стрелок различного вида. Все содержимое диаграммы классов однозначно определяется программным кодом: недаром диаграмму классов может автоматически построить любая уважающая себя среда разработки. Однако среда не может определить отличие между некоторыми отношениями и тем более не может расставить смысловые акценты, выделить важное и отбросить несущественные детали — все то, что нужно для правильной и быстрой передачи смысла. А это и есть цель создания диаграммы классов (как и любой другой).

Диаграмма классов может отображаться с разным уровнем детальности, в зависимости от того, какую идею вы хотите донести. От одних классов может остаться только название в прямоугольнике, у других классов будет пара главных свойств и методов, третьи классы будут раскрыты максимально детально, вплоть до реализации некоторых методов. Главная цель диаграммы — донести смысл чем-то более понятным, нежели обычный текст. Иногда даже важно не столько то, что на диаграмме есть, как то, чего на диаграмме нет! («Смотрите! Видите, тут нет никакой связи между этими двумя классами? То-то же!»)

Рассказ про диаграммы классов UML обычно требует много времени потому, что самое сложное в них — это понять разницу между различными видами отношений. А раз мы эту разницу уже поняли, нам остается только просто перечислить, как именно известные типы отношений отображаются в UML. Как будто здесь все понятно, только отношение наследования разделено на два разных типа стрелок. Когда вы разберетесь, что такое интерфейсы, вы узнаете, когда какую из двух этих стрелок в конкретном случае использовать (рис. 1.4).



Рис. 1.4. Отношения между классами в UML

Хотя в стандарте это не так, мы будем считать, что все стрелки у нас направлены и соединяют только два класса. Над началом стрелок композиции, агрегации и ассоциации могут быть надписаны имена свойств, потому что как еще один класс может агрегировать или композировать объекты других классов, кроме как храня свойства, в которых ссылки на эти объекты лежат? В некоторых случаях с разных концов некоторых стрелок может быть указана кардинальность (другими словами, число объектов, вступающих в связь), если это важно.

А для свойств и методов символами «+», «-», «#» и подобными может указываться их приватность или публичность.

Кроме этого, надо понимать, что не человек для правил, а правила для человека. Язык картинок нужен не для того, чтобы потешить самолюбие создателей стандартов, а для того, чтобы доносить информацию и лучше понимать друг друга. Поэтому если в конкретном случае нужно доносить какой-то смысл, который не вписывается в стандарт, то нужно смело этот стандарт менять, просто предупредив всех тех, кто ваши картинки будет читать. Так, например, создатели [7] мало того что основываются на более старом стандарте ОМТ, а не UML (который вышел после создания их нетленки), так еще и смело вносят в него изменения, потому что им нужно доносить смысл (рис. 1.5).

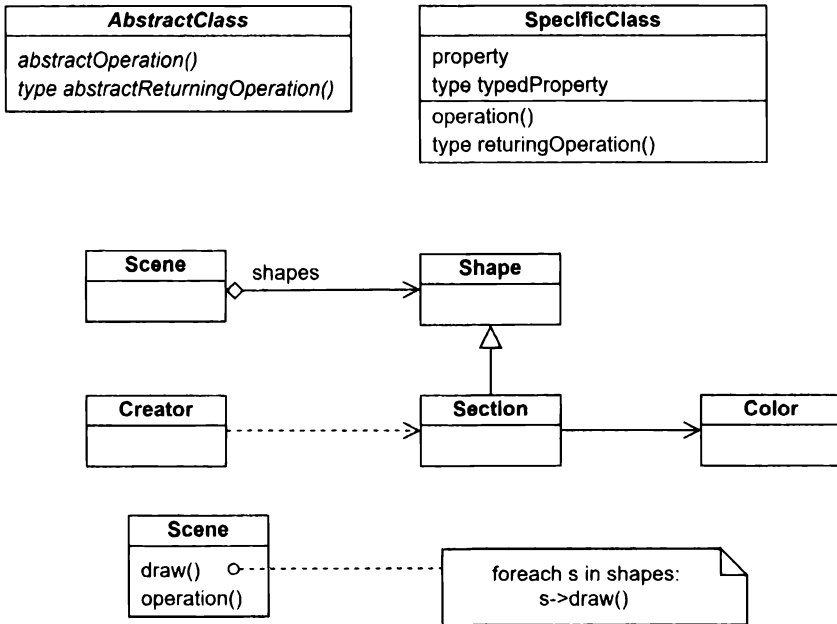


Рис. 1.5. Нотация диаграммы классов ОМТ из [7]

Их нотация, которой и мы будем пользоваться, описана в приложении в книге [7]. В самих классах тут в верхней части описано имя (к тому же курсивом для абстрактных классов), во второй секции описываются методы (тоже курсивом, если они абстрактные), а в нижней секции — свойства, с типом или без. В отношениях классов мы без труда опознаем наследование, агрегацию/композицию и ассоциацию. Но вот пунктирная стрелка в этой нотации означает отношение порождения, когда объекты одного класса создают объекты другого класса. Псевдокод в примечаниях, которые позволяют раскрыть содержание некоторых методов, — тоже, я считаю, хорошая идея.

Итак, смело используйте любую существующую нотацию или придумывайте свою систему, лишь бы она была, эта система. Если уж с самого начала у вас была какая-то тактика, то вы должны ее придерживаться!

Диаграмма классов описывает структуру системы взаимодействующих классов, но не несет и не может нести информации о том, как объекты этих классов взаимодействуют во времени: кто, когда и с кем. Такого рода информацию призвана доносить так называемая диаграмма последовательностей. Казалось бы, такую информацию могла бы доносить самая обычная блок-схема алгоритма, но диаграмма последовательности хороша еще и тем, что она позволяет описывать параллельно происходящие взаимодействия. Можно сказать, что блок-схема алгоритма отличается от диаграммы последовательностей тем же, чем простые программы с линейным потоком исполнения отличаются от событийно-ориентированных (рис. 1.6).

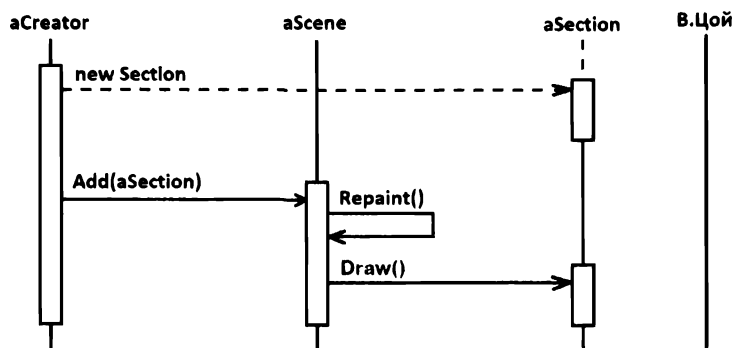


Рис. 1.6. Нотация диаграммы последовательностей из [7]

На диаграмме последовательностей отображаются не классы, а объекты! Это можно подчеркнуть добавлением префикса «а», ну или давая значащие имена конкретным объектам. Каждый объект имеет свою вертикальную, идущую сверху вниз «линию жизни» — пунктирную, когда объект еще не создан или уже сдох, и сплошную, когда объект жив. Прямоугольник на этой линии означает, что объект активен и сейчас что-то делает (другими словами, выполняется какой-то из его методов). Горизонтальная стрелка от одного объекта к другому (или к самому себе) означает передачу активности от одного объекта другому (другими словами, вызов одним объектом метода другого объекта). При этом, очевидно, другой объект тоже становится активным и некоторое время что-то там сам делает. Видите, как в этом примере в одно и то же время параллельно активны сразу три объекта?

Третьей, самой важной, на мой взгляд, диаграммой является диаграмма состояний, тесно связанная с таким понятием, как конечный автомат. Диаграмма состояний описывает какой-то сложный объект так, как будто весь его бесконечно богатый внутренний мир и все его переживания сводятся к реакциям на происходящие события, и все его поведение зависит только от того и определяется

однозначно только тем, в каком состоянии находится объект. В нотации диаграммы состояний этот сложный объект может находиться только в конечном числе конкретных состояний. Объект находится в состоянии и продолжает в нем оставаться, пока какое-то событие не заставит его перейти в другое состояние и находиться уже в нем.

Скругленные прямоугольники на диаграмме состояний обозначают собственно состояния, а подписанные стрелки, их соединяющие, — события, которые переводят систему из одного состояния в другое. Важно четко понимать разницу между состояниями и событиями. Событие — это нечто, происходящее в конкретный момент времени, не имеющее длительности и требующее реакции. Можно танцевать от противного и определять событие как нечто, что переводит систему из одного состояния в другое. «Дождь пошел» — это событие, «дождь закончился» — тоже событие, «дождь идет» — это не событие, это состояние, потому что оно имеет длительность. В то же время «дождь идет очередной час» — это событие, потому что оно локализовано в конкретный момент времени. Состояния на диаграмме состояний бывают сложно-составные, как состояние «Работа» здесь, потому что в рамках этого состояния микроволновка может находиться в более мелких простых состояниях. Черный круг и такой же черный круг, но обведенный, указывают соответственно на состояние, с которого все начинается и которым все заканчивается (рис. 1.7).

Диаграмма состояний — такой удачный инструмент еще и потому, что ее можно детализировать до таких мелких состояний, с которыми уже могут быть связаны конкретные электрические триггеры, а с ними — конкретные детали оборудова-

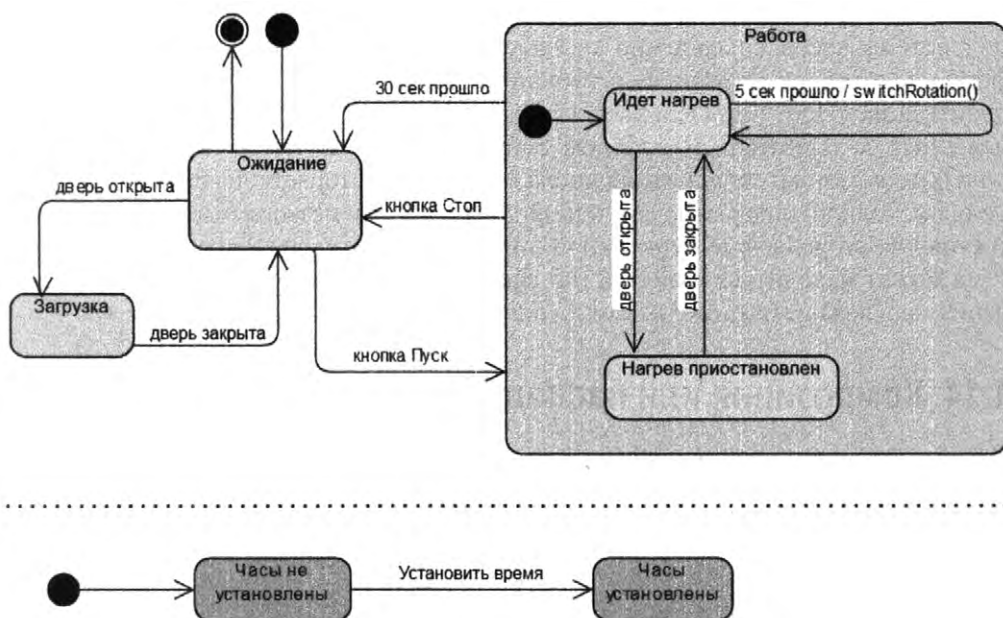


Рис. 1.7. Диаграмма состояний упрощенной микроволновки

ния. Например, с состоянием «Идет нагрев» у микроволновки может быть связан конкретный транзистор, питающий микроволновочный магнетрон. Поэтому одну и ту же диаграмму состояний микроволновки может начать создавать аналитик с достаточно абстрактных состояний, а потом инженер сможет ее же детализировать вплоть до чего-то очень близкого к железу.

Важно, чтобы на диаграмме состояний были все такие состояния, в которых гипотетически может находиться моделируемый вами объект, и которые требуют, с вашей точки зрения, различного поведения. Кроме этого, на диаграмме состояний должны быть все возможные и рассматриваемые вами события, на которые требуется реакция. Это очень важно, потому что, если какой-то стрелки из состояния А в состояние Б на вашей диаграмме нет, это означает, что объект не может непосредственно перейти от состояния А в состояние Б. Например, на этой диаграмме нет стрелки из состояния «Загрузка» в состояние «Идет нагрев», поэтому с открытой дверцей можно сколько угодно нажимать кнопку «Пуск», — нагрев не начнется, пока в состоянии «Загрузка» не произойдет событие «Дверь закрыта». А само событие «Дверь закрыта» приводит к разной реакции в зависимости от того, в каком состоянии («Загрузка» или «Нагрев приостановлен») находится система.

Диаграмма состояний организована так, что в каждый момент времени система находится исключительно в каком-то одном из указанных состояний. Это очень быстро приводит к проблемам при моделировании, когда появляются так называемые перпендикулярные состояния, которые друг от друга не зависят. Например, если у микроволновки есть часы, то она может еще и находиться в одном из двух состояний: «Часы установлены и тикают» и «Часы не установлены и показывают 12:00». Достаточно недолгих размышлений, чтобы понять, что эта подсистема часов совершенно независима от всех остальных состояний, но, формально говоря, каждое из состояний, например «Идет нагрев», должно раздвоиться на «Идет нагрев, а часы установлены» и «Идет нагрев, а часы не установлены», с идентичным набором событий и переходов, и вы очень быстро получите комбинаторный взрыв числа событий и потеряете логику. Чтобы этого не происходило, диаграмма состояний допускает так называемые ортогональные состояния, разделяемые пунктирной линией. Для ортогональных состояний система может находиться в любом (но каком-то одном) состоянии первого «измерения» и в любом (но тоже каком-то одном) состоянии другого измерения.

1.14. Композиция или наследование?

Давайте создадим вот такие несколько классов:

```
class Point {  
public:  
    int x;  
    int y;  
    ...  
};
```

```
class Circle : public Point {  
public:  
    int radius;  
    ...  
};  
  
class Section {  
public:  
    Point pbeg;  
    Point pend;  
    ...  
};  
  
class Section2 : public Point {  
public:  
    Point another;  
    ...  
};
```

...и нарисуем, как выглядят в памяти их объекты (рис. 1.8).

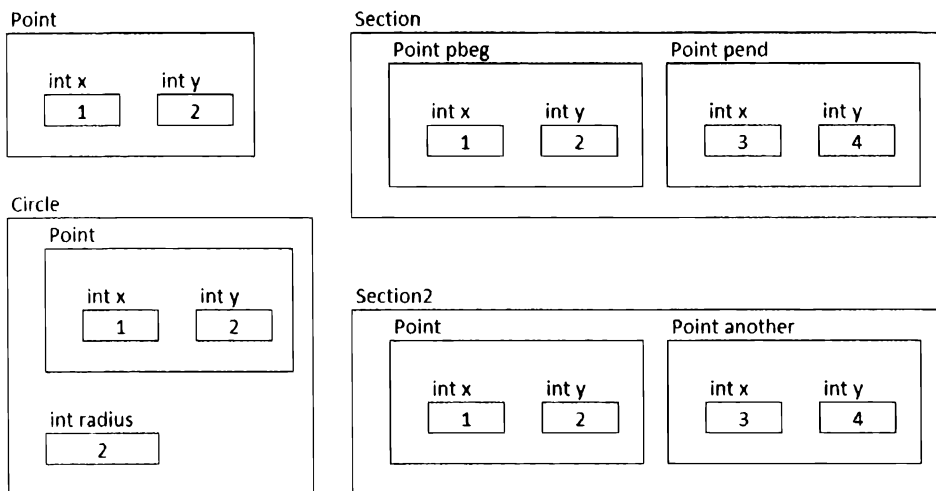


Рис. 1.8. Разница между композицией и агрегацией

Объект `Point` представляет собой объект, внутри которого находятся два целочисленных поля `x` и `y`, тут все просто. Объект `Circle` наследуется от `Point`, а значит, и является в то же самое время `Point`. То есть внутри `Circle`, если покопаться, можно найти `Point` (внутри которого, опять-таки, два целочисленных поля `x` и `y`), и рядом добавлено еще одно числовое поле `radius`.

А вот что представляет собой объект `Section`? Аналогично тому, как `Point` содержит внутри себя числовое поле, `Section` содержит внутри себя два объектных поля, два объекта `Point`, которые уже, в свою очередь, — ну и т. д.

А теперь мы в другом варианте `Section2` взяли и унаследовали `Section` от класса `Point`, а второй экземпляр `Point` `another` добавили в качестве поля.

Смотрите сами, чем отличаются на картинке `Section` и `Section2`. Вроде все выглядит так, будто различие чисто синтаксическое. При работе с `Section` мы к точкам начала и конца отрезка будем обращаться единообразно через дополнительные поля, а при работе с `Section2` мы к началу отрезка будем обращаться напрямую, а к концу — через дополнительное поле:

```
Section s;  
s.pbeg.x = 1;  
s.pbeg.y = 2;  
s.pend.x = 3;  
s.pend.y = 4;  
Section2 s2;  
s2.x = 1;  
s2.y = 2;  
s2.another.x = 3;  
s2.another.y = 4;
```

Но в обоих случаях внутри `Section` оказываются два объекта `Point`. Эта похожесть и приводит к множественным спорам по поводу того, что лучше, композиция или наследование, нужно ли всегда использовать композицию вместо наследования и т. д.

Прежде всего нужно понимать, что это отношения разные по смыслу, но с похожей технической стороной дела. Проблемы, как всегда, начинаются тогда, когда кто-то полностью игнорирует техническую сторону дела, упершись в идеологические предпочтения, — или, наоборот, использует технические решения, игнорируя суть и смысл происходящего, даже если они противоречат здравому смыслу.

Композиция — это отношение «состоит из...», а наследование — это отношение «является частным случаем...» или «ведет себя как...»; это их идеологическое отличие. И композиция, и наследование — это способ получить внутри одного класса функциональность другого класса; это их технологическое сходство. Наследование связывает классы во время компиляции, композиция может связывать классы во время выполнения.

Таким образом, наследование — это очень жесткий способ связывать классы, композиция гораздо гибче. Но работать с функциональностью чужого класса при наследовании гораздо проще, чем при использовании композиции. Для объекта `Circle` `c` мы просто пишем `c.x` и получаем доступ к унаследованной функциональности класса `Point`. Если бы `Circle` не наследовал от `Point`, а композировал его, то тот же вызов вместо `c.x` выглядел бы как-то так: `c.point.x`.

С другой стороны, одна из основных претензий к наследованию (публичному чаще всего) заключается в том, что если класс `A` унаследовался от `B`, чтобы получить его какую-то функциональность, то он неизбежно получает всю функциональность `B`. Пользователь сможет работать с классом `A` так, как он работал

бы с Б, а это может быть совсем не то, что проектировщик системы задумывал. Кроме этого, если потом в класс А добавятся новые публичные поля, то они сразу появятся и в классе Б, изменяя, таким образом, его интерфейс.

Поэтому рекомендуется использовать следующее правило. Если создаваемый класс Б действительно является частным случаем или вариацией А, а значит, интерфейс Б и должен совпадать с интерфейсом А, то лучше использовать наследование. Если создаваемому классу Б или его пользователю нужна только часть функциональности класса А, то лучше воспользоваться композицией/агрегацией. Мы еще вернемся к этой теме, когда будем обсуждать SOLID-принципы в *разд. 2.28*. Помните, у проблемы «наследование vs композиция» есть и техническая, и идеологическая грань!

1.15. Помещение объектов в переменные различных типов

В любой уважающей себя бюрократической организации есть отдел, который отслеживает жизненный цикл всякого барахла (например, мебели) и рисует для этого на нем инвентарные номера. Предположим, у нас в университете уже есть парты и аудиторные доски. Очевидно, что раз у них у всех есть некоторые общие свойства, то логично вынести эти свойства в базовый класс, а конкретные классы от него унаследовать:

```
class Furniture {
public:
    int inventory_number;
    int location;
};

class Desk : public Furniture{
public:
    int student_capacity;
};

class Whiteboard : public Furniture{
public:
    int area;
};
```

Кроме рисования инвентарных номеров на мебели, где-то в недрах организации нужно поддерживать список всех имеющихся парт и всех имеющихся досок:

```
Desk* desk_list[100500];
Whiteboard* wblist[100500];
```

Очевидно, что чем дальше, тем больше будет разных отдельных списков для столов, стульев, шкафов и т. д. А можно ли вместо всех этих отдельных списков

хранить только один список со всей мебелью? На программный язык этот вопрос переводится так: а можно ли указатель на один объект хранить где-то в указателе другого типа?

```
Desk *desk = new Desk();
Whiteboard *whiteboard = new Whiteboard();
Desk *d = whiteboard;
Whiteboard *w = desk;
```

Проверяем и убеждаемся: это не компилируется. Компилятор отслеживает типы, и в указатель на парту записать адрес доски — нельзя. Почему? Да потому, что доска не является партией! А если вот так:

```
Desk *desk = new Desk();
Whiteboard *whiteboard = new Whiteboard();
Furniture *f1 = whiteboard;
Furniture *f2 = desk;
```

А вот так можно! В указатель на мебель можно записать адрес парты, потому что парта является мебелью! А вот обратное уже неверно и не скомпилируется:

```
Desk *desk = new Desk();
Furniture *f = desk;
Desk *d = f;
```

Да, мы вроде бы только что во второй строке в переменную `f` положили адрес парты, почему же в третьей строке тот же самый адрес нельзя обратно положить в переменную `d`? Это бюрократия, детка, у нее есть четкие правила, и они звучат так. В переменную класса `A` можно положить любой объект класса `A` или любого `B` из его классов-потомков. Объект класса `A` можно положить в любую переменную класса `A` или любого `B` из его классов-предков. Это одно и то же правило, просто прочитанное с разных сторон.

Проще всего это представить себе следующим образом. Допустим, у нас есть бумажный список инвентарных номеров парт и бумажный же список инвентарных номеров всей мебели. Можем мы не глядя взять любой номер из списка парт и записать его в список мебели? Да, можем: парты в список мебели записывать можно. Можем мы не глядя взять любой номер из списка мебели и записать его в список парт? Нет, не можем: в списке мебели может оказаться все что угодно. Бюрократические правила запрещают нам так делать, даже если мы на 99,9% уверены, что выбираемый номер из списка мебели является партией. И есть еще один нюанс.

```
Furniture *f;
if (rand() / RAND_MAX > 0.5)
    f = new Desk();
else
    f = new Whiteboard();
f->inventory_number;    // ок
```

```
f->location;    // ок
f->student_capacity; // не ок
f->area;        // не ок
```

Компилятор в принципе может не знать, какой именно объект лежит в какой-то переменной-указателе: как мы видели, там может лежать не только объект самого класса, но и классов-потомков. Поэтому еще один (вполне обоснованный) бюрократический запрет: при обращении к переменной-указателю класса А мы можем получить доступ только к свойствам и методам класса А, даже если мы сами уверены, что там лежит объект класса-потомка Б:

```
Furniture *f = new Desk();
f->student_capacity;    // неверно, хотя осмысленно
```

Обратите внимание, в переменной `f` лежит все-таки объект класса `Desk`, т. е. парта! Не надо думать, что при помещении парты в список мебели она магическим образом перестает быть партой, а физически становится каким-то другим объектом. Объект как лежал по своему месту в памяти, так и лежит, с ним ничего не происходит, потому что мы манипулируем только адресами объектов, но не ими самими. И у объекта по адресу, хранимому в `f`, на самом деле есть свойство `student_capacity`, просто нам не дают к нему доступ. Но C++ позволяет нам оперировать и самими объектами тоже:

```
Desk d;
Furniture f;
f = d;
f.student_capacity;    // неверно и бессмысленно
```

Вот здесь в третьей строке компилятор, вздохнув, на самом деле отрежет от объекта `Desk` все лишнее, а то малое, что осталось, засунет в объектную переменную `f`. И у объекта, который в этой переменной будет лежать, уже не будет свойства `student_capacity` вообще. Такое действие именно так и называется, *object slicing*.

Итак, объекты-потомки можно записывать в переменные-предки. Но зачем это делать, если от этого как будто проблем только прибавляется? Записав адрес объекта-потомка в переменную класса-предка, мы сами себя ограничиваем в том, как с этим объектом мы дальше будем работать, — ради чего себя так ограничивать? Есть две основные цели таких действий:

- ❑ это позволяет использовать один и тот же контейнер для хранения объектов разных классов. Так, вместо отдельных списков для парт, досок, стульев, шкафов и другой мебели и необходимости при появлении нового типа мебели создавать еще один список, можно использовать один единый список мебели, в котором можно хранить сразу все. В том числе и те подклассы мебели, которые еще не появились, но, возможно, появятся позже;
- ❑ это позволяет коду работать с теми объектами, которые даже не были еще созданы тогда, когда этот код писался, а значит, позволяет использовать без изменений старый код с новыми объектами.

Последнее требует дополнительных пояснений. Дело в том, что объект помещается в переменную не только тогда, когда мы явно это делаем с помощью присваивания. Где-то давно у вас был такой код:

```
class Point {
public:
    int x;
    int y;
    int distTo(const Point &p2){
        return (x - p2.x)*(x - p2.x) + (y - p2.y)*(y - p2.y);
    }
};
```

Здесь функция `distTo` принимает как будто объект класса `Point`. Но на самом деле, учитывая все описанные выше правила, эта функция примет не только `Point`, но и, например, `ColoredPoint`, и вообще любой другой потомок `Point`, о котором этот код даже не знал, когда был написан! Мы еще вернемся к этой теме, когда будем изучать интерфейсы (интерфейсные типы) в *разд. 2.6*.

1.16. Полиморфизм

Наконец-то мы добрались до самой важной, на мой взгляд, темы в ООП, на которой, я считаю, держится вообще все. Эта тема — полиморфизм. Это еще одно очень многозначное слово, которое в общем и целом обозначает использование одних и тех же символов для обозначения или вызова разной (порой непредсказуемой) функциональности.

Мы не будем здесь описывать разные типы полиморфизма до тех пор, пока вы с ними не столкнетесь, и не будем ввязываться в академические споры по поводу различных классификаций видов полиморфизма, — вы сами будете встречать разные идеи в разных местах и находить то общее, что все эти идеи и определения объединяет.

Мы будем здесь говорить о полиморфизме как об одной конкретной технике использования виртуальных функций. И чтобы понять, зачем они нужны, рассмотрим следующий пример:

```
class Animal {
private:
    string name;
public:
    void sound() {
        printf("\n");
    }
};

class Cat : public Animal {
public:
    void sound() {
```

```
    printf("Mew-mew \n");
}
void catchMouse(){
}
};
class Dog : public Animal {
public:
    void sound() {
        printf("Wow-wow \n");
    }
    void chaseCat(){
    }
};
```

Один базовый класс `Animal`, определяющий метод `sound`, и два класса-потомка `Cat` и `Dog`, которые оба этот метод переопределяют и добавляют еще каждый по своему собственному методу. Теперь создадим коллекцию объектов — классов `Cat` и `Dog`, потому что где вы видели животное, которое не является при этом представителем какого-то более конкретного типа, в нашем случае кошкой или собакой? В *разд. 1.1* мы в этом случае говорили о том, что класс `Animal` является абстрактным, а не конкретным, потому что он является только «общей частью» каких-то конкретных классов и не предназначен для создания объектов.

```
const int ZOO_SIZE = 10;
Animal *zoo[ZOO_SIZE];
for (int i = 0; i < ZOO_SIZE; i++)
    if (rand() / RAND_MAX > 0.5)
        zoo[i] = new Cat();
    else
        zoo[i] = new Dog();
```

Мы создали массив из 10 указателей на `Animal` и заполнили его в случайном порядке объектами классов `Cat` и `Dog`, при этом мы сами не знаем, в какой «клетке» какой объект находится. Можете просто представить себе ряд таких клеток с животными, накрытых тканью так, что не видно, в какой клетке кто сидит. Что будет, если вы пройдете вдоль ряда таких клеток и произведете на каждую не-сильное физическое воздействие? Интуитивно ожидаемое поведение заключается в том, что мы должны услышать из каждой клетки «звук», соответствующий тому животному, которое там находится:

```
for (int i = 0; i < ZOO_SIZE; i++)
    zoo[i]->sound();
```

Однако этот код в C++ поведет себя контринтуитивным образом: мы не увидим никаких сообщений о звуках, только пустые строки. Это происходит потому, что в C++ в случае, если объект класса-потомка находится в переменной класса-предка, и при этом мы вызываем у этой переменной метод, объявленный в предке и переопределенный в потомке, C++ вызывает все-таки метод предка:


```
Cat *c = new Cat();
Animal *a = new Cat();
c->sound();           // метод Cat::sound()
a->sound();           // метод Animal::sound()
```

При обращении `c->sound()` вызывается, как и ожидалось, метод `Cat::sound()`, но при вызове `a->sound()` неожиданно вызывается метод `Animal::sound()`. Подчеркну еще раз: это противоречит ожидаемому нами поведению, нашему опыту из реальной жизни. Поэтому давным-давно было реализовано и решение данной проблемы: метод `sound` в классе `Animal` нужно сделать «виртуальным» с помощью соответствующего ключевого слова:

```
class Animal {
private:
    string name;
public:
    virtual void sound() {
        printf("\n");
    }
};

class Cat : public Animal {
public:
    void sound() override {
        printf("Mew-mew \n");
    }
    void catchMouse(){
    }
};

class Dog : public Animal {
public:
    void sound() override {
        printf("Wow-wow \n");
    }
    void chaseCat(){
    }
};
```

Видите, в методе `Animal::sound()` появилось ключевое слово `virtual`? Смысл его заключается в следующем: объявляя какой-то метод как `virtual`, мы как бы говорим: «Этот метод `Animal` потом будет перекрываться в потомках, и если для какого-то указателя `Animal` он будет вызываться, то вызови не базовый метод, а метод того объекта, который по этому адресу лежит».

Если какой-то метод в базовом классе был объявлен как `virtual`, то он как бы «заражает» всю свою дочернюю иерархию: все его потомки наследуют этот метод как виртуальный и могут его перекрывать (а могут и не перекрывать).

Заметьте, мы даже слово тут используем другое: «перекрывать», а не «переопределять», как мы говорили раньше. Но это только вопрос терминологии, которая не является общепринятой. Иногда эти два слова используются как синонимы, но лично я все-таки предпочитаю говорить «перекрывать» про виртуальные методы, а «переопределять» про обычные.

Еще вы могли заметить, что в перекрытых методах у потомков появилась добавка в виде `override`, что так и переводится — «перекрывать». Вообще говоря, она там не обязательна: вы можете ее убрать, и поведение C++ не поменяется, но ее использование может сберечь вам несколько нервных клеток:

```
class Dog : public Animal {
public:
    void saund() override {
        printf("Wow-wow \n");
    }
    void chaseCat(){
    }
};
```

Если вдруг случайно опечататься и написать `saund` вместо `sound` (или ошибиться в наборе аргументов), без слова `override` компилятор подумает, что вы просто объявили новый метод, а так он всполошится и заставит вас ошибку исправить.

Все написанное выше справедливо для C++. На самом деле в других языках, которые не вынуждены тащить за собой весь этот тяжелый груз *legasy*, все методы и так по умолчанию виртуальные, т. е. срабатывают в соответствии с ожидаемым интуитивно поведением. Но в C++ эта разница есть, и вот она, если совсем кратко: табл. 1.3.

Таблица 1.3

Создание объекта	Вызов <code>a->sound()</code> в случае, если <code>sound...</code>	
	...невиртуальный	...виртуальный
<code>Animal *a = new Animal()</code>	<code>Animal::sound</code>	<code>Animal::sound</code>
<code>Animal *a = new Cat()</code>	<code>Animal::sound</code>	<code>Cat::sound</code>
<code>Cat *a = new Animal()</code>	???	???
<code>Cat *a = new Cat()</code>	<code>Cat::sound</code>	<code>Cat::sound</code>

Я думаю, вы уже поняли, что все четыре комбинации сделать не получится: третья строка в этой таблице неправильная: объект класса `Animal` нельзя складывать в переменную класса `Cat`. Зато, если посмотреть на все остальные строки, можно увидеть, что отличие поведения виртуального метода от неvirtуального проявляется только в одном случае: когда объект класса-потомка лежит в переменной класса-предка и у предка вызывается виртуальный метод, перекрытый

в потомке. Так, например, если объект `Cat` находится в переменной `Cat`, то никакой разницы между виртуальными и неvirtуальными методами не будет.

Итак, зачем нужны виртуальные методы в C++ технически, когда их действие проявляется? Виртуальные методы нужны в тех случаях, когда объекты-потомки могут/будут лежать в переменных классов-предков и мы хотим со всеми ними работать единообразно, не зная, какому конкретно классу-потомку они принадлежат, а просто вызывая при этом общие для всех них методы, определенные в классе-предке. При этом мы хотим, разумеется, чтобы при вызове этого общего для всех них метода на самом деле для каждого объекта вызывался метод свой специфический:

```
for (int i = 0; i < ZOO_SIZE; i++)
    zoo[i]->sound();
```

Мы не хотим здесь знать, кто в какой клетке сидит, — но, с другой стороны, мы хотим, чтобы каждая клетка «реагировала» по-своему, в соответствии с тем, какое животное там сидит. Как этот код изменится, если в будущем появится новый класс-потомок `Animal`, т. е. какое-то новое животное? А никак! Мы вызываем здесь метод `sound()` у каждой ячейки массива `zoo`, которая имеет тип `Animal*`, и при этом там может оказаться любой (даже неизвестный науке) зверь.

В этом суть того полиморфизма, который нам особенно важен: работать с разными объектами единообразно, через единый интерфейс, но при этом ожидать различного поведения в соответствии с (неизвестным нам) конкретным типом объекта. Это понятие полиморфизма связано с другими близкими понятиями: «динамическая диспетчеризация» и «позднее связывание». Они означают, что с точки зрения компилятора принять решение о том, какой код будет вызван в строке вида `x->foo()` в случае, если `foo` — виртуальная функция, можно будет только «позже», «динамически», во время выполнения программы. Потому что для виртуального метода `foo()` то, какой конкретно кусок кода будет вызван, зависит только от одного: от реального типа объекта `x` (еще, кстати, это называют «одинарной диспетчеризацией»).

Кстати, а зачем нам вообще нужен метод `sound()` у базового класса `Animal`, если он все равно ничего не выводит на экран (не знает, что выводить)? Может быть, надо просто удалить этот пустой метод? Если вы попытаете это сделать, вы поймете, зачем он нужен: если его удалить, то любая попытка вызвать `zoo[i]->sound()` не скомпилируется. Так что этот метод нужен только для того, чтобы просто быть. Он должен быть пустым в базовом классе, для того чтобы классам-потомкам было, что перекрывать. Он может так и остаться пустым, — тогда какие-то из потомков, которые не перекроют его, просто унаследуют пустой метод, который ничего не делает. Но есть и альтернатива: этот метод можно сделать больше чем пустым, можно удалить у него реализацию вообще.

```
class Animal {
private:
    string name;
```

```
public:
    virtual void sound() = 0;
}
};
```

Такой метод без реализации называется абстрактным (или еще говорят, что это «чистый виртуальный метод»), а класс, содержащий хотя бы один абстрактный метод, называется абстрактным классом. Фишка в том, что для абстрактного класса невозможно создавать объекты. Поэтому, если мы унаследуемся от абстрактного класса, но забудем перекрыть хотя бы один абстрактный метод, объекты нашего класса создавать будет нельзя, компилятор не позволит. В этом и задумка.

Полиморфизм и хранение объектов в переменных базовых классов позволяют писать более гибкий код, но при этом они затрудняют отладку и статический анализ кода. Предположим, у нас есть примерно такая функция `foo`, и внутри нее при вызове у объекта `a` метода `boo` происходит, например, деление на ноль, ну или просто как-то не так код работает именно в этой строке:

```
void foo(A *a) {
    a->boo();
}
```

Как найти тот код, который при этом выполняется? Без полиморфизма мы бы просто пошли в класс `A`, нашли бы там метод `boo()`, а если не нашли бы, то обратились бы к его единственному предку и т. д. В случае полиморфизма мы начнем с того же самого, но как только найдем в `A` или его предке метод `boo()` и выяснится, что он виртуальный (еще часто говорят не «виртуальный», а «полиморфный»), мы поймем, что в функцию `foo` могли передать объект любого из потомков класса `A`, и ищи-свищи, какого именно!

Есть еще одна ситуация, в которой без `virtual` не обойтись. Посмотрите на эти два простых класса:

```
class Base {
public:
    Base() {
        printf("Base()\n");
    }
    ~Base() {
        printf("~Base()\n");
    }
};

class Desc : public Base{
private:
    int *arr;
public:
    Desc() {
```

```
arr = new int[42];
printf("Desc()\n");
}
~Desc() {
    delete arr;
    printf("~Desc()\n");
}
};
```

Какой-то примитивный класс-предок, от него наследуется класс-потомок, где в конструкторе выделяется память под массив `int`, а в деструкторе удаляется. Создадим и удалим объект `Desc` и посмотрим с помощью отладочного вывода, все ли в порядке:

```
Desc *x = new Desc();
delete x;
```

Судя по отладочному выводу, все в порядке — при создании объекта `Desc` вызываются конструкторы `Base` и `Desc`, при удалении его вызываются деструкторы `Desc` и `Base`. Но изменим немного создание объекта:

```
Base *x = new Desc();
delete x;
```

Мы создаем тот же самый объект `Desc` и видим, что вызываются правильные конструкторы, но мы складываем этот объект в переменную базового класса, и при удалении начинаются неожиданности. Действительно, в этом случае вызывается только деструктор базового класса `Base`. Проблема ли это? Конечно! В деструкторе `Desc` у нас освобождалась выделенная под массив память, поэтому, если деструктор `Desc` не вызывается, мы встречаем утечку памяти! Попробуем понять, почему так. Представьте, что `delete` был бы методом, который есть у каждого класса. Какой бы метод тогда вызвался в этом случае?

```
Base *x = new Desc();
x->delete();
```

Вы правы, это зависело бы от того, виртуальная функция `delete` или нет. Если она объявлена в `Base` как виртуальная и перекрыта в `Desc`, то вызовется именно `Desc::delete`, а иначе вызовется базовая версия `Base::delete`. Да, у нас `delete` — это вообще не метод, но вот зато деструктор — это метод, и логика его вызова именно такова. Можно представить себе, что `delete` где-то внутри вызывает метод-деструктор, и от того, виртуальный ли он, будет зависеть, какой именно деструктор вызовется.

Таким образом, все, что нам нужно, — это объявить деструктор виртуальным в базовом классе (во всех потомках он тогда автоматически будет виртуальным, и, учитывая, что с деструктором сложно лажануть в названии или параметрах, можно сэкономить на использовании слова `override` в потомках):

```
class Base {
public:
    Base() {
        printf("Base() \n");
    }
    virtual ~Base() {
        printf("~Base() \n");
    }
};
...
```

Заметьте, кстати, что в случае виртуального деструктора вызывается не только он, но и все деструкторы предков в обратном порядке, как положено. Итак, общее правило: виртуальный деструктор в базовом классе обязательно нужен тогда, когда вы создаете объекты классов-потомков и храните их в переменных классов-предков. В этом случае при удалении объекта-потомка, лежащего в переменной класса-предка, вызовется полная корректная последовательность деструкторов, а не только деструктор класса-предка.

После деструктора класса-потомка автоматически вызывается деструктор класса-предка, это не требует вмешательства. Но можно ли из перекрытого виртуального метода `Desc::foo()` вызвать унаследованный метод `Base::foo()`? Можно, конечно, так и пишите: `Base::foo()`, тут никаких изменений. Но еще нужно отметить, что вызов виртуального метода «проваливается» до самого глубокого перекрытого метода из тех, которыми обладает вызываемый объект, не только если мы вызываем метод, обращаясь к переменной. То же самое верно и для вызова метода изнутри другого метода. Пример:

```
class Base {
public:
    void foo() {
        boo();
    }
    virtual void boo() {
        printf("Base::boo() \n");
    }
};

class Desc : public Base{
public:
    void boo() override{
        printf("Desc::boo() \n");
    }
};
```

Снова класс-предок с одним неvirtуальным методом `foo()`, который внутри вызывает свой же виртуальный метод `boo()`. Класс-потомок перекрывает метод `boo()`, но не перекрывает, а просто наследует в неизменном виде метод `foo()`.

Что будет, если создать объект-потомок, положить в переменную базового класса и вызвать метод `boo()`?

```
Base *x = new Desc();
x->boo();
```

В этом случае, как мы весь этот раздел обсуждаем, очевидно, вызовется метод дочернего класса `Desc::boo()`, потому что в этом техническая суть полиморфизма и состоит. А что будет, если вызовем унаследованный неvirtуальный метод `Base::foo()`?

```
Base *x = new Desc();
x->foo();
```

Понятное дело, что раз метод `foo` у нас один (только в базовом классе), то вызовется именно он. Но вот как только метод `Base::foo()` попытается внутри себя вызвать метод `boo()`, выяснится, что метод `foo` вызвали не абы у кого, а у объекта `Desc`, а значит, полиморфизм сработает и здесь и неvirtуальный метод `Base::foo()` изнутри себя, сам того не зная, вызовет метод `Desc::boo()`.

Это становится более очевидным в тех языках, где всегда в явном виде используется `self` или `this` внутри метода. В этом случае очевидно, что при вызове кода выше указатель `this` будет иметь тип `Base*`, но указывать при этом будет на объект `Desc*`, а значит, полиморфизм сработает так, как должен:

```
class Base {
public:
    void foo() {
        this->boo();
    }
    ...
};
```

И отдельного внимания заслуживает оригинальный способ выстрелить себе в ногу путем использования виртуального метода в конструкторе или деструкторе. Если совсем кратко, то суть проблемы такова: вызывая виртуальный метод `boo()` внутри конструктора `Base` в процессе создания объекта `Desc`, вы можете получить как вызов `Base::boo()` в одних языках (неожиданно, но не фатально), так и вызов `Desc::boo()` в других языках (ожидаемо и фатально: объект `Desc` в этот момент еще недосоздан, до его конструктора дело не дошло, и попытка обратиться к его свойствам приведет к крашу или UB). Короче говоря, не делайте так!

1.17. Приведение типов

Приведение типов есть важная функциональность любого объектно-ориентированного языка. Под приведением типов часто имеют в виду всякую постороннюю, не относящуюся к ООП фигню, типа преобразования `float` в `int` или пре-

образований `string` во `float` и обратно (что, на наш взгляд, вообще не является приведением типов). В этом разделе мы говорим о частных случаях преобразований указателей на объекты одних типов к указателям на объекты других типов. Сразу оговоримся: приведение типов в том смысле, в котором мы будем его использовать, не изменяет тип самого объекта и не может превратить кошку в собаку, «сделать ножку маленькой, душу — большой, а сердце — справедливым». Если объект был создан объектом класса `X`, он и погибнет объектом класса `X`, потому что — судьба!

Вернемся к нашему зоопарку, т. е. базовому классу `Animal` и двум потомкам, `Cat` и `Dog`:

```
class Animal {
private:
    string name;
public:
    virtual void sound() {
        printf("\n");
    }
};

class Cat : public Animal {
public:
    void sound() override {
        printf("Mew-mew \n");
    }
    void catchMouse(){
    }
};

class Dog : public Animal {
public:
    void sound() override {
        printf("Wow-wow \n");
    }
    void chaseCat(){
    }
};
```

Будем считать, что у нас уже создан массив из указателей на объекты, заполненный в каком-то произвольном порядке разными потомками `Animal`:

```
const int ZOO_SIZE = 10;
Animal *zoo[ZOO_SIZE];
for (int i = 0; i < ZOO_SIZE; i++)
    if (rand() / RAND_MAX > 0.5)
        zoo[i] = new Cat();
```



```
else  
    zoo[i] = new Dog();
```

Как и раньше, мы можем пройти по всем элементам массива и у всех единообразно вызвать какой-то общий метод. Для этого мы и храним все объекты в одном массиве, для этого в базовом классе и объявлен виртуальный метод:

```
for (int i = 0; i < ZOO_SIZE; i++)  
    zoo[i]->sound();
```

А как, например, у всех объектов массива вызвать метод `catchMouse()`? Вот такой код, очевидно, не сработает, потому что типом `zoo[i]` является `Animal*`, а у `Animal` нет метода `catchMouse()`. Если мы по какой-то причине уверены, что `zoo[i]` хотя и является `Animal*`, но совершенно точно, 146% указывает на `Cat`, то в C++ можно воспользоваться или старым «сишным» приведением типов, или приведением `static_cast`:

```
((Cat*)zoo[i])->catchMouse();  
static_cast<Cat*>(zoo[i])->catchMouse();
```

И то и другое (аналоги есть и в других объектно-ориентированных языках) можно назвать небезопасным приведением типов: на входе был указатель `Animal*`, на выходе появляется указатель `Cat*`. Помните, мы говорили о том, что можно инвентарный номер из списка парт перенести в список мебели, а обратное недопустимо? Ну так вот, такое небезопасное приведение типов как раз и представляет собой эту обратную операцию: взять инвентарный номер из списка мебели и записать его в список парт.

Если вам повезет, и это действительно окажется парта, то все будет в порядке. В нашем случае, если нам повезет, и `zoo[i]` на самом деле окажется `Cat*`, то метод успешно вызовется и отработает как надо (на самом деле, если метод не обращается к полям класса, он может отработать и в других случаях, но гарантии нет). А если в `zoo[i]` будет не `Cat*`, а что-то другое, то программа скрашится или будет UB. Поэтому такое приведение называют небезопасным.

Кроме этого, подобные C-style-приведения вообще считаются фу-фу-фу, потому что и искать их в коде потом проблема, и вообще (казалось бы, причем тут кривая раздутаая грамматика языка?).

В любом случае если уж мы хотим использовать `static_cast`, то тогда хорошо бы перед его использованием проверить, является ли указываемый объект `Cat`. На самом деле мы сейчас собираемся изобрести велосипед, но в процессе этого изобретения пройдем через несколько поучительных уроков и, возможно, поймем суть объектно-ориентированного подхода к решению таких проблем.

Итак, мы хотим иметь возможность узнать, кем именно является каждый очередной объект в массиве `zoo[i]`. Все, что мы об этих объектах знаем, — что они разные и по-разному себя ведут: у них у всех есть виртуальный метод `sound()`, но у разных объектов он распечатывает на экране разный текст. Вызывать этот

метод, а потом читать экран — дело неблагодарное, поэтому мы должны заранее подготовиться, чтобы опознавать разные объекты нам было лучше. Пусть эти объекты что-то характерное для себя не печатают на экран, а возвращают в качестве результата какой-то виртуальной функции, например имя собственного класса!

```
class Animal {
public:
    virtual string classname() {
        return "Animal";
    }
};

class Cat : public Animal {
public:
    string classname() override {
        return "Cat";
    }
};

class Dog : public Animal {
public:
    string classname() override {
        return "Dog";
    }
};
```

Теперь мы можем у любого объекта запросить имя его класса и перед небезопасным приведением сделать проверку:

```
for (int i = 0; i < ZOO_SIZE; i++)
    if (zoo[i]->classname() == "Cat")
        zoo[i]->catchMouse();
```

На самом деле и это не скомпилируется: в третьей строке бюрократ от компиляторов все равно скажет, что, несмотря на то, что строкой выше что-то там проверяется, `zoo[i]` все равно является `Animal*` и вызывать `catchMouse()` нельзя. Короче говоря, проверяй не проверяй, а небезопасно приводить все равно придется:

```
for (int i = 0; i < ZOO_SIZE; i++)
    if (zoo[i]->classname() == "Cat")
        static_cast<Cat*>(zoo[i])->catchMouse();
```

Вот теперь компилятор доволен: небезопасное приведение все равно осталось, но оно осложнено проверкой, и мы сами взяли на себя ответственность за то, что приведение не приведет к ошибке тогда, когда предыдущая проверка пройдена успешно. Да, мы не у всех объектов в `zoo` вызовем `catchMouse()`, а только у объектов класса `Cat`, — но так и должно быть!

Но есть одна небольшая проблемка. Что если появится потомок класса Cat?

```
class Manul : public Cat {
public:
    string classname() override {
        return "Manul";
    }
};
```

Будет ли для него вызван кодом выше метод `catchMouse()`, если объекты класса `Manul` окажутся среди `zoo[i]`? Очевидно, что нет. А должен ли? Это вопрос сложнее, но если мы предположим, что этот метод должен вызываться для всех объектов, у которых он есть, то он должен вызываться и у `Manul` как у потомка `Cat`. Тогда, получается, нам придется менять проверку?

```
for (int i = 0; i < ZOO_SIZE; i++)
    if ((zoo[i]->classname() == "Cat") ||
        (zoo[i]->classname() == "Manul"))
        static_cast<Cat*>(zoo[i])->catchMouse();
```

Я думаю, вы уже поняли: всякий раз, когда будет появляться какой-то новый потомок `Cat`, нам придется менять подобные проверки очень во многих местах кода, с каждым новым классом они будут расти, и поэтому это плохое решение.

Вот теперь мы добрались до одного приема, на который вам стоит обязательно обратить внимание. Чтобы понять, в чем проблема этого кода, нужно понять, из чего он состоит. А состоит он из следующих действий: сначала мы запрашиваем у объекта полную информацию о нем (имя его класса), потом производим кое-какие вычисления (сравниваем со строками и вычисляем логическое выражение), а затем на основании этой информации принимаем решение о том, может ли объект вызывать метод `catchMouse()`.

Но кто лучше всех знает, может ли он выполнять метод `catchMouse()`? Конечно, сам объект! По факту получается, что мы нарушаем инкапсуляцию объекта, вынося логику вида «умеет ли выполнять такое-то действие» за пределы объекта. Эта логика должна выполняться внутри! Мы не должны запрашивать у объекта информацию и принимать какое-то решение за него. Мы должны, наоборот, дать ему достаточно информации для того, чтобы он сам смог принять решение. Этот принцип будет упоминаться в следующей главе под умным названием «Информационный эксперт». При этом решение может быть разное в зависимости от того, какой вопрос мы будем ему задавать. Мы можем прямо спросить у объекта, можно ли у него вызвать метод `catchMouse()`, или мы можем спросить у объекта, можно ли его приводить к классу `Cat` (т. е. является ли он объектом класса `Cat` или его потомком). Для этого, безусловно, метод `classname` не годится, нужен другой:

```
class Animal {
public:
    virtual bool isA(const string &who) {
```

```
    return who == "Animal";
}
};
class Cat : public Animal {
public:
    bool isA(const string &who) override {
        return (who == "Animal") || (who == "Cat");
    }
};
class Dog : public Animal {
public:
    bool isA(const string &who) override {
        return (who == "Animal") || (who == "Dog");
    }
};
```

Теперь мы не можем у любого объекта запросить имя его класса, но надо ли оно нам? Зато мы можем у любого объекта узнать, является ли он экземпляром нужного нам для приведения класса, и перед небезопасным приведением сделать проверку:

```
for (int i = 0; i < ZOO_SIZE; i++)
    if (zoo[i]->isA("Cat"))
        static_cast<Cat*>(zoo[i])->catchMouse();
```

Казалось бы, что поменялось? На первый взгляд, проверка условия осталась, просто она переехала внутрь метода `Cat::isA`, и все равно логическое условие будет где-то разрастаться? Но нет, аналогичную функцию у класса `Manul` можно реализовать вот так:

```
class Manul : public Cat {
public:
    bool isA(const string &who) override {
        return (who == "Manul") || (Cat::isA(who));
    }
};
```

То есть метод `Manul::isA`, если ему передать какую-то строку `who` (т. е. спросить у него: «Являешься ли ты `who`?»), проверяет, равна ли строка `who` значению «Manul», а если не равна, то не возвращает ли `true` унаследованная функция `Cat::isA`? Зачем такие сложности? Дело в том, что `Manul` должен отвечать «да» не только на вопрос: «Являешься ли ты `Manul`?», но и на вопросы: «Являешься ли ты `Cat`?» и «Являешься ли ты `Animal`?» (и отвечать «нет» на все остальные).

Аналогичным образом, объект `Cat` должен отвечать «да» не только на вопрос: «Являешься ли ты `Cat`?», но и на вопрос: «Являешься ли ты `Animal`?» (и отвечать «нет» на все остальные), — это именно то, что делает метод `isA` у каждого класса.

Самое главное, для чего мы это все делали, находится вот здесь. Этот код больше не будет меняться; даже если будут появляться другие потомки класса `Cat`, он будет корректно их обрабатывать.

```
for (int i = 0; i < ZOO_SIZE; i++)
    if (zoo[i]->isA("Cat"))
        static_cast<Cat*>(zoo[i])->catchMouse();
```

Конечно, это все, как я и говорил, — сплошной велосипед. Такие механизмы для динамического приведения типов, при котором производится проверка, принадлежит ли объект какому-то заданному классу или его потомкам, реализованы в любом языке с ООП. Где-то это простые проверки `instanceof` с дальнейшим аналогом небезопасного приведения, где-то это отдельные конструкции `is` и `as`, одна из которых проверяет, а другая проверяет и приводит. В самом C++ стандартный велосипед, встроенный давным-давно, выглядит так:

```
for (int i = 0; i < ZOO_SIZE; i++)
{
    Cat *c = dynamic_cast<Cat*>(zoo[i]);
    if (c != nullptr)
        c->catchMouse();
}
```

Конструкция `dynamic_cast<Cat*>(zoo[i])` берет указатель `zoo[i]` типа `Animal*` и, используя хранимую в нем информацию времени выполнения (RTTI), выясняет, хранится ли на самом деле по этому адресу объект класса `Cat*` или его потомков. Если это так, то конструкция просто возвращает тот же самый адрес, но уже «под другим шильдиком»: тот же адрес, но с другим типом, это теперь указатель не на `Animal`, а на `Cat`, а значит, этот адрес можно поместить в переменную `Cat *c`. Если же по адресу `zoo[i]` находится какой-то другой объект, не потомок `Cat`, то успешное приведение невозможно и `dynamic_cast` в этом случае просто возвращает `nullptr`.

В следующей строке мы проверяем, не `nullptr` ли вернулся, и если не он, то с полной уверенностью вызываем метод `catchMouse()`, и вся эта ситуация называется «безопасное приведение типов». Можно было написать чуть короче, но тогда проверка выполнялась бы два раза, а это и без того небыстрая (по компьютерным меркам, конечно) операция:

```
for (int i = 0; i < ZOO_SIZE; i++)
    if (dynamic_cast<Cat*>(zoo[i]))
        (dynamic_cast<Cat*>(zoo[i]))->catchMouse();
```

И наконец, можно было бы написать еще короче, но в данном случае мы вообще не проверяем на успешность приведения. А если у нас есть основания верить, что приведение будет всегда успешным, зачем мы тогда вообще используем `dynamic_cast`, если в этом случае можно было бы просто заменить на `static_cast`?

```
for (int i = 0; i < ZOO_SIZE; i++)  
    dynamic_cast<Cat*>(zoo[i])->catchMouse();
```

Итак, зачем нужно безопасное приведение типов? Чтобы безопасно приводить типы. А зачем вообще нужно приводить типы? Мы уже говорили: не для того, чтобы преобразовывать объекты одних классов в другие, это невозможно. Приведение типов в узком смысле ООП нужно тогда, когда мы объекты классов-потомков храним в переменных классов-предков, но при этом нам нужно проверить возможность вызова и вызвать какой-то метод, который специфичен только для некоторых из классов, которые мы единообразно храним.

Кстати, существует и альтернативный подход к решению этой проблемы. Мы могли с самого начала просто взять и метод `catchMouse()` объявить, причем виртуальным, в самом базовом классе. Не как абстрактный, а просто с пустой реализацией, примерно так:

```
class Animal {  
public:  
    virtual void catchMouse() {  
    }  
};
```

В самом классе `Cat` мы бы его перекрыли и заполнили нужной функциональностью, а все остальные классы унаследовали бы его от базового класса пустым. Это привело бы к тому, что приведение типов стало бы ненужным, потому что мы могли бы просто вызывать этот метод для всех объектов `zoo[i]`:

```
for (int i = 0; i < ZOO_SIZE; i++)  
    zoo[i]->catchMouse();
```

Раз этот метод виртуальный, то для тех из объектов `zoo[i]`, которые являются `Cat` или его потомками, вызвался бы перекрытый метод с полезной нагрузкой, а для всех остальных вызывался бы унаследованный метод, который ничего не делает. В результате мы получили бы тот же самый эффект — полезная нагрузка метода `catchMouse()` вызвана только для объектов `Cat` и потомков. Это еще и работать будет быстрее, потому что одно дело — глянуть в таблицу виртуальных методов и просто прыгнуть внутрь пустого метода и сразу вернуться оттуда обратно, а другое — так же смотреть в таблицу виртуальных методов, потом прыгать в какой-то код, который начинает разбираться, являешься ли ты `Cat` или нет.

Работать последний приведенный выше код будет быстрее, но идеологически это означало бы, что все-все-все объекты нашей иерархии, в том числе и собаки, умеют ловить мышей, потому что у них есть этот метод. Решайте сами, что вам важнее: скорость и эффективность работы — или правильная модель предметной области.

Из статьи в статью, из комментария в комментарий, из страницы в страницу по просторам программистского Интернета гуляет одно и то же рассуждение: `dynamic_cast` — плохой (приведение типов — плохо), это признак плохого дизайнера, срочно заменяйте его на обычные виртуальные функции! Ах, если бы

можно было плохой дизайн определять так легко, просто по наличию конкретных ключевых слов в тексте программы! На самом деле утверждение «dynamic_cast — это плохой дизайн» верно только тогда, когда это действительно плохой дизайн, а не тогда, когда это просто dynamic_cast. Например, подобный код — это действительно совершенно неправильное использование dynamic_cast:

```
for (int i = 0; i < ZOO_SIZE; i++)
    if (dynamic_cast<Cat*>(zoo[i]))
    {
        // 1
    }
    if (dynamic_cast<Dog*>(zoo[i]))
    {
        // 2
    }
    if (dynamic_cast<Whale*>(zoo[i]))
    {
        // 3
    }
    if (dynamic_cast<Bear*>(zoo[i]))
    {
        // 4
    }
```

Очевидно, что есть какая-то ситуация (происходящая внутри этого цикла), где каждое из животных должно вести себя по-своему, но эта ситуация, по-видимому, применима ко всем животным. В таком случае, безусловно, никакого dynamic_cast не должно быть, а должна быть виртуальная функция у Animal, которая у всех животных перекрывается по-своему. Но всегда ли это так? Нет, не всегда! Если какая-то специфическая операция целесообразна и применима только к малой части потомков Animal, то засовывать ее в базовый класс, как советуют в этих ваших интернетах, и будет, на мой взгляд, ошибкой дизайна:

```
for (int i = 0; i < ZOO_SIZE; i++)
{
    FlyingAnimal* fa = dynamic_cast<FlyingAnimal*>(zoo[i]);
    if (fa)
        fa->fly();
}
```

Неужели в такой ситуации вы тоже считаете, что метод fly должен был быть объявлен в базовом классе Animal? Типа ежики тоже умеют летать, только низко, недалеко и при должной мотивации? Наверное, правильно будет сказать так: если вы пытаетесь выполнить dynamic_cast и затем вызвать операцию, которая целесообразна/определена для всех или большинства потомков базового класса, — то это плохой дизайн и нужно перенести этот код в виртуальный метод

базового класса. А если, наоборот, вы объявили в базовом классе виртуальный метод с пустой реализацией (не абстрактный!), который перекрывается только в одном из потомков, — то именно это и будет плохим дизайном и нужно вместо этого использовать приведение типов (по факту в форме проверки на поддержку интерфейса).

1.18. Передача и возврат объектов из функций в C++

Создаваемые нами функции и методы принимают данные для работы. Данные совсем не обязательно будут простыми, рано или поздно нам понадобится передавать в функции и методы одних объектов другие объекты. Описанные далее проблемы большей частью касаются именно C++, потому что именно в C++ существует целых три способа передать объект внутрь функции:

```
void in1(Object obj) {  
    printf("inside function in1 \n");  
};  
void in2(Object *obj) {  
    printf("inside function in2 \n");  
};  
void in3(Object &obj) {  
    printf("inside function in3 \n");  
};
```

Первая функция принимает объектную переменную, вторая принимает указатель, а третья принимает ссылку. Во всех функциях для исследования этого вопроса должен быть отладочный вывод, а передаваемый объект `Object` обязательно должен иметь всю коллекцию стандартных конструкторов и деструктор, причем все с отладочным выводом.

Создайте два объекта класса `Object` и попробуйте передать их все в эти три функции и объяснить (а еще лучше — заранее предсказать) тот результат, который вы получите:

```
Object o;  
Object *po = new Object();  
in1(o);  
in2(&o);  
in3(o);  
in1(*po);  
in2(po);  
in3(*po);
```

Вы увидите, что при передаче объекта в первую функцию компилятор создает копию исходного объекта с помощью конструктора копирования. Это означает, во-первых, дополнительные потери времени и памяти при вызове этой функции,

а во-вторых, что любые изменения объекта внутри функции никак не повлияют на переданный в нее объект, потому что будут выполняться именно с копией (если, конечно, вы не сделали конструктор копирования с поверхностным копированием внутри). При выходе из первой функции созданная копия будет уничтожена. Это поведение не свойственно большинству других ЯП при работе с объектами. При передаче объекта во вторую и третью функции он не копируется, и функция работает с исходным объектом (и может его изменить), а при выходе из функции соответственно ничего не удаляется, раз не создавалось. Это поведение аналогично большинству других ЯП при работе с объектами.

С возвращением объектов из функции все гораздо сложнее и интереснее. Вот вам целых шесть вариантов возвращения объектов из функции (рис. 1.9). Обратите внимание, что во всех случаях мы пытаемся вернуть не тот объект, который нам в функцию передали (у этих функций нет аргументов), а тот, что мы создаем внутри функции.

<pre>Object out1() { Object o("data"); return o; };</pre>	<pre>Object out2() { Object *o = new Object("out2"); return *o; };</pre>
<pre>Object* out3() { Object o("out3"); return &o; };</pre>	<pre>Object* out4() { Object *o = new Object("out4"); return o; };</pre>
<pre>Object& out5() { Object o("out5"); return o; };</pre>	<pre>Object& out6() { Object *o = new Object("out6"); return *o; };</pre>

Рис. 1.9. Шесть любимых способов выстрелить себе в ногу

Вызывать первую функцию будем примерно так:

```
Object o1 = out1();
```

Как всегда, отладочный вывод должен вам подсказывать, что происходит, но происходить может разное в разных компиляторах и в разных режимах оптимизации. При работе с первой функцией внутри нее «статически», «на стеке» создается локальный объект. Затем вы увидите, как конструктором копирования этот локальный объект копируется в создаваемый тут же объект `o1`, а созданный ранее локальный объект удаляется. Впрочем, если изменить вызов функции `out1`, то все станет еще интереснее:

```
Object o1;
o1 = out1();
```

Мы тут перекрыли возможность компилятору пойти по короткому пути, и он сначала создаст объект `o1`, потом внутри функции `out1` создаст локальный объ-

ект, потом конструктором копирования создаст временный объект для возврата результата из функции, потом уничтожит локальный объект, потом побитово скопирует результат в `o1`, потом уничтожит этот возвращенный из функции временный объект. Как будто, тут много лишних созданий-уничтожений объектов? На самом деле компилятор не обязан буквально выполнять наш код и при мало-мальски включенной оптимизации он сам от лишних созданий-удалений избавится, и это будет самый простой и надежный способ вернуть объект.

Теперь будем вызывать вторую функцию. Примерно так:

```
Object o2 = out2();
```

При работе со второй функцией внутри нее «динамически», «на куче», создается объект. Затем он конструктором копирования копируется в создаваемый тут же объект `o2`, и мы выходим из функции. Подождите, а что же с динамически созданным объектом? А он остается висеть где-то в памяти, неуничтоженный, и освободить эту память больше никто не может. Привет, утечка! А если аналогично первой функции изменить вызов?

```
Object o2;  
o2 = out2();
```

Опять компилятор вынужден сначала создавать объект `o2`, потом внутри функции `out2` он создаст динамический объект, потом конструктором копирования создаст временный объект для возврата результата из функции, потом побитово скопирует результат в `o2`, потом уничтожит этот возвращенный из функции временный объект... И снова динамически созданный объект останется висеть в памяти и никуда не денется — утечка памяти.

Теперь будем вызывать третью функцию. Примерно так:

```
Object *o3 = out3();
```

При работе с третьей функцией внутри нее статически создается локальный объект. Затем вы увидите, как прямо перед выходом из функции этот локальный объект уничтожается... А что же тогда возвращается и помещается в переменную `o3`? Вы правы, туда складывается адрес локального объекта — объекта, который уже уничтожен. Любая попытка обратиться к свойствам или методам этого объекта вызовет краш программы или UB. Так что здесь мы имеем дело с классическим примером обращения к чужой памяти, без вариантов.

Теперь будем вызывать четвертую функцию. Примерно так:

```
Object *o4 = out4();
```

Тут мы увидим один из самых кратких выводов. После захода внутрь функции мы увидим, как создается динамический объект и его адрес из функции возвращается и помещается в переменную `o4`. Никаких лишних копирований объектов, но потенциальная (только потенциальная!) утечка памяти, потому что функция нам вместе с адресом объекта `o4` вручила поручение о его своевременном уничтожении, когда он будет больше не нужен, и мы об этом не должны забывать.

Избавиться от этого единственного недостатка данного способа нам поможет `unique_ptr`.

Теперь будем вызывать пятую функцию. Примерно так:

```
Object &o5 = out5();
```

При работе с пятой функцией очень похожая ситуация с функцией номер 3: внутри нее статически создается локальный объект, прямо перед выходом из функции этот локальный объект уничтожается, но ему успевает назначиться новое имя `o5` (потому что именно так работают ссылки). Любая попытка обратиться к свойствам или методам объекта `o5` после вызова функции вызовет краш программы или UB. Опять классический пример обращения к чужой памяти. Попытка вызвать эту функцию как `Object o5 = out5()`, т. е. с присваиванием результата не ссылке, а объекту, все равно вызовет краш или UB: компилятор попытается создать объект `o5` вызовом конструктора копирования, — но, вот беда, к этому моменту объект, созданный внутри функции, с которого компилятор пытается сделать копию, уже уничтожен.

И наконец, будем вызывать шестую функцию. Примерно так:

```
Object &o6 = out6();
```

После захода внутрь шестой функции мы увидим, как создается динамический объект и его имя назначается ссылке `o6` после выхода из функции. Никаких лишних копирований объектов, этот вариант похож на четвертый вариант, но проблема с удалением объекта, ответственность за который повисла на нас, стала гораздо тяжелее: ссылки не удаляют те объекты, на которые они указывают! Значит, как только мы выйдем за пределы области видимости переменной `o6`, объект останется висеть в памяти — привет, утечка! Удалить такой объект можно, но очень извратным способом: `delete &o6`.

Все описанное выше у вас может воспроизвестись, а может и не воспроизвестись, в зависимости от того, какой компилятор с какими настройками вы используете. Наблюдаемое поведение справедливо, например, для Visual Studio 2013 с отключенной оптимизацией, но уже при включении оптимизации компилятор включает различные механизмы типа `copy elision`, для того чтобы постараться избежать созданий лишних «промежуточных объектов».

Так как же правильно возвращать из функции созданный внутри нее объект? Похоже, что самый правильный и надежный тут только первый вариант, с возвратом по значению? А может, вообще нужно создавать объект заранее в вызывающем коде и передавать его функции для заполнения, а сама функция ничего создавать на всякий случай не должна?

В [6] рекомендуется возвращать объекты по значению (первый вариант) и передавать право/обязанность владения с использованием умных указателей, если по значению передавать нельзя и требуется именно работа со ссылками. Если требуется работа именно с обычными указателями, то там же рекомендуют явно разделять указатели на «владеющие» и «невладеющие»:

```
owner<X*> compute(args)    // тут явно вместе с объектом
                             передается право владения
{
    owner<X*> res = new X{};
    // ...
    return res;
}
```

Там же в [6] предлагается следующее правило: если объект передается куда-то просто указателем, то считается, что им владеет вызывающий код, а вызываемый его уничтожить права не имеет.

1.19. Проблемы работы с памятью и не только, в C++ и не только

В конце концов, ошибки, возникающие при работе с объектами, сводятся всего лишь к двум вариантам. Такие простые два варианта и такой огромный спектр неправильных действий, которые к одному из них могут привести!

- ❑ Ошибка первого рода: обращение к памяти, которая нам уже/еще не принадлежит. Объекта нет, а мы пытаемся к нему обращаться.
- ❑ Ошибка второго рода: необращение к памяти, которая нам принадлежит. Объект еще есть, а мы уже не обращаемся к нему, потому что не можем.

Первый вариант возникает тогда, когда объект уже был удален где-то в одном участке кода, но другой участок кода об этом не знает и пытается к нему обращаться. Получается обращение к памяти, которая нам уже не принадлежит (хоть на чтение, хоть на запись), и система этого не допустит — здравствуй, *access violation*! Мы видели такие ошибки при работе с композицией и поверхностным копированием.

Второй вариант возникает тогда, когда объект был создан, но его адрес больше не хранится ни в какой переменной и некому его удалить. Может показаться, что эта проблема и не проблема вовсе, потому что она не вызывает видимую ошибку в программе. На самом деле утечка памяти (а это именно она) плоха тем, что проявляется она не сразу, и поэтому отлаживать и исправлять такие ошибки гораздо сложнее.

Казалось бы, решение простое: надо «всего лишь» вовремя удалять создаваемые объекты — и все. И все действительно просто для тех объектов, которые вам нужны локально для выполнения какой-то задачи: вы их можете создать в одной функции и в ней же удалить. Гораздо сложнее проблема тогда, когда объекты создаются динамически в соответствии с какими-нибудь непредсказуемыми событиями (например, действиями пользователя) и удаляться должны тоже в непредсказуемое время (например, в соответствии с какими-нибудь другими действиями пользователя).

Проблема корректной работы с памятью — это проблема владения. Владеет объектом тот, кто отвечает за его время жизни (вовремя создать и вовремя удалить), и проблема владения объектами — это центральная проблема всех технологических решений, которые придумываются для борьбы с ошибками первого и второго рода.

Если какой-то объект передается в функцию, становится ли функция ответственной за его время жизни (а значит, должна ли она его удалять, когда он больше не будет ей нужен) или нет? Если один объект хранит ссылку на какой-то другой объект, становится ли он ответственным за время жизни этого объекта? А что если объект, ссылка на который хранится где-то, будет удален? Если какой-то объект возвращается из функции, то возвращается ли вместе с ним право владения и обязанность следить за временем его жизни? Как быть, если объект одновременно может быть нужен в нескольких местах кода, каждое из которых может про другое место не знать? Должен быть кто-то еще, кто следит, а есть ли еще кто-то, кому конкретный объект нужен? Вот только малая часть тех вопросов, которые возникают у создателей технологий для борьбы с проблемами первого и второго рода!

Для решения проблем с ручной работой с памятью нужны технологии и правильные практики. Правильные практики при работе с C++ во множестве описаны, например, в [6] и включают, например, такие эвристические правила, как «правило трех», «правило пяти», «RAII» и «никогда не используй голые указатели». Из технологий автоматического управления динамической памятью можно выделить подсчет ссылок и сборщик мусора, которые часто используются совместно.

Сборщик мусора, а точнее, сборщик недостижимого мусора, — это технология автоматического управления памятью, при которой в исполняемой программе периодически запускается специальный код, который находит и уничтожает «недостижимые» объекты, т. е. такие объекты, которые код не может использовать просто потому, что не существует переменных, через которые до этих объектов можно «достучаться». В любой момент времени у программы есть определенный набор «корневых объектов» — это глобальные переменные, а также локальные переменные и параметры функции для всех функций, которые в данный момент выполняются (т. е. которые находятся в текущий момент в стеке вызовов *call stack*). Эти объекты могут содержать ссылки на другие объекты, а те в свою очередь — на новые объекты и т. д. Получается, что в любой момент времени есть своеобразный ориентированный граф связей, включающих все потенциально достижимые объекты. Если при этом где-то хранить список всех объектов, то разница между ними покажет те объекты, которые синтаксически уже недостижимы — именно их сборщик мусора и уничтожает. Заметим, речь идет только о синтаксической недостижимости: возможно, есть объекты, которые синтаксически достижимы, но никогда в соответствии с логикой программы не будут использованы, — такие объекты сборщик мусора детектировать и удалить не сможет.

Проблемы со сборщиком мусора связаны, во-первых, с тем, что он, запускаясь, замораживает выполнение программы на некоторое время, по закону подлости в самый неподходящий момент. Во-вторых, удаляемые объекты могут удаляться в произвольном порядке и, самое главное, для программиста не понятно, когда именно объект будет удален (а значит, например, когда именно будут освобождены ресурсы, которые освобождаются в его деструкторе). В некоторых языках есть возможность вручную вызвать срабатывание сборщика мусора, чтобы он отработал в то время, когда пользователь задержки не заметит.

Подсчет ссылок использует другой принцип. В этой технологии каждый объект кроме всего прочего хранит еще и число активных на него ссылок. Если ссылка на объект *X* присваивается какой-то новой переменной *Y*, то у объекта *X* его внутренний счетчик ссылок увеличивается на единицу. Если после этого переменной *Y* присваивается ссылка на какой-то другой объект *Z*, то у объекта *X* счетчик уменьшится на единицу, а у объекта *Z* увеличится на единицу. Если локальная переменная *Y* выходит из области видимости, она неявно зануляется, а значит, уменьшается на единицу счетчик у того объекта, на который она раньше указывала. Ровно в тот момент, когда счетчик ссылок какого-то объекта становится равным нулю (на него больше не ссылается никакая переменная), объект удаляется с вызовом своего деструктора.

При подсчете ссылок объекты удаляются сразу, когда становятся недостижимыми, поэтому не требуется больших и неожиданных пауз, как при сборке мусора. Удаление объекта происходит предсказуемо для программиста (если он, конечно, не потерял забытую ссылку на этот объект где-то еще). Самая простая реализация механизма подсчета ссылок гораздо проще, чем самая простая реализация сборщика мусора.

Однако при этом в каждом объекте появляется дополнительное поле для счетчика ссылок, любое присваивание требует дополнительных действий, и самое важное — если вдруг появятся объекты или группы объектов, хранящие ссылки друг на друга (недостижимые острова взаимосвязанных объектов), их счетчики ссылок никогда не будут занулены, а значит, такие объекты никогда не будут удалены. Реализация механизмов поиска таких островов объектов — гораздо сложнее, чем сам механизм. К тому же и в случае обнаружения такого острова взаимосвязанных объектов не понятно, в каком порядке их нужно удалять.

При использовании механизма подсчета ссылок часто используют так называемые сильные и слабые ссылки. Первые участвуют в подсчете ссылок и влияют, таким образом, на время жизни объектов, а «слабые» ссылки необходимы только для связи объектов. Взаимосвязанные объекты возникают везде — хотя бы в любой иерархии связанных объектов. В такой иерархии, например, объекты-родители будут хранить сильные ссылки на детей, а дети будут хранить слабые ссылки на родителей, чтобы всю иерархию при необходимости можно было бы удалить.

В этом разделе речь шла про работу с памятью, но на самом деле написанное справедливо для любого ограниченного ресурса (примером которого является

память): всякие файлы-кавайлы, мьютексы-шмьютексы, gdi-бэдзи и т. д. Все это нужно вовремя выделять и освобождать, и к ООП это имеет отношение только потому, что мы все эти ресурсы оборачиваем в объекты, выделение ресурсов привязываем к конструкторам, а освобождение — к деструкторам (вновь отправлю вас к концепции, которая называется RAII). Это важно как минимум потому, что если за временем жизни объектов при автоматическом создании следит сам компилятор, то он удалит объекты с вызовом деструктора в любом случае — и при нормальной работе, и при любом исключении (а о том, чтобы не было исключений внутри деструкторов, нужно заботиться даже больше, чем о собственном здоровье!)

1.20. Технологии и практики управления долгоживущими объектами в C++

Еще раз подчеркнем важную мысль из предыдущего раздела. Главная проблема работы с ООП в C++ — это проблема работы с памятью, занимаемой объектами. Проблема корректной работы с памятью — это проблема владения. Владеет объектом тот, кто отвечает за его время жизни (вовремя создать и вовремя удалить), и проблема владения объектами должна стать вашей главной, центральной *obsession* при разработке надежных программ.

Технически проблема времени жизни объектов в C++ актуальна по большей части только для нелокальных объектов с непредсказуемым временем жизни. Если объект создан локально «на стеке», достаточно нигде больше его не сохранять, и тогда, куда бы он ни передавался, как только код выйдет за пределы области, в которой объект был создан (или произойдет исключение), он автоматически удалится.

К сожалению, это не всегда возможно: часто есть какие-то объекты, которые создаются и уничтожаются как реакция на непредсказуемые события (системные или пользовательские), соответственно время их жизни не ограничено тем блоком кода, в котором они создаются. Другая проблема заключается в том, что поток выполнения программы не всегда предсказуем для программиста: всякие возникающие исключения могут нарушить планы программиста вовремя удалить объект, который он создал.

И самое неприятное — это то, что на объекте (или указателе) не написана его будущая судьба: всегда ли он будет локальным или нелокальным, кем и когда он создается или будет создаваться, и, глядя на обычный указатель или ссылку на объект, никак нельзя предсказать политику его, как это называется, *lifetime management policy*.

В этом случае для надежной работы с памятью в C++ (а именно, начиная с C++11) могут использоваться так называемые умные указатели `unique_ptr` и `shared_ptr` (и другие, но не `auto_ptr`). Этот раздел посвящен только очень краткому знакомству с ними, потому что:

- ❑ это все слишком специфично для C++;
- ❑ глубокое понимание их требует глубокого погружения в детали C++, связанные с move-семантикой;
- ❑ но рассказ не будет полон без краткого упоминания их, потому что технология подсчета ссылок, аналогичная `shared_ptr`, является стандартом в большинстве других языков;
- ❑ сами умные указатели в том виде, в каком они есть сейчас, появились сравнительно недавно, поэтому правильные практики их использования еще долго будут устаканиваться.

Умные указатели — это технология, суть и декларация. Технология — это самое простое. Умные указатели работают так, как работают, их синтаксису и семантике использования можно просто научиться. Но если не понимать суть той проблемы, которую они решают, то можно просто усугубить (спрятать) свою проблему. Умные указатели сами по себе не дают гарантий корректной работы с памятью: неправильная с ними работа — очень простой путь выстрелить себе в ногу. Гарантию дает только понимание их сути и правильное использование. Кроме этого, использование соответствующих умных указателей — это декларация, документация того, как можно и нужно работать с конкретным классом или конкретной функцией; описание того, что она может/должна, а что не может или не должна делать.

Умные указатели, и `unique_ptr`, и `shared_ptr` — это классы-обертки над обычными указателями, точно так же, как какой-нибудь ваш `Storage` может быть классом-оберткой над обычным `vector`. Зачем нужны какие-то обертки, зачем нужны умные указатели? Чем плохи, «глупы» обычные указатели?

Умные указатели, если совсем на пальцах, утверждают: «Обычные указатели — зло, потому что их слишком легко копировать, а копирование указателей — зло». Почему создание копий указателей — зло? Да потому что пока указатель на объект один, есть единственная точка доступа к нему и единственная точка, где этот объект нужен, где понятно, жив он или нет, и понятно, как и когда его удалять. Как только вы создаете несколько копий указателя, возникают вопросы:

- ❑ если он нужен в нескольких местах, то в каком месте и когда его будут/могут/должны удалять;
- ❑ как в одном месте понять, жив ли объект, или его уже удалили в другом месте?

Раз создание копий указателя — зло, то, может, надо запретить создание копий? Этот подход к решению проблемы и демонстрирует `unique_ptr`: вы ему отдаете обычный указатель, забываете про этот указатель и дальше работаете через `unique_ptr`:

```
#include <memory>

using namespace std;
```



```

class Point {
private:
    int x;
    int y;
public:
    Point() : x(0), y(0) {
        printf("Point()\n");
    }
    Point(int x, int y) : x(x), y(y) {
        printf("Point(int x, int y)\n");
    }
    Point(const Point &t) : x(t.x), y(t.y) {
        printf("Point(const Point &t)\n");
    }
    void dump(){
        printf("%d, %d\n", x, y);
    }
    ~Point(){
        printf("~Point()\n");
    }
};

int _tmain(int argc, _TCHAR* argv[])
{
    unique_ptr<Point> p(new Point(1,2));
    p->dump();
    return 0;
}

```

Как видим, сначала динамически создается новый объект `Point`, потом его адрес передается в создаваемый умный-указатель `unique_ptr<Point>`. Для существующего указателя других вариантов у вас нет. Но в этом конкретном случае, раз мы создаем объект `Point` прямо здесь и сейчас, то более эффективно и правильно само создание поручить шаблонной функции `make_unique<>()`, которая сделает то же самое:

```

unique_ptr<Point> p = make_unique<Point>(1,2);
p->dump();

```

Обратите внимание, с точки зрения синтаксиса языка `p` — это объект, поэтому, вообще говоря, все (служебные) функции у него вызываются через точку: `p.get()`, `p.release()`, `p.reset()`, но у него есть перекрытый оператор «->», который позволяет обращаться к хранящемуся внутри указателю (а значит, и к самому умному указателю) как к обычному указателю: `p->dump()`.

А вот в чем главная особенность его:

```

unique_ptr<Point> p2;
p2 = p; // ошибка

```

Если сначала создать «пустой» умный указатель `p2`, т. е. не «управляющий» пока никаким объектом, а потом попробовать записать в него другой умный указатель, то ничего не выйдет — компилятор такое не пропустит.

```
unique_ptr<Point> p3(p); // ошибка
```

Это же касается и других способов получить копию объекта, например с помощью конструктора копирования, — такое тоже не пройдет. Конечно, способ обмануть компилятор и выстрелить себе в ногу всегда есть: можно сначала создать указатель, а потом передать его в два умных указателя, которые оба будут считать, что они «уникальные»:

```
Point *p = new Point(1,2);  
unique_ptr<Point> p1(p);  
unique_ptr<Point> p2(p);
```

Это, конечно, тоже вызовет ошибку, и главная беда тут будет в том, что мы будем использовать `unique_ptr`, не понимая его суть, предназначение. А оно именно в том и заключается, чтобы он стал единственным владельцем, единственной точкой доступа к созданному объекту `Point`, той точкой, которая ответственна за хранение и удаление объекта `Point`.

По этой же причине технически можно создать не статический, а динамический умный указатель: `unique_ptr<Point>* pp = new unique_ptr<Point>(p)`, и его даже можно будет сколько угодно копировать, но это все будет нарушением самой идеи, ради которой все задумывалось.

Использовать `unique_ptr` для хранения конкретного объекта `Point` нужно для того, чтобы не только технически (постараться) обеспечить отсутствие «нелицензионных указателей» на этот объект, но для того, чтобы подчеркнуть: «этот кусок кода, что держит этот `unique_ptr`, является владельцем объекта `Point` и отвечает за его время жизни». Как отвечает? Да очень просто: `unique_ptr<Point> p`, объявленный в примере выше, — это обычный локальный объект, и он будет удален, как только мы выйдем из зоны его видимости. А при его удалении он удалит и хранимый в нем `Point`:

```
{  
    {  
        unique_ptr<Point> p = make_unique<Point>(1, 2);  
        p->dump();  
    }  
    // p и Point удаляются здесь  
}
```

Подождите, но чем тогда это отличается от простого локального объекта, зачем так много лишнего?

```
{  
    {  
        Point p(1, 2);
```

```
p.dump();  
} // p и Point удаляются здесь  
}
```

Разница появляется тогда, когда нам нужно не просто работать с этим объектом локально, но передавать его куда-то в другую функцию или возвращать его откуда-то из третьей функции. В случае `Point` это неизбежно потребует или передачи/возврата копий, или передачи/возврата адресов (т. е. ссылок и указателей). И то и другое возможно, но имеет понятные недостатки.

```
void pass_object(unique_ptr<Point> p)  
{  
    p->dump();  
}  
  
int _tmain(int argc, _TCHAR* argv[])  
{  
    unique_ptr<Point> p = make_unique<Point>(1, 2);  
    pass_object(p);  
}
```

Этот код не скомпилируется. Если бы он скомпилировался, то в какой-то момент у нас оказалось бы две копии `unique_ptr` с одной и той же точкой внутри: одна копия в основной функции, другая — в функции `pass_object`, и суть была бы растоптана. Технически, чтобы этот код скомпилировался, надо передавать в функцию не `p`, а `move(p)`:

```
pass_object(move(p));
```

И эта добавка `move()` декларирует: теперь ответственность за время жизни `Point` переходит туда, в функцию `pass_object()`. У нас появляются два умных указателя — один в основной программе, а другой в функции `pass_object()`, но при этом указатель на один-единственный существующий `Point` в каждый момент времени будет только в одном из них.

Если вы будете отлаживать этот код по шагам, то заметите, что внутри `pass_object()` объект доступен и работает, но при выходе из функции ее аргумент будет уничтожен (как и любой другой аргумент любой другой функции), и вместе с собой он уничтожит и объект `Point`. В итоге после возврата из функции выяснится, что умный указатель остался на месте, но вот объекта `Point` в нем уже нет: он передан в функцию `pass_object()` и там «потреблен».

Еще раз. Умный указатель `unique_ptr` берет на себя ответственность за время жизни того объекта, который он хранит, и не допускает больше никого к этой ответственности. Но он не запрещает программисту передавать объект куда-то, он просто заставляет в явном виде (с помощью функции `move()`) подчеркивать, что передается не только объект, но и ответственность за его время жизни. Можно ли вернуть объект из функции обратно? Легко, с помощью той же самой функции `move()`:

```
unique_ptr<Point> pass_object(unique_ptr<Point> p)
{
    p->dump();
    return p; // возвращается move(p);
}

int _tmain(int argc, _TCHAR* argv[])
{
    unique_ptr<Point> p = make_unique<Point>(1, 2);
    p = pass_object(move(p));
    p->dump();

    return 0;
}
```

Функция возвращает `return p`, но по смыслу там происходит (вызовом конструктора перемещения) `return move(p)`. Таким образом, один-единственный объект `Point` внутри разных умных указателей будет «перекидываться» из одного места в другое, как горячая картошка, но всегда будет одно и только одно место, где этот умный указатель будет «непустой», потому что в этот момент именно в нем и будет лежать `Point`.

Можно ли в функцию передать не умный указатель, а объект, находящийся в нем? Можно: у умного указателя есть метод `get()`, возвращающий указатель на хранимый объект. Но спросите себя, что это будет означать? А что если функция, куда вы передали этот указатель, удалит или, того хуже, куда-то его сохранит? Разные альтернативные способы передать куда-нибудь умный указатель приведены, например, в статье [8].

До сих пор мы обсуждали только `unique_ptr`, но это не единственный подход к решению проблемы существования множества копий одного и того же указателя. Вспомним, почему это проблема. Потому что если есть несколько копий одного и того же указателя, то:

- ☐ в каком месте и когда его будут/могут/должны удалять?
- ☐ как в одном месте понять, жив ли объект, или его уже удалили в другом месте?

Подход `unique_ptr` дает одно решение: запретить иметь несколько копий одного указателя. Другой умный указатель `shared_ptr` предлагает другое решение. Он говорит: «Давайте так: кто последний останется, тот и ~~выключит свет~~ удалит объект». Этот подход позволяет множеству существующих `shared_ptr` хранить указатель на один и тот же объект вместе с подсчетом количества таких `shared_ptr`. Как только появляется новый `shared_ptr`, хранящий ссылку на все тот же объект, счетчик ссылок для этого указателя увеличивается на единицу, а как только очередной `shared_ptr` удаляется, счетчик ссылок уменьшается. Как только он достигнет нуля — объект будет удален. Другими словами, это все тот же старый добрый `reference counting`.

```

shared_ptr<Point> pass_object(shared_ptr<Point> p)
{
    p->dump();
    return p;
}

int _tmain(int argc, _TCHAR* argv[])
{
    shared_ptr<Point> p = make_shared<Point>(1, 2);
    p = pass_object(p);
    shared_ptr<Point> p2 = p;
    p2->dump();

    return 0;
}

```

Как видим, с `shared_ptr` никаких проблем с передачей его в функцию и возврата из нее нет, как нет их и с присваиванием в другую переменную. Счетчик ссылок автоматически увеличивается и уменьшается при любом таком присвоении и удалении очередного локального умного указателя соответственно.

Умный указатель `unique_ptr` декларирует: «Этот объект — мой, до тех пор пока я явно его кому-то не передам. Пользоваться — пользуйтесь на здоровье, но отвечаю за его время жизни я, этот объект меня не переживет». Аналогичным образом, `shared_ptr` декларирует: «Этот объект — нужен как минимум мне, но может быть и кому-то еще, я не против. Но пока он мне нужен — его никто не сможет удалить».

С `shared_ptr` возможна другая проблема — циклические ссылки. Если два объекта внутри содержат поля в виде умных указателей друг на друга, то они никогда не будут удалены. Для решения этой проблемы используется третий тип умного указателя: `weak_ptr` (аналогичные возможности есть и в других языках со встроеной технологией подсчета ссылок).

Он никак не влияет на владение объектами и не предотвратит удаление объекта, на который никто, кроме него, не ссылается. Но, в отличие от обычного указателя, он всегда знает, «жив» ли еще тот объект, на который он ссылается. Таким образом, комбинируя `shared_ptr` и `weak_ptr`, можно создавать циклические структуры данных, которые, однако, не приводят к утечке памяти, например дерево:

```

class BinaryTree
{
    weak_ptr<BinaryTree> parent;
    unique_ptr<BinaryTree> left;
    unique_ptr<BinaryTree> right;
};

```

Тут любой узел бинарного дерева владеет своими левым и правым поддеревьями, но не владеет своим родителем, хотя и хранит указатель на него. На самом

деле делать на умных указателях структуры типа деревьев, графов или даже просто связанных списков — не самая правильная идея, потому что удаление такой структуры через цепочку вызовов деструкторов у умных указателей может привести к переполнению стека, если структура окажется большой. Тогда все равно придется делать вручную нормальный деструктор, в котором самостоятельно обходить все узлы и удалять, что нужно. А зачем тогда умные указатели?

Итак, в заключение этого очень краткого раздела про умные указатели подведем итоги. Если есть возможность не работать с указателями — лучше вообще с ними не работать, обходясь локальными объектами. Даже при том, что у вас каждому `new` в коде соответствует правильный `delete` во всех возможных вариантах исполнения этого кода, которые вы предусмотрели, неожиданное исключение посередине между ними может привести к тому, что до `delete` дело не дойдет и память утечет.

Если указатели нужны, то лучше использовать умные указатели. Умные указатели — про описание и декларацию времени жизни объектов. В зависимости от того, кто и как будет объектом владеть, нужно использовать разные умные указатели.

Старайтесь по возможности всегда использовать по умолчанию `unique_ptr`. Содержащийся в нем обычный указатель можно передавать в те функции, которые должны этот объект использовать, но не должны с ним ничего делать. Сам `unique_ptr` вполне хорош до тех пор, пока не станет очевидно, что владеть этим объектом должно несколько объектов сообща. При композиции, например, более чем очевидно, должен использоваться именно `unique_ptr`. Фабрика, скорее всего, должна тоже возвращать `unique_ptr`, подчеркивая тот факт, что теперь за время жизни созданного ею объекта отвечает не она. К тому же `unique_ptr` просто меньше памяти требует и быстрее работает, чем `shared_ptr`. С другой стороны, `shared_ptr` используется тогда, когда нет очевидного единственного владельца, заранее известного; когда объект в равной мере нужен разным другим объектам. Но в этом случае посмотрите, не образуется ли циклов «владения», и определите для циклических ссылок, какая из них будет «сильной» (`shared_ptr`), а какая — «слабой» (`weak_ptr`).

Статья [8] при этом советует передавать в функции не умные указатели, а сами объекты, потому что это позволяет функции работать с объектом независимо от того, как вызывающий код «рулит» его временем жизни (если функция предполагала, например, что ей передадут `shared_ptr`, и раньше так и было, но потом в результате рефакторинга `shared_ptr` в вызывающем коде превратился в `unique_ptr`, неужели нужно переписывать и функцию?). Понятно, что передавать объект по указателю, а не по ссылке, стоит тогда, когда есть возможность или потребность передачи `nullptr`. Передавать `unique_ptr` по значению можно и нужно только в ту функцию, которая должна его «потребить» и не вернуть. Передавать `shared_ptr` по значению есть смысл только тогда, когда принимающая функция планирует присоединиться к управлению этим объектом и поэтому и так сохранит копию.

ГЛАВА 2

ПРИЕМЫ ООП

2.1. Демистификация паттернов ООП

Шаблоны проектирования в ООП — они же просто «паттерны ООП» — известны тем, что вызывают очень разную реакцию в сообществе программистов. Кто-то — обычно это молодые и неопытные программисты, — узнав о паттернах, становятся ярыми евангелистами Этой Универсальной Парадигмы и создают код, в котором фабрики фабрик команд клонируют декораторы наблюдателей. Другие — обычно это очень опытные программисты — считают само использование паттернов антипаттерном, а любую попытку о них «в сотый раз» рассказать — публичной пропагандой нетрадиционных ~~отношений~~ методов разработки.

На самом деле паттерны ООП — это просто типовые приемы работы с объектами, и, по сути, любой, кто разрабатывает объектно-ориентированные программы и при этом обладает склонностью к самоанализу, рано или поздно самостоятельно переизобретет самые популярные паттерны. Если при этом он сможет их достаточно внятно описать, дать им понятные имена и написать на них диссертацию, то он сможет повторить путь, который прошла «Банда четырех» с их нетленкой [7], манера написания которой (весьма академическая) очень многих раздражает (тем не менее многие согласны ее терпеть). Тем же, кого (в отличие от автора) литературная стезя не манит, достаточно просто овладеть всем спектром приемчиков, чтобы хотя бы не изобретать велосипеды. Не зря [7] называется именно «Приемы ООП», а слово «паттерны» идет уже только подзаголовком. На «Банде четырех» литература не исчерпывается: желающие могут ознакомиться, например, с [9] или [10].

Итак, паттерн — это:

- ☐ какая-то типовая проблема, с которой сталкиваются многие из раза в раз при разработке объектно-ориентированных программ;
- ☐ основная идея решения: как правило, это идея о том, как и какие должны взаимодействовать объекты; шаблон, по которому должен строиться конкретный код, решающий проблему;
- ☐ диаграмма классов, описывающая идею решения;

- название проблемы и идеи решения;
- фрагменты кода, иллюстрирующие идею решения и ее конкретное применение.

Паттерн — это типовое решение какой-то типовой проблемы и название. Название важно, потому что, используя названия паттернов, которые знают оба собеседника, они могут общаться, передавая идеи разработки кода гораздо быстрее, чем используя даже сам код. Типовое решение — это даже не код, это идея о том, какие объекты и как должны взаимодействовать, чтобы типовая проблема решалась способом, на который можно будет смотреть без боли, который можно будет допиливать деталями, и код не развалится от первой же доделки.

2.2. Обычные приемы и антиприемы ООП, паттерны и антипаттерны

ООП — это технология и набор типовых техник по использованию ее возможностей. Под самыми обычными приемами ООП можно понимать вообще что угодно, но я бы выделил вот такие:

- *инкапсуляция* — как технология смыслового объединения разной информации об одном и том же в рамках одного объекта, с сокрытием деталей или без такового;
- *класс* — как описание свойств большого числа однотипных объектов;
- *наследование* — как возможность воспользоваться уже готовым классом, но дополнять и расширять его, и все вытекающие отсюда способы заменять и расширять функциональность — например, вызов унаследованных методов предка;
- *композиция* — как альтернативный прием использования функциональности уже существующего класса, как стандартный способ борьбы со сложностью методом «разделяй и властвуй»;
- *полиморфизм* — как возможность работать с множеством разнообразных объектов единообразно через интерфейс их общего предка, но получать разное поведение.

Этот список можно множить, и очень сложно сказать, в какой момент эти приемы перестанут быть обычными приемами и использованием обычного здравого смысла и станут паттернами — эта классификация скорее носит просто исторический характер. Промежуточное положение на этой оси, на мой взгляд, например, занимают так называемые GRASP-паттерны, или GRASP-принципы (General Responsibility Assignment Software Patterns или Principles, и эта неуверенность в последней букве как раз и подчеркивает их промежуточное положение). Кто захочет изучить их в оригинале, обратится к книге Крэга Лармана [11]. Как обычно, названия этих принципов могут выступать в качестве средства коммуникации, но с ними сложнее, чем с классическими паттернами: уж больно

близки они к обычному здравому смыслу, и тем важнее не путать незнание названий этих приемов и невладение соответствующей техникой.

Информационный эксперт (Information expert) — это GRASP-принцип, утверждающий, что код должен быть написан там, где естественным образом есть максимум информации для решения имеющейся проблемы. Например, в программе, где нужно создавать круги на форме по нажатию кнопки мыши и выделять их той же кнопкой мыши, есть соблазн, имея координаты мыши *e.X* и *e.Y*, написать примерно такой код:

```
for (const auto &s : shapes){  
    if (s->getX() == e.X && s->getY() == e.Y)  
        s->setSelected(true);  
}
```

Это как раз и есть нарушение принципа информационного эксперта, граничащее с нарушением инкапсуляции класса *Shape*. Вместо того, чтобы запрашивать у объекта *Shape* информацию о его координатах и потом принимать решение о том, выделять его или нет, мы должны были, наоборот, передать ему всю необходимую информацию для принятия решения, потому что именно он обладает или должен обладать максимумом информации о том, должен ли он выделяться при нажатии на него мыши («нужно ли попадать точно в координаты или с запасом?», «должен ли круг выделяться по прямоугольной или круглой области?», «что нужно вообще делать при нажатии?», «нужно ли выделяться, если ты уже выделен» и т. п.):

```
for (const auto &s : shapes){  
    s->onClick(e.X, e.Y);  
}
```

Создатель (Creator) — это GRASP-принцип, определяющий, где и кем должен создаваться какой-то объект. В частности, объект какого-то класса *A* должен создаваться именно классом *B*, а не кем-то еще, тем вероятнее, чем сильнее связаны эти классы, в частности, если:

- ☐ В композирует *A* (это очевидно просто по определению);
- ☐ В агрегирует набор объектов *A*;
- ☐ В активно использует объекты *A*;
- ☐ В владеет той информацией, которая нужна для создания *A* (в соответствии с принципом информационного эксперта).

Контроллер (Controller) — это GRASP-принцип, который во многом связан с идеей принципа MVC (см. разд. 2.31) и утверждает, что обработка связанных событий должна производиться единообразно в одном объекте-контроллере, который не должен быть сильно связан и не должен брать на себя работу ни слоя программы, генерирующего события (например, с графического интерфейса), ни слоя программы, непосредственно выполняющего реакцию на эти события.

Слой абстракции, косвенный доступ (Indirection) — это GRASP-принцип, утверждающий, что для того, чтобы разорвать жесткую связь между двумя компонентами программы и позволить использовать их независимо друг от друга, нужно просто добавить слой абстракции между ними, организовать непрямо́й доступ. Но, как известно, добавление нового слоя абстракции в программу решает почти все проблемы, кроме одной проблемы — слишком большого числа слоев абстракции.

Принцип слабого зацепления (Low coupling) — это GRASP-принцип, требующий снижать связность между объектами как меру того, насколько сильно один класс зависит от его предположений о каком-то другом конкретном классе. Например, мы скажем, что класс А слишком сильно связан с классом В, если класс А так использует класс В, что попытка заменить В на какой-то класс С связана с такими большими проблемами и усилиями, что за это никто из команды программистов даже не готов браться. При этом речь идет именно о связности между классами, а не между объектами. Например, если два объекта связаны циклически, но через интерфейсы, то это еще не говорит о нарушении принципа низкой связности, как это было бы, хранили бы они прямые ссылки друг на друга по родным классам.

Принцип высокой связности (High cohesion) — это GRASP-принцип, призванный уравнивать стремление снизить до нуля какие-либо зависимости классов друг от друга. Он утверждает, что сильно связанная бизнес-логика взаимодействия между объектами реального мира должна выражаться в сильной связности между описывающими их программными классами. Можно сказать, что это своеобразное применение принципа инкапсуляции по отношению к помещению связанных по смыслу классов в один модуль.

Полиморфизм (Polymorphism), из книги слова не выкинешь, — это тоже GRASP-принцип, но мы его уже 100 500 раз обсудили.

Устойчивость к изменениям (Protected Variations) — это GRASP-принцип, позволяющий добиться устойчивости к изменениям в разных классах путем выработки интерфейсов, которые они должны поддерживать. Другими словами, этот принцип утверждает: вносите изменения в код сколько угодно и делайте 100 500 вариантов какой-то функциональности — лишь бы эти варианты и эти изменения были «защищены» в том смысле, что не видны снаружи. Разумеется, чтобы это работало, интерфейсы должны выделяться таким образом, чтобы их приходилось менять как можно реже, в идеале — никогда.

Чистая выдумка (Pure Fabrication) — это GRASP-принцип, согласно которому, несмотря на то, что при наивном объектно-ориентированном подходе объекты в программе должны быть моделями и напрямую соответствовать объектам реального мира, мы можем и должны при необходимости смело создавать объекты, которые в реальном мире не существуют. Зачем? Что они моделируют, если их нет в той предметной области, с которой мы работаем? Они ничего не моделируют, они инкапсулируют связанную функциональность для достижения слабого зацепления и высокой связности.

Кроме приемов работы с объектами и паттернов часто принято выделять, наоборот, антиприемы и антипаттерны: часто, к сожалению, встречающиеся типовые плохие приемы работы с объектами. Антипаттерн приводит к проблемам, но при этом достаточно заманчив, чтобы сначала казаться подходящим решением. Список антипаттернов приводить бесполезно, потому что ~~автор хочет писать о хорошем, а не о плохом~~ он будет слишком обширным и изменчивым.

2.3. Template Method

Вы будете смеяться, но *шаблонный метод* (англ. template method) — это тоже паттерн, и не упомянут он в предыдущем разделе только потому, что присутствует в книге [7]. Посмотрите на эти три класса:

```
class BeforeX {
public:
    void TemplateMethod() {
        printf("1\n");
        printf("2\n");
        printf("hello\n");
        printf("4\n");
        printf("5\n");
    }
};
```

```
class BeforeY {
public:
    void TemplateMethod() {
        printf("1\n");
        printf("2\n");
        printf("world\n");
        printf("4\n");
        printf("5\n");
    }
};
```

```
class BeforeZ {
public:
    void TemplateMethod() {
        printf("1\n");
        printf("2\n");
        printf("!!!\n");
        printf("4\n");
        printf("5\n");
    }
};
```

Они пока никак не связаны друг с другом, но, очевидно, имеют очень похожий метод с одинаковым названием `TemplateMethod`. Похожий, но не идентичный: в каждом из классов этот метод имеет отличающуюся часть в середине. Всегда, когда мы видим одинаковый или почти одинаковый код, мы отделяем одинаковую часть от отличающейся, и здесь мы поступим так же:

```
class AfterBase {
public:
    virtual void doSpecific() { }
    void TemplateMethod() {
        printf("1\n");
        printf("2\n");
        this->doSpecific();
        printf("4\n");
        printf("5\n");
    }
};

class AfterX : public AfterBase {
public:
    void doSpecific() override {
        printf("hello\n");
    }
};

class AfterY : public AfterBase {
public:
    void doSpecific() override {
        printf("world\n");
    }
};

class AfterZ : public AfterBase {
public:
    void doSpecific() override {
        printf("!!!\n");
    }
};
```

Теперь у нас появился общий класс `AfterBase`, задача которого — один раз и в одном месте описать то общее, что объединяло три класса и было написано в трех местах. Общая часть кода осталась в методе `TemplateMethod()`, но на месте отличающейся части прописан вызов виртуального метода `doSpecific()`, который в том же классе объявлен пустым (но не абстрактным, хотя это тоже вполне возможно). У общего класса `AfterBase` появилось три потомка: `AfterX`, `AfterY` и `AfterZ`, которые наследуют без изменений метод `TemplateMethod()`, но перекры-

вают метод `doSpecific()`, описывая в нем каждый только свою изменяющуюся часть.

```
BeforeX *bx = new BeforeX();  
BeforeY *by = new BeforeY();  
BeforeZ *bz = new BeforeZ();  
bx->TemplateMethod();  
by->TemplateMethod();  
bz->TemplateMethod();
```

```
AfterX *ax = new AfterX();  
AfterY *ay = new AfterY();  
AfterZ *az = new AfterZ();  
ax->TemplateMethod();  
ay->TemplateMethod();  
az->TemplateMethod();
```

```
delete bx; delete by; delete bz;  
delete ax; delete ay; delete az;
```

Достаточно теперь проверить и убедиться, что классы до и после изменений выводят на экран одно и то же. Теперь при вызове метода `ax->TemplateMethod()` у объекта `ax` класса `AfterX`, вызывается метод `TemplateMethod()` класса `AfterBase`, но он в середине своего кода обращается к виртуальной функции `doSpecific()`, которая, по причине своей виртуальности, вызывается именно для `AfterX`.

Где здесь шаблонный метод и почему он шаблонный? Очевидно, что шаблонный метод здесь — это `TemplateMethod()`, а шаблонным он называется потому, что он ведет себя по-разному в разных классах `AfterX`, `AfterY` и `AfterZ`, но всегда действует по одному и тому же шаблону.

Суть паттерна «Шаблонный метод» заключается в том, что какое-то шаблонное действие в базовом классе формулируется в виде отдельных операций в общем шаблонном методе. Потомки базового класса могут перекрывать эти отдельные операции, тогда наследуемый без изменений шаблонный метод у этих потомков будет работать по-другому. Мораль: чтобы изменить поведение какого-то метода у класса-потомка, не обязательно перекрывать и переписывать сам метод. Если он написан «в терминах» каких-то виртуальных операций, то достаточно перекрыть эти отдельные виртуальные операции. Сам метод при этом переписывать не обязательно, он наследуется без изменений, но начинает себя вести по-другому.

У такого простого паттерна и диаграмма классов соответствующая. На ней видно, что в базовом классе объявляется неvirtуальный шаблонный метод `TemplateMethod` и какие-то виртуальные примитивные операции, которые в нем используются (рис. 2.1). Конкретный класс-потомок перекрывает эти примитивные операции, заставляя наследуемый `TemplateMethod()` вести себя по-другому.

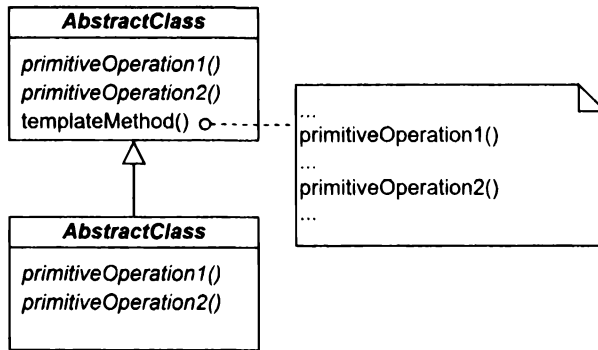


Рис. 2.1. Диаграмма классов паттерна «Шаблонный метод»

2.4. Prototype

Паттерн «Прототип» относится к семейству порождающих паттернов. Другими словами, он описывает способ создания новых объектов. Как мы умели создавать объекты раньше? По сути, у нас был только один способ: взять какой-то существующий класс и специальной командой создать новый объект этого класса, заполняя или не заполняя его свойства. Но давайте посмотрим на этот код:

```

class Animal {
public:
    virtual Animal *clone() = 0;
    virtual ~Animal() { };
};
  
```

Класс `Animal` с виртуальным абстрактным методом `clone` и виртуальным деструктором. Попробуем выжать максимум информации из такого короткого определения. Про виртуальный деструктор вроде бы и так все ясно, а вот если посмотреть на метод `clone`, то видно, что он не берет никаких параметров, но возвращает указатель на объект класса `Animal`. С одной стороны, если никаких объектов ему не передают, но требуют от него вернуть какой-то объект, да к тому же и называют говорящим именем `clone`, то, скорее всего, этот метод будет возвращать новый объект. С другой стороны, метод `clone`, а значит, и весь класс `Animal` — абстрактные, это значит, что объектов этого класса не может существовать. Но как тогда методу `clone` такие объекты возвращать? Вывод один — возвращать он будет не объекты класса `Animal`, а реально существующие объекты классов-потомков:

```

class Dog : public Animal {
public:
    Animal *clone() override {
        return new Dog(*this);
    }
};
  
```

```
class Cat : public Animal {
public:
    Animal *clone() override {
        return new Cat(*this);
    }
};
```

Вот они, два класса-потомка. Оба наследуются от `Animal`, оба перекрывают метод `clone`, и в реализации этого метода каждый из них создает с помощью конструктора копирования и возвращает свою собственную копию. Объект `Dog` создает новый объект `Dog`, объект `Cat` создает новый объект `Cat`. Казалось бы, зачем? Если есть объект `d` класса `Dog`, то и так мы всегда можем написать `new Dog(d)` и создать его копию безо всякого метода?

```
void main()
{
    Animal *cages[2] = {new Cat(), new Dog()};
    ...
}
```

Теперь в одной части основной программы где-то создаются два объекта `Cat` и `Dog` и помещаются в массив `cages` указателей на `Animal`. А в другой части программы элементы массива `cages` используются для создания новых копий объектов с помощью метода `clone()`. Зачем? Зачем писать `cages[0]->clone()` для создания нового `Cat`, если мы могли просто написать `new Cat()`?

```
{
    Animal *newAnimal1 = cages[0]->clone(); // создается Cat
    Animal *newAnimal2 = cages[1]->clone(); // создается Dog
    ...
}
```

Почему этот паттерн называется «Прототип»? Где здесь прототипы? Для чего они используются? Какому приему нас учит этот паттерн? Что вообще такое прототип? Прототипом называется объект, который был создан первым, и создан не для использования его по назначению, а для создания затем на его основе копий, которые уже будут использованы по назначению. Очевидно, что в нашем случае массив `cages` — это массив прототипов, и два первых объекта `Cat` и `Dog`, которые в него были сложены, и стали теми самыми прототипами. Паттерн этот показывает нам, что вместо того, чтобы в явном виде писать `new SomeClass()`, создавать объекты можно и по-другому: клонируя уже существующие.

Заполнение массива `cages` и использование этого массива могут находиться совсем в разных частях программы. Знаем ли мы в момент вызова `cages[0]->clone()`, с каким именно классом работаем? Нет, не знаем, и это ключевой урок, которому нас этот паттерн учит: чтобы создать объект какого-то класса, оказывается, совсем не обязательно знать, о каком именно классе идет речь. Достаточно иметь готовый объект этого класса (а раз он может находиться в перемен-

ной своего предка, то мы можем и не знать, с каким именно классом имеем дело). Что единственное мы должны знать о том объекте, с которым работаем? То, что у него есть метод `clone()`.

Зачем нам вообще может понадобиться «не знать», объект какого класса мы создаем? Разве может быть «добродетелью» такое незнание? Знать, объект какого класса мы создаем, означает на самом деле жесткую привязку нашей программы к конкретному классу:

```
// 1
a = new Cat();
...
// 2
b = new Cat();
...
// 3
c = new Cat();
...
```

Если у вас по всей программе прописано создание объектов класса `Cat`, то что если понадобится везде заменить `Cat` на `Dog`? Правильно: придется изменять каждое место, где жестко прописано использование класса `Cat`. А если у вас для создания объектов используются прототипы?

```
// прототип
Animal *p = new Cat();
...
// 1
a = p->clone();
...
// 2
b = p->clone();
...
// 3
c = p->clone();
...
```

Теперь для того, чтобы везде создавать `Dog` вместо `Cat`, достаточно изменить создание прототипа только в одном месте. Вся остальная программа не только не изменится, она вообще может и не узнать, что теперь создает объекты совсем других классов!

Теперь посмотрим на диаграмму классов этого паттерна, она аналогична диаграмме из книги [7] (рис. 2.2).

На этой схеме `Prototype` — это наш класс `Animal`, а его потомки `SpecificPrototype1` и `SpecificPrototype2` соответственно это наши `Cat` и `Dog`. Здесь же диаграмма классов нам намекает, что клиент (не пугайтесь, это не клиент ресторана: в терминах диаграмм классов — это просто тот код, которому нужно использовать прототипы) держит у себя ссылку `prototype` на базовый класс `Prototype` и в какой-то своей операции, где ему нужно создавать новые

объекты, не создает их с помощью `new`, а вызывает `prototype->clone()`. При этом раз `clone()` — это виртуальная функция, то вызовется на самом деле `SpecificPrototype1::clone()` или `SpecificPrototype2::clone()`, и создан будет экземпляр конкретного класса, но клиент ничего про конкретные классы не узнает. Как и задумывалось.

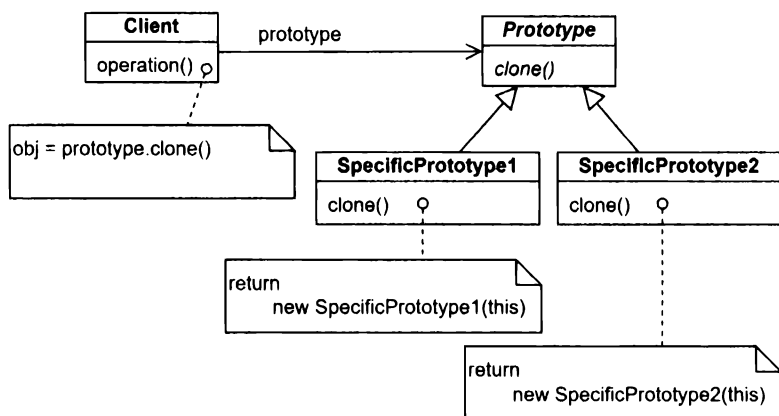


Рис. 2.2. Диаграмма классов паттерна «Прототип»

Собственно, как и в большинстве паттернов, правильно нарисованная диаграмма классов заменяет долгое описание идеи. Ключевая идея в диаграмме классов здесь проявляется тем, что клиент хранит ссылку на базовый класс `Prototype` и обращается к нему за созданием объектов с помощью функции `clone()`, создавая объекты тех классов, про которые ничего не знает.

2.5. Бесклассовое (прототипно-ориентированное) ООП

Описанная паттерном `Prototype` ситуация на самом деле шире, чем просто конкретный прием. Сам подход к созданию новых объектов путем клонирования уже существующих получает свое развитие в так называемом бесклассовом или прототипно-ориентированном программировании. В «привычном» ООП мы сначала создавали классы, а потом на основе класса можно было создавать объекты. В бесклассовом ООП, как ни странно, классов нет (хотя объекты, разумеется, есть) и для создания типовых и разнообразных объектов используются три механизма:

- ☐ создание пустого объекта без свойств и методов;
- ☐ добавление в любой объект новых свойств и методов;
- ☐ создание копии любого объекта.

Таким образом, там, где в классовом ООП мы сначала создавали бы класс, наполняли бы его свойствами и методами, а затем создавали бы множество объек-

тов, — в бесклассовом ООП мы сначала создаем пустой объект, наполняем его свойствами и методами, а потом создаем множество клонов. Как и везде, в результате мы получаем, плюс-минус километр, одно и то же — множество объектов с одинаковыми наборами свойств, но с потенциально разными значениями этих свойств. Конечно, дальше в каждый из таких «экземпляров» могут быть добавлены новые свойства, но правило «все объекты одного класса обязательно имеют одинаковый набор свойств» при этом не нарушается — ведь классов-то нет!

Там, где в классовом ООП мы наследовали новый класс от предка, добавляли в него новые свойства и затем уже создавали объекты класса-потомка, в бесклассовом ООП мы клонируем существующий объект, добавляем в него новые свойства и методы, а затем создаем множество клонов измененного объекта, — в этом смысле наследование в прототипном ООП тоже есть, потому что позволяет добиться той же цели.

Понятно, что ЯП, позволяющий «на лету» добавлять в объект свойства и методы, имеет, скорее всего, динамическую природу и допускает большую гибкость. Соответственно и вопросов по поведению таких объектов возникает сразу много.

Например, если объект был создан как клон какого-то прототипа, то при добавлении новых свойств в прототип, уже после создания клонов, появятся ли эти свойства у клонов или нет? Разные ЯП, реализующие бесклассовое ООП, будут по-разному отвечать на этот вопрос, и в зависимости от ситуации и то и другое поведение может быть как удобным, так и неудобным.

2.6. Интерфейсы

Давайте снова вернемся к примеру из паттерна Prototype и посмотрим на его абстрактный класс, только добавим в него какую-нибудь функциональность, которая могла бы быть у любого объекта класса `Animal`, например имя:

```
class Animal {
public:
    string name("");
    virtual Animal *clone() = 0;
    virtual ~Animal() { };
};
```

Помните, мы говорили о том, что для целей создания клонов коду совсем не обязательно знать, с каким объектом он имеет дело? И все, что нужно о каком-то объекте знать, — это то, что у него есть метод `clone()`. Предположим, у нас есть какая-то функция, которая должна внутри себя создать целый массив клонированных `Animal`. Очевидно, что нам придется в такую функцию передать прототип как указатель на `Animal`:

```
vector<Animal*> clones(Animal *prototype, int count) {
    vector<Animal*> vec(count);
```

```
for (size_t i = 0; i < vec.size(); i++)
    vec[i] = prototype->clone();
return vec;
}
```

Если вы не знаете, что такое `vector`, не волнуйтесь: раздел про шаблоны и STL будет уже очень скоро, а пока можете вместо `vector<Animal*> vec(count)` мысленно подставлять настоящий массив из указателей `Animal* vec[count]`.

Проблема заключается в том, что раз в эту функцию передается `Animal`, то эта функция может обращаться ко всем публичным свойствам и методам `Animal`, в том числе и, например, к `name`. В то время как внутри этой функции не должен требоваться доступ ни к чему, кроме метода `clone()`!

Как можно сделать так, чтобы функция и не могла с объектом делать ничего, кроме вызова метода `clone()`? Для этого у передаваемого объекта и должен быть только один метод `clone()`. Но при этом объект должен иметь какие-то другие полезные публичные свойства и методы. Разрешить это противоречие можно одним-единственным способом: добавив объекту `Animal` предка, у которого один единственный метод `clone()` и есть:

```
class ICloneable {
public:
    virtual ICloneable *clone() = 0;
};

class Animal : public ICloneable {
public:
    string name{ "" };
    virtual ~Animal() { };
};

vector<ICloneable*> clones(ICloneable *prototype, int count) {
    vector<ICloneable*> vec(count);
    for (size_t i = 0; i < vec.size(); i++)
        vec[i] = prototype->clone();
    return vec;
}
```

Класс `ICloneable` не зря так называется, потому что это специфический класс — он содержит только виртуальные абстрактные методы; он предназначен только для того, чтобы унаследованные от него классы были вынуждены перекрывать эти методы, и он называется так, потому что, имея какой-то объект, который принадлежит потомку этого класса, мы можем быть точно уверены, что этот объект «клонировуемый», т. е. его можно клонировать.

Такие классы называются интерфейсами. Само слово «интерфейс» очень многозначное. Как мы помним, раньше уже мы под этим словом в одном контексте могли понимать:

- ❑ визуальный интерфейс приложения;
- ❑ ту часть прибора, которая предназначена для взаимодействия с пользователем, в отличие от внутреннего устройства прибора, для доступа не предназначенного;
- ❑ набор публичных свойств и методов объекта, в отличие от всех остальных защищенных членов.

Вот теперь к этому списку можно добавить еще одно возможное понимание слова «интерфейс» — специальное понятие ООП, специальная сущность ООП, состоящая из набора виртуальных абстрактных методов. В некоторых языках для такого явления имеется специальное ключевое слово `interface`, но C++ такого слова не имеет, поэтому в C++ интерфейс — это просто класс, которому мы даем специфическое название и содержание, чтобы отличать его от обычных классов. Интерфейс предназначен для того, чтобы наследоваться от него и перекрывать его методы в потомках: в этом случае мы даже будем специально говорить не «класс X наследуется от интерфейса IX», а «класс X поддерживает или реализует интерфейс IX». Интерфейсы в ООП нужны для того, чтобы:

- ❑ ограничивать возможности кода, который работает с объектом, и тем самым позволять подменять реальный объект, с которым производится работа. Если код использует объект `x` через переменную его «родного» класса `X`, то код может вызывать у `x` любые свойства и методы `X`. Если же код использует тот же объект `x`, но находящийся в переменной интерфейсного класса `IX`, то код сможет вызывать только методы, описанные в интерфейсе `IX`. Подразумевается, что этих методов гораздо меньше и они меняются гораздо реже. В этом случае, если в переменной `x` интерфейсного класса `IX` окажется не объект класса `X`, а объект класса `Y`, не понадобится вносить никакие другие изменения при условии, что класс `Y` тоже наследуется от `IX`. Общий принцип: ограничь себя в работе с объектом сейчас, — тогда меньше ограничений будет в работе с объектом в будущем;
- ❑ описывать требования к объекту, с которым будет производиться работа. Это позволяет разграничить работу программистов по написанию кода различных частей объектно-ориентированной системы, если начать с формализации и описания интерфейсов, которые должны поддерживать объекты, которыми они обмениваются.

При этом работа с интерфейсами не лишена проблем. Так, например, в коде выше мы не просто вынесли метод `clone()` в интерфейс, мы поменяли его тип: теперь он возвращает не `Animal*`, а `ICloneable*`. И сама функция `clones` стала еще более общей: она теперь возвращает вектор `ICloneable*`, а не вектор `Animal*`. Да, эта функция стала более универсальной: в нее можно передать объект-прототип типа `Cat`, и она вернет вектор объектов `Cat*`. Но это будет вектор объектов `Cat*` в виде указателей на `ICloneable`, и для работы с ними как с `Cat` (если такая работа понадобится) придется использовать приведение типов. Да, можно было не заходить так далеко и не менять тип возвращаемых объектов, но сам интерфейс тогда все-таки пришлось бы назвать по-другому:

```
class Animal;

class ICloneableAnimal {
public:
    virtual Animal *clone() = 0;
};

class Animal : public ICloneableAnimal {
public:
    string name{ "" };
    virtual ~Animal() { };
};

vector<Animal*> clones(IClonableAnimal *prototype, int count) {
    vector<Animal*> vec(count);
    for (size_t i = 0; i < vec.size(); i++)
        vec[i] = prototype->clone();
    return vec;
}
```

2.7. Множественное наследование

Понятно, что раз интерфейс описывает требования к объекту, а разных требований к разным объектам может быть очень много, то интерфейсов будет много, и некоторым объектам понадобится реализовывать сразу несколько интерфейсов. Для этого в тех языках, где интерфейс — это отдельная сущность и ключевое слово, классу разрешено указывать целый список интерфейсов, которые он обещает реализовать. В C++, где отдельных интерфейсов нет, придется использовать для этого так называемое множественное наследование.

При обычном наследовании класса Y от класса X класс Y получает все свойства и методы класса X (и затем доопределяет или переопределяет какие-то собственные). Множественное наследование не меняет общий принцип, но просто разрешает классу Y наследоваться не только от класса X, но и от произвольного набора классов. При этом класс X получит все свойства и методы от всех своих предков.

Зачем нам в принципе может понадобиться наследование? Чтобы получить в своем классе функциональность, которая уже существует в каком-то другом классе, — как уже говорилось раньше, можно (а может, и нужно) использовать композицию вместо наследования. Но если нужная функциональность находится в нескольких классах, то собрать ее в одном классе (не используя композицию) можно, только наследуясь сразу от нескольких предков.

Почему многие считают, что множественное наследование — это плохо? В некоторых языках оно даже просто запрещено средствами языка. Первая причина заключается в том, что при использовании множественного наследования могут

возникать странные неопределенные ситуации, когда объект наследуется от нескольких предков, которые, в свою очередь, имеют какого-то общего предка. Эта ситуация называется еще проблемой ромба:

```
class A {
public:
    int x;
    void foo() { };
};

class B : public A {
public:
    void foo() { };
};

class C : public A {
public:
    void foo() { };
};

class M : public B, public C {
public:
    void boo() { };
};
```

Вот здесь классы *B* и *C* наследуются от *A* и получают от него свойство *x* и перекрывают его метод *foo()*. Но затем класс *M* наследуется и от *B*, и от *C*! При создании объекта класса *M* возникает вопрос: какой из методов *foo()* будет вызываться для этого объекта?

```
M m;
m.foo();
m.x = 5;
```

А кроме этого, если посмотреть на область памяти, занимаемую объектом *m*, то станет понятно, что внутри объекта класса *M* содержится целых две копии целочисленного свойства *x* (рис. 2.3). Возникает вопрос: в какую именно область памяти мы записываем значение в выражении *m.x = 5*?

Разные языки решают эту проблему по-разному. Как уже было сказано, в некоторых языках множественное наследование просто запрещено (кроме случая «наследования» от нескольких интерфейсов, которое в этом случае и наследованием-то не называется). В C++ в этом случае можно явно указать, к какому из иксов мы обращаемся: *m.B::x = 5*, а можно воспользоваться более специфическим для C++ так называемым виртуальным наследованием (*virtual inheritance*). Можно придумать разные автоматические правила выбора поведения в такой ситуации, но сама ситуация, если задуматься, определенно «нехорошая» и, скорее всего, сигнализирует о том неправильно выбранном проектном решении

иерархии классов. Кстати, в случае с дублированием свойства даже композиция не поможет, потому что два свойства *x* внутри объекта класса *M* окажутся и при композиции тоже.

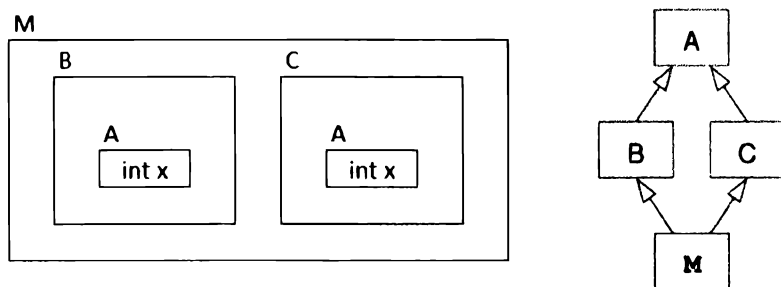


Рис. 2.3. Содержимое объекта *M* в памяти и диаграмма классов для проблемы ромба

Еще одна причина относиться настороженно к множественному наследованию от реальных объектов (не интерфейсов) — это подозрение на то, что идея множественного наследования нарушает первый принцип из SOLID, а именно принцип единственной ответственности (см. разд. 2.28). Ведь если каждый из классов *B* и *C* имеет свое четко обозначенное предназначение, то класс *M* как их комбинация по определению получает предназначения два! Так что подозрение обоснованное, но на самом деле не следует смешивать техническую процедуру множественного наследования как возможность языка и принцип единственной ответственности, который на то и принцип, а не математическое определение, чтобы давать направление мысли, а не четкий критерий. Классы, с которыми мы работаем, бывают очень разные по смыслу и предназначению, и комбинация разнородных классов в одном совсем не обязательно нарушает принцип единственной ответственности.

2.8. Примеси (миксины, mixins)

Завершая обсуждение интерфейсов и множественного наследования, нужно затронуть и еще одно понятие, связанное с ними по смыслу. Примесь (или миксин, или *mixin*, или *mix-in*) — это такой класс, который содержит свойства и методы, инкапсулирующие какую-то функциональность, но не предназначен для создания объектов. Вместо этого этот класс «подмешивается» к другим классам, для того чтобы у них тоже эта функциональность появилась. В зависимости от языка миксины могут быть реализованы по-разному. В каких-то языках они могут присутствовать как отдельные сущности, и у них могут быть свои ключевые слова для определения и использования. В других языках, например в C++, миксин — это такой же класс, как и все остальные, «подмешиваться» он будет путем обычного наследования, и называть его миксином будем мы сами только потому, что он для нас несет смысл, не похожий на смысл обычного наследования.


```
class NotifierMixin {
private:
    vector<string> observers;
public:
    void add_observer(string name) {
        observers.push_back(name);
    }
    void notify() {
        for (auto o : observers)
            // ...
    }
};
```

Вот этот класс явно не предназначен для создания самостоятельных объектов. Но если какой-то другой класс его «примешает», то он его обогатит вектором каких-то наблюдателей и какой-то функциональностью, с ними связанной. Мы еще встретимся с этим примером при обсуждении паттерна «Наблюдатель» в *разд. 2.20*.

2.9. Singleton

Один из самых простых паттернов и пример такого паттерна, у которого идея и название гораздо полезнее (для обмена смыслами между разработчиками), чем конкретный код. Суть паттерна «Одиночка» заключается в том, что он описывает такой класс, для которого может существовать только один-единственный объект (еще и ноль объектов тоже, конечно). Казалось бы, если нам нужен только один объект какого-то класса, то почему бы просто не создавать больше чем один объект? Так тоже можно, конечно, но гораздо лучше постараться средствами языка гарантировать то, что такой объект будет только один. В каждом языке это можно сделать по-разному: в C++, например, можно реализовать его с помощью статического атрибута.

Но прежде отвлечемся и разберемся, как работают статические атрибуты и статические локальные переменные.

```
class A {
public:
    int x{0};
    static int y;
};
```

```
int A::y = 0;
```

Здесь объявляется класс `A` с обычным целочисленным атрибутом `x` и со статическим целочисленным атрибутом `y`. Фишка в том, что статический атрибут разделяется между всеми объектами класса. Обычный атрибут `y` каждого объекта — свой собственный, а статический — общий. Он один на все объекты этого клас-

са, он существует в единственном экземпляре, поэтому попытка изменить его, обратившись через один объект, изменит его для всех объектов сразу:

```
A a1;
A a2;
a1.x = 5;
a1.y = 6;
a2.x = 7;
a2.y = 8;
```

Присваивание `a2.y = 8` приводит к тому, что `a1.y` сразу тоже получает значение 8. И даже более того, для обращения к статическому атрибуту какого-то класса совсем не обязательно использовать какой-либо объект. К статическому атрибуту можно обращаться напрямую через имя класса, при этом объекты даже не обязательно создавать:

```
A::y = 42;
```

Статические атрибуты «привязаны» к самому классу, а не к объектам класса. Этим они похожи на глобальные переменные, но не засоряют глобальное пространство имен. Начальное значение статического атрибута, которое он получает прямо при запуске программы, мы задали сами в коде выше: `int A::y = 0;`.

Теперь мы можем вернуться к самому коду для паттерна «Одиночка»:

```
class Zoo {
private:
    static Zoo *_zoo;
protected:
    Zoo() {
        cout << "Zoo::Zoo()\n";
    }
public:
    static Zoo *get_instance() {
        if (_zoo == nullptr)
            _zoo = new Zoo();
        return _zoo;
    }
    void do_something() {
        cout << "Zoo::doSomething()\n";
    }
};
```

```
Zoo *Zoo::_zoo = nullptr;
```

Класс `Zoo` является типичным синглтоном-одиночкой. В классе объявлен один приватный статический атрибут-указатель `_zoo` на объект того же класса `Zoo`. Напоминаю, что статический атрибут существует в единственном экземпляре для всех возможных объектов класса `Zoo`, и даже вообще не для всех объектов

класса `Zoo`, а просто в единственном экземпляре. В нашем случае его начальное значение равно `nullptr`.

В самом классе объявлен защищенный конструктор `Zoo()`. Это означает, что создавать объекты класса `Zoo` в самой программе не получится, потому что вот такой код просто не скомпилируется:

```
Zoo z;  
Zoo *pz = new Zoo();
```

Что же тогда можно делать с этим классом, если не создавать его объекты? А в классе объявлена публичная, но статическая же функция `get_instance()`. Раз она объявлена статической, то вызывать ее можно прямо у класса, не создавая объекты:

```
Zoo::get_instance()->do_something();
```

При первом вызове этой функции условие `if` внутри нее будет истинно, поэтому будет создан и помещен в переменную `_zoo` первый и единственный экземпляр класса `Zoo` (раз мы находимся внутри метода `Zoo`, нам доступен его защищенный конструктор). При всех последующих вызовах `get_instance()` новые объекты создаваться не будут, а будет всегда возвращаться указатель на один и тот же первый созданный объект, у которого мы и будем вызывать какие-нибудь методы типа `do_something()`.

Таким образом мы не только обеспечили создание единственного экземпляра этого класса, но и создали его «ленивым» образом при первом обращении к нему.

Другой, более корректный вариант реализации этого паттерна в C++, правильно работающий с многопоточностью, можно сделать с помощью простой статической локальной переменной, которая, как и любые статические переменные, сохраняет свое значение между вызовами функции:

```
class Zoo {  
protected:  
    Zoo() {  
        cout << "Zoo::Zoo()\n";  
    }  
public:  
    static Zoo &get_instance() {  
        static Zoo zoo;  
        return zoo;  
    }  
    void do_something() {  
        cout << "Zoo::do_something()\n";  
    }  
};
```

Здесь даже проверок никаких не надо: автоматически при первом заходе в функцию `get_instance()` будет инициализирована переменная `zoo`, а в дальнейшем она будет просто возвращать свое исходное значение.

Где могут, в принципе, понадобиться объекты, существующие в единственном экземпляре? Это может быть какой-нибудь конфиг, логгер, менеджер памяти и вообще любой распределитель ресурса, которым нужно управлять исключительно из одной точки. Но при этом надо понимать, что на самом деле «Одиночка» не очень далеко ушел от глобальной переменной, и активное использование синглтона внутри разных функций приведет к тому, что эти функции будут зависеть от глобального состояния, а значит, их сложно будет отлаживать.

Синглтон представляет собой, по сути, глобальную переменную, со всеми ее недостатками. Поэтому использовать синглтоны нужно очень редко и в случае крайней необходимости. К примеру, распространенная практика использовать синглтон для системы логгирования совершенно не подходит для случая, когда вы пишете библиотеку, которая может быть использована в проекте, в котором могла быть реализована своя система логгирования со своим синглтоном. В таком случае в библиотеке лучше добавить возможность зарегистрировать callback-и или специальные объекты с виртуальными функциями, которые внешний проект может переопределить и связать со своей реализацией системы логгирования. То же самое можно сказать и про аллокаторы памяти, сетевые соединения и многие другие подсистемы. Так в чем же преимущества синглтонов по сравнению с глобальными переменными? Во многих языках преимуществ нет, но в C++ синглтоны позволяют обойти проблему под названием Static Initialization Order Fiasco. Суть этой проблемы в том, что порядок инициализации глобальных переменных в C и C++ не определен. Если одна глобальная переменная во время своей инициализации обращается к другой глобальной переменной — программа может упасть, а может и не упасть: как повезет. Отлаживать такие ошибки очень сложно. Вторая часть проблемы, о которой часто даже не задумываются, — порядок вызова деструкторов глобальных переменных при завершении программы, который также не определен.

Синглтоны являются единственным надежным решением, если для инициализации одной «глобальной» переменной нужно обращаться к другой, т. к. каждый объект-одиночка создается в момент первого обращения. Порядок вызова деструкторов при завершении программы тоже будет правильным — от последнего к первому в порядке их инициализации.

2.10. Chain of Responsibility

Паттерн «Цепочка ответственности» описывает такую ситуацию, когда непонятно, кто именно должен обработать какой-то запрос, потому что это может быть один из обработчиков, организованных в цепочку, а нам известен только крайний получатель в этой цепочке. С похожей ситуацией можно было столкнуться когда-то давно, когда старосты получали стипендию в бухгалтерии на всю группу сразу и надо было, например, передать ведомость старосте какой-то группы. Можно было тогда прийти в день получения стипендии, грустно посмотреть на огромную очередь из старост, выстроившихся к далекому окошеч-

ку, узнать, кто тут последний, и попросить его передать ведомость старосте группы ПО-216.

Впрочем, давайте посмотрим на код.

```
#include <iostream>
#include <stdio.h>
using namespace std;

class Handler {
public:
    virtual bool handle(int request) = 0;
    virtual ~Handler() {};
};
```

Вы что-нибудь поняли? Класс называется «обработчик», мы видим в нем единственную виртуальную функцию без реализации с названием «обработать», с единственным параметром с названием «запрос», поэтому похоже, что функция будет принимать и обрабатывать число и возвращать логический true или false. Раз эта функция абстрактная, мы можем догадаться, что она создана только для того, чтобы перекрывать ее в классах-потомках. Пожалуй, больше из этого кода пока не выжать.

```
class DivisionChecker : public Handler {
private:
    int value;
    Handler *next;
public:
    DivisionChecker(int _value, Handler *_next) {
        value = _value;
        next = _next;
    }
    bool handle(int request) override {
        if (request % value == 0)
            return true;
        else
            return next->handle(request);
    }
    ~DivisionChecker() override {
        if (next != nullptr)
            delete next;
    }
};
```

Что мы видим здесь? Класс `DivisionChecker`, потомок класса `Handler`, видимо, какой-то реальный обработчик. Мы видим у него в полях какое-то числовое `value` и какой-то указатель `next` на другой объект `Handler`. Но раз класс `Handler` абстрактный, то `next` будет хранить указатель на объекты-потомки класса

Handler, вполне возможно, и другие объекты класса `DivisionChecker`. Конструктор `DivisionChecker` просто инициализирует значения своих полей, тут ничего интересного, а дальше, мы видим, перекрывается абстрактная функция `handle`.

Что же `DivisionChecker` делает, если его просят «обработать» какое-то число, передав его в качестве параметра `request`? Мы видим, что он проверяет, делится ли `request` нацело на то число, которое он сам хранит в `value`, и если делится, то сразу возвращает `true`. А если не делится, то он вызывает функцию `handle` у другого обработчика `next`, ссылку на которого хранит. Возвращает сам то, что этот другой обработчик вернет. Переводит стрелки, короче. Делегирует работу. Перенаправляет задание. Настоящий менеджер! А кто же крайним останется?

```
class DefaultHandler : public Handler {
    bool handle(int request) override {
        printf("%d is prime\n", request);
        return false;
    }
};
```

Вот он, наш обычный работяга, `DefaultHandler`, еще один потомок класса `Handler`. У него нет никакой своей `property`, только унаследованный и перекрытый метод `handle`. Если у него этот метод вызывают, он сообщает, что переданное ему число является простым, и возвращает `false`.

Теперь давайте посмотрим на основную программу. Что мы видим?

```
void main() {
    Handler *queue = new DefaultHandler;
    for (int i = 2; i < 100; i++)
        if (queue->handle(i) == false)
            queue = new DivisionChecker(i, queue);
    delete queue;
    system("pause");
}
```

В ней сначала создается переменная-указатель `queue`, потом создается объект класса `DefaultHandler` и адрес его складывается в `queue`. Короче говоря, `queue` указывает на объект `DefaultHandler` как-то так (рис. 2.4).

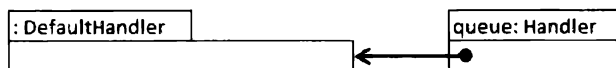


Рис. 2.4. Первый созданный обработчик

Потом начинает работать цикл, переменная `i` получает значение 2, и у объекта `queue` вызывается метод `handle` с параметром 2. Чей метод вызывается? Правильно, метод класса `DefaultHandler`. Что он делает? Сообщает, что число 2 простое, и возвращает `false`. У нас как раз в цикле стоит проверка, и если возвраще-

но false, то создается новый объект, но уже класса `DivisionChecker`. Этому объекту в конструктор передается число 2 и адрес того объекта, куда сейчас указывает переменная `queue`, т. е. адрес старого `DefaultHandler`. Адрес полученного объекта заносится обратно в переменную `queue`. В результате этих манипуляций получаем следующее (рис. 2.5).

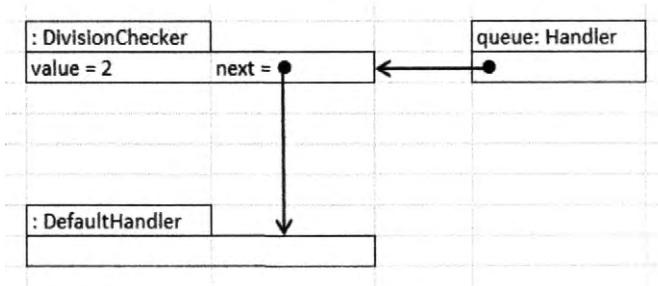


Рис. 2.5. Второй созданный обработчик

Начинает работать следующая итерация цикла, `i` получает значение 3, и снова у объекта `queue` вызывается метод `handle` с параметром 2. Но только теперь `queue` указывает на другой объект и вызывается метод класса `DivisionChecker`. Что он делает? Проверяет, что 3 не делится на 2, хранящееся у него в `value`, и раз оно не делится, то вызывает `handle` у того объекта `next`, который хранит. А чей это будет метод? Метод многострадального `DefaultHandler`. Что он делает снова? Сообщает, что число 3 простое, и возвращает false. Это false возвращается обратно в `DivisionChecker`, а из него обратно в основную программу, как раз к проверке условия. Раз возвращено false, то создается новый объект, снова класса `DivisionChecker`, ему в конструктор передается число 3 и адрес того объекта, куда сейчас указывает переменная `queue`, т. е. адрес созданного ранее `DivisionChecker`. Адрес полученного объекта заносится обратно в переменную `queue` и получаем такую структуру ссылающихся друг на друга объектов (рис. 2.6).

Посмотрим еще одну итерацию цикла: `i` равно 4, у объекта `queue` вызывается метод `handle` с параметром 4, `queue` указывает на `DivisionChecker`. Тот проверяет, что 4 не делится на 3, хранящееся у него в `value`, и раз оно не делится, то вызывает `handle` у того объекта `next`, который хранит, а это другой `DivisionChecker`! Он проверяет, что 4 делится на 2, хранящееся, в свою очередь, у него в `value`, и раз оно делится, то сразу возвращает true. Это true через пару возвратов прилетает обратно в основную программу, как раз к проверке условия, условие не срабатывает, поэтому никаких новых объектов не создается, и цикл продолжает работу своей дорогой.

В итоге, когда цикл доработает до конца, мы увидим на экране список простых чисел, не превышающих 100, а в памяти окажется такая структура из объектов (рис. 2.7).

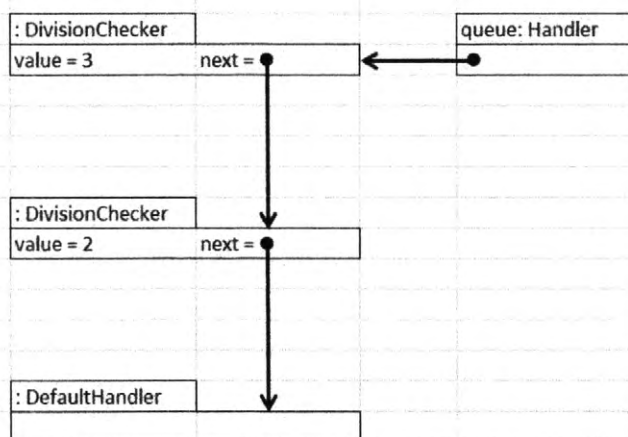


Рис. 2.6. Три обработчика в цепочке

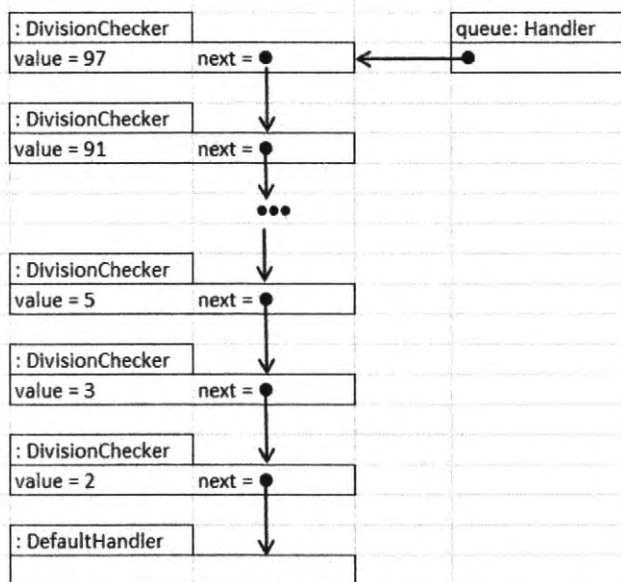


Рис. 2.7. Цепочка связанных объектов готова

В чем смысл этого паттерна? Где здесь цепочка ответственных и за что они ответственны? Так вот же, это цепочка обработчиков `DivisionHandler`! Почему цепочка? Потому что каждый из них знает только следующего. Почему ответственности? Потому что он передает следующему ответственность за решение задачи, за исполнение запроса.

Где еще можно встретить пример такого поведения? Например, на лекции в университете. Приходит преподаватель в аудиторию, и ему нужен журнал

группы, чтобы вписать отсутствующих. Должен преподаватель знать, у кого сегодня этот журнал? Конечно, нет. А что он должен знать? Он должен знать одного старосту, и все. Преподаватель такой: «Эй, староста! Давай гони сюда журнал!». А староста такой встает и говорит: «А меня сегодня нет, я на Госуслугах записался и пошел вакцинироваться! Но журнал должен быть у замстаросты! Эй, замстаросты, давай журнал!». А замстаросты такая встает и говорит: «А меня тоже сегодня нет, я корзинку с пирожками понесла бабушке. Но журнал должен быть у замзамстаросты! Эй, замзамстаросты!..».

Должен преподаватель самостоятельно разбираться в хитросплетениях групповой иерархии власти? Нет, не должен. Он просто обращается к единственной точке входа, где должен выполняться его запрос, а кто именно из цепочки объектов его запрос выполнит, его не особо волнует.

Итак, идея паттерна Chain of Responsibility в том, что, вызывая какую-то функциональность у некоторого объекта, мы ждем, что она просто выполнится, а на самом деле за этим объектом может находиться цепочка из возможных исполнителей, где каждый исполнитель может или выполнить запрос сам, или обратиться к тому единственному следующему исполнителю, которого он знает. Как пишут в умных книгах, разрывается жесткая связь между отправителем запроса и его исполнителем (не знаю, говорит вам сейчас эта фраза что-нибудь или нет). Важно, что цепочка исполнителей может формироваться на лету и перестраиваться в процессе работы программы, что у нас и происходит (рис. 2.7).

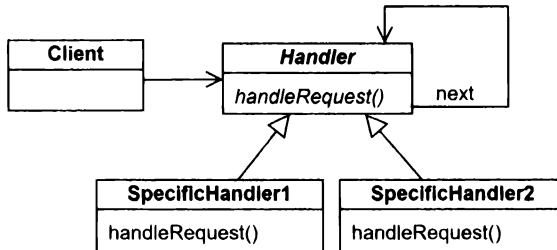


Рис. 2.8. Диаграмма классов паттерна Chain of Responsibility

Диаграмму классов (рис. 2.8) можно найти везде, в том числе и в книге [7]. Но рекомендую сначала построить диаграмму классов для нашей программы, потом посмотреть на диаграмму классов паттерна из книги [7], сопоставить классы друг с другом и кое-что заметить, связанное со свойством `next` у `DivisionChecker`.

Это классический вариант паттерна Chain of Responsibility, но есть еще один часто используемый прием, который, как мне кажется, тоже можно к этому паттерну отнести. Давайте посмотрим на другую программу.

```

#include <iostream>
#include <stdio.h>
using namespace std;
  
```

```
class Handler {
public:
    virtual bool handle(int request) = 0;
};

class DefaultHandler : public Handler {
public:
    bool handle(int request) override {
        printf("%d is prime\n", request);
        return false;
    }
};
```

Как будто, пока все то же самое. Абстрактный базовый класс, объявляющий одну абстрактную функцию, один потомок, который, получая число, сразу объявляет его простым и возвращает false.

```
class DivisionChecker2 : public DefaultHandler {
public:
    bool handle(int request) override {
        if ((request % 2 == 0) && (request != 2))
            return true;
        else
            return DefaultHandler::handle(request);
    }
};
```

А вот и отличие. Новый потомок, но не `Handler`, а `DefaultHandler`! Это значит, что он перекрывает не абстрактный метод `handle` базового класса `Handler`, а вполне себе рабочий метод `handle` класса `DefaultHandler`! Что же он делает? Он проверяет, делится ли переданное ему число на 2, и если делится, но при этом не является самим числом 2, то сразу возвращает `true`. А вот если это условие не выполняется, то он, как и раньше, вызывает другой метод, но не у какого-то другого объекта, а у самого себя же, — унаследованный метод `DefaultHandler::handle`. А дальше?

```
class DivisionChecker3 : public DivisionChecker2 {
public:
    bool handle(int request) override {
        if ((request % 3 == 0) && (request != 3))
            return true;
        else
            return DivisionChecker2::handle(request);
    }
};
```

Новый потомок, и снова не от базового класса, а от `DivisionChecker2`. Проверяет переданное число на делимость на 3 и вызывает унаследованный метод `DivisionChecker2::handle`. Думаю, вы уже поняли, что дальше будут еще два потомка и основная программа:

```
class DivisionChecker5 : public DivisionChecker3 {
public:
    bool handle(int request) override {
        if ((request % 5 == 0) && (request != 5))
            return true;
        else
            return DivisionChecker3::handle(request);
    }
};

class DivisionChecker7 : public DivisionChecker5 {
public:
    bool handle(int request) override {
        if ((request % 7 == 0) && (request != 7))
            return true;
        else
            return DivisionChecker5::handle(request);
    }
};

void main() {
    Handler *queue = new DivisionChecker7();
    for (int i = 2; i < 100; i++)
        queue->handle(i);
    delete queue;
    system("pause");
}
```

Тут, как видим, никакой цепочки из объектов не создается, но программа делает то же самое. И все равно тут можно увидеть цепочку ответственности, но цепочку не из объектов, а из классов, цепочку «предок-потомок», где запрос может быть при определенных условиях обработан в классе-потомке или делегирован (передан предку) в иерархии. Такой прием сплошь и рядом используется при создании обработчиков событий. Там, если нужно изменить поведение какого-то обработчика, типа `mouseDoubleClickEvent` в Qt, мы в потомке перекрываем этот метод, проверяем в нем, должны ли мы поведение при конкретном событии менять, и если да, то выполняем нужное нам действие, а если нет, то вызываем унаследованный обработчик, передавая управление дальше по цепочке ответственных Chain of Responsibility.

2.11. Делегирование, объектная шизофрения

Раз уж мы произнесли в предыдущем разделе это слово «делегирование», пришла пора разобраться в том, что оно обозначает. Мы говорим, что один объект делегирует другому объекту какую-то задачу, если кто-то полагал, что эту задачу он должен был решать сам, но сам решать он нее отказывается и поручает решать ее кому-то другому. В коде это чаще всего выглядит так, что у объекта

есть какой-то метод `foo()`, но при его вызове объект просто вызывает метод `foo()` у другого объекта:

```
class Junior {
public:
    void work() {...}
};

class Senior {
private:
    Junior *assigned;
public:
    Senior(Junior *ass) {
        assigned = ass;
    }
    void work() {
        assigned->work();
    }
}
```

Мы говорим: «один объект делегирует другому объекту какую-то задачу», но откуда он узнает, кому другому? Чаще всего в таких случаях, как показано в коде выше, используется агрегация, и делегирующий делегирует задачу тому объекту, ссылку на которого он получил когда-то и хранит. На самом деле иногда разделяют два схожих понятия: «делегирование» и «перенаправление» (*англ.* forwarding), и разница между ними сложно уловима. При делегировании обычно сохраняется информация о том, кто был изначальным исполнителем этой задачи, и для результата это важно. Умными словами то же самое обычно утверждается примерно так: «при делегировании важно сохранять контекст делегирующего объекта». Например, если Васе нужна именная ложка и Вася делегировал/перенаправил своему знакомому Пете задачу сходить в ближайшую мастерскую и купить именную ложку. В итоге если все пойдет нормально и без исключений, то возможны два варианта. Если Вася в итоге окажется с ложкой, на которой написано: «Вася», то программисты скажут, что он задачу делегировал. А если у Васи в руках окажется ложка с надписью «Петя», то это не делегирование было, а перенаправление. Формально при делегировании тот, кому делегировали задачу, должен помнить о том, кто ее делегировал. В коде это часто видно по тому, что в метод делегируемого объекта передается ссылка на делегирующий объект (характерно называемая `sender`, `self` или как-то так). Вот пример:

```
class Senior;

class Junior {
public:
    void work(Senior *sender) {...}
};
```

```

class Senior {
private:
    Junior *assigned;
public:
    Senior(Junior *ass) {
        assigned = ass;
    }
    void work() {
        assigned->work(this);
    }
}

```

Делегирование позволяет на лету менять поведение объекта, в результате чего разные объекты одного и того же класса при вызове одного и того же метода могут в итоге вести себя по-разному, и является очень часто используемой техникой при разработке объектно-ориентированных приложений.

Идея делегирования может быть реализована очень по-разному, но идея всегда прослеживается: исходный объект в разных ситуациях делегирует выполнение задачи разным объектам и добивается разного поведения. Если вы хотя бы раз создавали форму с кнопками, вы уже обязательно столкнулись с делегированием и воспользовались им, даже не заметив этого:

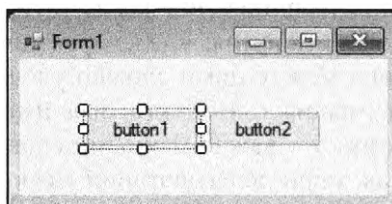


Рис. 2.9. Формочка с кнопками

Например, если вы будете создавать простое графическое приложение Windows Forms Application (можно в любом другом подходящем фреймворке), бросите на формочку две кнопки (рис. 2.9), то что произойдет по двойному щелчку этих кнопок?

```

partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }
    ...
    private void InitializeComponent()
    {
        this.button1 = new System.Windows.Forms.Button();

```

```
        this.button2 = new System.Windows.Forms.Button();  
        ...  
    }  
    ...  
    private System.Windows.Forms.Button button1;  
    private System.Windows.Forms.Button button2;  
}
```

Во-первых, появится класс-потомок `Form1` стандартного класса формы `Form`. Нам ведь не нужна стандартная форма? Она пустая и ничего делать не умеет. У этого класса появляются два поля стандартного типа `Button`, которые создаются в конструкторе (да, это композиция). Но стандартные кнопки тоже ничего не умеют, они при нажатии ничего не делают. Возможно, нам нужно было создать сначала несколько потомков класса `Button`, перекрыть у них метод, отвечающий за нажатие? Думаю, вы понимаете, что создавать по новому классу для каждой кнопки, которая должна вести себя особенным образом, — это перебор. Вот тут нам на помощь и приходит делегирование.

```
public partial class Form1 : Form  
{  
  
    private void InitializeComponent() {  
        this.button1 = new System.Windows.Forms.Button();  
        this.button2 = new System.Windows.Forms.Button();  
        ...  
        this.button1.Click +=  
            new System.EventHandler(this.button1_Click);  
        this.button2.Click +=  
            new System.EventHandler(this.button2_Click);  
    }  
    ...  
    private void button1_Click(object sender, EventArgs e)  
    {  
    }  
    private void button2_Click(object sender, EventArgs e)  
    {  
    }  
    ...  
}
```

У нашего потомка формы создались два метода, `button1_Click` и `button2_Click`, и произошло делегирование: «метод» `button1.Click` у кнопки `button1` был «делегирован» в метод `Form1.button1_Click` и аналогично со второй кнопкой. Обратите внимание: `Click` был «методом» кнопки, — а работу в итоге будет выполнять метод формы. Это и есть делегирование.

С понятием и техникой делегирования связана проблема под названием «объектная шизофрения». Под неправильным названием, понятное дело, потому что

шизофрения — она совсем не про то, про что принято думать в массовой культуре. Правильно было бы называть эту проблему «объектное раздвоение личности». Допустим, в методе `button1_Click` написан код: `Width = Width + 10`. Вопрос заключается в следующем: чья ширина при этом должна возрасти на 10 пикселей? Если Пете делегировали задачу по набиванию имени на ложку, то чье имя он должен набить: свое или имя того, кто ему эту задачу делегировал? Говоря умными словами, при использовании делегирования в контексте какого из объектов должен выполняться код: делегирующего или делегируемого?

```
{
    private void button1_Click(object sender, EventArgs e)
    {
        Width = Width + 10;
    }

    private void button2_Click(object sender, EventArgs e)
    {
        (sender as Control).Width =
            (sender as Control).Width + 10;
    }
}
```

На самом деле эта проблема скорее академическая, чем практическая: наверняка кому-то очень хотелось изобрести проблему и включить ее в свою диссертацию. С точки зрения языка никакого раздвоения личности нет: если внутри метода написано обращение к имени свойства или тем более к имени свойства с использованием `this` (в C++), то речь идет о свойстве того класса, которому сам метод принадлежит. Если требуется обратиться к свойству делегирующего объекта, то нужно это указать явно с использованием ссылки на делегирующий объект, которая — вот удача-то! — как раз при делегировании и передается.

2.12. Strategy

Паттерн «Стратегия» — это еще один из простейших приемов, позволяющих отказаться от жесткого прописывания в классе разных вариантов поведения. Вообще, большинство паттернов — про то, как бы что-то такое откуда-нибудь взять и куда-нибудь вынести, в идеале — в отдельный объект. В частности паттерн «Стратегия» — про то, как в отдельный объект вынести алгоритм перемножения массива чисел. Нельзя ведь просто так взять и перемножить все числа в массиве? Можно перемножать все числа, а можно перемножать числа до первого встретившегося нуля; перемножать с начала в конец или с конца в начало и т. д. Давайте посмотрим вот на этот код:

```
class MultStrategy {
public:
    virtual double multiply(double *arr, int cnt) = 0;
};
```

Что такое стратегия (в данном случае — стратегия перемножения массива чисел)? Можно было бы сказать, что стратегия перемножения — это просто функция, которая содержит внутри алгоритм. И это действительно почти так: но с функциями в объектно-ориентированном языке не очень удобно работать. Их ни передать куда-нибудь, ни вернуть откуда-нибудь, — можно даже сказать, что функции в объектно-ориентированном языке не являются *first class citizens*. Поэтому приходится функцию оборачивать объектом. Стратегия — это объект-обертка над функцией, у которого есть метод `multiply(double *arr, int cnt)` для перемножения чисел в массиве.

```
class ForwardMultStrategy: public MultStrategy {
public:
    double multiply(double *arr, int cnt) override {
        double res = 1;
        for (int i = 0; i < cnt; i++)
            res = res * arr[i];
        return res;
    }
};

class BackwardMultStrategy: public MultStrategy {
public:
    double multiply(double *arr, int cnt) override {
        double res = 1;
        for (int i = cnt-1; i >= 0; i--)
            res = res * arr[i];
        return res;
    }
};
```

А вот два потомка абстрактного класса-стратегии, а именно две стратегии, две возможные реализации, собственно, перемножения: для примера, одна реализация перемножает с начала в конец, а другая — с конца в начало. Обратите внимание, что стратегии в параметрах передается массив, числа из которого нужно перемножать, и его размер.

```
class Container {
private:
    int cnt;
    double *values;
public:
    MultStrategy *strategy;
    Container(int _cnt): cnt(_cnt) {
        values = new double[cnt];
        for (int i = 0; i < cnt; i++)
            values[i] = double(rand()) / RAND_MAX + 0.5;
        strategy = nullptr;
    }
};
```



```
double get_product() {
    if (strategy != nullptr)
        return strategy->multiply(values, cnt);
    else
        return 0;
}

virtual ~Container() {
    delete[] values;
}

};
```

Еще один класс — контейнер, который хранит числа и которому может понадобиться их перемножить. Главная его идея — он сам не занимается перемножением, у него для этого хранится указатель `strategy` на кого-то из потомков `MultStrategy`, но кого именно — контейнер сам не знает.

```
void main() {
    Container container;

    container.strategy = new ForwardMultStrategy();
    printf("%f\n", container.get_product());
    delete container.strategy;

    container.strategy = new BackwardMultStrategy();
    printf("%f\n", container.get_product());
    delete container.strategy;
}
```

В основной программе создается контейнер, и потом контейнер настраивается объектом-стратегией. Возможно, это должно происходить прямо при создании объекта-контейнера, но совершенно точно не внутри объекта-контейнера, потому что он не должен ничего знать про конкретные методы перемножения, это не его дело. В любой момент, когда контейнеру понадобится перемножить хранимые в нем числа, он обращается к своему текущему объекту-стратегии и просит его выполнить перемножение. При этом ему приходится передавать в стратегию указатель на массив с данными, потому что сам контейнер имеет данные, но не знает, как их перемножать, а стратегия, наоборот, знает, как перемножать, но не имеет данных.

Таким образом, основная идея этого паттерна заключается в том, чтобы инкапсулировать, «обернуть», «вынести» разные возможные алгоритмы какого-то сложного действия в отдельные объекты, чтобы потом иметь возможность изменять алгоритм поведения путем просто замены объекта. Заменить один объект на другой — гораздо проще ведь, чем заменить один метод на другой? Представьте себе, что алгоритм перемножения был бы прописан напрямую в контейнере как-то так:

```
class Container {
private:
    int cnt;
    double *values;
public:
    // ...
    double get_product() {
        double res = 1;
        for (int i = cnt-1; i >= 0; i--)
            res = res * values[i];
        return res;
    }
};
```

Если бы вдруг нам понадобилось реализовать несколько алгоритмов, то что пришлось бы делать? Первый вариант — изменять уже существующий класс и реализовывать какой-то `if` в зависимости от текущего состояния контейнера, хранящегося в атрибуте `mult_method`:

```
class Container {
private:
    int cnt;
    double *values;
    int mult_method{ 0 };
public:
    // ...
    double get_product() {
        if (mult_method == 0) {
            double res = 1;
            for (int i = cnt - 1; i >= 0; i--)
                res = res * values[i];
            return res;
        }
        if (mult_method == 1) {
            double res = 1;
            for (int i = 0; i < cnt; i++)
                res = res * values[i];
            return res;
        }
        return 0;
    }
};
```

Другой вариант, если мы не можем позволить себе изменять уже существующий класс: тогда придется наследоваться от него и перекрывать метод `get_product()`:

```
class Container {
protected:
```

```

int cnt;
double *values;
public:
    // ...
    double get_product() {
        double res = 1;
        for (int i = cnt - 1; i >= 0; i--)
            res = res * values[i];
        return res;
    }
};

class AnotherContainer: public Container {
public:
    // ...
    double get_product() {
        double res = 1;
        for (int i = 0; i < cnt; i++)
            res = res * values[i];
        return res;
    }
};

```

И то и другое — так себе варианты: они требуют изменения существующих классов при появлении нового варианта алгоритма. Паттерн «Стратегия» тем и хорош, что дает хороший, годный вариант для гибкой замены алгоритма поведения класса без изменения самого класса, дописывая новый, а не изменяя старый код.

Диаграмма классов паттерна соответствует первоисточнику, она очень проста (рис. 2.10).

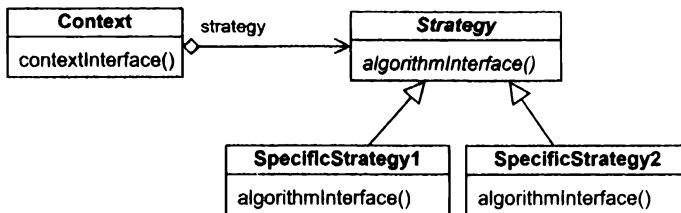


Рис. 2.10. Диаграмма классов паттерна «Стратегия»

Здесь есть тот, кому нужно воспользоваться неким алгоритмом (Context), и есть три варианта реализации этого алгоритма (ConcreteStrategy A, B и C). Тот, кто хочет им воспользоваться, хранит ссылку strategy на одного из потомков класса Strategy и при необходимости вызова алгоритма вызывает у strategy метод AlgorithmInterface(). Обратите внимание, тут никак не прописаны детали о том,

как именно стратегии передаются данные, с которыми она должна работать. Возможно, как в нашем случае, эти данные передаются напрямую параметром (у нас метод `multiply(double *arr, int cnt)` принимает массив чисел), но, возможно, в стратегию передается прямо ссылка на сам объект `Context` (у нас тогда метод перемножения выглядел бы так: `multiply(Container *container)`, принимая весь контейнер вместо массива чисел), и стратегия уже сама забирает нужные ей данные из `Context` с помощью каких-то методов `ContextInterface()`, присутствующих на диаграмме.

2.13. Adapter

Паттерн «Адаптер» предназначен для того, чтобы обеспечить работу через какой-то интерфейс с тем объектом или классом (два разных варианта паттерна), который изначально этот интерфейс не поддерживает, а изменить его, чтобы он поддерживал, нельзя. Давайте посмотрим на новый код:

```
class IArray {
public:
    virtual int getCount() = 0;
    virtual int getValue(int index) = 0;
};
```

```
class CArray : public IArray {
private:
    int *_values;
    int _size;
public:
    CArray(int size) {
        _size = size;
        _values = new int[_size];
        for (int i = 0; i < _size; i++)
            _values[i] = rand();
    }
    int getCount() override {
        return _size;
    }
    int getValue(int index) override {
        return _values[index];
    }
    virtual ~CArray() {
        delete[] _values;
    }
};
```

Пока все просто и понятно: есть интерфейс `IArray`, определяющий пару виртуальных абстрактных методов для работы с массивом целых чисел. И есть класс

CArray, который этот интерфейс реализует, обеспечивая хранение данных (в нашем случае — случайных целых чисел) и доступ к ним через интерфейс массива.

На этом моменте уже стоит остановиться. Что такое массив? Это не самый простой вопрос, потому что отвечать на него можно двояко. Массив — это набор однотипных элементов, располагаемых последовательно в памяти. Массив — это набор однотипных элементов, обращаться к которым можно по индексу. Первый ответ описывает массив как реальную структуру данных, хранящуюся где-то в памяти: этот ответ касается деталей реализации и ничего даже не говорит о том, как с массивом работать! Второй ответ описывает массив как так называемый абстрактный тип данных: он, наоборот, описывает, как с массивом работают, и ничего не говорит о том, как на самом деле все в памяти хранится.

Что такое утка? Может быть, утка — это не столько «ценный мех и два килограмма диетического мяса», сколько биологическое существо с генотипом и фенотипом утки? Или утка — это что-то, что умеет летать, плавать и крякать, и все равно, как она там устроена внутри? Чувствуете разницу между двумя этими взглядами на объект? Один взгляд направлен на внутреннюю структуру объекта, а другой взгляд направлен только на внешний интерфейс объекта, его поведение.

Что такое массив как структура данных? Кусок памяти, хранящий элементы последовательно так, что можно сразу сказать, по какому смещению от начала массива будет храниться элемент с каким номером. Именно эту структуру данных и реализует объект CArray (по сути у нас он просто обертка над обычным массивом языка C).

Что такое массив как абстрактный тип данных? Это нечто, к элементам чего мы можем обращаться по номеру, а также (чтобы не обратиться по неправильному номеру) узнавать, сколько именно в нем элементов. Именно это понятие и описывает интерфейс IArray: с этой точки зрения любой класс, который этот интерфейс реализует, можно считать массивом — независимо от того, как данные хранятся внутри, и вообще, хранятся ли!

Первое понятие по-английски называется *data structure*, а второе понятие — *abstract data type*, и разницу между этими двумя понятиями лучше всего показывает именно паттерн «Адаптер».

```
class IList {
public:
    virtual void first() = 0;
    virtual int currentValue() = 0;
    virtual void next() = 0;
    virtual bool isEOL() = 0;
};
```

А вот другой интерфейс: интерфейс списка *IList*. Я думаю, вы уже догадались, — это еще один абстрактный тип данных. Как должна летать, плавать и

крякать структура данных, чтобы мы были готовы назвать ее списком? Согласно этому интерфейсу списком можно назвать любую структуру данных, которая позволяет выполнять с ней четыре основных действия:

- ☐ получить текущий элемент;
- ☐ перейти к следующему элементу;
- ☐ выяснить, не выскочили ли мы за пределы списка;
- ☐ перейти к первому элементу.

Обратите внимание, тут нет, например, функции для получения следующего элемента — ее можно заменить путем последовательного вызова `next()` и `currentValue()`. Ни одну из этих функций нельзя выбросить, убедитесь в этом сами: без любой из этих функций мы не сможем, имея список, пройти по нему и получить все его элементы. Имея только эти четыре функции, можно написать через них любую другую функцию работы со списком. Математики бы сказали, что эти четыре функции образуют базис в пространстве функций работы со списком.

Заметим еще, что как `IArray` есть интерфейс не любого массива, а, можно сказать, массива `read-only`, так и `IList` есть интерфейс не любого списка, а односвязного списка `read-only`, но это детали.

```
void foo(IList *list) {
    for (list->first(); !list->isEOL(); list->next())
        printf("%d ", list->currentValue());
    printf("\n");

    list->first();
    for (int i = 0; i < 5; i++) {
        printf("%d ", list->currentValue());
        list->next();
    }
    printf("\n");
}
```

А вот и функция, которая работает со списком, вызывая только методы интерфейса `IList` (больше ей ничего не доступно). Как видим, функции интерфейса `IList` сделаны специально так, чтобы их было удобно засунуть в `for` для того, чтобы пройти по всем элементам, — но их можно и вызывать отдельно. Но даже итерируясь по индексу, мы не сможем у списка получить элемент по индексу, и так и должно быть.

Ну и главный вопрос, который тут возникает: а можно ли как-то заставить функцию `foo` получать и работать с объектом `CArray`?

```
CArray a(10);
foo(&a);
```

Напрямую это не получится, потому что `foo` требует указатель на `IList`, а объект `CArray` к этому указателю не приводится (потому что этот интерфейс не реализует). Тем более нельзя добавлять нужные методы в класс `Carray`, — он уже написан, используется кем-то и менять его нельзя. Казалось бы, мелочь какая — ну не получится объект в функцию передать, и фиг с ним. На самом деле проблема глобальнее: у вас может быть написана уже большая программа, которая работает с каким-то объектом (например, со списком), а нужно заставить ее работать с другим объектом (например, с массивом), и желательно при этом не переписывая ее всю.

Если при этом вся программа работает со списком, используя все методы, определенные в каком-нибудь условном `CList`, реализующем интерфейс `IList`, то у вас реальные проблемы! А если используются только методы `IList`, то жить гораздо проще, и я думаю, вы уже понимаете почему. Мы именно для того и создавали интерфейсы и ограничивали сами себя, работая с объектами через интерфейс, чтобы перечень используемых методов был как можно меньше. Именно это нам позволит сейчас эффективно использовать паттерн «Адаптер».

```
class CListAdapter : public IList, private CArray {
private:
    int _current_index;
public:
    CListAdapter(int size) : CArray(size) {
        _current_index = 0;
    }
    void first() override {
        _current_index = 0;
    }
    int currentValue() override {
        return getValue(_current_index);
    }
    void next() override {
        _current_index++;
    }
    bool isEOL() override {
        return _current_index == getCount();
    }
    ~CListAdapter() override { }
};
```

Вот она, главная идея паттерна! Нельзя менять класс `CArray` — создадим его потомка, который будет приватно наследоваться от `CArray`, а публично наследоваться от `IList`. Этот потомок `CListAdapter` и будет адаптером! В самом классе объявим целочисленную переменную для хранения номера «текущего» элемента (у списка всегда есть «текущий» элемент, а у массива «текущего» элемента нет — кто-то сказал бы, что у списка присутствует «состояние»). В конструкторе

ре адаптера мы просто вызываем конструктор массива и обнуляем внутренний счетчик.

Дальше мы должны, как и обещали, реализовать все четыре метода класса `IList`. Метод `first()` просто сбрасывает внутренний счетчик в нуль, метод `next()` увеличивает его на единицу, метод `isEOL()` проверяет, не выскочил ли счетчик уже за пределы массива (если индекс равен числу элементов массива, по этому индексу уже обращаться нельзя), а метод `currentValue()` возвращает элемент по текущему индексу-счетчику.

Подождите, откуда метод `currentValue()` берет сами данные, чтобы возвращать их? Можно ли сказать, что класс `CListAdapter` занимается хранением данных? И да и нет. Хранением данных занимается класс `CArray`, как и раньше, но `CListAdapter` от него наследуется, поэтому можно сказать, что он данные и хранит. Совершенно как Крокодил Гена нес в итоге чемодан, хотя на самом деле чемодан нес Чебурашка, а Гена нес Чебурашку. Важно, что когда у `CListAdapter` просят вернуть текущий элемент в методе `currentValue()`, `CListAdapter` вызывает унаследованный от `CArray` метод `getValue(_current_index)` для того, чтобы на самом деле к данным обратиться.

Можно даже сказать, что класс `CListAdapter` сам делает очень мало, а большей частью делегирует выполнение работы хранящемуся внутри него объекту `CArray`. Еще можно сказать, что `CListAdapter` перенаправляет вызовы методов интерфейса `IList` в вызовы каких-то других (но соответствующих) методов класса `CArray`, и это тоже будет частью описания этого паттерна.

Вот теперь то, что не получалось раньше, становится возможным:

```
CListAdapter list(10);  
foo(&list);
```

Числа на самом деле хранятся в массиве, но функция `foo` работает с ними так, как будто они хранятся в списке! Важный вопрос, на котором я еще раз хочу заострить внимание: является ли класс `CListAdapter` списком или нет?

Снова и снова мы приходим к обсуждению этого кардинального отличия. Ответ на этот вопрос зависит от того, с какой стороны вы смотрите на объект, изнутри или снаружи, и соответственно что вкладываете в понятие «является». С одной стороны (изнутри), `CListAdapter`, безусловно, не является списком, потому что внутри он массив. С другой стороны (снаружи), благодаря приватному наследованию от `CArray` и публичному от `IList` любой наблюдатель скажет, что `CListAdapter`, безусловно, является списком, потому что он выглядит как список и с ним можно работать как со списком. Получается, имея всего два интерфейса, `IArray` и `Ilist`, и два класса, `CArray` и какой-нибудь `CList`, можно получить целых четыре комбинации: объект, который:

- ☐ выглядит снаружи как массив, и внутри он тоже массив;
- ☐ выглядит снаружи как массив, а внутри он при этом список;

- ❑ выглядит снаружи как список, и внутри он тоже список;
- ❑ выглядит снаружи как список, а внутри он при этом массив.

Прямую иллюстрацию адаптерам, если кому-то все еще не очень понятна идея этого паттерна, можно найти в реальной жизни. Если у меня есть зарядное устройство с европейской вилкой, у меня возникнут проблемы по приезду в какую-нибудь страну с британскими розетками. Какое решение предлагает мне рассмотренный паттерн? Правильно: обратиться на фабрику, где производят зарядные устройства, чтобы они изменили свой техпроцесс, встроили в вилку переходник и продали мне новое устройство!.. Подождите, как же так? Мне нужно новое зарядное устройство для новых розеток? Посмотрим еще раз на этот пример:

```
CListAdapter list(10);  
foo(&list);
```

Подождите, у меня уже есть мое зарядное устройство, оно мне нравится, я не хочу покупать новое только для того, чтобы работать с другими розетками! Действительно, мы передаем в функцию только что созданный объект класса `CListAdapter`, который внутри себя, в конструкторе, создает новый `CArray`. Но что если у нас уже есть существующий `CArray`?

```
CArray arr(10);  
CListAdapter list(10);  
foo(&list);
```

В этом коде создаваемый `CListAdapter` создаст свой собственный массив, и функция `foo` будет работать именно с ним, а не с уже существующим массивом `arr`. Два массива нам, конечно, не нужны. И для случая, когда у нас есть не только программа, которая уже привыкла работать с `IList`, но и уже существующий объект класса `CArray`, нам понадобится слегка модифицированный адаптер:

```
class CListAdapter : public IList {  
private:  
    int _current_index;  
    CArray *_arr;  
public:  
    CListAdapter(CArray *arr) {  
        _arr = arr;  
    }  
    virtual void first() {  
        _current_index = 0;  
    }  
    virtual int currentValue() {  
        return _arr -> getValue(_current_index);  
    }  
}
```

```
virtual void next() {  
    _current_index++;  
}  
virtual bool isEOL() {  
    return _current_index == _arr -> getCount();  
}  
virtual ~CListAdapter() { }  
};
```

Заметили отличия? Теперь `CListAdapter` наследуется только публично от `IList`, но не от `CArray`. Зато он принимает уже существующий объект `CArray` в конструкторе и сохраняет его в своих свойствах. Всякий раз, когда, например, `CListAdapter` просят вернуть текущий элемент, там, где он раньше просто вызывал унаследованный метод `getValue()`, теперь он вызывает тот же метод `_arr -> getValue()`, но уже не у себя самого, а у хранимого объекта `_arr`.

```
CListAdapter list(&arr);  
foo(&list);
```

Теперь `CListAdapter` не создает массив, он принимает уже существующий массив и будет работать с ним без копирования. Возвращаясь к нашему примеру, теперь нам не понадобится покупать новый зарядник или вносить какие-то изменения в его конструкцию: нам понадобится только простой и дешевый переходник, который мы сможем использовать с существующим зарядником!

Эти два варианта паттерна «Адаптер» (кстати, вы видите здесь старую добрую проблему выбора между композицией и наследованием?) называются соответственно, адаптер уровня класса и адаптер уровня объекта. Они оба реализуют одну и ту же основную идею паттерна. В первом случае у нас есть класс, который хорошо работает, но имеет неподходящий нам интерфейс, и мы должны сделать на его основе другой класс, который будет работать так же, но просто иметь другой интерфейс. Во втором случае у нас есть не класс, а готовый объект, который имеет неподходящий нам интерфейс, и наша задача, не меняя сам объект, как-то научиться с ним работать через новый интерфейс. Можно еще сказать, что в первом случае мы имеем дело с «толстым» адаптером, потому что в первом примере у нас адаптер внутри и является хранящим числа массивом `CArray`, а во втором случае у нас «тонкий» адаптер, потому что он не хранит сам большой массив чисел, а хранит просто ссылку на него.

Диаграммы классов для первой и второй вариаций паттерна приведены на рис. 2.11 и 2.12.

Вы уже, конечно, разобрались с соответствием этих абстрактных диаграмм нашей программе. `Adaptee` — это у нас `CArray`, `Target` — это у нас `IList`, а `Adapter` — это у нас `CListAdapter`. Вопрос только, где на этой схеме `IArray`? А вот его-то тут как раз и нет, но гипотетически он мог бы быть тут нарисован как предок `IAdapteeInterface` у класса `Adaptee`.

В одном случае `Adapter` (`CListAdapter`) наследуется публично от `Target` (`IList`), а неявно — от `Adaptee` (`CArray`). В другом случае он наследуется публично от

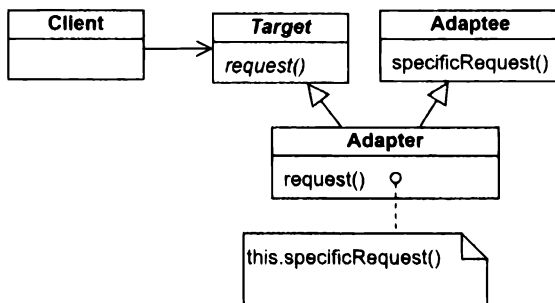


Рис. 2.11. Диаграмма классов для адаптера уровня класса

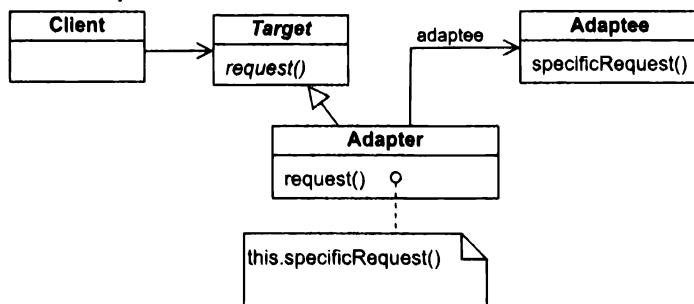


Рис. 2.12. Диаграмма классов для адаптера уровня объекта

Target (IList) и содержит ссылку adaptee на класс Adaptee (CArray). Обе вариации полезны в зависимости от решаемой задачи.

2.14. Iterator

«Итератор» — это очень полезный и поучительный паттерн про организацию работы с контейнерами. Контейнер — это что-то, что хранит значения, для целей примера пусть это будут числа. Мы говорили выше, что хранить значения можно очень по-разному, например в массиве или в списке. На самом деле этот перечень гораздо шире — контейнерами могут быть не только разные виды списков (односвязные, многосвязные), но и разные деревья, ассоциативные массивы, хеш-таблицы и т. д. В реальности, в полном соответствии с двусторонним взглядом на объекты, о котором говорилось в предыдущем разделе, можно говорить о том, что все контейнеры могут отличаться друг от друга тем:

- как они физически хранят данные (т. е. какую структуру данных реализуют);
- как они позволяют работать с данными (т. е. какому абстрактному типу данных соответствуют).

Так, например, какая-нибудь очередь и какой-нибудь стек внутри могут хранить данные в одной и той же структуре данных, но внешний интерфейс у них соответствует разному предназначению и вариантам использования именно очереди и именно стека.

Какой бы ни был контейнер внутри и какой бы интерфейс он ни предоставлял снаружи, часто бывает полезно иметь один общий универсальный способ «обхода» всех элементов контейнера, особенно это касается той ситуации, когда вы, не меняя способ работы с данными (не меняя абстрактный тип данных), хотите поменять внутреннюю структуру хранения данных. Понятно, что если ваша программа работает со стеком, то, поменяв стек на очередь, вы можете случайно изменить способ обхода графа, с которыми имеете дело, и это вряд ли вам нужно. Но если ваша программа работает, например, с массивом, а позже вы выяснили, что вам приходится часто вставлять элементы в разные места массива (для которого эта операция имеет линейную сложность $O(n)$), то очень заманчиво было бы заменить массив на список (для которого та же операция имеет линейную сложность $O(1)$), не изменив способ работы с ним. Да, вы правы: вспомнив предыдущий раздел, мы можем сказать, что итератор выступает в роли адаптера к контейнеру. Но, во-первых, это только одно предназначение адаптера, а во-вторых, давайте потихоньку:

```
class CIterator {
public:
    virtual void first() = 0;
    virtual int getCurrentItem() = 0;
    virtual void setCurrentItem(int item) = 0;
    virtual void next() = 0;
    virtual bool isEOL() = 0;
    virtual ~CIterator() { }
};
```

Кто такой итератор? Итератор — это кто-то, позволяющий универсальным способом обойти все элементы хранилища-контейнера (целых чисел, в данном случае) с помощью набора стандартных действий: перейти на первый элемент, перейти на следующий элемент, проверить, не выскочили ли мы за пределы доступных элементов, получить значение текущего элемента и заменить значение текущего элемента. Это мы просто прочитали содержание абстрактного предка для всех возможных итераторов. Кто-то может сказать: да это же просто методы любого списка, методы интерфейса `IList` из предыдущего раздела! Да, похоже это так. Кто-то может счесть это совпадением, а кто-то — просто признаком того, что способ работы со списком есть самый удачный способ обхода любого контейнера. Вопрос для тех, кто любит задумываться: а самому списку тогда нужен ли итератор?

```
class CArray {
private:
    int *_values;
    int _size;
public:
    CArray(int size) {
        _size = size;
        _values = new int[_size];
    }
```

```

int getCount() {
    return _size;
}
int getValue(int index) {
    return _values[index];
}
void setValue(int index, int item) {
    _values[index] = item;
}
CIterator *createIterator(string);
virtual ~CArray() {
    delete[] _values;
}
};

```

Вот он, наш подопытный контейнер — снова обычный массив целых чисел, и отличается он от прошлых массивов только одной (не реализованной пока) функцией `createIterator`, которая, судя по ее типу, берет строку и возвращает указатель на какой-то объект класса `CIterator` (или его потомков, которые вот-вот появятся).

Используя паттерн «Адаптер» из предыдущего раздела, мы могли бы унаследовать `CArray` от `CIterator`, и это тоже позволило бы работать с массивом универсальным способом, но, как я уже говорил, универсальный способ — это только одна из наших целей.

```

class CRegularArrayIterator : public CIterator {
private:
    CArray *_arr;
    int _current_index;
public:
    CRegularArrayIterator(CArray *arr) {
        _arr = arr;
        _current_index = 0;
    }
    void first() override {
        _current_index = 0;
    }
    int getCurrentItem() override {
        return _arr->getValue(_current_index);
    }
    void setCurrentItem(int item) override {
        _arr->setValue(_current_index, item);
    }
    void next() override {
        _current_index++;
    }
}

```

```
bool isEOL() override {
    return _current_index == _arr->getCount();
}
};

class CPositiveOnlyArrayIterator : public CIterator
{
private:
    CArray *_arr;
    int _current_index;
public:
    CPositiveOnlyArrayIterator(CArray *arr) {
        _arr = arr;
        _current_index = 0;
    }
    void first() override {
        _current_index = 0;
        while (!isEOL() && getCurrentItem() <= 0) {
            _current_index++;
        }
    }
    int getCurrentItem() override {
        return _arr->getValue(_current_index);
    }
    void setCurrentItem(int item) override {
        _arr->setValue(_current_index, item);
    }
    void next() override {
        if (!isEOL())
            _current_index++;
        while (!isEOL() && getCurrentItem() < 0) {
            _current_index++;
        }
    }
    bool isEOL() override {
        return _current_index == _arr->getCount();
    }
};
```

Вот они, `CRegularArrayIterator` и `CPositiveOnlyArrayIterator`, два конкретных класса-итератора. Оба они получают и хранят у себя указатель на какой-то существующий массив `CArray`, оба помнят о своей текущей позиции внутри этого массива. Оба они — потомки абстрактного класса `CIterator` и поэтому перекрывают и реализовывают все его функции. Делают они это очень похожим на адаптер способом: увеличивая хранимый индекс текущего элемента по мере движения по массиву и обращаясь к методам хранимого массива `_arr` непосред-

ственно за данными. `CRegularArrayIterator` при этом, действительно, просто как адаптер проходит по всем элементам, а `CPositiveOnlyArrayIterator` при выборе первого или переходе к следующему элементу пропускает все элементы в поисках положительных.

```
CIterator* CArray::createIterator(string type) {
    if (type == "REGULAR")
        return new CRegularArrayIterator(this);
    else if (type == "POSITIVEONLY")
        return new CPositiveOnlyArrayIterator(this);
    else
        throw "Wrong iterator type!";
}
```

Вот теперь мы можем реализовать метод `createIterator()`. Он, в зависимости от переданной строки, создает и возвращает какой-то из этих двух итераторов. При создании итератора он передаст ему в конструктор себя (`this`), чтобы у итератора был доступ к хранимым данным. Ну а теперь посмотрим, а как же работать с итераторами:

```
CArray arr(10);
for (int i = 0; i < arr.getCount(); i++) {
    arr.setValue(i, rand() - RAND_MAX / 2);
}
```

Ну, во-первых, имея класс `CArray`, будучи знакомы с итераторами, мы все равно можем работать с его объектами по-старинке. Итератор дает новые возможности, но не закрывает исходный интерфейс класса (в отличие от адаптера из предыдущего раздела, где, вспомните, `CListAdapter` наглухо приватизировал методы массива).

```
CIterator *it = arr.createIterator("REGULAR");
for(it->first(); !it->isEOL(); it->next()) {
    printf("%d ", it->getCurrentItem());
}
printf("\n");

CIterator *it2 = arr.createIterator("POSITIVEONLY");
for(it2->first(); !it2->isEOL(); it2->next()) {
    printf("%d ", it2->getCurrentItem());
}
printf("\n");

delete it;
delete it2;
```

Но, во-вторых, мы можем все-таки работать с массивом через итератор. Сначала мы создаем итератор одного типа и используем его в цикле для обхода всех элементов. Потом создаем итератор второго типа и используем его для того же

самого. Циклы `for` в обоих случаях выглядят совершенно идентично (так и задумывалось!), но вот действие они производят разное: один цикл проходит по всем элементам, а другой — только по положительным.

Возникает вопрос: а нафига козе баян? Зачем нужен этот промежуточный объект, если достаточно было просто адаптера? Или, вообще, почему бы не работать напрямую с массивом?

Чтобы лучше понять паттерн, надо представить себе в картинках ситуацию и проблему, которую он решает. Лучше всего при этом, если ситуация будет какая-нибудь каждодневная, привычная всем. Например, допустим, у вас есть бочки с радиоактивными отходами. Вряд ли вы захотите хранить их дома: лучше для этого построить какое-нибудь хранилище, в котором эти бочки будут храниться, в каком-то определенном внутренней структуре порядке.

Хотите вы самостоятельно заходить в это хранилище и непосредственно передвигать радиоактивные бочки своими руками? Вряд ли. Что вместо этого предлагает вам делать рассматриваемый паттерн? Где в этой картинке итератор? Очевидно, что в этой схеме итератор — это робот, который ездит внутри хранилища отходов и может непосредственно передвигать бочки с отходами! С каждым роботом связан пульт, который вы держите в руках, и на этом пульте ровно пять кнопок: «езжай к первой бочке», «езжай к следующей бочке», «мигни, если доехал до конца бочек», «возьми текущую бочку», «поставь текущую бочку». Используя этот пульт, можно выполнить любую требуемую операцию с бочками, не подвергая себя воздействию альфа-, бета-, гамма- и эпсилон-излучения.

В чем особенность такого способа работы, что он дает нам? Во-первых, мы можем ничего не знать о том, как именно бочки организованы внутри здания: они могут там лежать вповалку, в ряд или в несколько рядов на нескольких уровнях, — мы этого не знаем и не должны знать. Если этот способ хранения изменится, — мы ничего и не заметим, потому что у нас в руках всегда будет пульт с теми же самыми пятью кнопками. Да, если хранилище внутри станет многоуровневым, то и роботы внутри должны будут измениться, чтобы уметь подниматься наверх и доставать бочки с верхних этажей. Должны ли мы знать при этом, какой именно робот нам нужен, чтобы он мог справиться с бочками в конкретно организованном хранилище? Нет, не должны: это дело самого хранилища, мы хотим иметь в руках совершенно стандартный пульт с пятью кнопками.

Откуда вообще взялся наш робот с пультом? Как он выглядит, какого он цвета? Мы ничего этого не знаем, потому что робота сделали не мы сами. На стене хранилища есть большая красная кнопка с надписью «выделить робота», по нажатии которой открывается окошечко и вываливается пульт, с которым связан какой-то выделенный нам робот внутри хранилища. Мы даже не видим этого робота и не знаем, как он будет работать, но стандартный пульт с пятью кнопками — у нас в руках.

Еще раз: что можно делать при таком способе работы? Во-первых, можно работать с разными хранилищами единообразно с помощью одного и того же пульта.

Во-вторых, можно с одним и тем же хранилищем работать по-разному с помощью разных роботов, но одинаковых пультов: робот может по команде «езжай к следующей бочке» ехать на самом деле в обратном направлении, и мы, сами не зная того, переберем все бочки в обратном порядке вместо прямого. В-третьих, можно отложить пульт и сходить пообедать, — робот останется там, где был, или вообще передать пульт кому-то другому, и он сможет продолжить с той позиции, где остановились мы. В-четвертых, можно одновременно орудовать двумя пультами и соответственно двумя роботами. Ну и в-пятых, можно передать кому-то пульт как универсальный способ указать на какую-то конкретную бочку внутри хранилища вместо указания вида «третья бочка справа в восьмом ряду снизу», которое, очевидно, зависит от реализации хранилища.

Я надеюсь, вы следили за тем, как эта аналогия ложится на приведенный выше код. Робот — это какой-то из конкретных потомков `CIterator`, например `CPositiveOnlyArrayIterator`, а пульт к нему со стандартными кнопками — это сам абстрактный класс `CIterator`. Итераторы создаем не мы сами, это очень важно! Мы хотим пользоваться абстрактными функциями из `CIterator` и не хотим знать никаких деталей о конкретных роботах и хранилищах. А кто лучше всего знает структуру хранилища и соответственно детали реализации необходимых итераторов для него? Разумеется, само хранилище! Поэтому у нашего контейнера есть «красная кнопка» в виде метода `createIterator()`, а мы не знаем, какой конкретно итератор нам будет возвращен, — да и не хотим знать. Мы даже не знаем, будет ли контейнер создавать новый итератор или они у него предсозданы, и мы получим просто любой незанятый итератор, — это тоже скрыто от нас абстракцией.

Итак, суть паттерна «Итератор» заключается в том, что в отдельном объекте-итераторе инкапсулируется способ и текущее состояние обхода элементов некоторого контейнера — это все малопонятная абстракция, поэтому просто имейте в виду: итератор — это объект-робот, который по вашим командам передвигается по хранилищу.

Каких целей позволяет достичь итератор при работе с контейнером? Таких целей как минимум пять, и они уже озвучены:

- ☐ работать с разными типами контейнеров (вектор, список, ассоциативный массив и т. д.) единообразно;
- ☐ работать с одним и тем же типом контейнера по-разному (обходить спереди назад, сзади наперед, прыгая через один элемент и т. д.);
- ☐ сохранять текущее состояние обхода, останавливать и восстанавливать обход;
- ☐ организовывать одновременно два обхода по одному и тому же контейнеру (адаптер уровня класса этого, например, не может);
- ☐ использовать итератор как универсальный «указатель» на один из элементов контейнера.

Вот как выглядит диаграмма классов паттерна «Итератор» (рис. 2.13).

Тут все очевидно и уже обсуждалось выше: конкретный контейнер умеет создавать свой собственный конкретный итератор, который знает, какой контейнер и как ему придется обходить, потому что хранит на него ссылку. Клиент не знает, ни с каким конкретным контейнером имеет дело, ни какой конкретно итератор получает для работы, но может с помощью этого итератора обрабатывать все элементы контейнера.

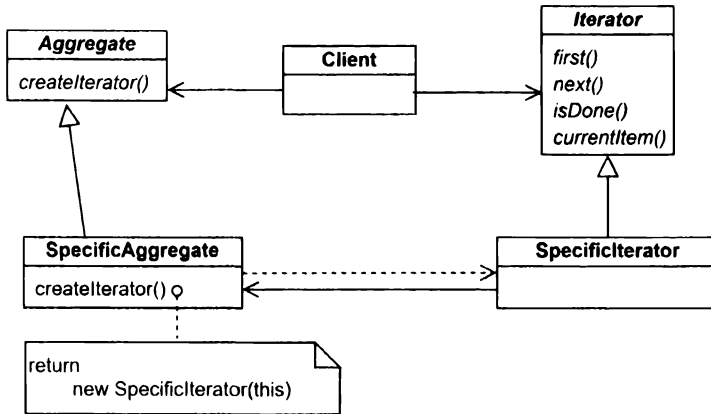


Рис. 2.13. Диаграмма классов паттерна «Итератор»

Плодотворная идея итератора проникла во многие библиотеки и часто реализована более или менее стандартным способом во многих языках. Давайте, например, сравним, как выглядит использование нашего «велосипедного» итератора:

```

CArray arr(10);
CIterator *it = arr.createIterator("REGULAR");
for(it->first(); !it->isEOL(); it->next()) {
    it->setCurrentItem(rand());
}
delete it;
  
```

и как выглядит использование итераторов при работе с библиотекой STL:

```

list<int> values(10);
list<int>::iterator i;
for (i = values.begin(); i != values.end(); i++)
    *i = rand();
  
```

Что такое `list<int>::iterator`? Это способ получить итератор, специфический для конкретного контейнера `list<int>`, аналогично нашему `arr.createIterator("REGULAR")`. Что такое `begin()`? Это функция контейнера, возвращающая итератор, «стоящий» на первом элементе. Прямого аналога у нас нет, но ближе всего функция `it->first()` уже у самого итератора. Что такое `end()`? Это аналогичная функция контейнера, возвращающая итератор, «стоящий» за пределами последнего элемента. Что такое `i++`, ведь `i` здесь — это не

число, это итератор? Очевидно, что это просто аналог `it->next()`. Ну и наконец, что такое `*i`, ведь `i` здесь — это не указатель? Очевидно, что это просто аналог `it->getCurrentItem(...)` или `it->setCurrentItem(...)`. Реализация, как видим, немного отличается, но общая идея — та же самая. Объект итератора `i` можно использовать как универсальный указатель на какое-то место в контейнере.

2.15. Шаблоны C++.

Осторожно по минному полю

Но, кажется, в конце предыдущего раздела мы немного забежали вперед. Самое время осторожно познакомиться с шаблонами C++ и их идеей. Вот был у нас класс `CArray`, предназначенный для хранения массива целых чисел. А что если понадобится хранить не целые числа, а `float`? Придется копипастить, создавать другой класс с другим именем, который будет отличаться только в нескольких местах использованным типом:

```
class CArrayInt {
private:
    int *_values;
    int _size;
public:
    CArrayInt(int size) {
        _size = size;
        _values =
            new int[_size];
    }
    int getCount() {
        return _size;
    }
    int getValue(int i) {
        return _values[i];
    }
    void setValue(int i,
                  int it)
    {
        _values[i] = it;
    }
    virtual ~CArrayInt() {
        delete[] _values;
    }
};
```

```
class CArrayFloat {
private:
    float *_values;
    int _size;
public:
    CArrayFloat(int size) {
        _size = size;
        _values =
            new float[_size];
    }
    int getCount() {
        return _size;
    }
    float getValue(int i) {
        return _values[i];
    }
    void setValue(int i,
                  float it)
    {
        _values[i] = it;
    }
    virtual ~CArrayFloat() {
        delete[] _values;
    }
};
```

Один код получается из другого автозаменой одного типа на другой (правда, не все `int` заменяются, потому что они используются в качестве индекса). Чтобы отказаться от этой копипасты (а точнее, переложить ее на компилятор), можно использовать шаблоны: заменить `int` не на `float`, а на какое-то имя `T` и просто указать компилятору, что это не настоящий тип, а просто имя, которое мы потом будем на что попало заменять:

```
template <class T>
class CArray {
private:
    T *_values;
    int _size;
public:
    CArray(int size) {
        _size = size;
        _values = new T[_size];
    }
    int getCount() {
        return _size;
    }
    T getValue(int index) {
        return _values[index];
    }
    void setValue(int index, T item) {
        _values[index] = item;
    }
    virtual ~CArray() {
        delete[] _values;
    }
};
```

Если не обращать внимания на первую строку, `template <class T>`, то все оставшееся есть просто шаблон, который можно взять, заменить там `T` на `int` и получить первый класс, а если заменить `T` на `float`, то получится второй класс. Собственно говоря, компилятор так и делает. Мы не можем создавать сами по себе объекты класса `CArray` (потому что это не класс, это «шаблонный класс», т. е. шаблон для создания класса), но мы можем создавать объекты варианта класса `CArray`, где `T` заменено на что-то, нужное здесь и сейчас нам:

```
CArray<int>    arr1(10);
CArray<float>  arr2(10);
CArray<Base*>  arr3(10);
```

Как только препроцессор увидел `CArray<float>`, он скопипастил `CArray`, назвал его, условно, `CArrayfloat` и заменил в нем `T` на `float`. Как только препроцессор увидел `CArray<Base*>`, он скопипастил `CArray`, назвал его, условно, `CArrayBasePtr` и заменил в нем `T` на `Base*` и т. д. Шаблон, написанный один раз, позволяет отказаться от копипасты очень похожих классов, отличающихся только какими-то типами данных (а возможно, не только типами, но еще и какими-то константными значениями). При компиляции все равно код будет размножен и скомпилирован, но мы этого уже не увидим.

Вот это «мы этого уже не увидим» и является на самом деле главной проблемой при работе с шаблонами. Если ошибка возникнет в исполняющемся коде, то не факт, что нам будет просто найти то место, где происходит ошибка, потому что

она возникает в размноженном коде, который мы даже не видим! Кроме этого, работа с шаблонами требует повышенного умения предвидения, потому что нужно мысленно представлять себе, что получится при подстановке всего в шаблон. Например, попробуйте представить, что получится при подстановке `Base*` в одном случае и `Base` — в другом и как будут работать функции `getValue` и `setValue` с передаваемыми в них значениями (создайте у класса `Base` весь набор конструкторов с отладочным выводом, чтобы убедиться):

```
CArray<Base*> arr3(10);  
CArray<Base> arr4(10);
```

Шаблоны в C++ — это очень полезная технология, но я предлагаю вам пользоваться ею только вот так ограниченно, как показано здесь, и не увлекаться. Начав с таких простых шаблонов для создания контейнеров, можно заглянуть слишком глубоко в эту кроличью нору, и вы не успеете заметить, как оперируете шаблонами шаблонов шаблонов шаблонов, а ваши сообщения об ошибках не умещаются в один текстовый экран.

2.16. Стандартные контейнеры STL

Мы уже несколько раз создавали свой собственный класс-контейнер для хранения разных типов данных. Конечно же, такая важная задача давным-давно имеет множество стандартных решений, и в C++ для этого чаще всего используются стандартные контейнеры библиотеки STL. В большинстве случаев контейнеров для STL вам будет достаточно для решения ваших задач — нужно просто знать, какие есть и как их правильно использовать. Они все шаблонные, поэтому храните в них те типы данных, которые вам нужны.

Контейнеры могут отличаться тем, как именно они хранят данные внутри. В этом смысле такие контейнеры представляют собой то, что называется «типы данных». Другие контейнеры представляют собой «обертку» над существующим контейнером и отличаются друг от друга интерфейсом (как наши `IList` и `IArray` в паттерне «Адаптер» оба позволяли работать по-разному с одним и тем же массивом, хранящимся внутри). Такие контейнеры ближе к понятию `Abstract Data Type (ADT)`.

Говоря о любом контейнере, надо иметь в виду именно эти две вещи: как этот контейнер хранит данные внутри (и к каким последствиям это ведет) и какой интерфейс или типовой воркфлоу этот контейнер подразумевает снаружи. Это нужно, чтобы случайно не обращаться к элементам `list` по индексу через `[]` и не вставлять случайно элементы в середину `vector`. Еще при работе с любым контейнером надо держать в голове, что если у вас есть какие-нибудь указатели на его элементы (прямые указатели на область памяти или индексы), то для разных контейнеров при разных операциях они могут «слетать», а могут и не «слетать». Например, если вы держали указатель на какой-то из элементов `vector`, то при добавлении элемента в `vector` этот указатель может стать неправильным (пото-

му что `vector` переедет в другое место памяти). А для `deque`, например, это не так: указатель останется верным при добавлении элементов что в начало, что в конец.

Контейнер `std::array<T>` (где `T` — любой тип) представляет собой развитый аналог обычного C-массива, который имеет фиксированный размер, задаваемый при создании, и хранит свое содержимое прямо «внутри себя», в той памяти, где создан он сам. Поэтому передать по значению куда-то (например, в функцию) `std::array<что-нибудь-жирное>` — значит скопировать кучу памяти. Внешне `array` выглядит как массив и подразумевает работу по индексу через `[]`.

Контейнер `std::vector<T>` представляет собой развитый аналог динамического C-массива, который тоже обязан занимать непрерывную область памяти, но сам автоматически растет при добавлении элементов (и делает это так, чтобы выделять или перемещать память пореже). Контейнер `vector` хранит «внутри себя» только указатель на свой массив и его размер, а сам массив лежит где-то в динамической памяти. При передаче по значению куда-то (например, в функцию) `std::vector<что-нибудь-жирное>` копирует весь массив, как и `std::array<T>`. Внешне `vector` тоже выглядит как массив и подразумевает работу с элементами по индексу через `[]`. При добавлении или удалении элемента указатели на другие элементы и итераторы становятся невалидными.

Контейнер `std::deque<T>` хранит внутри несколько динамических массивов и при необходимости роста не перераспределяет заново память (что требует скопировать все уже существующие элементы), а добавляет очередной динамический массив. Еще одна фишка `deque` — динамические массивы прирастают не только к концу массива, но и к началу тоже, поэтому в `deque` легко добавлять (без перераспределения) элементы и в начало, и в конец (у `vector` легко добавлять только в конец). При добавлении или удалении элемента в начало или конец указатели на другие элементы остаются валидными, а итераторы становятся невалидными. В иных случаях инвалидируются и те и другие.

Контейнер `std::list<T>` представляет собой двусвязный список, в который легко можно добавлять элементы в любое место без перераспределения существующих. Создатели STL намеренно не реализовали у этого контейнера операцию доступа по индексу `[]`, потому что она бы потребовала линейного времени, и работа с контейнером выполняется по интерфейсу списка чаще всего с использованием итераторов для работы с «серединкой» списка и отдельными функциями для работы с крайними элементами. При добавлении или удалении элементов в список (в любое место) итераторы и указатели на элементы остаются валидными.

Контейнер `std::forward_list<T>` представляет собой односвязный список, в который легко можно добавлять элементы в любое место без перераспределения существующих. У этого контейнера тоже недоступна операция доступа по индексу `[]`, а доступные операции похожи на операции с `list`, только если с `list` можно работать одновременно с началом и с концом списка, то у `forward_list`

функции аналогичны только для начала списка. При добавлении или удалении элементов в список (в любое место) итераторы и указатели на элементы остаются валидными.

Контейнер `std::set<T>` представляет собой контейнер-множество (внутри чаще всего реализованный сбалансированным деревом, но в данном случае это не важно), который, как и любое множество, содержит уникальные элементы и операции, специфические именно для множества: добавление элемента, удаление элемента, поиск элемента и т. д. При добавлении или удалении элементов во множество итераторы и указатели на элементы остаются валидными.

Контейнер `std::map<K, T>` представляет собой удобный ассоциативный массив (внутри тоже чаще всего реализованный сбалансированным деревом), доступ к элементам которого производится операцией `[]` аналогично массиву, только ключом является не целое число, а какой-то другой тип, заданный шаблонным параметром `K`. При добавлении или удалении элементов итераторы и указатели на элементы остаются валидными.

Контейнеры `std::unordered_set<T>` и `std::unordered_map<K, T>` являются аналогами `std::set<T>` и `std::map<K, T>`, но реализованы как хеш-таблица, а не дерево. Во многих сценариях они работают быстрее, но имеют и свои недостатки: при добавлении или удалении элементов, хотя указатели на элементы и остаются валидными, итераторы становятся невалидными.

И наконец, два контейнера, `std::stack<T>` и `std::queue<T>`, в явном виде являются просто оберткой над каким-то другим контейнером (по умолчанию `std::deque`) и реализуют в чистом виде две абстрактные структуры данных: стек и очередь.

2.17. Decorator

Теперь вернемся к паттернам и рассмотрим еще один достаточно простой — «Декоратор».

```
class CShape {
public:
    virtual void draw() = 0;
    virtual ~CShape() { };
};

class CPoint : public CShape {
private:
    int _x, _y;
public:
    CPoint(int x, int y) {
        _x = x; _y = y;
    }
};
```

```
void move(int dx, int dy) {
    _x = _x + dx; _y = _y + dy;
}
void draw() override {
    printf("(%d, %d)", _x, _y);
}
~CPoint() { }
};
```

Как видим, здесь объявлен абстрактный класс `CShape` с единственным абстрактным методом `draw()`, а в классе-потомке `Point`, содержащем координаты точки, этот метод перекрыт. В методе `draw()` у точки производится «рисование» точки на экране. Дополнительно есть еще метод `move()`, который перемещает точку, изменяя ее координаты. Пока ничего неожиданного.

```
class CDecorator : public CShape {
private:
    CShape *_shape;
public:
    CDecorator(CShape *shape) {
        _shape = shape;
    }
    virtual void draw() {
        printf("*");
        _shape->draw();
        printf("*");
    }
    ~CDecorator() {
        delete _shape;
    }
};
```

Вот он, декоратор! Это еще один потомок `CShape`, который при создании получает и внутри себя хранит указатель на какой-то другой, уже существующий объект-потомок `CShape` (раз сам `CShape` абстрактный, объекты его «родного» класса существовать не могут). Мы видим, что в конструктор `CDecorator` приходит указатель на существующий объект, а в деструкторе этот объект удаляется, т. е. получается, что декоратор принимает на себя бремя владения временем жизни того объекта, который в него передан, но не это главное.

Главное — класс `CDecorator` как потомок `CShape` обязан перекрыть абстрактный метод `draw()`, и он это делает, но специфическим образом: в своем методе `draw()` декоратор вызывает метод `draw()` того объекта, который хранит, но, кроме этого, делает что-то еще. В данном случае, раз речь идет о методе рисования, декоратор «пририсовывает» слева и справа от точки свои звездочки.

```
void main() {
    vector<CShape*> shapes(3);
```



```

shapes[0] = new CPoint(1, 1);
shapes[1] = new CDecorator(
    new CPoint(2, 2));
shapes[2] = new CDecorator(
    new CDecorator(
        new CPoint(3, 3)));

for (const auto &i : shapes)
    if (i != nullptr) {
        i->draw();
        printf("\n");
    }

vector<CShape*>::iterator i;
for (i = shapes.begin(); i != shapes.end(); i++)
    if (*i != nullptr)
        delete *i;
}

```

В основной программе мы создаем `shapes` как вектор STL (см. предыдущий раздел) из трех указателей (раз именно трех, то вполне подошел бы и `array`). В нулевую ячейку мы складываем создаваемый объект класса `CPoint`, в первую ячейку — создаваемый объект класса `CDecorator`, в который сложен создаваемый объект `CPoint`, а во вторую ячейку — создаваемый объект класса `CDecorator`, в который сложен создаваемый объект класса `CDecorator`, в который сложен создаваемый объект класса `CPoint`.

Обратите внимание: раз в конструктор `CDecorator` передается не `CPoint*`, а `CShape*`, то `CDecorator` может хранить не `CPoint`, а какой-то другой декоратор, и это позволяет не только «обернуть» какую-то точку декоратором, но и обернуть ее в несколько декораторов в разном порядке.

Дальше мы просто работаем со всеми объектами вектора `shapes`, при этом делаем это двумя разными способами — используя итераторы STL во втором случае и нотацию C++11 для итерации по циклам в первом. Нотация с итераторами описана в конце раздела по паттерн «Итератор».

Итак, в чем основная идея паттерна «Декоратор»? Она заключается в динамическом (во время выполнения программы) изменении поведения объекта. Думаю, вы уже догадались, что, несмотря на название «Декоратор», которое намекает на внешний вид, декорирование, украшательства и наш пример, который тоже связан с отрисовкой точки на экране и изменением этой отрисовки, речь в «Декораторе» совсем не обязательно про рисование.

- Декоратор изменяет какую-то (малую?) часть поведения объекта (например, только один метод), не меняя остального поведения (делегуя исходному объекту выполнение всей остальной функциональности).

- ❑ Декоратор подменяет собой исходный объект, потому что обладает тем же самым интерфейсом, что и он. Код, который раньше использовал исходный объект, теперь будет работать с декоратором и не будет этого замечать.
- ❑ Исходный объект может быть «обернут» не в один декоратор, а в несколько. Цепочка декораторов, обернутых вокруг объекта, может изменять поведение того нужной комбинацией, потребность в которой возникла «на лету».
- ❑ Вместо использования декоратора можно было просто унаследоваться от исходного класса и перекрыть нужный метод, получив класс-декоратор. Но если возможных классов-декораторов будет много (для разных вариантов изменения поведения объекта), то попытка собрать из них все возможные комбинации-декораторы, к тому же во всех возможных порядках, приведет к комбинаторному взрыву количества порождаемых классов. Декоратор вместо этого предлагает не порождать новый класс-потомок для каждой нужной комбинации декораторов, а собирать их «на лету» как матрешку.

Второй пункт в этом списке на самом деле не совсем верен. `CDecorator` обладает интерфейсом `CShape`, а не интерфейсом `CPoint`. Поэтому, например, вот этот код не скомпилируется:

```
for (const auto &i : shapes)
    i->move(42, 42);
```

Метод `move()` — это метод `CPoint`, у декоратора его не вызвать. Впрочем, у нас ведь и так вектор не точек, а `CShape`, поэтому можно было бы воспользоваться приведением типов:

```
for (const auto &i : shapes) {
    CPoint *p = dynamic_cast<CPoint*>(i);
    if (p != nullptr)
        p->move(42, 42);
}
```

Проблема в том, что это сработает только для первой точки, которая у нас лежит в векторе «сама по себе». Те точки, которые обернуты в декораторы, откажутся динамически приводиться к `CPoint`. В этом смысле вызывающий код «не заметит», что работает с декоратором, а не с самой точкой, только если он работает через базовый интерфейс `CShape` и ни в коем случае не пытается приводить ничего к конкретному классу: тут его ожидает сюрприз.

В принципе, если заранее предполагать, что вместо объектов могут быть декораторы (но неизвестно, какие именно и сколько вложенных друг в друга), можно было бы обойти эту проблему, организовав у базового объекта интерфейс для обхода всех декораторов:

```
class CShape {
public:
    ...
    virtual CShape* get_real_object() = 0;
    ...
};
```

У точки и у декоратора этот метод можно было бы перекрыть примерно так:

```
class CPoint : public CShape {
    ...
    virtual CShape* get_real_object() override {
        return this;
    }
    ...
};

class CDecorator : public CShape {
    ...
    virtual CShape* get_real_object() override {
        return _shape->get_real_object();
    }
    ...
};
```

При вызове этого метода точка возвращает саму себя, потому что она и есть «настоящая», а декоратор запрашивает «реальный объект» у того объекта, который он декорирует. Теперь можно смело вызывать приведение типов, и оно работает:

```
for (const auto &i : shapes) {
    CPoint *p = dynamic_cast<CPoint*>(i->get_real_object());
    if (p != nullptr)
        p->move(42, 42);
}
```

Может, вместо этого можно было бы просто унаследовать CDecorator от CPoint, а не от CShape? Ведь метод move() тогда появился бы у декоратора автоматически? Да, но в этом случае, если у нас появятся CCircle и CRectangle и им тоже понадобятся те же самые декораторы, их придется дублировать. Кроме того, если CDecorator наследуется от CPoint, то он сам и будет CPoint и возникает неопределенность, а на кого тогда ему хранить указатель?

Диаграмма классов для декоратора аналогична приведенной в книге [7] (рис. 2.14).

Мы видим, она описывает более сложную ситуацию, чем в нашем примере. Авторы книги [7] показывают этой диаграммой классов, что декоратор может не только добавлять и изменять какую-то функциональность у объекта, но и добавлять какое-то свое состояние (т. е. новые данные).

Представьте, что вам требуется добавлять простые объекты-круги на форму и выделять их (например, рамкой). Ваша программа в этом случае будет состоять, скорее всего, из трех частей: в первой части будет описан класс для объектов-кругов, во второй части будет описано хранилище кругов, а в третьей части — основная форма и взаимодействие с пользователем. Если сначала реализовать добавление кругов на форму, а потом посмотреть, что нужно добавить, чтобы реализовать их выделение по щелчку мыши, то можно заметить, что это можно

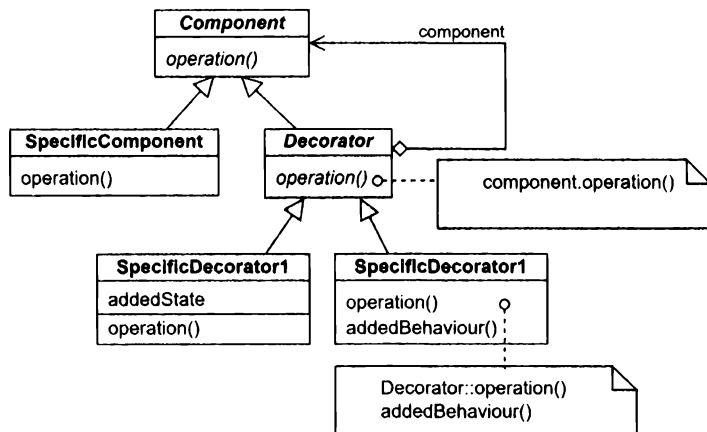


Рис. 2.14. Диаграмма классов паттерна «Декоратор»

сделать несколькими способами, которые отличаются тем, кто должен «помнить» о том, выделен какой-то конкретный объект или нет.

Первый способ: изменить сам объект круга так, чтобы он хранил внутри новый флаг, выделен он или нет:

- ☐ изменяется сам класс круга и его функция отрисовки;
- ☐ хранилище кругов остается неизменным, при отрисовке оно, как и раньше, просто вызывает у всех хранимых объектов функцию отрисовки;
- ☐ основная программа по щелчку мыши выясняет, на какой объект мы кликнули, и меняет у него флаг выделенности.

Но как быть, если, к примеру, сам класс круга уже менять нельзя, а выделение для него реализовать нужно?

Второй способ: изменить хранилище кругов: разделив его на две части, в одной из которых будут храниться выделенные круги, а в другой — все остальные:

- ☐ сам класс круга и его функция отрисовки не изменяются;
- ☐ хранилище кругов при отрисовке вызывает у всех хранимых объектов функцию отрисовки, а для тех объектов, что попали в «выделенную» часть хранилища, дополнительно рисует рамку;
- ☐ основная программа по щелчку мыши выясняет, на какой объект мы кликнули, и помещает его в «выделенную» часть хранилища (или обратно).

Многовато изменений, вам не кажется? Именно тут и может пригодиться третий способ, с помощью декоратора:

- ☐ сам класс круга и его функция отрисовки не изменяются;
- ☐ хранилище кругов остается неизменным, при отрисовке оно, как и раньше, просто вызывает у всех хранимых объектов функцию отрисовки;

- основная программа по щелчку мыши выясняет, на какой объект мы кликнули, и заменяет объект-круг в хранилище на него же, но обернутым в объект-декоратор, который и реализует рисование рамки.

Если посмотреть, сколько изменений вносится в программу во всех трех случаях, то выяснится, что в третьем случае мы при реализации новой возможности (выделение объектов по щелчку мыши) большей частью дописываем новый код, но не меняем уже существующий. Это иллюстрация к принципу О из SOLID (см. разд. 2.28).

2.18. Proxy

Паттерн «Заместитель» — про отложенное создание «тяжелых объектов». И диаграммой классов, и кодом он очень похож на предыдущий паттерн «Декоратор»:

```
class CShape {
public:
    virtual void draw() = 0;
    virtual ~CShape() { };
};

class CPoint : public CShape {
private:
    int _x, _y;
public:
    CPoint(int x, int y) {
        _x = x; _y = y;
    }
    virtual void move(int dx, int dy) {
        _x = _x + dx; _y = _y + dy;
    }
    virtual void draw() {
        printf("CPoint::draw() : drawing (%d, %d)\n", _x, _y);
    }
    ~CPoint() { }
};
```

Базовый абстрактный класс и класс точки — точно такие же, как в прошлом примере, поэтому тут смотреть особо не на что. Точка хранит свои координаты, может изменять их с помощью метода `move()` и может рисоваться.

```
class CProxyPoint : public CShape {
private:
    int _x, _y;
    CPoint *point;
```

```
public:
    CProxyPoint(int x, int y) {
        _x = x; _y = y;
        point = nullptr;
    }
    virtual void draw() {
        if (point == nullptr)
            point = new CPoint(_x, _y);
        point->draw();
    }
    ~CProxyPoint() {
        if (point != nullptr)
            delete point;
    }
};
```

Вот он, заместитель! Класс `CProxyPoint` наследуется от `CShape` так же, как декоратор, но хранит ссылку не на `CShape`, а на `CPoint`, и эта ссылка в конструкторе зануляется, т. е. при создании заместителя ее нет. Но вместо этого в `CProxyPoint` передается и сохраняется вся информация, необходимая для создания настоящего объекта `CPoint`, когда он понадобится.

А когда он реально понадобится? А тогда, когда у точки первый раз вызовут перекрытый метод `draw()`. Именно тогда, если хранимый указатель `point == nullptr`, и будет создан новый объект класса `CPoint`. Все дальнейшие вызовы метода `draw()` приведут только к тому, что будет просто вызываться `CPoint::draw()` у хранимого настоящего объекта.

Таким образом, заместитель реализует то, что называется «отложенное» или «ленивое» создание. Если по каким-то причинам нет возможности создать все 100 500 «тяжелых объектов», а, главное, есть основания считать, что реальная функциональность, работоспособность понадобится не сразу от всех 100 500 объектов, а только от части, а от каких-то объектов может и не понадобится вообще, — вот тогда идея этого паттерна и выстреливает: создавать не сами «тяжелые» объекты, а их «легкие» заместители, обладающие тем же самым интерфейсом. Идея этого паттерна позволяет нам не отслеживать, какой из объектов уже понадобился и был заменен на реальный объект, а какой еще нет. Кто лучше всех способен за этим следить и об этом помнить? Конечно, сам объект!

Мы сталкиваемся с отложенными «тяжелыми» действиями объектов в очень многих случаях. Открывая какую-нибудь страницу при слабом сетевом соединении или папку с большим количеством файлов с большими изображениями, мы сразу видим вместо настоящих картинок их заместители — объекты с рамками вместо изображений, которые постепенно, по мере возможности, заменяются на изображения. Если при этом перемотать в конец папки, браузер может перестать превращать рамки в картинки у первых файлов и переключиться на последние,

потому что именно от них теперь понадобилась, образно говоря, функция `draw()`.

```
void main() {  
    vector<CShape*> shapes(3);  
    shapes[0] = new CPoint(1, 1);  
    shapes[1] = new CProxyPoint(2, 2);  
    shapes[2] = new CPoint(3, 3);  
  
    for (const auto &i : shapes)  
        if (i != nullptr)  
            i->draw();  
  
    vector<CShape*>::iterator i;  
    for (i = shapes.begin(); i != shapes.end(); i++)  
        if (*i != nullptr)  
            delete *i;  
}
```

Здесь мы создаем снова вектор из трех указателей на `CShape`, но создаем только две точки и одного заместителя. Третья точка будет создана только при первом вызове метода `draw()`, а если он никогда не произойдет, то так никогда и не будет создана.

К паттерну «Заместитель» в полной мере применимы те же претензии, что и к «Декоратору». Во-первых, `CProxyPoint` наследуется от «легкого» `CShape`, а не от «тяжелого» `CPoint`, иначе в нем не было бы смысла. Но это же значит, что `CProxyPoint` не обладает интерфейсом `CPoint` и не является им, а значит, его нельзя будет при необходимости «задинамиккастить» к `CPoint`. Во-вторых, `CProxyPoint` приносит пользу только в самом начале своей жизни — откладывая необходимость создания «тяжелого» объекта. После того как последний уже создан, от `CProxyPoint` — сплошной вред: он просто делегирует созданному им `CPoint` вызовы всех методов, замедляя, таким образом, эти вызовы. Если представить себе какой-то контейнер, где `CProxyPoint` изначально хранился, после создания им `CPoint` хорошо было бы вообще, чтобы он смог как-то самого себя в хранилище заменить на свой `CPoint`, а сам самоудалиться.

Итак, заместитель обладает (почти) интерфейсом замещаемого объекта и создаст замещаемый объект только тогда, когда от него потребуется выполнить какую-то тяжелую функциональность (например, загрузить и отрисовать большую картинку). Пока такой функциональности не понадобилось, какие-то методы замещаемого объекта заместитель вполне может выполнять сам. Так, например, заместитель для картинки вполне может сообщать ее размеры без загрузки самой картинки.

Диаграмма паттерна подчеркивает его основную идею: хранение ссылки и создание реального объекта тогда, когда понадобится на самом деле вызвать его функциональность (рис. 2.15).

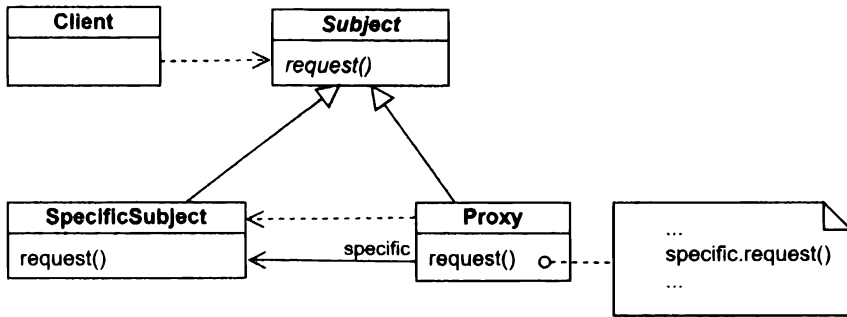


Рис. 2.15. Диаграмма классов для паттерна Proxy

2.19. Composite

Паттерн «Компоновщик» позволит вам в вашем графическом редакторе реализовать такую полезную функциональность, как группировка. Посмотрим, как он этого добивается:

```

class CShape {
public:
    virtual void move(int dx, int dy) = 0;
    virtual void draw() = 0;
    virtual ~CShape() { };
};

class CPoint : public CShape {
private:
    int _x, _y;
public:
    CPoint(int x, int y) {
        _x = x; _y = y;
    }
    void move(int dx, int dy) override {
        _x = _x + dx; _y = _y + dy;
    }
    void draw() override {
        printf("CPoint::draw() : drawing (%d, %d)\n", _x, _y);
    }
    ~CPoint() { }
};

```

Уже привычный нам базовый класс для фигуры и конкретная фигура — точка с координатами. В этом случае в базовом классе объявлены и функция для рисования, и функция для передвижения.


```

class CGroup : public CShape {
private:
    vector <CShape*> _shapes;
public:
    CGroup() {
        printf("CGroup::CGroup()\n");
    }
    void addShape(CShape *shape) {
        _shapes.push_back(shape);
    }
    void move(int dx, int dy) override {
        printf("CGroup::move(%d, %d)\n", dx, dy);
        for (const auto &shape : _shapes)
            shape->move(dx, dy);
    }
    void draw() override {
        printf("CGroup::draw()\n");
        for (const auto &shape : _shapes)
            shape -> draw();
    }
    ~CGroup() {
        printf("start CGroup::~~CGroup()\n");
        for (const auto &shape : _shapes)
            delete shape;
        printf("finish CGroup::~~CGroup()\n");
    }
};

```

Вот он, объект-компоновщик! Это CGroup, который является оберткой над vector. Зачем вообще нужна обертка, если vector и сам хорош? Дело в том, что CGroup наследуется от CShape, как и сам CPoint, а значит, он должен дополнительно реализовывать интерфейс CShape, методы move() и draw().

Объект CGroup хранит внутри вектор указателей на CShape, и когда его просят отрисоваться вызовом метода draw(), он просто вызывает метод draw() у всех хранимых им объектов CShape. Аналогично и для метода move(). Обратите внимание: класс CGroup берет на себя ответственность за время жизни хранимых им объектов, потому что удаляет их в деструкторе.

```

void main() {
    CGroup *shapes = new CGroup;
    shapes->addShape(new CPoint(1,1));
    shapes->addShape(new CPoint(2,2));

    CGroup *childshapes = new CGroup;
    childshapes->addShape(new CPoint(3,3));
    childshapes->addShape(new CPoint(4,4));
}

```

```
shapes->addShape(childshapes);  
  
shapes->move(10,10);  
shapes->draw();  
  
delete shapes;  
}
```

В основной программе мы создаем сначала группу `shapes` и добавляем в нее две точки `CPoint`. Потом создаем еще одну группу, `childshapes`, в которую добавляем еще две точки `CPoint`. А потом помещаем вторую группу в первую! Разве такое возможно? Действительно, метод `CGroup::addShape(CShape *)` требует указатель на `CShape`, а сама группа хранит `vector<CShape*>`, а значит, она может хранить среди них указатели на другие группы, потому что сама группа тоже потомок `CShape`!

Теперь при вызове метода `move()` или `draw()` у группы она просто пробегает по всем своим хранящимся элементам и вызывает у всех элементов соответственно методы `move()` или `draw()`. Суть паттерна «Компоновщик» заключается в том, что:

- ☐ группа `CGroup` хранит у себя коллекцию каких-то других элементов базового класса;
- ☐ но при этом сама является элементом базового класса, поэтому может хранить внутри себя и другие группы;
- ☐ раз она сама является объектом-потомком базового класса, то с ней можно работать как с одним (любым другим) объектом-потомком базового класса;
- ☐ при этом, ведя себя как единый объект, она чаще всего (но не всегда и не обязательно) просто делегирует выполнение этой функциональности реально хранимым объектам.

Обратите внимание при этом, что у нас в примере в основной программе создаются группы, но они складываются в переменные своего собственного класса. Так, у нас написано `CGroup *shapes = new CGroup()`, но не `CShape *shapes = new CGroup()`. Это сделано потому, что метод `addShape()`, который мы хотим вызывать, объявлен и имеется только у группы, а не у `CShape`. По этой же причине мы не можем написать где-то внутри метода `CGroup` что-то вроде `_shapes[i]->addShape(...)`; даже если мы полностью уверены в том, что `_shapes[i]` хранит группу. Посмотрите теперь на диаграмму классов этого паттерна из книги [7] (рис. 2.16).

Как видим, здесь предлагается объявить метод добавления объектов в группу — прямо в базовом классе-предке. Это значит, что этот метод будет у всех объектов-потомков: и у группы, и у конечных объектов. В нашем случае такой метод будет у `CPoint`, но смысл и суть гипотетического метода `CPoint::addShape()` не очень ясна. В этом случае, видимо, понадобится как-то узнать при вызове мето-

да `addShape()`, успешно ли он был вызван, и уж точно нельзя будет его вызывать вот так опрометчиво, как мы делали в нашем примере `x->addShape(new CPoint(1,1))`, чтобы не допустить утечку памяти.

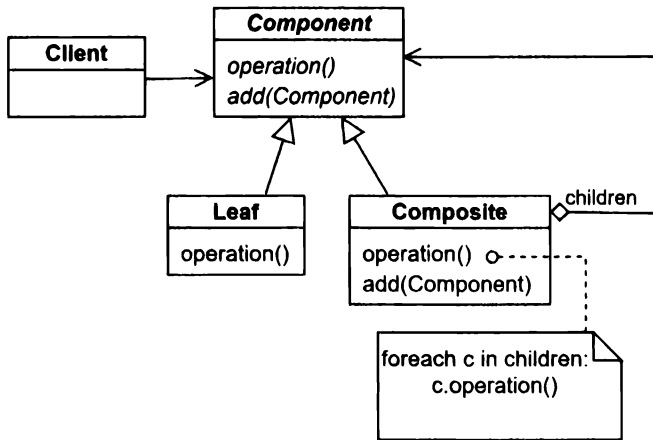


Рис. 2.16. Диаграмма классов паттерна Composite

Паттерн Composite вам понадобится в первую очередь, когда в вашем графическом редакторе вы будете реализовывать такую функциональность, как группировка выделенных объектов. Ожидаемое пользователем поведение заключается в том, что множество выделенных объектов и так ведет себя как единый объект: они перемещаются и изменяются все вместе. Однако при выполнении операции «группировка» они становятся единым объектом в том смысле, что ни один из них теперь нельзя выбрать отдельно, а при выборе одного выделяется сразу весь составной объект. Обратите внимание, что в большинстве случаев сгруппировать можно другие группы (в полном соответствии с идеей этого паттерна), а при разгруппировке объектов группа разваливается на подгруппы, если они там были, а не на отдельные объекты (хотя и такую операцию при желании можно тоже реализовать).

Последнее, на что стоит обратить внимание в этом паттерне, — это подозрительно похожая реализация методов `draw()` и `move()` у группы. Сравните:

```

void move(int dx, int dy) override {
    for (const auto &shape : _shapes)
        shape->move(dx, dy);
}

void draw() override {
    for (const auto &shape : _shapes)
        shape->draw();
}
  
```

Ну правда же, обидно писать код, который так повторяется? По мере появления новых методов у класса `Shape` будут множиться соответствующие методы у `CGroup`, реализация которых чаще всего будет как под копирку. Может быть

можно как-то заменить все эти методы на один? Да, можно, но нам придется подождать паттерна Visitor (см. разд. 2.26).

2.20. Observer сам по себе и как пример mixin

Паттерн «Наблюдатель» — один из самых широко используемых и известных паттернов, причем совсем не обязательно даже в рамках объектно-ориентированной программы. Суть его в том, что он реализует такой механизм, как подписка. Зачем вообще подписка нужна? Дело в том, что когда нам нужна какая-то информация, мы можем не знать, когда она появится.

Предположим, вы — студенты, на носу сессия, и вы, конечно, хотите узнать список вопросов к экзамену по ООП. А преподаватель еще не приготовил список таких вопросов. Вы, конечно, можете все вместе и по очереди каждый день доставать преподавателя вопросами вида: «Уже готов список билетов к экзамену?», — но понятно, что преподаватель не будет радоваться каждый день отвечать на одно и то же, да и вам это будет утомительно.

Что вместо этого предлагает паттерн «Наблюдатель»? Он предлагает сообщить преподавателю, что вы ждете список билетов к экзамену, и передать ему необходимую информацию для того, чтобы оповестить вас тогда, когда это событие произойдет (т. е. список вопросов появится). Паттерн «Наблюдатель» нужен тогда, когда требуется развернуть поток информации в обратном направлении, потому что потребитель этой информации не может ее успешно запрашивать, потому что просто не знает, когда уже можно запрашивать, а когда еще нельзя.

Подписка и принцип подписки с нами везде. Наши смартфоны подписываются на сайт Госуслуг, чтобы вовремя узнать о назначенном штрафе, а студенты подписываются в YouTube на канал преподавателя, чтобы добавить тому просмотры и успешно сдать экзамен. Давайте смотреть на код:

```
class CObject;
```

```
class CObserver {  
public:  
    virtual void onSubjectChanged(CObject *who) = 0;  
    virtual ~CObserver() { }  
};
```

Класс CObject — это какой-то объект, за которым можно наблюдать, с которым может что-то произойти, требующее нашей реакции. Класс CObserver — это класс-наблюдатель, который должен отреагировать на что-то, что может произойти с CObject. Наблюдатель знает, как реагировать, но не знает когда. Объект знает, когда уже надо реагировать, но не знает, как именно. Паттерн «Наблюдатель» разводит их по разным классам и позволяет гибко, уже в процессе выполнения программы, связывать вместе тех, кто знает «когда», и тех, кто знает «как».

У абстрактного класса-наблюдателя `CObserver` нет ничего, кроме виртуальной абстрактной функции `onSubjectChanged`, которая и будет перекрываться и выполнять необходимые для реакции действия.

```
class CObject {
private:
    vector<CObserver*> _observers;
public:
    void addObserver(CObserver *o) {
        _observers.push_back(o);
    }
    void notifyEveryone() {
        for (const auto &o : _observers)
            o->onSubjectChanged(this);
    }
    virtual ~CObject()
    { }
};
```

Класс-объект `CObject`, за которым можно наблюдать, раньше у нас был заявлен, но не определен, а вот теперь пришло его определение. Наблюдаемый объект `CObject` хранит вектор наблюдателей `_observers` и позволяет в него добавляться другим наблюдателям с помощью своей функции `addObserver()`. По сути, в этой идее подписаться на кого-то означает попросить его добавить себя в список подписчиков: именно это и происходит в функции `addObserver()`.

Еще у класса-объекта `CObject` есть метод `notifyEveryone()`, который проходит по всем наблюдателям, хранящимся в списке наблюдателей, и уведомляет их всех, вызывая по очереди у каждого его метод `onSubjectChanged()`. Собственно, на этих двух методах вся суть паттерна «Наблюдатель» и исчерпывается, но давайте посмотрим, как он работает.

```
class CAnimal : public CObject {
public:
    string name;
    string lastsound;
    CAnimal(char *aname) {
        name = aname;
        lastsound = "";
    }
    void say(string s) {
        lastsound = s;
        printf("CAnimal %s : '%s'\n",name.c_str(), s.c_str());
        notifyEveryone();
    }
};
```

Пример наблюдаемого объекта — какое-то животное в зоопарке. Класс `CAnimal` наследуется от `CObject`, добавляет какое-то свое внутреннее состояние в виде

имени и последнего произнесенного звука. Когда с объектом класса `CAnimal` что-то происходит, например оно издает звук методом `say()`, то все наблюдатели должны незамедлительно на это отреагировать. Поэтому в этом методе, как и во всех остальных подобных, класс `CAnimal` вызывает свой собственный унаследованный метод `notifyEveryone()`, в котором, как мы помним, вызывается метод `onSubjectChanged()` у каждого наблюдателя.

```
class CChild : public CObserver {
public:
    string name;
    CChild(string cname) {
        name = cname;
    }
    void onSubjectChanged(CObject *who) {
        printf("CChild %s : 'Hurray! %s had just said : %s'\n",
            name.c_str(),
            ((CAnimal*)who)->name.c_str(),
            ((CAnimal*)who)->lastsound.c_str());
    }
};
```

А вот и пример конкретного наблюдателя: ребенок в зоопарке. У него тоже есть имя, и он знает, как реагировать на животных в зоопарке, но не знает, на кого и когда именно. Все, что у него есть, — это заготовленная реакция на это событие, описанная в его перекрытом методе `onSubjectChanged()`, унаследованном от `CObserver`.

```
void main() {
    CAnimal cat("Zorra");
    CAnimal dog("Tortilla");
    CAnimal unknown("Cheburashka");

    CChild vasya("Vasya");
    CChild petya("Petya");
    CChild kolya("Kolya");

    cat.addObserver(&vasya);
    cat.addObserver(&petya);
    dog.addObserver(&kolya);

    cat.say("mew-mew!");
    dog.say("wow-wow!");
    cat.say("wow-wow!");
    dog.say("mew-mew!");
    unknown.say("nobody cares about me");
}
```

Вот она, основная программа, где создаются наблюдаемые объекты `CAnimal`, создаются наблюдатели `CChild`, а затем какие-то из наблюдателей подписываются ждать изменений (т. е. наблюдать) в каких-то наблюдаемых объектах. Важно,

что кто на кого подписывается, становится известно только во время выполнения программы и может зависеть от какой-то дополнительной информации (о том, кто близко к кому подошел, и т. д.).

После этого, стоит какому-то из наблюдаемых объектов что-нибудь сказать, как мы незамедлительно увидим реакцию подписанных на него наблюдателей. При этом обратите внимание, что в метод `onSubjectChanged(Object *who)` наблюдателя передается информация о том, кто этот метод вызывает: это делает наблюдаемый объект `o->onSubjectChanged(this)` в своем методе `notifyEveryone()`. Поэтому наблюдатель узнает не только сам факт того, что произошло какое-то событие, требующее его реакции, но и то, кто это событие «испустил». Это позволяет одному наблюдателю подписаться на несколько наблюдаемых объектов и не путаться, на кого он реагирует.

Посмотрим теперь на диаграмму классов из книги [7] (рис. 2.17).

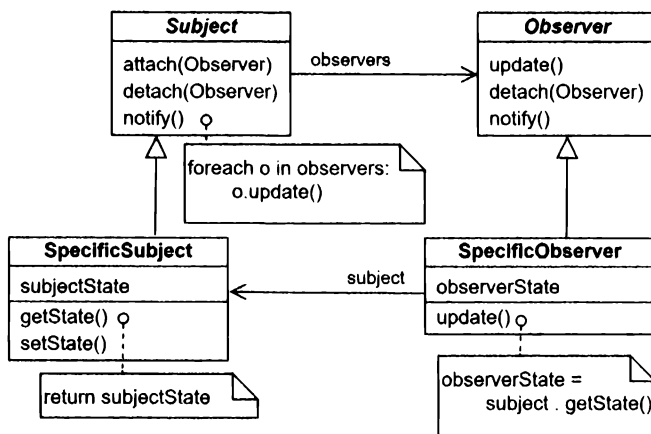


Рис. 2.17. Диаграмма классов паттерна Observer

Названия слегка перепутаны, но общую идею проследить можно. `Subject` и его метод `Notify()` на картинке — это наш класс `CObject` (при всем уважении к авторам, объект, за которым наблюдают, он точно объект, но никак не субъект) и его метод `notifyEveryone()`, `ConcreteSubject` — это наш `CChild`. `Observer` и его функция `Update()` у нас называются `CObserver` и `onSubjectChanged()`. Обратите внимание, что в книге метод `Update`, в отличие от нашего метода `onSubjectChanged()`, не принимает никаких параметров. Это значит, что наблюдатель должен помнить того, за кем он наблюдает, — это мы и видим в виде ссылки `subject`. Кстати, это проектное решение не позволяет наблюдателю наблюдать за несколькими объектами сразу. Ну, еще в книге в явном виде указано, что у наблюдаемого объекта есть какое-то состояние `subjectState` и что это состояние наблюдатель переносит себе (у нас аналогом такого состояния является атрибут `lastsound`).

Таким образом, паттерн «Наблюдатель» позволяет организовывать динамические связи между объектами, когда они объекты не «достают» другие объекты

постоянными запросами, а подписываются на их изменения и спокойно уведомляются тогда, когда происходит нечто, на что они должны отреагировать.

Могут два объекта одновременно выступать в роли наблюдателей и наблюдаемых, т. е. подписываться на изменения друг друга? Конечно, могут, но тут возникают две проблемы, которые нужно решать (первая актуальная и без взаимной подписки):

- ❑ при удалении объекта-наблюдателя он обязательно должен «отписываться» от всех, на кого он подписался, потому что после удаления объекта-наблюдателя его адрес становится невалидным, и наблюдаемые объекты не должны по нему обращаться;
- ❑ при взаимной подписке объектов друг на друга нужно избежать бесконечного цикла взаимных обновлений об изменении состояния.

Последнее, что можно сказать про этот паттерн, — он является, на мой взгляд, хорошим примером такого понятия, как `mixin`. Посмотрите на класс `CObject` — по факту, это и есть `mixin`, т. е. класс, не предназначенный для самостоятельной работы, но предназначенный для того, чтобы «подмешиваться» путем наследования к другим классам, принося с собой функциональность подписки в виде списка наблюдателей `_observers`, метода уведомления `notifyEveryone()` и нескольких методов для управления подпиской, типа `addObserver(CObserver *o)` и `removeObserver(CObserver *o)`.

2.21. Command

Теперь — к паттерну «Команда». Один из самых клевых, на мой взгляд, паттернов, с крутой идеей и множеством последствий, к которым она приводит. Как обычно, берем программу и начинаем читать.

```
#include <conio.h>
#include <map>
#include <stack>
using namespace std;

class Point {
private:
    int _x, _y;
public:
    Point(int x, int y) {
        _x = x; _y = y;
    }
    void move(int dx, int dy) {
        _x = _x + dx; _y = _y + dy;
    }
}
```



```

void report() {
    printf("Point is: %d %d\n", _x, _y);
}
virtual ~Point() { }
};

```

Что мы тут видим? Класс для описания точки с двумя координатами, конструктором, который координаты инициализирует методом для сдвига точки (сиречь изменения координат на заданное приращение) и методом для рисования точки на экране. Как? Вы считаете, что это нельзя назвать рисованием точки в заданной точке экрана? Включите абстрактное мышление и воображение! Пока все понятно. Читаем дальше.

```

class Command {
public:
    virtual void execute(Point *selection) = 0;
    virtual void unexecute() = 0;
    virtual Command *clone() = 0;
    virtual ~Command() {};
};

```

А вот здесь уже все серьезно. Абстрактные классы нам зачем даны? Чтобы описать то общее, чем будут объединены все их потомки. Значит, у нас будет много частных случаев «Команды», много команд, но что станет их все объединять? То, что у всех их можно будет:

- ☐ вызывать метод `execute`, передавая туда указатель на какую-то существующую точку, и они будут ее казнить;
- ☐ вызывать метод `unexecute` непонятного назначения;
- ☐ вызывать метод `clone` без параметров, и они будут возвращать указатель на какую-то другую команду.

Негусто, но с абстрактными классами всегда все так... абстрактно. Читаем дальше.

```

class Command {
public:
    virtual void execute(Point *selection) = 0;
    virtual void unexecute() = 0;
    virtual Command *clone() = 0;
    virtual ~Command() {};
};

```

```

class MoveCommand: public Command {
private:
    Point *_selection;
    int _dx; int _dy;

```

```
public:
    MoveCommand(int dx, int dy) {
        printf("MoveCommand::MoveCommand(%d, %d)\n", dx, dy);
        _dx = dx;
        _dy = dy;
        _selection = nullptr;
    }

    void execute(Point *selection) override {
        printf("MoveCommand::execute(Point *selection)\n");
        _selection = selection;
        if (_selection != nullptr) {
            _selection -> move(_dx, _dy);
        }
    }

    void unexecute() override {
        printf("MoveCommand::unexecute()\n");
        if (_selection != nullptr) {
            _selection -> move(-_dx, -_dy);
        }
    }

    Command *clone() override {
        printf("MoveCommand::clone()\n");
        return new MoveCommand(_dx, _dy);
    }

    ~MoveCommand() {
        printf("MoveCommand::~MoveCommand()\n");
    }
};
```

Наконец что-то конкретное, а не абстрактное! Это конкретный потомок абстрактного класса `Command`, а именно класс `MoveCommand`. У него есть свойство для хранения указателя на какую-то точку и два целочисленных свойства для хранения смещения. В конструктор передаются и инициализируются свойства и зануляется указатель на точку, а вот дальше посмотрим повнимательнее еще раз:

```
void execute(Point *selection) override {
    printf("MoveCommand::execute(Point *selection)\n");
    _selection = selection;
    if (_selection != nullptr)
        _selection -> move(_dx, _dy);
}
```

В метод `execute` у команды передается точка. В методе она запоминается во внутреннем указателе, и у нее вызывается метод `move` с тем смещением `_dx` и `_dy`, которое хранилось в команде. Спрашивается, зачем кому-то вызывать метод `execute`, чтобы вызвался метод `move` у точки, если можно было просто вызвать самостоятельно метод `move` у той же самой точки? Непонятно. Читаем внимательно дальше.

```
void unexecute() override {
    printf("MoveCommand::unexecute()\n");
    if (_selection != nullptr) {
        _selection->move(-_dx, -_dy);
    }
}
```

Что произойдет, если вызвать у такой команды метод `unexecute`? Как видим, команда просто вызывает тот же самый метод `move` у точки (какой точки? да той, которая должна храниться к этому моменту в указателе `_selection`), но с обратными знаками смещений. Если для одной и той же точки вызвать сначала `execute`, а потом `unexecute`, то координаты точки не изменятся. Это важно.

```
Command *clone() override {
    printf("MoveCommand::clone()\n");
    return new MoveCommand(_dx, _dy);
}
```

Ну и наконец, метод `clone`. Мы видим, что если его вызвать у команды `MoveCommand`, то он создает и возвращает новый, но точно такой же экземпляр класса `MoveCommand`. Вам это не напоминает какой-то другой паттерн, который вы уже видели?

Ну, как бы и все. Мы надували щеки, ходили вокруг да около одного и того же метода `move` у точки. Но ведь чтобы создать и сдвинуть точку, нужно всего лишь вызвать у нее метод `move`? Как-то так:

```
Point *selection = new Point(0,0);
selection->move(42, 42);
```

Нет! Нельзя просто так ~~взять и засунуть лампочку в рот~~ сдвинуть точку! У нас есть еще целая главная функция, давайте читать ее:

```
void main() {
    map<char, Command*> commands;
    commands['a'] = new MoveCommand(-10,0);
    commands['d'] = new MoveCommand(10,0);
    commands['w'] = new MoveCommand(0,-10);
    commands['s'] = new MoveCommand(0,10);

    Point *selection = new Point(0,0);
    selection->report();

    stack<Command*> history;

    char key;
    do {
        key = _getch();
        Command *command = commands[key];
```

```
if (command != nullptr) {
    Command *newcommand = command->clone();
    newcommand->execute(selection);
    history.push(newcommand);
}

if (key == 'z' && !history.empty()) {
    Command *lastcommand = history.top();
    lastcommand -> unexecute();
    delete lastcommand;
    history.pop();
}

selection -> report();

} while (key != 27);

while (!history.empty()) {
    delete history.top();
    history.pop();
}

delete commands['a'];
delete commands['d'];
delete commands['w'];
delete commands['s'];

delete selection;

system("pause");
}
```

Сначала мы создаем ассоциативный массив для указателей на команды, где привязанные к символам 'a', 'd', 'w' и 's' будут лежать четыре экземпляра класса MoveCommand, отличающиеся друг от друга смещениями. Это будут четыре прообраза для четырех возможных движений точки во все стороны света. Потом мы создаем собственно точку и рисуем ее на экране. А потом мы делаем что-то совсем странное: мы создаем стек для команд. Стек, как известно, это такая несправедливая структура данных, что кто в нее пришел последний, тот первый из нее вышел. А дальше начинается основной цикл ~~обработки сообщений~~ программы, и в нем:

```
key = _getch();
Command *command = commands[key];
if (command != nullptr) {
    Command *newcommand = command->clone();
```

```
newcommand->execute(selection);
history.push(newcommand);
}
```

Читаем с клавиатуры символ, нажатый пользователем, и ищем по коду символа в ассоциативном массиве команду. Если такая команда нашлась, то мы делаем три вещи:

- ❑ вызываем ее метод `clone`, который, как мы помним, создает и возвращает новый, точно такой же ~~как прототип~~ экземпляр команды, и я снова забыл, как этот паттерн называется;
- ❑ у полученного нового экземпляра команды, помещенного в переменную `newcommand`, вызываем метод `execute`, передавая в него нашу единственную точку;
- ❑ новый экземпляр команды заталкивается в стек с говорящим названием `history`.

```
if (key == 'z' && !history.empty()) {
    Command *lastcommand = history.top();
    lastcommand->unexecute();
    delete lastcommand;
    history.pop();
}
```

А вот если нажата клавиша `'z'`, то делается кое-что хитрое. Из стека `history`, если он не пустой, вытаскивается верхний объект-команда, у нее вызывается метод `unexecute`, и эта команда удаляется.

За пределами моего обзора осталась перерисовка точки после каждой выполненной команды и выход их цикла по нажатой клавише `Esc`, но в этом вы наверняка и так разобрались сами.

В итоге, что позволяет делать эта программа? Она рисует точку на экране и позволяет перемещать ее с помощью клавиш. При этом если в любой момент нажать клавишу `'z'`, то последнее действие будет отменено (какое бы оно ни было), а если снова нажать клавишу `'z'`, то будет отменено предыдущее действие и т. д.

Так что же, паттерн «Команда» — он просто про отмену действий? Нет, это слишком близорукий взгляд на вещи. Давайте подумаем и попробуем понять идею того, что мы сделали. Еще раз поднимем самый главный в этом паттерне вопрос. Почему нельзя просто создать объект и вызвать у него нужный метод?

```
Point *selection = new Point(0,0);
selection->move(42, 42);
```

Можно! Но всё дело в том, что вызов метода — это очень негибкая штука по сравнению, например, с объектом. Метод можно только вызвать и не вызвать, и все. А объект, как говорится, является *first class citizen*. Поэтому вместо такого простого кода нужно использовать вот такой, более сложный:

```
Point *selection = new Point(0,0);  
Command *command = new CMoveCommand(42,42);  
command->execute(selection);
```

Суть почти всех за редкими исключениями паттернов заключается в том, чтобы отнять и поделить: что-нибудь откуда-нибудь вытащить и инкапсулировать как отдельный объект. Так вот, суть паттерна «Команда» в том, что нельзя просто так брать и вызывать метод у объекта. Надо, чтобы каждый вызов метода представлял собой отдельный специальный объект! Обратите внимание: не метод `move` превращается в объект класса `CMoveCommand`, а конкретный вызов этого метода превращается в отдельный объект. Собрались вызывать метод `move` 20 раз — значит, надо создать 20 объектов класса `CMoveCommand`.

Что это дает и почему это важно. Представьте, что вы — директор маленького стартапа разработчиков-влогеров (хотел написать «разработчиков-программистов», но, как известно, сегодня эта профессия уже выходит из моды и все дети хотят стать влогерами). Утром вы приходите на работу и лично подходите к каждому своему сотруднику и говорите: «Арсений, ты сегодня снимаешь сюжет про то-то. Ты, Света, сегодня снимаешь сюжет про се-то. А ты, Денис, сегодня должен два сюжета снять про то, как они будут снимать сюжеты». Они сразу говорят: «Будет сделано, Олег Сергеевич» и начинают... эээ... работать (это у влогеров все равно так называется?).

На что это похоже в мире объектов? На прямой вызов метода. Есть тот, кто вызывает, и есть тот, кто сразу исполняет. Директивно, сразу, безо всяких посредников.

Потом вы получили MBA и начинаете внедрять новые технологии непрямого управления. Вместо того, чтобы утром подходить к каждому сотруднику, вы нанимаете ассистента и кожаный диван, и удобно на нем (диване, конечно) развалившись, утром пишете бумажки с поручениями. На одной бумажке — задача для Арсения, на другой — для Дениса, на третьей — для Светланы. Вы остаетесь на диване, а ассистент разносит бумажки по исполнителям, которые точно так же начинают работать.

В чем разница? Между вами и исполнителями появляется бумажка-задание, разрывается эта жесткая связь между тем, кто дает задание, и тем, кто его исполняет. Исполнитель может припоздать на работу — он увидит бумажку с заданием чуть позже и начнет работать. Директор может уехать на Таити в отпуск, наготовив достаточно бумажек с заданиями, — ассистент вполне справится с раздачей этих бумажек каждое утро, ничего не понимая в теме. Директор может не думать, кому поручить конкретное задание, — новомодная agile-команда сама разберется, кто его будет выполнять. Исполнитель, если он приболел, может перенаправить бумажку кому-то другому. Директор не сможет сказать, что он поручал совсем другое, а исполнитель не сможет сказать, что ему ничего не поручали, — вот она, бумажка с конкретным заданием. Ассистент может по пути создать копию этой бумажки и отправить в налоговую сложить в сейф для истории. Наконец, в дальнем углу сейфа могут лежать три конверта с тремя бумаж-

ками, как в известном анекдоте, помните? Этот анекдот, ведь он на самом деле — про паттерн «Команда»!

Итак, переводим всю эту аналогию обратно на язык классов, объектов и методов. Каждый конкретный объект-команда — это замена одного вызова метода. Да, неудобно: лишний уровень indirection. Но зато с объектом, в отличие от метода, можно делать все что угодно: отдать куда-то, получить откуда-то, сохранить, загрузить, скопировать и т. д. Это позволяет:

- разорвать жесткую связь между тем, кто инициирует действие, и тем, кто его будет выполнять. Тот, кто инициирует, не обязан знать того, кто будет выполнять, и наоборот;
- отложить, разнести во времени инициацию действия и его исполнение. Действия заготовлены заранее и отправлены по нужным адресатам. В час X действие запускается на выполнение, и в этот момент уже даже может быть неизвестно, что это за действие;
- сохранять, загружать, логгировать, клонировать, универсально работать со всеми действиями, не зная, что это конкретно за действия;
- добавлять новые действия, расширяя, а не изменяя существующий код.

Последний пункт очень важен. Что если я захочу добавить еще одну команду — по клавише <O> перемещаться в начало координат? Я, очевидно, добавлю нового потомка для `Command`: команду `ToOriginCommand`, я создам один объект для этой команды и помещу его в наш ассоциативный массив. Как изменится при этом основной цикл программы? Никак! Вот в этом вся и фишка — основная часть программы ничего не знает про конкретные команды, она никак от них не зависит и никак не меняется при добавлении новых команд. Команды как обработчики событий могут быть описаны в любом порядке, это ни на что не влияет. Программа только дописывается, расширяется для добавления новой функциональности, но не изменяется. Помните вторую букву в SOLID? Ну так она как раз про это!

На сдачу, как побочный эффект, мы получаем легкость реализации `undo`, отмены действий. Если все, что происходит, происходит в результате выполнения последовательности атомарных команд, каждая из которых умеет отменять, откатывать свое действие, то, сохранив копии этих действий и их порядок выполнения, мы всегда можем «проиграть» эту цепочку в обратном порядке и откатиться назад.

Вот классическая диаграмма классов для паттерна «Команда» (рис. 2.18). С `Command` и `SpecificCommand`, я думаю, и так все ясно, но будет поучительно, если вы попыдаетесь найти, где у нас в программе `Invoker`, `Client` и `Receiver`. Да, у нас отдельных классов таких нет, но мы же должны за деревьями лес видеть? Найдите в нашей программе те части, которые отвечают за создание команд (`Client`), за хранение и вызов команд (`Invoker`) и за получение и исполнение команд (`Receiver`), убедитесь, что они достаточно друг от друга независимы.

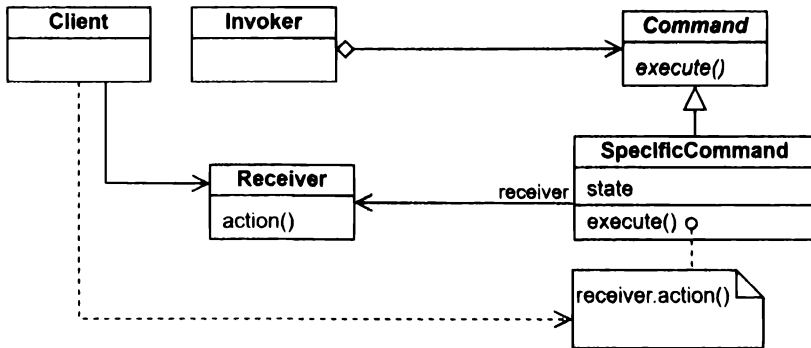


Рис. 2.18. Диаграмма классов паттерна «Команда»

2.22. Factory Method, Abstract Factory

Два следующих паттерна, «Фабричный метод» и «Абстрактная фабрика», относятся к семейству порождающих паттернов, т. е. описывают очередной хитро-вывернутый способ создания объектов. Один из самых простых способов проиллюстрировать их полезность — попробовать реализовать контейнер объектов, умеющих сохранять и загружать свое содержимое из файла.

Казалось бы, что сложного? Кто лучше всех знает, как загружать и восстанавливать свое состояние из текстового файла? Разумеется, сами объекты. Поэтому, чтобы сохранить все объекты какого-нибудь контейнера в файл, надо у каждого из них вызвать какую-то виртуальную функцию:

```
for (const auto &s : shapes)
    s->save(file);
```

Учитывая, что `save()` — наверняка виртуальная функция, не важно, какие именно классы объектов хранятся в контейнере `shapes`, вызовется именно функция `save()` нужного класса, и он правильно сохранит себя в файл. А вот с загрузкой все как-то сложнее:

```
for (const auto &s : shapes)
    s->load(file);
```

Это будет работать, если в контейнер уже созданы и помещены соответствующие объекты. Но мы только при чтении объекта узнаем, какой объект создать, значит, сначала нужно прочитать код, и только потом создать объект, чтобы было у кого вызывать функцию `load()`. Этим мы и займемся:

```
class CShape {
public:
    virtual void save(FILE *stream) = 0;
    virtual void load(FILE *stream) = 0;
    virtual ~CShape() { }
};
```


На этот раз абстрактный класс описывает фигуру, которая умеет сохраняться и загружаться из какого-то файлового потока, для этого и объявлены две соответствующие функции.

```
class CShapeArray {
private:
    vector<CShape*> shapes;
public:
    virtual CShape *createShape(char code) {
        return nullptr;
    }
    void loadShapes(string filename) {
        FILE *stream;
        int count;
        char code;
        CShape *shape;

        if ((stream=fopen(filename.c_str(), "r")) != nullptr) {

            fscanf(stream, "%d\n", &count);
            for (int i = 0; i < count; i++) {
                fscanf(stream, "%c\n", &code);
                shape = createShape(code);
                if (shape != nullptr) {
                    shape->load(stream);
                    shapes.push_back(shape);
                }
            }
            fclose(stream);
        }
    }
    virtual ~CShapeArray() {
        for (const auto &s : shapes)
            delete s;
    }
};
```

А вот и собственно класс-контейнер. Он инкапсулирует вектор указателей на потомков CShape, в деструкторе удаляет все хранимые объекты и реализует две функции: createShape() и loadShapes(). Функция createShape, на первый взгляд, очень странная: возвращать она должна указатель на CShape, а название у нее такое, как будто она должна создавать объекты-потомки класса CShape по переданному ей коду-символу, однако она ничего этого не делает, а всегда возвращает nullptr. Эта функция вполне могла бы вообще быть абстрактной.

Вторая функция — функция loadShapes(). Она берет имя файла, открывает его для чтения и первым делом читает строку с количеством будущих элементов

в хранилище. Затем в цикле по количеству элементов, она читает строку с кодом элемента в символьную переменную `code`, затем этот код передается в функцию `createShape()`, которая должна создать объект по этому коду, и если этот объект создан, то у него вызывается функция `load()`, которая должна прочитать все нужное содержимое этого объекта из файла, а сам объект добавить в контейнер.

Как будто, все выглядит почти логично, только не понятно, как это должно работать, ведь `createShape()` всегда возвращает `nullptr`, и поэтому условие никогда не работает. Но это все присказка, а сказка будет впереди.

```
class CPoint : public CShape {
private:
    int _x, _y;
public:
    CPoint(int x, int y) {
        _x = x;
        _y = y;
    }
    void save(FILE *stream) override {
        fprintf(stream, "P\n");
        fprintf(stream, "%d %d\n", _x, _y);
    }
    void load(FILE *stream) override {
        fscanf(stream, "%d %d\n", &_x, &_y);
    }
};

class CCircle : public CShape {
private:
    int _x, _y, _r;
public:
    CCircle(int x, int y, int r) {
        printf("CCircle::CCircle(int x, int y, int r)\n");
        _x = x;
        _y = y;
        _r = r;
    }
    void save(FILE *stream) override {
        fprintf(stream, "C\n");
        fprintf(stream, "%d %d %d\n", _x, _y, _r);
    }
    void load(FILE *stream) override {
        fscanf(stream, "%d %d %d\n", &_x, &_y, &_r);
    }
};
```

Вот два конкретных потомка CShape, классы CPoint и CCircle. Они перекрывают методы save() и load(), а значит, умеют сохранять и загружать свое собственное состояние из файлового потока.

```
class CMyShapeArray : public CShapeArray {
public:
    CShape *createShape(char code) override {
        CShape *shape = nullptr;
        switch (code) {
            case 'P':
                shape = new CPoint(0,0);
                break;
            case 'C':
                shape = new CCircle(0,0,0);
                break;
            default:;
        }
        return shape;
    }
};
```

А вот и потомок CShapeArray приехал, класс CMyShapeArray, который перекрывает функцию createShape() заново, и теперь она больше не возвращает всегда nullptr, а в зависимости от переданного ей символа-кода создает и возвращает новый объект класса CPoint или CCircle.

```
void main() {
    const string filename = "d:\\data.txt";
    FILE *stream;
    CShape *shape;

    if ((stream=fopen(filename.c_str(), "w" )) != nullptr) {
        fprintf(stream, "%d\n", 2);
        shape = new CPoint(10,20);
        shape->save(stream);
        delete shape;
        shape = new CCircle(30,40,5);
        shape->save(stream);
        delete shape;
        fclose(stream);
    }

    CMyShapeArray shapeArray;
    shapeArray.loadShapes(filename);
}
```

В основной программе мы сначала готовим файл data.txt, в который просим сохраниться один объект класса CPoint и один же объект класса CCircle. После

подготовки файла мы создаем контейнер `CMyShapeArray` и предлагаем ему восстановить свое содержимое из файла `data.txt`, что он успешно и делает.

На первый взгляд может показаться, что мы наплодили какие-то лишние классы. Зачем, например, нужен был абстрактный класс `CShapeArray`, если мы все равно не создавали его объекты, а только наследовались? Разве нельзя было реализовать его примерно так, реализовав функцию `createShape()` прямо в нем:

```
class CShapeArray {
private:
    vector<CShape*> shapes;
public:
    CShape *createShape(char code) override {
        CShape *shape = nullptr;
        switch (code) {
            case 'P':
                shape = new CPoint(0,0);
                break;
            case 'C':
                shape = new CCircle(0,0,0);
                break;
            default:;
        }
        return shape;
    }
    void loadShapes(string filename) {
        FILE *stream;
        int count;
        char code;
        CShape *shape;

        if ((stream=fopen(filename.c_str(), "r")) != nullptr) {

            fscanf(stream, "%d\n", &count);
            for (int i = 0; i < count; i++) {
                fscanf(stream, "%c\n", &code);
                shape = createShape(code);
                if (shape != nullptr) {
                    shape->load(stream);
                    shapes.push_back(shape);
                }
            }
            fclose(stream);
        }
    }
    virtual ~CShapeArray() {
        for (const auto &s : shapes)
```

```

delete s;
}
};

```

Как будто в этом случае никаких потомков ему не понадобилось бы! А мы зачем-то намеренно отказались от функции `createShape()`, сделав ее пустой. Проблема такой реализации функции `createShape()` заключается в том, что в ней жестко прописаны классы `CPoint` и `CCircle`, и никакие другие она создавать не умеет, следовательно, наш класс `CShapeArray` никакие другие загружать не сможет, а это, безусловно, косяк.

Именно поэтому мы реализовали `CShapeArray` без упоминания и какой-либо привязки к конкретным классам фигур (и вообще в отдельном файле), и эта привязка осуществляется уже в потомках `CShapeArray`, которые перекрывают функцию `createShape()`, но не трогают функцию `loadShapes()`, которая и так уже написана правильно.

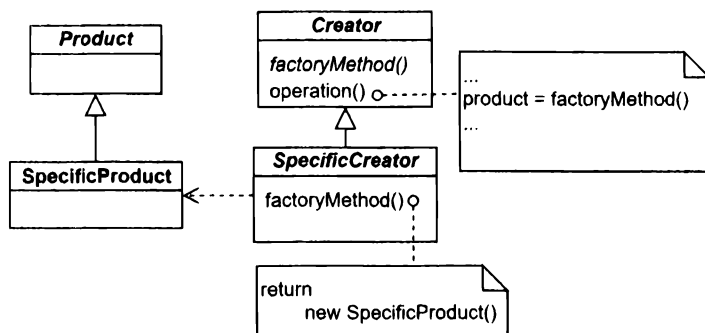


Рис. 2.19. Диаграмма классов паттерна «Фабричный метод»

Где в нашей программе `Product` и `ConcreteProduct` (рис. 2.19)? Разумеется, это `CShape` и парочка `CPoint` и `CCircle`. Где в нашей программе фабричный метод и почему он фабричный? Фабрика в реальной жизни — это что-то производящее продукты, а фабричный метод — это метод, производящий объекты. У нас это, конечно же, `createShape()`. Метод `AnOperation()` с картинки соответствует нашему `loadShapes()`.

В чем идея этого паттерна? Какой особенный способ создания объектов предлагает нам этот порождающий паттерн? Идея фабричного метода заключается в следующем: если вам нужно создавать объекты, не создавайте их с помощью `new`, прописывая конкретный класс. Вынесите это создание в отдельный виртуальный метод (который тогда будет называться фабричным) и вызывайте его тогда, когда нужен будет новый экземпляр. Если вдруг потомки перекроют этот метод и научат его создавать новые объекты, то весь тот код, который фабричный метод использовал, неожиданно для себя научится создавать новые объекты, про которые он ничего не знал.

В нашем примере мы добиваемся того, что наше хранилище сможет создавать и загружать из файла объекты тех классов, про которые оно ничего не знало на

момент своего создания. Это происходит благодаря тому, что мы создаем маленький класс-наследник, который перекрывает `createShape()` и наполняет его нужными классами.

Итак, фабричный метод — это способ создавать объекты каких-то неизвестных нам классов, про которые знает только сам фабричный метод. Он инкапсулирует, прячет в себе логику выбора того, какой конкретно объект создавать, и этим он хорош. Единственная его проблема — что он метод. Методы — это не *first class citizens*, их нельзя ни передать в функцию, ни вернуть из функции, ни сложить в переменную, ни сравнить друг с другом, ни переименовать (если что, это только что было определение понятия «объект первого класса»).

Из-за этого, если понадобится куда-то передать или где-то сохранить наш метод, умеющий создавать объекты `CPoint` и `CCircle`, придется передавать объект `CMyShapeArray`, т. е. целый контейнер! Кроме этого, если подумать о самом `CMyShapeArray`, то это уже не «хранилище», а «хранилище-создавалище», оно предназначено не только для хранения и группировки элементов, но и для их порождения. Забегая вперед (до раздела 2.28), можно сказать, что это явное нарушение принципа S из SOLID — принципа единственной ответственности.

Как бы нам разделить эту функциональность по разным классам, чтобы был класс, который порождает объекты, а был тот, что хранит объекты. Собственно, это и есть ответ. Раз мы не можем передавать туда-сюда сам фабричный метод, нужно его обернуть в объект. То есть вынести его из `CShapeArray` в отдельный класс, в котором будет только этот сам фабричный метод. Этот класс и будет называться фабрикой.

Фабричный метод — это метод, создающий объекты. Фабрика — это объект, в котором есть только фабричные методы, т. е. объект, умеющий создавать другие объекты.

```
class CShapeFactory {
public:
    virtual CShape *createShape(char code) = 0;
    virtual ~CShapeFactory() { }
};
```

Вот он, общий предок для фабрик: объект, содержащий только виртуальный деструктор и фабричный метод `createShape()`.

```
class CMyShapeFactory : public CShapeFactory {
public:
    CShape *createShape(char code) override {
        CShape *shape = nullptr;
        switch (code) {
            case 'P':
                shape = new CPoint(0,0);
                break;
```

```

        case 'C':
            shape = new CCircle(0,0,0);
            break;
        default:;
    }
    return shape;
}
};

```

А вот и его потомок, конкретная фабрика, перекрывающая фабричный метод и наполняющая его содержанием. Как теперь изменится наш класс `CShapeArray`? Теперь в нем не будет собственного метода `createShape()`, потому что это «хранилище», а не «создавалище». Но вместо этого в метод `loadShapes()` кроме имени файла будет передаваться фабрика:

```

void loadShapes(string filename, CShapeFactory &factory) {
    FILE *stream;
    int count;
    char code;
    CShape *shape;

    if ((stream=fopen(filename.c_str(), "r")) != nullptr)
    {

        fscanf(stream, "%d\n", &count);
        for (int i = 0; i < count; i++)
        {
            fscanf(stream, "%c\n", &code);
            shape = factory.createShape(code);
            if (shape != nullptr) {
                shape->load(stream);
                shapes.push_back(shape);
            }
        }
        fclose(stream);
    }
}

```

Там, где раньше `loadShapes()` вызывала свой собственный метод `createShape()` для создания нового объекта неизвестного класса, теперь она обращается к фабрике, чтобы та создала нужный объект. В основной программе создается класс для конкретной фабрики, которая умеет создавать `CPoint` и `CCircle`:

```

class CMyShapeFactory : public CShapeFactory {
public:
    CShape *createShape(char code) override {
        CShape *shape = nullptr;

```

```
switch (code) {
    case 'P':
        shape = new CPoint(0,0);
        break;
    case 'C':
        shape = new CCircle(0,0,0);
        break;
    default:;
}
return shape;
}
};
```

При этом класс хранилища `CShapeArray` какой был, такой и остается, безо всяких потомков. В основной программе создается именно хранилище `CShapeArray`, а если ему нужно загрузить свое содержание из файла, то создается объект-фабрика `CMyShapeFactory` и передается в метод `CShapeArray::loadShapes()`:

```
CShapeArray shapeArray;
```

```
CMyShapeFactory factory;
shapeArray.loadShapes(filename, factory);
```

2.23. Abstract Factory еще раз

Попробуем отделить мух от котлет и еще раз посмотреть на этот паттерн отдельно, не говоря ничего про хранилища:

```
class CFood {
public:
    virtual void prepare() = 0;
    virtual ~CFood() { }
};

class CMeatFood : public CFood {
public:
    CMeatFood() {
        cout << "CMeatFood::CMeatFood()\n";
    }
    void prepare() override {
        cout << "CMeatFood::prepare()\n";
    }
};

class CVegetarianFood : public CFood {
public:
    CVegetarianFood() {
        cout << "CVegetarianFood::CVegetarianFood()\n";
    }
};
```



```
void prepare() override {
    cout << "CVegetarianFood::prepare()\n";
}
};
```

Пусть у нас есть два разных типа еды: мясная `CMeatFood` и вегетарианская `CVegetarianFood`. Им нужен общий предок `CFood`, чтобы можно было хранить их в одних и тех же переменных. Что такое фабрика? Фабрика — это объект, умеющий создавать другие объекты. В нашем случае одна фабрика, `CMeatFoodFactory`, умеет создавать мясную еду, а другая фабрика, `CVegetarianFoodFactory`, — вегетарианскую:

```
class CFoodFactory {
public:
    virtual CFood *createFood() = 0;
    virtual ~CFoodFactory() { }
};
class CMeatFoodFactory : public CFoodFactory {
public:
    CFood *createFood() override {
        return new CMeatFood();
    }
};
class CVegetarianFoodFactory : public CFoodFactory {
public:
    CFood *createFood() override {
        return new CVegetarianFood();
    }
};
```

Это легко себе представить: вот один завод, он производит консервы одного типа, а вот другой завод напротив, он производит другие консервы. Что такое фабрики, понять просто. А что такое абстрактная фабрика? А это их общий предок `CFoodFactory`! Что он производит? Ничего не производит, он абстрактный, он в реальности не существует. Зачем нужна абстрактная фабрика? Только для того, чтобы служить общим предком всех конкретных фабрик и позволять хранить их в одних и тех же переменных:

```
class CAnimal {
public:
    CAnimal() {
        cout << "CAnimal::CAnimal()\n";
    }
    virtual void feedWith(CFood *food) {
        cout << "CAnimal::feed()\n";
    }
    virtual ~CAnimal() { }
};
```

```
void main() {
    CFoodFactory *factory;
    if (rand() % 2 == 0)
        factory = new CMeatFoodFactory();
    else
        factory = new CVegetarianFoodFactory();

    // ...

    CFood *food = factory->createFood();
    food->prepare();

    CAnimal *student = new CAnimal();
    student->feedWith(food);

    delete food;
    delete factory;
    delete student;
}
```

Вот пример, с помощью которого можно продемонстрировать использование фабрик. Где-то в одной части программы создается какая-то конкретная фабрика, но мы можем даже не знать, какая именно (нам ее могли передать из какой-то другой функции, из динамической библиотеки и т. д.). Мы не знаем, какая именно это фабрика, потому что работаем с ней через стандартный интерфейс абстрактной фабрики `CFoodFactory`, поэтому все, что мы знаем об этой фабрике, — что это фабрика, т. е. у нее есть метод `createFood()`, а значит, с ее помощью можно создавать еду.

```
CFood *food = factory->createFood();
food->prepare();
```

```
CAnimal *student = new CAnimal();
student->feedWith(food);
```

Какая именно еда здесь создается? Неизвестно! Да объекту `student` и нет разницы, какая это еда, — главное, что ее можно есть! Наша программа может с помощью фабрики создавать объекты одного класса, а если мы хотим, чтобы она начала создавать объекты другого класса, все, что нам нужно, — подсунуть программе другую конкретную фабрику.

Итак, фабрика — объект, создающий другие объекты. Можно иметь много разных конкретных фабрик, они будут все создавать объекты разных конкретных типов. Но всем конкретным фабрикам нужен общий предок — абстрактная фабрика, чтобы со всеми ними можно было работать единообразно через общий интерфейс.

В нашем случае фабрика всегда создавала только один тип объектов, потому что содержала только один фабричный метод. Но это совсем не обязательно так:

в фабрике может быть несколько разных фабричных методов. Например, в нашей фабрике кроме `createFood()` мог бы быть какой-нибудь `createDrink()`. Диаграмма классов для фабрики из книги [7] это подчеркивает (рис. 2.20).

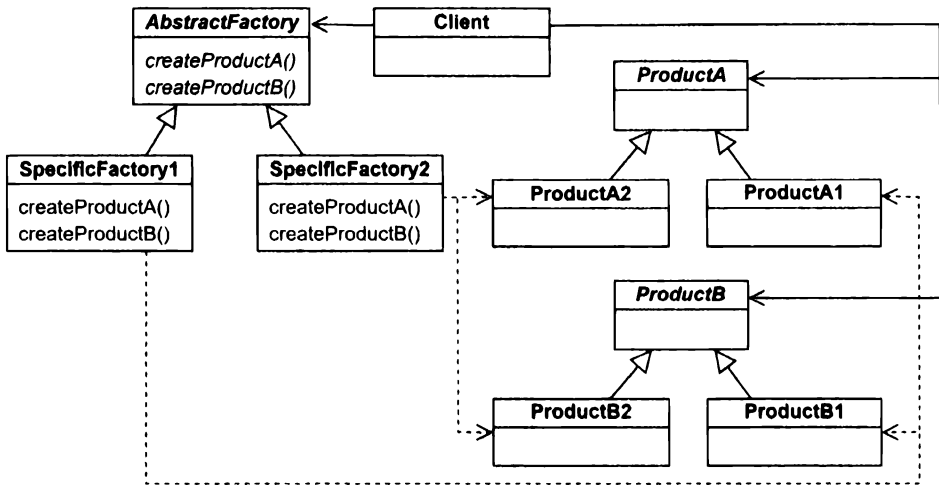


Рис. 2.20. Диаграмма классов паттерна Abstract Factory

Тут есть два типа продуктов, А и В, и у каждого два варианта, 1 и 2. Две фабрики, первая создает продукты А1 и В1, а вторая — А2 и В2. Самое главное на этой диаграмме — что клиент знает только про абстрактные продукты А и В и только про абстрактную фабрику, но ничего не знает ни про конкретные продукты, ни про конкретные фабрики.

2.24. Memento

Следующий небольшой паттерн, который мы разберем, называется «Храни-тель». Он часто используется для сохранения и восстановления состояния объ-ектов между так называемыми контрольными точками, а также для отката дей-ствий. Но суть его в другом. Представьте себе объект, у которого внутри есть какие-то приватные свойства, доступа к которым основная программа не имеет. Но при этом нам нужно иметь возможность делать «слепок» этих свойств, т. е. внутреннего состояния объекта, а позже восстанавливать прошлые значения этих свойств. Как сохранять значения свойств объекта, ничего про эти свойства не зная? Правильно. Надо поручить эту работу самому объекту. Он должен по нашей команде сохранить значения всех своих свойств в особый «конвертик», передать нам этот конвертик на хранение, а когда-нибудь потом, снова по нашей команде, восстановить свое состояние из конвертика, который мы ему вернем. Сам конвертик при этом для нас останется «непрозрачным», потому что доступ к его содержимому будет только у того, кто этот конвертик создаст.

```
class CMemento {
private:
    friend class CPoint;
    int _x, _y;
    void setState(int x, int y) {
        _x = x;
        _y = y;
    }
    void getState(int &x, int &y) {
        x = _x;
        y = _y;
    }
public:
    CMemento() {
        _x = 0; _y = 0;
    }
    virtual ~CMemento() { };
};
```

Вот он, этот конвертик CMemento. Он хранит состояние точки (координаты), но у него все приватное, кроме инициализирующего конструктора и деструктора, а значит, никто не сможет ничего из него прочитать. Никто — кроме класса CPoint, который объявлен дружественным, а значит, будет иметь доступ к приватным полям CMemento.

```
class CPoint {
private:
    int _x, _y;
public:
    CPoint(int x, int y) {
        _x = x; _y = y;
    }
    void move(int dx, int dy) {
        _x = _x + dx; _y = _y + dy;
    }
    void report() {
        printf("CPoint is: %d %d\n", _x, _y);
    }
    virtual CMemento *createMemento() {
        CMemento *memento = new CMemento();
        memento -> setState(_x, _y);
        return memento;
    }
    virtual void updateFromMemento(CMemento *memento) {
        memento -> getState(_x, _y);
    }
    virtual ~CPoint() { }
};
```

Теперь сам класс точки: у него есть координаты и методы для их изменения `move()` и отрисовки точки на экране `report()`. И, кроме этого, у точки есть метод `createMemento()`, с помощью которого можно попросить точку создать конвертик-`memento`, запечатлеть в нем свое текущее состояние и вернуть его нам. Точка также должна уметь восстанавливать свое состояние из конвертика-`memento`, который ей передадут в метод `updateFromMemento()`.

```
void main() {
    CPoint *p = new CPoint(10,20);
    p -> report();

    CMemento *memento = p -> createMemento();

    p -> move(5,5);
    p -> report();

    p -> updateFromMemento(memento);
    p -> report();

    delete memento;
    delete p;
}
```

Вот пример работы с «Хранителем». Мы создаем точку, рисуем ее на экране, потом просим зафиксировать свое состояния в «точке отката», т. е. объекте-`memento`. Затем передвигаем ее и снова рисуем на экране, а потом просим восстановить ее прошлое состояние. Собственно, суть паттерна «Хранитель» на этом и заканчивается: научить объект инкапсулировать свое внутреннее состояние в отдельный объект-конвертик, а также научить восстанавливать свое состояние из существующего объекта-конвертика. Вот диаграмма классов паттерна «Хранитель» (рис. 2.21).

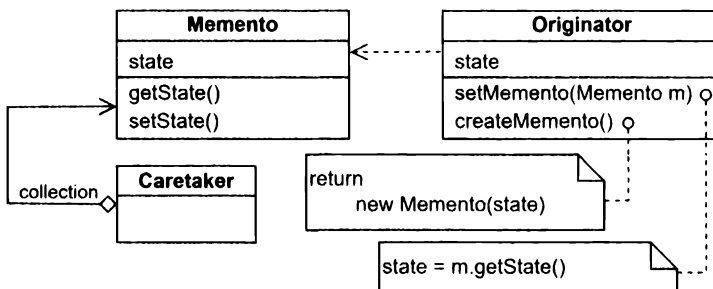


Рис. 2.21. Диаграмма классов паттерна «Хранитель»

Как уже было сказано, паттерн «Хранитель» можно использовать для отката предыдущих действий. Учитывая, что эту же функциональность можно получить «на сдачу» в паттерне «Команда», может возникнуть вопрос: какой же паттерн лучше для этого использовать. И тут можно легко увидеть существенную разницу между ними.

Если представить себе, что речь идет о каком-нибудь растровом редакторе изображений, то очевидно, что состоянием программы будет как минимум описание цвета каждого пиксела, т. е., по сути, все изображение целиком, — и это именно то, что придется у себя внутри сохранять паттерну Memento. Команда же в паттерне Command может сохранять у себя внутри только ту информацию, которая необходима для отмены только ее действия, — а значит, это могут быть не все пикселы, а только те, которые изменяются. А если команда, например, увеличивает яркость изображения, то команда вообще может (не во всех случаях, но во многих) сохранить только величину изменения этой яркости, что вообще несравнимо с объемом информации, который придется сохранять Memento. Таким образом, команде, как правило, требуется сохранять для отмены действия гораздо меньше информации.

С другой стороны, имея объект-memento, можно всегда «переключиться» на это состояние за одно-единственное действие. Тогда как в случае команды для того, чтобы вернуться в состояние, которое было много действий назад, придется одно за другим в обратном порядке отменять все прошедшие действия. Таким образом, с точки зрения отката действий и возврата к прошлым состояниям паттерн Memento как будто дает больше свободы и более удобен, но требует для этого гораздо больше памяти.

2.25. Bridge

Паттерн «Мост» многими изучающими ООП почему-то считается одним из самых сложных, хотя, по-моему, самое сложное в этом паттерне — понять, почему он называется именно «Мост». Хотя, соглашусь, паттерн на самом деле достаточно абстрактный, и сейчас вы в этом убедитесь.

```
class StorageImpl {
public:
    virtual float get(int index) = 0;
    virtual void set(int index, float value) = 0;
    virtual void push_back(float value) = 0;
    virtual void push_front(float value) = 0;
    virtual int size() = 0;
    virtual ~StorageImpl() { }
};
```

Класс StorageImpl, как мы видим, содержит функции, позволяющие получать откуда-то float-значение по индексу, устанавливать его где-то так же по индексу, вставлять float-значение в начало и конец, получать количество хранимых где-то там float-значений, ну и деструктор. Исходя из всего этого списка действий, мы можем предположить, что StorageImpl — это хранилище float-значений (точнее, общий предок, интерфейс для возможных хранилищ).

```
class ListStorageImpl : public StorageImpl {
private:
    list<float> values;
```

```

public:
    float get(int index) override {
        list<float>::iterator i;
        for (i=values.begin(); i!=values.end(); i++)
            if (index == 0)
                return *i;
            else
                index--;
        return 0;
    }
    void set(int index, float value) override {
        list<float>::iterator i;
        for (i=values.begin(); i!=values.end(); i++)
            if (index == 0) {
                *i = value;
                break;
            }
            else
                index--;
    }
    void push_back(float value) override {
        values.push_back(value);
    }
    void push_front(float value) override {
        values.push_front(value);
    }
    int size() override {
        return values.size();
    }
};

```

Теперь мы видим класс `ListStorageImpl`, потомок `StorageImpl`, который, видимо, является конкретным примером хранилища, внутри себя использующим `list<float>` для фактического хранения значений. Как потомку `StorageImpl` ему нужно перекрыть все методы `StorageImpl`, и он это делает, где-то удачно, а где-то нет. Так, например, `list<float>` не позволяет легко получать и устанавливать значения по индексу, и значит, приходится это вручную делать в цикле. Зато вставку элементов в начало и в конец, а также подсчет числа элементов у `list<float>` выполнить легко, поэтому эти методы у `ListStorageImpl` — просто обертка.

```

class VectorStorageImpl : public StorageImpl {
private:
    vector<float> values;
public:
    float get(int index) override {
        return values[index];
    }
}

```

```

void set(int index, float value) override {
    values[index] = value;
}
void push_back(float value) override {
    values.push_back(value);
}
void push_front(float value) override {
    values.insert(values.begin(), value);
}
int size() override {
    return values.size();
}
};

```

Еще один потомок `StorageImpl`, еще один пример конкретного хранилища, который фактически хранит значения не в `list<float>`, а в `vector<float>`. Как мы видим, ему тоже приходится перекрывать все абстрактные методы `StorageImpl`, но ему это делать проще, потому что и чтение, и запись по индексу у `vector<float>` реализованы «нативно». Что интересно, так это то, что и функция `insert` у вектора реализована своя собственная, хотя мы уже взрослые люди, в Деда Мороза не верим, и в константную реализацию этого метода у вектора — тоже.

Итак, у нас есть два потомка `StorageImpl`, которые реализуют все методы `StorageImpl`, но при этом мы заранее знаем, что методы `get-set` у `vector` имеют сложность $O(1)$, а у `list` — $O(N)$, а метод `insert`, наоборот, у `vector` имеет сложность $O(N)$, а у `list` — $O(1)$. Что дальше?

```

class Storage {
protected:
    StorageImpl *impl;
    map<string, int> ops;
public:
    Storage() {
        impl = new ListStorageImpl();
        ops["getset"] = 0;
        ops["pushback"] = 0;
        ops["pushfront"] = 0;
    }
    float get(int index) {
        ops["getset"]++;
        relocate();
        return impl->get(index);
    }
    void set(int index, float value) {
        ops["getset"]++;
        relocate();
        impl->set(index, value);
    }
}

```



```
void push_back(float value) {
    ops["pushback"]++;
    relocate();
    impl->push_back(value);
}

void push_front(float value) {
    ops["pushfront"]++;
    relocate();
    impl->push_front(value);
}

int size() {
    return impl->size();
}

void relocate();

virtual ~Storage() {
    delete impl;
}

};
```

Еще какой-то класс хранилища? Подождите, а предыдущие классы — они что, получается, не были хранилищами? Давайте аккуратно разбираться. Что мы здесь видим? Мы видим класс `Storage`, который ни от кого не наследуется и сам, несмотря на то, что называется «хранилище», фактически значений не хранит. Вместо этого он хранит указатель `impl` на какой-то объект класса `StorageImpl`. В частности, в своем конструкторе он создает и записывает в эту переменную новый объект класса `ListStorageImpl`. Еще этот класс определяет два счетчика для подсчета количества разных операций, которые с ним будут производить.

Затем класс `Storage` определяет те же самые операции для доступа к хранимым элементам-числам по индексу, для подсчета числа элементов и для вставки элемента в начало и конец хранилища. Но мы видим, что раз самих чисел в `StorageImpl` нигде не хранится, большую часть времени `StorageImpl` просто переадресовывает вызовы к хранимому внутри объекту `impl`, например возвращает `return impl->size()`, когда у него самого запрашивают размер. Еще мы должны увидеть, что в каждом методе соответствующий счетчик увеличивается, чтобы подсчитывать, сколько раз к хранилищу обращались каким способом, и вызывает функцию `relocate`, которую мы еще не реализовали.

Прежде чем двигаться дальше, нужно понять, что такое `Storage`, что такое `StorageImpl`, чем они друг от друга отличаются и кто из них достоин называться хранилищем. Я думаю, понять это можно на вот таком простом примере. Я живу в хрущевке, и тот, кто в таких квартирах жил, знает, что в них есть специальная небольшая комнатка без окон — кладовка, которую можно использовать, например, для хранения носков. Каждое утро можно подходить к этой комнате, где вам выдадут новую пару носков. Но если заинтересоваться, как носки внутри этой комнаты хранятся, то можно выяснить, что это бывает очень по-разному. У меня самого сначала носки хранились методом динамической кучи в углу,

и выбирать их по номеру было довольно долго, зато вставлять купленные носки в начало этой кучи было очень просто. Затем я заменил кучу на упорядоченный набор контейнеров по номерам, и жить стало проще. В конце концов я купил продвинутый электрифицированный вендомат по выдаче носков при нажатии большой красной кнопки, — но он, к сожалению, в стандартную кладовку хрущевки не влез.

Надеюсь, вы уже проследили аналогию? Чем был бы какой-нибудь `HeapStorageImpl`? Это механизм хранения носков в виде кучи. Что такое `VectorStorageImpl`? Это механизм хранения носков в виде пронумерованного набора контейнеров. Что такое `StorageImpl`? Это их общий предок, это общий интерфейс для механизма хранения и выдачи носков! А что такое `Storage`? А это сама комната для хранения носков, в которую можно установить любой механизм хранения и выдачи — лишь бы он поддерживал интерфейс `StorageImpl`. Когда мой вендомат для носков не влез в кладовку — просто его интерфейс оказался несовместим с интерфейсом `StorageImpl`.

Зачем все это нужно? Когда я подхожу утром к кладовке, я хочу просто получить очередную пару носков, и кладовка, как `Storage`, должна мне эту функциональность обеспечить. Меня в этот момент может не интересовать, какой именно механизм внутри кладовки отработал, — лишь бы он сделал это правильно и быстро.

Итак, есть кладовка, обеспечивающая меня носками, и есть конкретный механизм внутри нее, который может быть даже скрыт от меня, но который весь этот процесс и обеспечивает. Есть хранилище `Storage`, которое предоставляет возможность работать с хранимыми объектами (по индексу и вставкам), и есть конкретные механизмы `ListStorageImpl` и `VectorStorageImpl`, которые в приватной секции `Storage` скрыты от нас, но при этом обеспечивают выполнение всей работы. Почему этих механизмов может быть больше одного? Потому что хранить значения можно очень по-разному и работать с ними можно очень по-разному, и хорошо бы, чтобы то, как мы работаем с хранилищем, соответствовало тому, как элементы в хранилище хранятся.

```
void Storage::relocate() {
    if (ops["getset"] > ops["push_front"] &&
        dynamic_cast<ListStorageImpl*>(impl) != nullptr) {
        printf("Relocating List to Vector\n");
        StorageImpl *new_impl = new VectorStorageImpl;
        for (int i = 0; i < impl->size(); i++)
            new_impl->push_back(impl->get(i));
        delete impl;
        impl = new_impl;
    }

    if (ops["push_front"] > ops["getset"] &&
        dynamic_cast<VectorStorageImpl*>(impl) != nullptr) {
        printf("Relocating Vector to List\n");
```

```

StorageImpl *new_impl = new ListStorageImpl;
for (int i = 0; i < impl->size(); i++)
    new_impl->push_back(impl->get(i));
delete impl;
impl = new_impl;
}
}

```

Вот наконец функция `relocate()`. Она проверяет два условия. Если операций доступа по индексу было больше, чем операций вставки в начало хранилища, но при этом механизмом хранения является список (к которому очень невыгодно обращаться по индексу), то хранилище меняет свой внутренний механизм хранения со списка на вектор и переносит в него все данные.

Если же, наоборот, операций вставки в начало было больше, чем операций доступа по индексу, но при этом механизмом хранения является вектор (в начало которого вставлять значения очень невыгодно), то хранилище меняет свой внутренний механизм хранения с вектора на список.

```

class OrderedStorage : public Storage {
public:
    void sort() {
        for (int i = 0; i < impl->size(); i++)
            for (int j = i + 1; j < impl->size(); j++)
                if (impl->get(i) > impl->get(j)) {
                    float xi = get(i);
                    float xj = get(j);
                    set(i, xj);
                    set(j, xi);
                }
    }
};

```

Мало того, что мы можем порождать разные потомки `StorageImpl` для реализации разных, эффективных в разных условиях механизмов хранения. Теперь мы можем определять разные потомки самого хранилища, например упорядоченное хранилище `OrderedStorage`, которое будет пользоваться в произвольном порядке методами `get`, `set`, `push_back`, `push_front` и даже не задумываться при этом, какой именно механизм хранения элементов используется.

```

OrderedStorage storage;

printf("Inserting values\n");
for (int i = 0; i < 5; i++)
    storage.push_front(rand() % 100);

printf("Accessing values twice\n");
for (int i = 0; i < storage.size(); i++)
    printf("%.1f\n", storage.get(i));

```

```
for (int i = 0; i < storage.size(); i++)
    printf("%.1f\n", storage.get(i));

printf("Inserting values twice\n");
for (int i = 0; i < 5; i++)
    storage.push_front(rand() % 100);
for (int i = 0; i < 5; i++)
    storage.push_front(rand() % 100);

printf("Sorting values\n");
storage.sort();

printf("Accessing values once\n");
for (int i = 0; i < storage.size(); i++)
    printf("%.1f\n", storage.get(i));
```

Так же и основная программа может создать упорядоченное хранилище, пользоваться в произвольном порядке методами `get`, `set`, `push_back`, `push_front`, и хранилище внутри себя будет перестраивать свой собственный механизм реального хранения объектов.

Вот как диаграмма классов для паттерна «Мост» представлена в книге [7] (рис. 2.22).

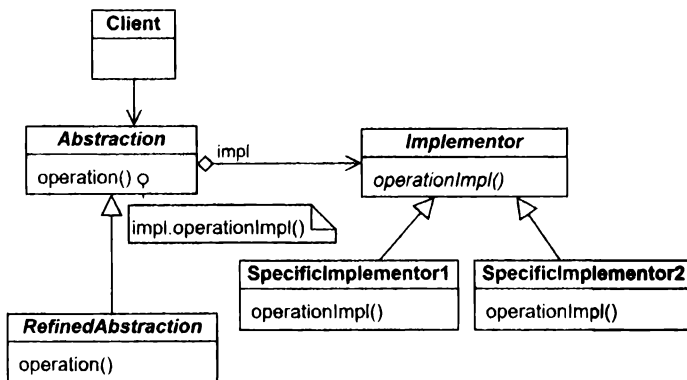


Рис. 2.22. Диаграмма классов паттерна «Мост»

Она показывает, что есть какой-то класс `Abstraction` (в нашем случае — хранилище `Storage`), который при вызове у него какой-то операции сам ее не выполняет, а делегирует выполнение хранимому механизму `impl`, указывающему на одного из потомков `Implementor` (у нас это `StorageImpl` и его потомки `ListStorageImpl` и `VectorStorageImpl`). И даже если у класса `Abstraction` будут потомки, например `RefinedAbstraction` (в нашем случае — `OrderedStorage`), то они тоже будут пользоваться методами механизма, сами не зная об этом.

Паттерн «Мост» еще иногда связывают с приемом, который называется `Private Implementation`, хотя обычно разделяют `Pimpl` с расшифровкой `Pointer to`

Implementation как конкретный прием разделения кода в языке C++ (C++ Pimpl, Opaque pointer, Compiler Firewall) и собственно идею паттерна «Мост», которую тоже иногда называют Pimpl, имея в виду уже Private Implementation. Почему private implementation? Потому что наш класс `Storage` отказывается сам предоставлять реализацию функций `get`, `set`, `push_back`, `push_front`, а делегирует их все куда-то во внутреннюю часть `StorageImpl` и его потомков `ListStorageImpl` и `VectorStorageImpl`. Цель Pimpl как приема C++ — засунуть реализацию куда подальше, чтобы пореже перекомпилировать нужно было, при этом реализация чаще всего одна-единственная. Цель Pimpl в виде паттерна «Мост» — обеспечить множественность этих возможных реализаций и подарить эту множественность своим потомкам.

Мы уже много раз говорили, что большая часть паттернов реализует одну и ту же схему: что-то откуда-то вытащить и оформить как отдельный объект. Можно сказать, что паттерн «Мост» или Pimpl действует по той же самой схеме: взять реализацию какого-то объекта и вынести ее в отдельный объект. Так, возможно, когда-то наш объект `Storage` был нормальным объектом, который все сам реализует. Однако у него вытащили всю реализацию методов `get`, `set`, `push_back`, `push_front` и оформили ее как отдельный объект (потомок `StorageImpl`). Зачем?

Представьте, что у нас был бы `Storage` с нормальной реализацией методов `get`, `set`, `push_back`, `push_front`. Что нам пришлось бы делать, если бы понадобилось изменить реализацию этих методов? Похоже, что только наследоваться от класса `Storage` и перекрывать все эти методы. С паттерном Bridge мы можем легко изменить реализацию этих методов, не трогая сам класс `Storage` и не порождая его потомков для этой цели. Более того, мы сможем, по сути, изменить реализацию этих методов у `Storage` прямо во время работы программы, заменив у него внутри указатель `impl`.

Но замена реализации во время работы программы — не самое главное: это мы просто получаем «на сдачу». Паттерн «Мост» позволяет во многих случаях избежать размножения классов, которое происходило бы, если бы у нас было только наследование. Судите сами: если бы у нас был `Storage` и нам понадобилось бы сделать три его варианта: один вариант хранил бы данные в списке, другой — в векторе, третий — в каком-нибудь дереве. Это было бы три класса: `VectorStorage`, `ListStorage` и `TreeStorage`. Потом мы бы сделали возможность упорядочивания элементов, но вместо одного `OrderedStorage` пришлось бы сделать сразу три новых класса: `VectorOrderedStorage`, `ListOrderedStorage` и `TreeOrderedStorage`. Понадобилось бы реализовать хранилище фигур, классы бы снова размножились: вместо одного `ShapeStorage` появились бы `VectorShapeStorage`, `ListShapeStorage` и `TreeShapeStorage` и т. д.

На самом деле весь этот комбинаторный взрыв произошел бы потому, что мы смешали бы иерархию нужных пользователю классов `Storage`, `OrderedStorage`, `ShapeStorage` с совсем другой иерархией: иерархией классов-механизмов реализации `VectorStorageImpl`, `ListStorageImpl` и `TreeStorageImpl`. Вместо этого эти две иерархии живут независимо друг от друга: появление нового механизма

DBStorageImpl, хранящего объекты в БД, не должно никак затронуть какой-нибудь класс упорядоченного хранилища OrderedStorage, но должно позволить и ему тоже хранить свои данные в БД.

Любую иерархию можно условно представить себе в виде пирамиды. Вот этот рисунок объясняет, почему этот паттерн называется именно «Мост» — это мост между двумя иерархиями, мост от вершины иерархии (условно) пользовательских объектов к вершине другой иерархии (условно) механизмов-impleментаций (рис. 2.23)!

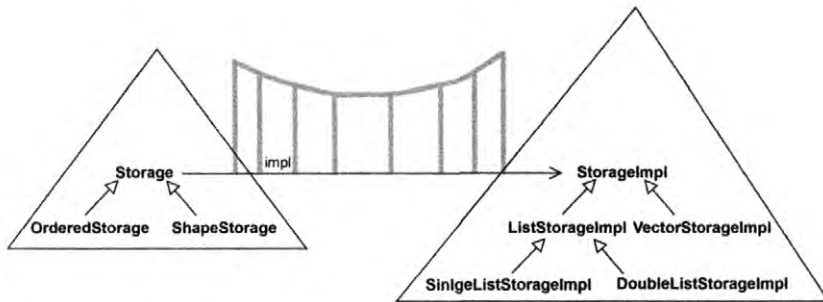


Рис. 2.23. Мост между двумя иерархиями

Таким образом, суть паттерна «Мост» — инкапсуляция реализации методов какого-то объекта в отдельный объект. Это позволяет:

- ❑ независимо друг от друга развивать иерархию самих объектов и иерархию возможных реализаций;
- ❑ на лету изменять реализацию какого-либо объекта;
- ❑ самому объекту на лету изменять свою собственную реализацию (что и реализовано в нашем примере).

Интересно, что диаграмма классов паттерна «Стратегия» — практически такая же, как у паттерна «Мост», и это неспроста, они действительно родственные. Если вы попытаете понять, в чем между ними разница, то, скорее всего, придете к выводу, что по факту паттерн «Мост» — это одновременная реализация паттерна «Стратегия» сразу у всех методов какого-то базового класса иерархии. Кроме этого, паттерн «Стратегия» больше направлен именно на динамическое изменение реализации какого-то метода, а паттерн «Мост», хотя и позволяет это делать, больше направлен на то, чтобы еще до запуска программы разделить объекты и их реализации по разным иерархиям, даже если не придется их менять на лету во время выполнения программы.

2.26. Visitor

Паттерн «Посетитель» иногда считается Вершиной Теоретически Постижимой Человеком Абстракции и Самым Сложным Паттерном, который если понять, то все остальные паттерны покажутся тривиальщиной. Студент, которому на экза-

мене попался в билете «Посетитель», сдает билет и сразу идет на пересдачу. На самом деле к паттерну «Посетитель» надо подбираться постепенно, тогда и его мотивация («Зачем вообще эта белиберда?»), и его абстракция станут вполне очевидны. Лучше всего разбираться в паттерне «Посетитель», разобравшись в паттерне «Компоновщик». Если помните, там у группы было два метода, примерно таких (у метода `move()` были, конечно, параметры, но я тут убрал их, чтобы было нагляднее):

```
void CGroup::move() override {
    for (const auto &shape : _shapes)
        shape->move();
}

void CGroup::draw() override {
    for (const auto &shape : _shapes)
        shape->draw();
}
```

Согласитесь, эти два метода подозрительно похожи до степени смешения, отличаясь только тем, какой метод они вызывают. Вот было бы здорово, если бы можно было вызывать метод с помощью какой-то строковой переменной, примерно так:

```
void CGroup::call(string what) override {
    for (const auto &shape : _shapes)
        shape->what();
}
```

Если `what` — это какая-то строка, то можно было бы отказаться от двух разных методов `move()` и `draw()` и вместо них вызывать примерно так: `call("draw")` и `call("move")`, и эти строки бы подставлялись на место `what` и получалось бы что-то вроде `shape->"move"()` и `shape->"draw"()`. Это, конечно, так не работает, потому что методы не являются *first class citizens*, их никуда нельзя передать. Но, как всегда с паттернами, это будет работать, если метод обернуть в объект и вытащить в отдельный класс:

```
class MethodBase {
public:
    virtual void call(CShape *shape) = 0;
};

class DrawMethod : public MethodBase {
public:
    virtual void call(CShape *shape) {
        shape->draw();
    }
};
```

```
class MoveMethod : public MethodBase {
public:
    virtual void call(CShape *shape) {
        shape->move();
    }
};

void CGroup::call(MethodBase &method) override{
    for (const auto &shape : _shapes)
        method.call(shape);
}
```

Теперь вместо вызова `CGroup::draw()` можно вызвать `CGroup::call(DrawMethod())`, а вместо вызова `CGroup::move()` можно вызвать `CGroup::call(MoveMethod())` и вообще, методы `move()` и `draw()` положить в переменную `m`, чтобы вызвать потом `CGroup::call(m)`, нельзя, а если эти методы обернуты соответственно в объекты `MoveMethod()` и `DrawMethod()`, то уже все можно.

Еще одна проблема паттерна «Компоновщик» заключается в том, что чем больше разных (но при этом похожих) операций у нас будет выполняться с фигурами, тем сильнее расплзаться будет базовый класс `CShape`. И что еще неприятнее — как только появляется какое-то новое действие, которое нужно выполнять со всеми объектами иерархии, приходится менять самый базовый класс `CShape`, добавляя в него новый виртуальный абстрактный метод, а добавление нового абстрактного метода в базовый абстрактный класс — это совсем не то действие, которое следует пушить в репозиторий перед тем, как уйти в отпуск! Этот метод придется сразу поддерживать во всех классах-потомках, о некоторых из которых вы можете даже не знать!

```
class CShape {
public:
    virtual void draw() = 0;
    virtual void move() = 0;
    virtual void rotate() = 0;
    virtual void resize() = 0;
    virtual void change_color() = 0;
    ...
}
```

Убедитесь, что вы врубались в приведенный выше финт ушами, прежде чем продолжать. А если вы готовы, то временно забудьте про слова `Visitor` и «Посетитель», потому что мы будем работать не с посетителями, а с обработчиками (на мой взгляд, именно так для пушей понятности следовало назвать этот паттерн):

```
class CPoint;
class CGroup;
```



```
class CHandler {
public:
    virtual void handlePoint(CPoint *p) = 0;
    virtual void handleGroup(CGroup *g) = 0;
    virtual ~CHandler() { }
};

class CShape {
public:
    virtual void Apply(CHandler *Handler) = 0;
    virtual ~CShape() { };
};
```

Итак, у нас будут какие-то классы `CPoint` и `CGroup` в качестве объектов, над которыми мы будем измываться. И у нас будет обработчик `CHandler`, который должен и уметь обрабатывать точку, и уметь обрабатывать группу. Это два разных метода, потому что и точка, и группа — объекты достаточно разные, чтобы их обрабатывать по-разному.

А теперь объявляется базовый класс `CShape`... Подождите, а где же у него вся коллекция разнообразных `move`, `draw`, `resize`, `rotate` и т. д.? А нет ее! Вместо них всех — единственный абстрактный метод `Apply()`, которому требуется объект класса `CHandler`. Помните наш метод `call()` в примере выше? Вот это как раз он.

```
class CPoint : public CShape {
public:
    int _x, _y;
public:
    CPoint(int x, int y) : _x(x), _y(y) {};
    void Apply(CHandler *Handler) override {
        Handler->handlePoint(this);
    }
};
```

Вот он, наш любимый `CPoint`. У него координаты и конструктор, который их устанавливает, все для примера публично. И перекрытый от `CShape` метод `Apply()`, куда передали какой-то обработчик, а этот метод не нашел ничего лучше, чем делегировать работу обратно в обработчик, сказав ему: «Раз ты у нас обработчик, то давай-ка ты обработай точку». — «Какую точку?» — возмутился обработчик. — «Меня!» — сказала точка.

```
class CGroup : public CShape {
public:
    vector<CShape*> _shapes;
public:
    CGroup() {
    }
```

```

~CGroup() {
    for (const auto &s : _shapes)
        delete s;
}
void addShape(CShape *shape) {
    _shapes.push_back(shape);
}
void Apply(CHandler *Handler) override {
    Handler->handleGroup(this);
}
};

```

А вот как выглядит переделанный, по сравнению с паттерном «Компоновщик», класс группы. Вы уже видите главное отличие — оно заключается в том же методе Apply, куда тоже передали какой-то обработчик, а этот метод не нашел ничего лучше, чем делегировать работу обратно в обработчик, сказав ему: «Раз ты у нас обработчик, то давай-ка ты обработай группу». — «Какую группу?» — возмутился обработчик. — «Меня!» — сказала группа.

Что за фигня, скажете вы? Зачем вся эта двойная косвенность? Ведь вместо `CPoint *p = new CPoint(); p->Apply(обработчик);` можно было бы сразу вызвать `обработчик->handlePoint(p)`, а вместо `CGroup *g = new CGroup(); g->Apply(обработчик);` можно было бы сразу вызвать `обработчик->handleGroup(g)!` И это действительно так, если вы точно знаете, имеете вы дело с точкой или с группой. А если все, что у вас есть, — это какой-то `CShape *s`, и кто в нем прячется — то ли `CPoint`, то ли `CGroup`, не понятно, — тогда вам ничего не остается, кроме как вызвать у объекта `Apply(обработчик)`, и пусть полиморфизм решает, провалимся мы при этом в `CGroup::Apply` или в `CPoint::Apply`.

```

class CDrawer : public CHandler
{
    void handlePoint(CPoint *p) override {
        printf("CPoint drawing at (%d, %d)\n", p->_x, p->_y);
    }
    void handleGroup(CGroup *g) override {
        for (const auto &s : g->_shapes)
            s->Apply(this);
    }
};

class CMover : public CHandler
{
private:
    int _dx, _dy;
public:
    CMover(int dx, int dy) : _dx(dx), _dy(dy) {};

```

```

void handlePoint(CPoint *p) override {
    printf("CPoint moving on (%d, %d)\n", _dx, _dy);
    p->_x = p->_x + _dx; p->_y = p->_y + _dy;
}

void handleGroup(CGroup *g) override {
    for (const auto &s : g->_shapes)
        s->Apply(this);
}
};

```

Вот они, два конкретных обработчика: Рисователь и Передвигатель, потомки CHandler. Если Рисователя просят обработать точку — он рисует точку. Если Передвигателя просят обработать точку — он передвигает точку, меняя ее координаты. А вот что делает Рисователь, когда его просят обработать группу? В группе могут содержаться и точки, и другие группы, поэтому гипотетически Рисователь мог бы поступить вот так:

```

void CDrawer::handleGroup(CGroup *g) override {
    for (const auto &s : g->_shapes)
        if (s is CPoint)
            handlePoint(&s);
        if (s is CGroup)
            handleGroup(&s);
}

```

...но так, конечно, делать нельзя, ибо именно для того, чтобы не писать такие ветвистые if-ы, и придумали полиморфизм, поэтому CDrawer::handleGroup просто предлагает очередному своему объекту *s* (то ли точке, то ли группе) «принять» обработчика (отсюда на самом деле и настоящее название этого паттерна — «Посетитель», потому что он ходит по гостям и все его принимают и заставляют работать). По факту произойдет то же самое: если *s* окажется точкой, то *s->Apply(this)* вызовет CPoint::Apply(CDrawer *this), а этот метод, в свою очередь, вызовет CDrawer::handlePoint(s), и мы придем к тому же самому результату, просто за два шага, а не за один.

```

void main()
{
    CGroup *shapes = new CGroup();
    shapes->addShape(new CPoint(1, 1));
    shapes->addShape(new CPoint(2, 2));
    CGroup *childshapes = new CGroup();
    childshapes->addShape(new CPoint(3, 3));
    childshapes->addShape(new CPoint(4, 4));
    shapes->addShape(childshapes);

    CDrawer dr;
    CMover mv(10, 10);
}

```

```

shapes->Apply(&dr);
shapes->Apply(&mv);
shapes->Apply(&dr);

delete shapes;
}

```

Ну и вот, наконец, основная программа. Здесь, как и в примере с паттерном «Компоновщик», создается группа из трех элементов, в нее складываются две точки и группа, состоящая, в свою очередь, из еще двух точек. А вот затем вместо того, чтобы вызвать `shapes->draw()`, как мы это делали раньше, создается обработчик (в терминах книги [7] — Посетитель) `CDrawer` и отправляется обрабатывать (в терминах книги [7] — посещать) группу `shapes`:

```

CDrawer dr;
shapes->Apply(&dr);

```

Аналогичным образом, когда нужно вызвать у всех объектов иерархии (иерархии объектов, не классов!) метод `move()`, создается обработчик `CMover` и отправляется «обходить»/«посещать» все объекты корневой группы `shapes`.

Главное, чего мы добились, и заключается в основной идее паттерна «Посетитель»: если нужно добавить новую функциональность, например метод `resize()`, всем объектам какой-то иерархии классов (`CShape`, `CPoint`, `CGroup`) не обязательно добавлять новый виртуальный метод в базовый класс иерархии `CShape` и перекрывать его в `CPoint` и `CGroup`. Если так сделать, то получится, что новая функциональность изменения размера `resize()` будет «размазана» ровным слоем по всей иерархии: метод там, метод сям и т. д. Достоинство паттерна «Посетитель» в том, что новая добавляемая функциональность `resize()` будет сосредоточена в одном новом месте — в новом объекте `CResizer`, а уже существующие объекты `CShape`, `CPoint` и `CGroup` менять не придется, в полном соответствии с принципом О из SOLID (см. разд. 2.28).

Да, мы добиваемся этого, во-первых, добавляя лишний слой абстракции (что, как известно, всегда только затрудняет понимание), замедляя вызовы всех методов да и просто получая больше кода. Но даже с размером кода уже видно, где мы постепенно будем получать экономию. Сравните методы `CDrawer::handleGroup()` и `CMover::handleGroup()` — вы видите разницу? Правильно. Ее нет! А что мы делаем, если обнаруживаются два одинаковых метода у разных объектов? Правильно. Выносим их в общего предка:

```

class CHandler {
public:
    virtual void handlePoint(CPoint *p) = 0;
    virtual void handleGroup(CGroup *g) {
        for (const auto &s : g->_shapes)
            s->Apply(this);
    }
}

```

```

virtual ~CHandler() { }
};
class CDrawer : public CHandler
{
    void handlePoint(CPoint *p) override {
        printf("CPoint drawing at (%d, %d)\n", p->_x, p->_y);
    }
};
class CMover : public CHandler
{
private:
    int _dx, _dy;
public:
    CMover(int dx, int dy) : _dx(dx), _dy(dy) {};
    void handlePoint(CPoint *p) override {
        printf("CPoint moving on (%d, %d)\n", _dx, _dy);
        p->_x = p->_x + _dx; p->_y = p->_y + _dy;
    }
};
};

```

То есть если у большинства наших обработчиков процесс обработки группы заключается только в том, что обработчик отправляется «посещать» один за другим все объекты группы, то именно такое поведение мы и напишем один раз в базовом обработчике и потом не будем трогать.

Диаграмма классов для паттерна «Посетитель» в книге [7] выглядит так, как на рис. 2.24.

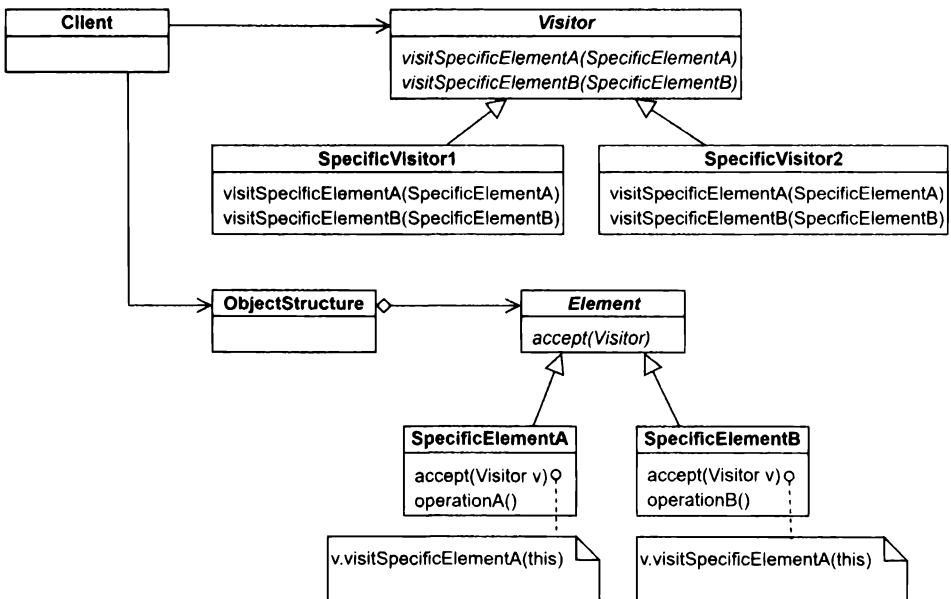


Рис. 2.24. Диаграмма классов для паттерна «Посетитель»

Что? Вам не нравится эта диаграмма классов? Ну так не зря этот паттерн считается одним из самых сложных. Давайте хотя бы попробуем расставить соответствие. Класс `Visitor` и его потомки `ConcreteVisitor1` и `ConcreteVisitor2` у нас называются соответственно `CHandler`, `CDrawer` и `CMove`. У них есть методы `handlePoint` и `handleGroup`. Класс `Element` и его потомки `ConcreteElementA` и `ConcreteElementB` у нас называются `CShape`, `CPoint` и `CGroup`, у них есть только один метод `Apply()`. Надеюсь, так стало понятнее.

2.27. Одинарная и двойная диспетчеризация

Паттерн «Посетитель» часто упоминается в связи с таким понятием, как *двойная диспетчеризация*. Под диспетчеризацией в программировании понимается такая ситуация, когда в коде написан вызов какого-то метода (а если без привязки к ООП, то не метода, а просто функции), а компилятору (или, что чаще, среде выполнения) нужно решить, на какую ветку кода отправить процессор выполнять инструкции.

Возьмем пример, приведенный в начале предыдущего раздела. Что такое одинарная (динамическая) диспетчеризация? Это простой обычный полиморфизм. Вот встречает компилятор вызов:

```
void foo(CShape *s) {  
    s->move();  
}
```

В какую ветку кода нужно отправиться процессору? Как мы знаем, это зависит от реального класса переданного в эту функцию объекта `s`, что может быть неизвестно до момента вызова этой функции уже во время работы программы. Придет в эту функцию `CPoint` — диспетчер отправит процессор выполнять `CPoint::move()`, придет `CGroup` — отправит диспетчер выполнение в функцию `CGroup::move()`. Это обычный полиморфизм, и это одинарная диспетчеризация, которая выполняется по одному признаку — реальному классу объекта `s`. Что такое двойная диспетчеризация? Это такая диспетчеризация, которая выполняется по двум, а не по одному признаку, и оба признака могут быть неизвестны до момента выполнения программы. Как она может гипотетически выглядеть? Примерно вот так:

```
void foo(CShape *s, string method) {  
    s->method();  
}
```

Если бы вдруг это было возможно в таком явном виде в C++, то какой бы код тут вызывался, от чего бы это зависело? А это зависело бы от двух признаков: и от реального класса объекта `s`, и от содержания строковой переменной `method`. Если `method` окажется равным «`move`», а в объекте `s` скрывался бы `CGroup`, то выполняться бы начал метод `CGroup::move()`. Если `method` окажется равным «`draw`»,

а в объекте `s` скрывался бы `Cpoint`, то выполняться бы начал метод `CPoint::draw()`.

Какие-то языки динамическую двойную диспетчеризацию могут поддерживать непосредственно, но в C++ такой синтаксис невозможен, это недостаточно динамический язык. Как можно реализовать в C++ что-то похожее на двойную диспетчеризацию? Первый шаг в этом направлении может выглядеть примерно так:

```
void foo(CShape *s, string method) {
    s->call(method);
}
```

...для этого в каждом потомке `CShape` придется реализовать метод `call` с каким-то большим `switch`:

```
void CGroup::call(string method) {
    switch (method)
    {
        case "move":
            CGroup::move();
            break;
        case "draw":
            CGroup::draw();
            break;
    }
}

void CPoint::call(string method) {
    switch (method)
    {
        case "move":
            CPoint::move();
            break;
        case "draw":
            CPoint::draw();
            break;
    }
}
```

А дальше, если вы попытаете отказаться от переменной `method` в виде строки и заменить ее на объект, то в итоге придете к тому, что называется паттерном «Посетитель».

То, что мы выше рассмотрели, — это только один из частных случаев двойной диспетчеризации, которая, в свою очередь, есть частный случай множественной диспетчеризации. Чтобы понять идею последней в целом, нужно вернуться к самым основам, с которых мы начинали. Вспомним, что `x.foo()` есть просто альтернативная форма записи `foo(x)`, а `x.foo(y, z)` есть просто альтернативная форма записи `foo(x, y, z)`, где первый аргумент `x`, как `this` или `self`, имеет осо-

бенный смысл «разруливающего код». Одинарная диспетчеризация возникает тогда, когда выбор нужного для исполнения в `foo(x, y, z)` кода производится только по первому («разруливающему») параметру `x`, а именно по его типу. Множественная диспетчеризация возникает в поддерживающих ее языках тогда, когда при вызове `foo(x, y, z)` производится выбор между множеством альтернативных функций `foo(..., ..., ...)` и этот выбор зависит от типов и/или значений многих параметров `x`, `y`, `z`, а не только первого.

2.28. SOLID-принципы

Изучение SOLID через считанное число разделов после того, как был изучен обычный полиморфизм, — это, на мой взгляд, мягко говоря, преждевременно. С другой стороны, в курсе по ООП, что обзорном, что тем более детальном, нельзя эту тему обойти стороной, потому что она — предмет не только опыта, но и кругозора, и на собеседованиях по ООП она может стать если не предметом, то поводом для обсуждения. Не расстраивайтесь, если здесь и сейчас обсуждаемые в этом разделе понятия и принципы покажутся вам слишком сложными или непонятными. Любое обсуждение проектирования (а SOLID, в отличие от паттернов, — это не конкретные приемы, а именно принципы) требует наличия опыта, набитых шишек, успешных и неуспешных решений. Всего этого у вас пока нет. Поэтому этот раздел должен загрузить в ваш мозг новые понятия и «пищу для ума», которая будет перевариваться постепенно и всплывать по мере накопления вами своего программистского опыта в области ООП.

SOLID — это английская аббревиатура для пяти известных принципов, применяемых при разработке объектно-ориентированных программ, описанных Р. Мартином в статьях и книгах, например в [12] (конечно же, совершенно случайно так совпало, что первые буквы, будучи соединенными, дали значащее слово английского языка):

- ❑ принцип единственной ответственности (Single responsibility principle, SRP);
- ❑ принцип открытости/закрытости (Open-closed principle);
- ❑ принцип подстановки Лисков (Liskov substitution principle);
- ❑ принцип разделения интерфейса (Interface segregation principle);
- ❑ принцип инверсии зависимостей (Dependency inversion principle, DIP).

Как и с любыми другими принципами, всегда найдутся евангелисты, которые поднимут их на флаг или превратят в кувалду и соблюдение их будут считать самоцелью, а не средством. Цель внедрения этих принципов — борьба с излишней сложностью при разработке ПО. У вас может быть сложная предметная область, и ей будет соответствовать сложная структура программы (так называемая *естественная сложность*). Некорректное, неграмотное, непрофессиональное использование ООП приводит к тому, что сложность программы начинает сильно превышать сложность той проблемы, которую программа призвана решать

(так называемая *привнесенная* сложность). В статье [13] я писал о том, что это происходит из-за того, что в программировании очень легко повышать сложность и очень сложно понижать сложность. Так вот, использование принципов хорошо тогда, когда оно позволяет уменьшать сложность кода, и плохо, когда наоборот. Это главный, на мой взгляд, критерий. Ну и так же, как с паттернами, сами по себе принципы SOLID часто являются по существу проявлением здравого смысла в разработке ПО, но при этом могут использоваться для вежливой коммуникации: «Ты тут, похоже, нарушаешь SRP» — это краткая форма долгого и в этом случае ненужного объяснения, что в этом коде не так.

Как и с любыми другими принципами, с SOLID возможны две проблемы, ошибки первого и второго рода. Любой из этих принципов можно нарушить, а можно настолько рьяно, буквально и чрезмерно его применять, что качество кода это будет только ухудшать. Когда говорят, что кто-то нарушает какой-то из принципов SOLID, стоит выяснить, ошибка первого или второго рода имеется в виду. Опыт разработки даст вам ответственность применять эти принципы там, где они нужны, и смелость отказываться от них там, где они не нужны. Но для этого как минимум нужно знать, про что они есть.

Есть мнение, что любую идею лучше изучать в первоисточнике, и в этом случае за детальным описанием вы пойдете к «Дядюшке Бобу» в [12]. Однако можно сказать, что сам первоисточник написан как раз евангелистом (читать можно, но осторожно), и я бы рекомендовал знакомиться с идеей через более критическое осмысление, например цикл статей Сергея Теплякова [14] на тему SOLID или его книгу [10] и не только. В книге есть и паттерны тоже, но рекомендовать их можно, на мой взгляд, уже для продвинутого изучения.

Принцип единственной ответственности в разных формулировках утверждает, что у любого класса должна быть одна-единственная, четко определенная зона ответственности, область применения, решаемая задача. Класс должен заниматься решением одной целостной задачи. Первоисточник еще утверждает о наличии «одной и только одной причины для внесения изменений в класс», но мне это объяснение, если честно, кажется неубедительным. Этот принцип, скорее всего, нарушается тогда, когда в одном классе смешиваются явно разнородные, разноуровневые с точки зрения здравого смысла обязанности, которые часто можно в описании противопоставлять друг другу (то, про что вы можете сказать: «X — это одно, а Y — это совсем другое»). Например, хранение информации — и отображение информации, ввод данных — и вывод данных, подготовка данных — и расчет с использованием этих данных, низкоуровневой — и высокоуровневой код. Почему это становится проблемой? Да потому что это получаются не очень зависимые между собой части, собранные в одном классе, каждая из которых может использоваться где-то отдельно сама по себе — или в комбинации с другими классами. Я бы предложил вот такой критерий: если какую-то часть класса вы можете вынести в отдельный класс и использовать отдельно саму по себе или в комбинации с другими классами, то ее нужно вынести в отдельный класс.

Принцип открытости/закрытости утверждает, что программные сущности (классы, модули, функции и другие) должны быть открытыми для расширения, но закрытыми для модификации. В чем проблема модификации класса? Очевидно, в том, что класс может уже использоваться в каких-то очень далеких от вас частях кода, о которых вы можете ничего не знать. Меняя класс, вы легко можете сломать работу кода, который его использует. Что тогда делать? Мы знаем, как писать ООП-код, который использует уже существующий код, — например, используя композицию или наследование. Есть какой-то класс, который почти такой, какой нужен? Не меняйте его, говорит нам этот принцип, — лучше унаследуйтесь и добавьте новую функциональность в потомках. Означает ли это, что даже при обнаружении какой-то ошибки в классе вы не должны его изменять? Конечно, нет! Ошибки и исправления могут и должны вноситься в существующий класс или модуль, даже если кто-то там уже использует эти ошибки и их исправление сломает его поведение. Принцип открытости/закрытости утверждает, что разрабатывать хорошую, годную программную систему — одновременно расширяемую, в которой появляется новая функциональность, и устойчивую, в которой добавление новой функциональности не ломает тех, кто использует только старую, — можно путем выбора такой архитектуры, когда новая функциональность появляется дописыванием новых классов, а не изменением существующих. Ну и просто в соответствии с принципом инкапсуляции вы должны иметь возможность безопасно (без неожиданностей для тех, кто класс использует) изменить внутреннюю реализацию класса, не меняя его интерфейс, поэтому под принципом открытости/закрытости иногда понимают еще и открытость класса к использованию через его интерфейс и закрытость внутренней реализации класса.

На самом деле, мы уже много где видели следование этому принципу в предыдущих разделах. Например, в паттерне «Декоратор» мы не изменяли существующий класс, когда хотели, чтобы он сам научился рисовать вокруг себя рамку, а дописывали новый класс-обертку `Decorator`, который в итоге позволял добиться того же самого, но оставив исходный класс `Point` неизменным. В паттерне «Команда» для того, чтобы научить программу выполнять новое действие с объектом (к тому же отменяемое!), не надо было изменять сам объект, — нужно было дописывать новый класс для новой команды, а основной цикл в программе при этом даже не меняется. Наконец, в паттерне «Посетитель» мы вообще делаем как будто невозможное — вместо того, чтобы добавить новый виртуальный метод в базовый класс иерархии (сложно представить себе в ООП более «опасное» действие), добавляем нового обработчика, который по факту приводит к тому же самому результату. Развитие кода путем дописывания нового кода, а не изменения существующего, — вот, на мой взгляд, главная идея этого принципа.

Принцип подстановки Лисков в разных, более и менее формальных формулировках утверждает, что вместо любого класса можно подsunуть любого из его потомков и при этом ничего не должно сломаться. Это не значит, что ни в коем случае нельзя наследоваться и изменять при наследовании поведение существующего класса. Это значит только, что бывает такое наследование, которое на-

рушает принцип подстановки Лисков, а бывает такое, которое не нарушает. А что вообще означает в начале абзаца «ничего не должно сломаться»? Очевидно, что речь идет о чем-то, что с объектом делают: обращаются к свойствам, вызывают методы, — т. е. речь идет о каком-то контракте того, что можно делать с объектом. И принцип подстановки Барбары Лисков утверждает: контракт класса-потомка не должен противоречить контракту класса-предка. Если что-то умел предок, то это же должен уметь и потомок (и, возможно, что-то еще свое, конечно!). Все, что требуется потомку, обязательно должно было требоваться и от предка (и, возможно, что-то еще). Обратите внимание: в двух последних предложениях — предок и потомок находятся в разных частях предложения.

Сама проблема, для которой формулировался принцип подстановки Лисков, возникает из-за того, как по-разному мы можем понимать и для каких разных целей мы можем использовать такой совершенно конкретный механизм, как наследование одного класса от другого.

С одной стороны, техническая реализация такова, что при наследовании класса `Cat` от класса `Animal` класс `Cat` просто получает все свойства и методы класса `Animal` и право записываться в переменные, предназначенные для хранения объектов класса `Animal`. С другой стороны, вот это «право записываться в переменные, предназначенные для хранения объектов класса `X`» связано совсем с другим: с пониманием наследования как отношения «является частным случаем». Мы утверждаем, что потомок `Cat` имеет право записываться в переменные, предназначенные для хранения объектов класса `Animal` потому, что `Cat` есть частный случай `Animal`, т. е. `Cat` в то же самое время есть и `Animal` тоже.

И два этих понимания могут входить в противоречие друг с другом, как раз и нарушая принцип подстановки Лисков. Классическим примером здесь является так называемая проблема круга-эллипса. `Circle` должен быть потомком `Ellipse` или наоборот? С одной стороны, математика утверждает, что `Circle` является частным случаем `Ellipse`, потому что любой круг — это эллипс, но не любой эллипс — это круг. Тогда, получается, следуя логике классов `Animal-Cat`, нужно `Circle` сделать потомком `Ellipse`. Но тогда получается, что у круга будут свойства и методы эллипса, такие как `RadiusX`, `RadiusY`, `StretchX`, `StretchY`, которые для круга не имеют смысла и не могут быть выполнены. С другой стороны, если мы выберем `Ellipse` потомком `Circle`, реализуем у `Circle` свойство `RadiusX`, а `Ellipse` будет его наследовать и добавлять свое свойство `RadiusY`, то вроде бы лишних свойств ни у кого не будет, но тогда получится, что эллипс есть частный случай круга, и тот код, который использовал круг (например, рисовал его внутри квадрата и рассчитывал, что он там уместится), при использовании «частного случая круга» вдруг неожиданно обнаружит, что этот «круг» больше не умещается в квадрате, хотя его радиус все еще равен половине стороны квадрата. «Проблема круга-эллипса» показывает, что некоторые взаимосвязи вида «является частным случаем» между классами просто нельзя описать прямым наследованием, не нарушив принцип подстановки Лисков. В данном случае, если мы хотим этому принципу следовать, мы должны оба класса, и `Ellipse`, и `Circle`,

унаследовать от какого-то общего предка, а не друг от друга. И дублировать в них общий код типа `RadiusX`, спросите вы?

Принцип разделения интерфейса, если его объяснять совсем на пальцах, применяет концепцию принципа единственной ответственности к интерфейсам, причем не к тем, кто эти интерфейсы реализует (это классы, реализующие интерфейсы, и за ними «бдит» принцип единственной ответственности), а к тем, кто эти интерфейсы использует. Формулировка принципа может быть примерно такая: код не должен зависеть от методов, которые он не использует или не должен использовать. Если мы работаем с каким-то классом через какой-то интерфейс, то в этом интерфейсе не должны быть методы, которые мы не используем или не должны использовать. Если вдруг такое случается, то такой интерфейс должен быть разделен на более мелкие части, и мы должны работать с классом через одну из этих частей.

```
class IShape {
public:
    void move() = 0;
    void draw() = 0;
    void save(FILE *f) = 0;
    void load(FILE *f) = 0;
}

void saveToFile(string filename, IShape *shape) {
    // ...
    shape->draw();
    shape->move();
    // ...
}
```

Вот тут у нас метод `saveToFile()` берет и вместо сохранения фигуры рисует ее или двигает. Почему он так делает? Да потому, что может! Имеет ли место тут быть нарушение принципа единственной ответственности? Нет, этот принцип применим к классу, а мы тут класса не видим вообще. Возможно, когда у нас появится класс `CShape`, реализующий класс `IShape`, у нас появится повод к нему «прикопаться» на предмет того, а не слишком ли ты много на себя берешь: и рисоваться, и сохраняться, и передвигаться, — но пока такого класса нет. А нарушение принципа разделения интерфейса уже есть. Как его исправить, чтобы функция `saveToFile()` видела только ту часть объекта, которая ответственна за сериализацию? Да просто разделить интерфейс:

```
class IShape {
public:
    void move() = 0;
}

class IViewItem {
public:
    void draw() = 0;
}
```

```

class ISerializeable {
public:
    void save(FILE *f) = 0;
    void load(FILE *f) = 0;
}

void saveToFile(string filename, ISerializeable *shape) {
    // ...
    shape->draw(); // ошибка
    shape->move(); // ошибка
    // ...
}

```

Обратите внимание, что в функцию `saveToFile()` вполне может передаваться все тот же самый объект, но функции видна только та его часть, та его сторона, те его методы, которые ей необходимы для выполнения своей задачи.

Принцип инверсии зависимостей традиционно считается самым сложным и непонятным студентами, изучающими ООП. Я бы даже сказал, что причиной этого является то, что, в отличие от всех остальных принципов SOLID, этот принцип не основывается на таком же очевидном здравом смысле. Еще проблема чаще всего в том, что не всегда при объяснении этого принципа делается акцент на его главной идее (очень спорной, как по мне). Поэтому, даже рисуя картинку этого принципа на экзамене, можно ошибиться, потому что мы привыкли, что схему со стрелочками можно покорежить, но пока все указывает туда же, куда надо, схема со стрелочками считается правильной.

Вы видите разницу между этими двумя схемами (рис. 2.25 и 2.26)? Те же самые стрелочки соединяют те же самые квадратики, но одна картинка верная, а другая нет. При этом, как по мне, так они обе непонятные, потому что догадаться, почему класс `Mechanism` реализует не интерфейс `MechanismInterface`, а интерфейс `PolicyInterface` (картинка в 99% случаев воспринимается именно так), может, наверное, только сам Роберт Мартин Фаулер!

Это все, конечно, заговор, цель которого — помешать нам понять, что именно имел в виду «Дядюшка Боб» в его книге, но чтобы его обойти, надо читать первоисточник [12]. Согласно первоисточнику принцип инверсии зависимостей состоит из двух частей, и со второй частью, как правило, проблем нет.

Вторая часть звучит так: «Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций». Если под абстракциями понимать абстрактные классы и интерфейсы, а под деталями — те классы, которые их реализуют, то смысл этого принципа — избежать ситуации, когда интерфейсы зависят от какого-нибудь конкретного класса. В более сильной форме это утверждение даже требует, чтобы везде, где это только возможно (но не там, где это невозможно) нужно стараться работать с объектами через интерфейсы:

```

class ISection {
public:

```

```
CPoint get_middle() = 0;  
// IPoint get_middle() = 0;  
}
```

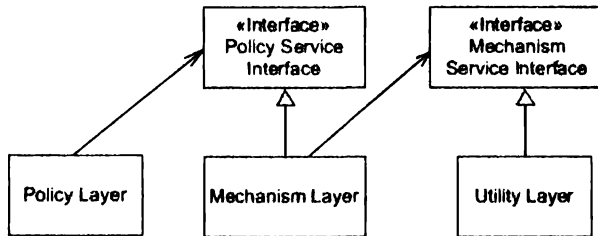


Рис. 2.25. Ненастоящая картинка DIP из Википедии

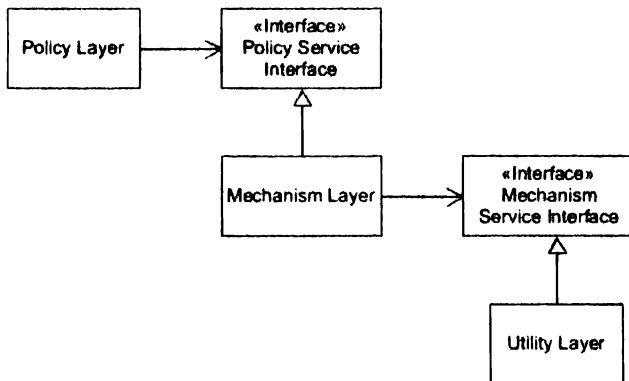


Рис. 2.26. Настоящая картинка DIP из Википедии

Интерфейс отрезка, например, может позволять возвращать середину отрезка, но если записано, что он возвращает конкретный класс `CPoint`, а не интерфейс `IPoint`, то это и будет нарушением принципа инверсии зависимостей. Почему? Потому что тогда интерфейс не может вернуть ничего другого, кроме `CPoint`, и будет от него зависеть. Пользователю этого интерфейса, возможно, будет вполне достаточно методов, описанных в `IPoint`, и тогда `CPoint` можно было бы заменить на любой другой объект, реализующий этот интерфейс, но тут мы сами себе закрываем эту возможность. К тому же при описании такого интерфейса можно случайно потащить за собой импортированный выше класс `CPoint` и весь его модуль и неожиданно начать зависеть от всей кодовой базы сразу. Цель же этой второй части принципа инверсии зависимостей — получить этаким слоем абстракций, описывающий взаимодействие разных частей кода (по сути, набор интерфейсов), который никак не зависит от конкретных классов и поэтому эти самые конкретные классы можно взаимозаменять.

А вот с первой частью принципа все хуже. Она звучит обманчиво понятно и скрывает за собой ту идею, которую далее в своей книге описывает Мартин:

«Модули верхнего уровня не должны зависеть от модулей нижнего уровня. И те и другие должны зависеть от абстракций».

Вот такая ситуация может у нас гипотетически сложиться (рис. 2.27). В одном модуле определен класс точки `Point`, в другом модуле определен класс отрезка `Section`, который создает и хранит ссылки на `Point` (композиция?), в третьем модуле определен класс графической сцены, который создает и хранит ссылки на `Section` (композиция?). В итоге модуль (верхнего уровня) сцены зависит от модуля (среднего уровня) отрезка, который зависит от модуля (нижнего уровня) точки.

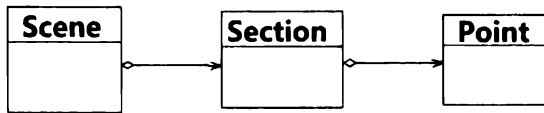


Рис. 2.27. Исходная ситуация: классы зависят друг от друга

Помня вторую часть принципа, мы должны сказать: «Конкретные объекты не должны зависеть друг от друга, они должны все зависеть от интерфейсов», и переписать код примерно так (рис. 2.28).

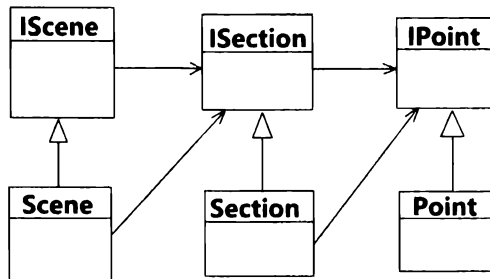


Рис. 2.28. Классы больше не зависят друг от друга, все зависит от интерфейсов

Теперь конкретные классы не зависят от конкретных классов, они зависят от интерфейсов. И интерфейсы тоже зависят от интерфейсов. Каждый класс сам по себе в силу обычной инкапсуляции предоставляет интерфейс в виде набора публичных свойств и методов и прячет свою реализацию, но мы, по сути, тут добавляем новый слой абстракции: у каждого класса отдельно выделяется его интерфейс и отрезок `Section` теперь зависит не от точки `Point`, а от интерфейса `IPoint`. Тут на самом деле спряталось очень важное изменение. Раз теперь отрезок `Section` не зависит от `Point`, он не может его создавать, как это было раньше:

```

class Section {
private:
    Point p_beg;
    Point p_end;

```

```
public:
    Section() { ... }
    ...
}
```

Теперь эту композицию придется менять на агрегацию. Это значит, что `Section` теперь должен получать или фабрику, с помощью которой он будет создавать `IPoint`, или уже два готовых `IPoint`:

```
class Section {
private:
    IPoint *p_beg;
    IPoint *p_end;
public:
    Section(IPoint *p_beg, IPoint *p_end) {
        this->p_beg = p_beg;
        this->p_end = p_end;
    }
    ...
}
```

И многие при обсуждении принципа инверсии зависимостей на этом останавливаются, хотя резонный вопрос: а при чем тут инверсия? Что тут инвертируется? Ответ на этот вопрос можно получить, если разнести все эти классы и интерфейсы по модулям. Как мы поступили бы, ничего не зная про принцип DIP (рис. 2.29)?

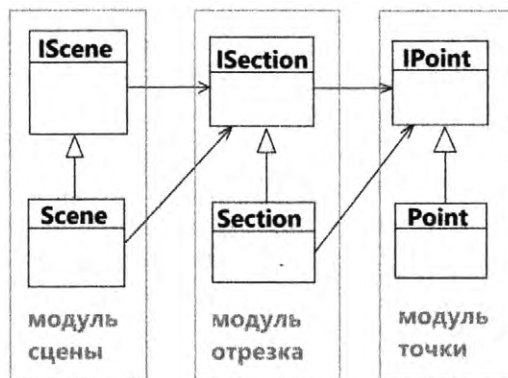


Рис. 2.29. Разбиение по модулям без DIP

Во-первых, это сохраняет всю ту ситуацию, для разрешения которой придумывался принцип DIP: модуль сцены зависит от модуля отрезка, а модуль отрезка зависит от модуля точки. Во-вторых, это нелогично и без всякого DIP. Если мы включаем интерфейс отрезка `ISection` в модуль отрезка, то как только кому-то понадобится интерфейс `ISection`, он будет вынужден подключить этот модуль вместе с самим `Section`, что сразу убивает весь смысл.

Так вот главная (первая) идея принципа инверсии зависимостей в применении к этому примеру заключается в том, что мы смотрим совсем под другим углом, например, на `ISection`. Применяя принцип DIP, мы говорим, что `ISection` — это не интерфейс отрезка, а это интерфейс того, что классу `Scene` именно нужно от отрезка. А раз так, то этот интерфейс надо, во-первых, переименовать, а во-вторых, перенести в модуль сцены (рис. 2.30).

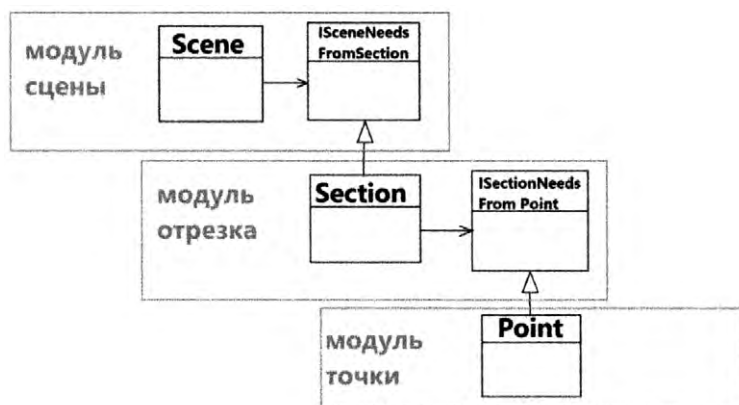


Рис. 2.30. Разбиение по модулям в DIP

Теперь не точка определяет, какой интерфейс она предоставляет, а отрезок определяет, какой интерфейс от точки ему нужен, а точка вынуждена ему соответствовать. И теперь действительно, все зависимости (между модулями!) переворачиваются с ног на голову: теперь модуль отрезка зависит от модуля сцены, а модуль точки — от модуля отрезка. Когда смотрите на картинку с принципом инверсии зависимостей, всегда смотрите, нарисованы ли там границы модулей. Если не нарисованы, то автор картинки просто не понял Мартина.

В этом, собственно, и состоит идея инверсии зависимостей в задумке Мартина. Модули верхнего уровня (начальники) определяют абстрактные интерфейсы того, что им нужно от модулей среднего уровня (подчиненных), а модули среднего уровня вынуждены подстраиваться и зависеть от этих требований. А раньше было как? Подчиненные что-то там умели, а начальники зависели от того, что умели подчиненные, и вынуждены были под их умения подстраиваться. Видите разницу? А по факту это все только вопрос того, что гибко, а что жестко и не может меняться: что гибче — требования заказчиков или умения исполнителей?

Да, конечно, тут все равно остаются вопросы. Во-первых, так ли уж правильно описывать не интерфейс того, что должен представлять собой `Point`, а интерфейс того, что нужно от `Point` кому-то другому? Или нужно и то и то? Если отрезку `Section` нужно от `Point` что-то одно, а кругу `Circle` нужно от `Point` что-то другое, то, как правильно отмечено в статьях [14], должен ли бедный `Point` наследоваться сразу от двух (трех, пяти, десяти) интерфейсов? Или это все-таки

должны быть зафиксированные в документах (в интерфейсе `IPoint`) сервисы, которые `IPoint` может предоставлять, — и все?

Во-вторых, все-таки остается открытым вопрос помещения в один модуль и интерфейсов, и конкретных классов. Возможно, лучше все-таки просто выносить интерфейсы в отдельные интерфейсные модули (рис. 2.31)? А возможно, тут вообще интерфейсы не нужны? А когда нужны? Об этом всегда нужно думать, чтобы борьба со сложностью не стала сложнее борьбы с абстракциями.

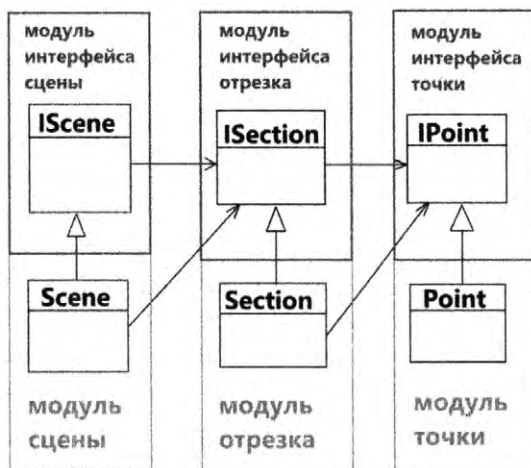


Рис. 2.31. Разбиение по модулям с интерфейсными модулями

Не могу сказать, что для меня вопрос правильного применения принципа инверсии зависимостей окончательно закрыт. С одной стороны, идея автора этого принципа понятна. С другой стороны, очень многие его рассуждения просто не выдерживают никакой критики. Например, дальше в своей книге он дает пример использования этого принципа для выстраивания «правильной» зависимости между такими объектами, как кнопка, включающая лампу, и лампа, управляемая кнопкой. Я специально так описал оба этих объекта, которые он приводит, чтобы показать, что вообще не очевидно, кто тут от кого должен зависеть, и кто из них тут низко- или высокоуровневый. Любому, кто достаточно владеет приемами ООП, будет претить любая жесткая зависимость между этими объектами, потому что каждый из них может существовать отдельно друг от друга, и подключаются они друг к другу даже в реальности — каким-то третьим объектом, проводом, который особым способом соединяет их. И это мы даже не вспомнили паттерн «Наблюдатель», который в данном случае очень напрашивается!

2.29. Объектная интроспекция и рефлексия

Интроспекция и рефлексия — два связанных понятия, описывающих возможность программного кода анализировать (интроспекция) и изменять (рефлексия) свою собственную внутреннюю структуру. Очень частный и ограниченный слу-

чай этой технологии называется *интроспекция типов* (type introspection), иногда он связан с автоматическим (или не очень) включением в состав класса расширенной информации о типе (RTTI — runtime type information) и означает только возможность спросить у объекта: «Ты кто такой? ~~Давай, до свидания!~~». Помните, мы давным-давно реализовывали виртуальную функцию `classname()`, позволяющую у любого объекта выяснить его тип? Так вот, по сути, мы реализовывали свой собственный упрощенный вариант интроспекции типов и RTTI. Возможность динамического приведения типов требует наличия в языке этой возможности, поэтому ответ на вопрос: «Зачем нужна интроспекция типов?» тут очевиден.

Расширенное понятие рефлексии, относящееся не только к типам, включает в более широком смысле интроспекцию (introspection) и вмешательство (intercession, но это второе слово используют редко, обычно в этом случае говорят просто о рефлексии), позволяющее исследовать и манипулировать всеми программными сущностями исполняемого кода: и объектами, и функциями, и модулями.

Когда говорят об объектной интроспекции и рефлексии, имеют в виду возможность, имея произвольный объект, узнать, какие у него есть свойства и методы и какие они значения имеют (интроспекция), и изменять значения его свойств и вызывать его методы (рефлексия), не пользуясь при этом стандартным способом приведения к нужному типу (потому что тот может быть просто недоступен и неизвестен):

```
void foo(Unknown *u) {
    SomeClass *o = dynamic_cast<SomeClass *>(u);
    if (o != nullptr){
        o->someMethod(...);
        ...
    }
}
```

Этот код, безусловно, использует интроспекцию типов, это не объектная интроспекция и не рефлексия, потому что код, условно, знает, чего можно ждать: делает предположение о типе объекта, и если оно оправдывается, то использует его обычным порядком. Как выглядит изменение свойства `someProp` и вызов метода `someMethod()` у объекта с использованием объектной рефлексии? Это зависит от того, как она реализована. Например, вот так:

```
void foo(Reflectable *r) {
    Prop *p = r->getPropByName("someProp");
    if (p != nullptr){
        p->setValue(Value(p->getValue()->AsDouble() * 2));
    }

    Method *m = r->getMethodByName("someMethod");
    if (m != nullptr){
        list<pair<string, Value>> params;
```

```
    params.push_back(make_pair("x", Value(5)));
    params.push_back(make_pair("y", Value(6)));
    m->invoke(params);
}
}
```

Понятно, что для того, чтобы это все даже просто компилировалось, должны быть известны классы `Value`, `Prop`, `Method` и, разумеется, сам интерфейс типа `Reflectable` для того, чтобы можно было обращаться к объекту через рефлексию:

```
class Value {
public:
    virtual double AsDouble();
    virtual double AsString();
    Value(double);
    Value(string);
    ...
};

class Prop{
public:
    virtual Value* getValue();
    virtual void setValue(Value);
};

class Method{
public:
    virtual void invoke(list<pair<string, Value>>);
};

class Reflectable {
public:
    virtual Prop* getPropByName(string name);
    virtual Method* getMethodByName(string name);
};
```

Сравните с тем, как легко было бы сделать то же самое без использования интроспекции...

```
void foo(NonReflectable *r) {
    r->someProp = r->someProp * 2;
    r->someMethod(5, 6);
}
```

...и станет очевидно, что согласиться грызть такой кактус можно только при наличии хоть какой-нибудь серьезной выгоды. И такая выгода может появиться там, где с большим количеством до предела разнообразных объектов необходи-

мо работать единообразно, ничего не зная про их типы. Где такое может понадобиться?

Ну, например, если вы делаете какой-нибудь визуальный редактор и хотите реализовать подобное окно инспектора свойств (рис. 2.32) с возможностью исследовать и менять все свойства у любого выбранного объекта. Содержимое этого окна будет меняться всякий раз, когда пользователь будет выбирать в рабочем поле новый объект. Очевидно, что без рефлексии придется где-то в коде писать большой `switch-case` вида «если выбран объект такого-то класса, то добавь в инспектор свойств вот такой список, а если выбранный объект имеет другой класс, то добавь в инспектор свойств другой список» и т. д. Понятно, что синхронизация такого кода с реальным содержанием классов будет кошмаром. И такое окно свойств будет вообще невозможно, если программа работает с классами объектов, которые приходят из какой-нибудь динамически подключаемой библиотеки, потому что вообще ничего не знает о классах объектов, которые в этой библиотеке могут быть реализованы.

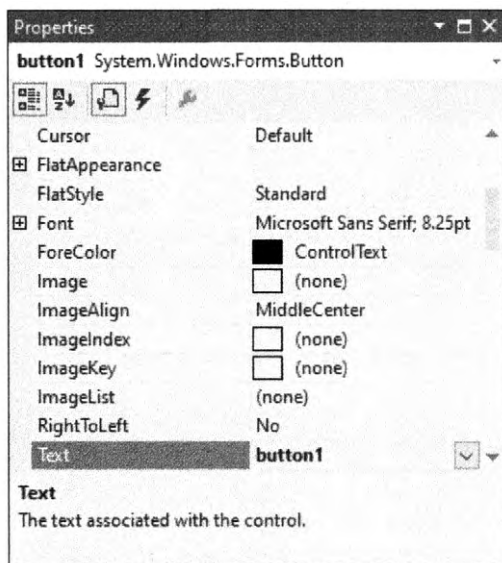


Рис. 2.32. Окно свойств

А что если библиотека реализована вообще на другом языке? А вы хотите обратиться к какому-то объекту из нее и работать с ним. Какой минимум функций от такого объекта будет вам нужен, чтобы, «зацепившись» за этот минимум, иметь возможность выяснить, какие функции у этого объекта есть? Наверное, этим минимумом будет функция вида `QueryInterface`, которая позволяет узнать у объекта, поддерживает ли он какой-то заданный интерфейс (по сути, это аналог нашей функции `getMethodByName`, а точнее, просто аналог `dynamic_cast` или `isA`). Сами интерфейсы тогда, конечно, должны иметь не имя, а какой-то глобальный уникальный идентификатор (он так и называется: `guid`), а функция

QueryInterface должна быть независимой от языка. То, что я сейчас описал, называется старой технологией COM (Component Object Model), придуманной 30 лет назад, и представляет собой, в определенном смысле, форму интроспекции типов.

2.30. Событийно-управляемое приложение: событие, типы событий, сообщение, обработчик, механизм обмена сообщениями, очередь сообщений

Этот раздел представляет собой небольшой ликбез по созданию GUI-приложений, т. е. приложений с графическим пользовательским интерфейсом, и предназначен он для тех, кто раньше создавал только консольные приложения с линейным потоком выполнения. В таких приложениях команды выполняются последовательно, одна за другой (с учетом циклов, ветвлений и прочих команд, изменяющих порядок их выполнения), и порядок выполнения команд в определенных пределах можно предсказать. Когда последняя команда заканчивает свое выполнение, приложение завершает работу.

Графические приложения чаще всего работают совсем по-другому: в рамках событийно-ориентированного подхода. В таких программах все, что происходит, происходит исключительно как реакция на какие-либо события, и если никаких событий не происходит, то приложение не делает вообще ничего и не занимает процессорное время. Раз происходящие события и их порядок зависят от субъективной воли пользователя («хочу — нажму эту кнопку, хочу — ткну сюда мышкой»), то и предсказать, в каком порядке какой код будет выполняться в таких приложениях, гораздо сложнее. Я попробую в рамках этого короткого раздела описать основные приемы, благо они большей частью всегда одни и те же, несмотря на то, что GUI-приложения можно создавать на очень разных языках и в рамках очень разных GUI-фреймворков. Примеры в этом разделе будут создаваться в рамках фреймворка Windows Forms в среде Visual Studio на языке C#.

Эта книга — про ООП, но в этом разделе мы почти не будем создавать собственные объекты и вообще не будем создавать собственные классы, мы будем учиться создавать GUI-приложения. Запустите студию, создайте новое решение, новый проект, выберите шаблон для C# и Windows Forms Application и, как говорится, follow the white rabbit. И вообще, событийно-управляемые приложения совсем не обязательно будут объектно-ориентированными. Разработчики с большим опытом наверняка приведут примеры типа такой content management system, как Drupal, которая (в мое время в старых версиях была, а сейчас, конечно, я уже не знаю, как обстоят дела) как раз событийно-ориентированная, но не объектно-ориентированная. Но я отвлекся.

Когда проект будет создан, его уже можно запускать. Поздравляю вас: первое свое GUI-приложение вы уже создали. Несмотря на то что при запуске появится

просто пустая форма, эта форма уже умеет очень многое: сворачиваться и разворачиваться, распахиваться на весь экран и закрываться, у нее даже есть меню по иконке в верхнем левом углу! Она умеет так много, потому что мы используем и будем использовать в дальнейшем готовые заранее разработанные графические объекты-компоненты (они еще называются **controls**, стандартные элементы управления): вся наша GUI-программа будет из них состоять. Вашей задачей чаще всего будет собрать эти компоненты вместе, организовав их нужное взаимодействие для достижения цели. Все чаще всего нужные вам компоненты находятся в окне **Toolbox** в секциях **Common Controls**, **Containers** и **Menus & Toolbars**.

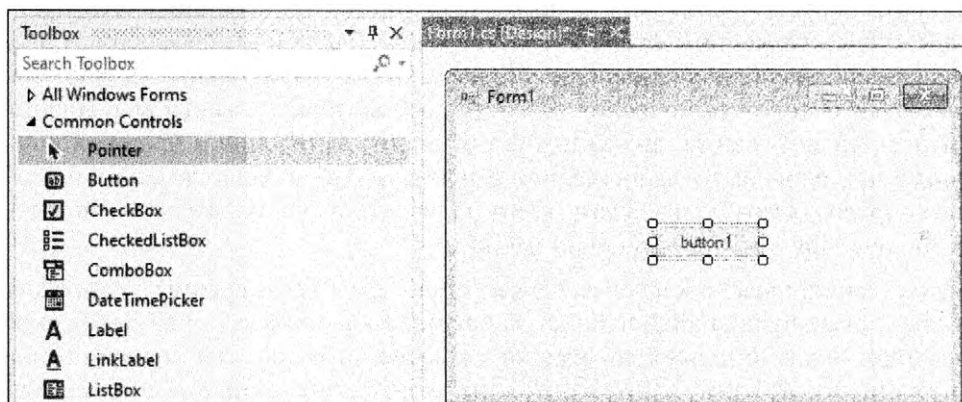


Рис. 2.33. Перетащите кнопку из списка объектов на форму

Возьмите в окне **Toolbox** кнопку, перетащите ее на вашу форму и запустите приложение снова. Вы увидите, что кнопка появилась в вашем окне, ее даже можно нажимать, она реагирует на нажатие, изменяя немного свой внешний вид, но пока больше ничего полезного не происходит (рис. 2.33). Сама кнопка выглядит совершенно обычно, я бы даже сказал, банально, и если вы хотите, чтобы ее внешний вид изменился, вы должны изменить свойства этой кнопки, т. е. свойства этого конкретного объекта-экземпляра какого-то стандартного класса **Button**.

Хочу еще раз подчеркнуть: весь внешний вид и все особенности поведения каждой конкретной кнопки (которых у вас может быть много) целиком и полностью определяются значениями ее свойств. А значения эти можно изменять как во время работы (мы еще говорим «в run-time»), так и до запуска программы (мы это называем «в design-time»). Во время работы программы это у вас будет какой-то объект, например `button1`, и вы будете просто изменять значения свойств, как у любого другого объекта, например `button1.Text = "Hello World!"`. А до запуска программы вы выберете нужную кнопку на форме, откроете окно редактора свойств и просто введете нужное значение нужному свойству (рис. 2.34).

Кроме свойства `Text` вы найдете там же все 100 500 свойств, полностью определяющих внешний вид и поведение кнопки: от шрифта и текста до цвета, поло-

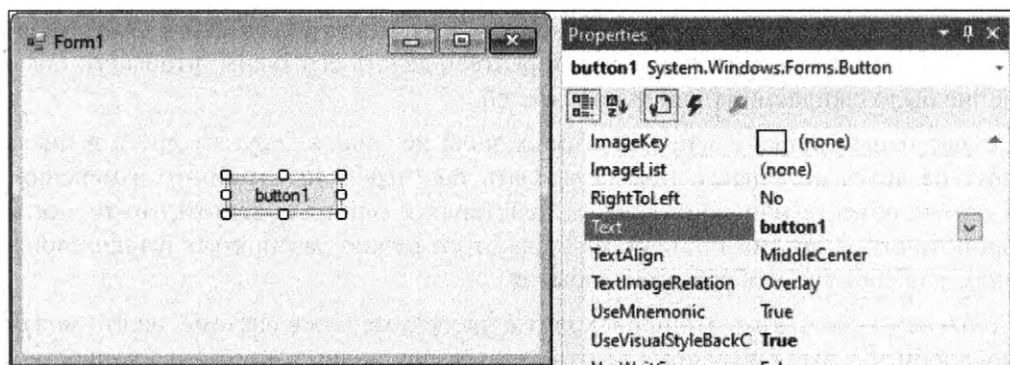


Рис. 2.34. Задание значения свойств в design-time

жения и флага блокировки. Если вы создадите две кнопки с абсолютно одинаковыми свойствами, то они будут визуально неотличимы друг от друга.

Бросьте на форму парочку объектов типа `TextBox`. Все объекты, которые вы добавляете, получают автоматически назначаемые имена, обычно по какой-то системе, связанной с типом и номером объекта. Разумеется, раз объекты имеют разные имена, то где-то у каждого из этих объектов есть свойство, которое за имя отвечает: оно называется `Name` (рис. 2.35). Имена объектов можно и нужно изменять, иначе очень скоро вы потеряетесь в своих автоматически названных `TextBox13` и `button42`. Придумайте сами какую-нибудь систему именований и скрупулезно придерживайтесь ее: переименуйте и форму, и кнопку, и оба текстовых поля.

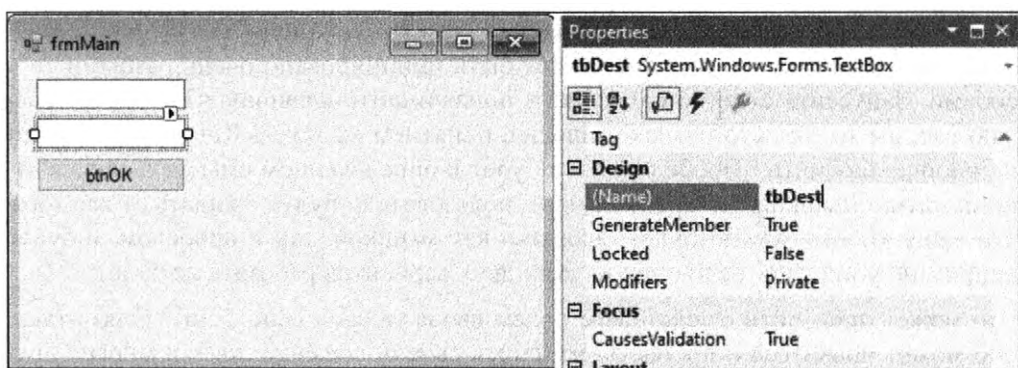


Рис. 2.35. Имена объектов

Снова запустите приложение и убедитесь, что кнопка и текстовые поля на месте. Убедитесь, что сами текстовые поля уже умеют многое: в них можно вписывать текст, в них работает `<Ctrl>+<C>`, `<Ctrl>+<V>`, работает выделение и т. д. Эти классы и их стандартное поведение создавали не вы, но их создатели сделали их такими, чтобы любой пользователь вашего приложения не нашел в этих объектах ничего неожиданного, типа стирания всего текста по комбинации клавиш

<Ctrl>+<C>. Это очень важный момент: всегда, когда вы создаете приложение с пользовательским интерфейсом, старайтесь делать его таким, чтобы его поведение было ожидаемым для пользователей.

Сейчас наша кнопка и наши текстовые поля не зависят друг от друга и никак друг на друга не влияют. Как же сделать так, чтобы при каких-то изменениях в одном объекте или при каких-то действиях с одним объектом что-то могло происходить с другими объектами? Для этого нужно разобраться в таких понятиях, как событие и обработчик события.

Событие — это что-то, что происходит в программе или в системе, на что может понадобиться ваша реакция. Событие всегда привязано к какому-то конкретному моменту, оно может уже произойти или еще не произойти, оно не может происходить хоть какое-то протяженное время. Напомню уже приведенный ранее пример: «дождь пошел» — это событие, «дождь закончился» — тоже событие, «дождь идет» — это не событие (это состояние, если вы вспомните диаграмму состояний), потому что оно имеет длительность. В то же время «дождь идет очередной час» — это событие, потому что оно локализовано в конкретный момент времени.

У каждого графического объекта есть целый список событий, потому что нам может понадобиться реагировать на очень разные происходящие с ним события. Например, что может такого произойти с кнопкой? Кнопка может появиться на экране, пропасть с экрана, изменить свое положение или цвет. Кнопку могут нажать, на кнопку могут навести указатель мыши, с кнопки могут увести указатель мыши, кнопку могут нажать мышкой и т. д. При этом, если вам показалось, что первое и последнее упомянутое в предыдущем предложении — это одно и то же, то это не так: кнопку могут нажать мышкой, а кнопку могут просто «нажать», и это последнее событие может быть инициировано очень разными способами. Запустите свое приложение и понажимайте клавишу <Tab>. Поздравляю вас, вы только что познакомились с понятием «фокус»! Когда фокус будет на кнопке, нажмите <Пробел> — это будет в определенном смысле эквивалентно нажатию мышкой. По крайней мере, пользователи будут ожидать от вас того, что вашу кнопку можно будет «нажать» как мышкой, так и пробелом, и будут неприятно удивлены, если один из этих двух вариантов работать не будет.

Что может произойти с текстовым полем ввода `TextBox` (рис. 2.36)? Опять-таки, оно может появиться и пропасть, изменить свое положение, цвет и многие другие свойства, на него могут навести и нажать мышкой. То есть очень многое из того, что может произойти и с кнопкой, и с многими другими объектами. Весь список событий, которые могут произойти с объектом, вы можете видеть где-то рядом со списком свойств.

Однако каждый из используемых объектов вообще-то предназначен для довольно конкретной цели, и поэтому у него чаще всего есть его главное событие — то, на которое чаще всего нужно будет реагировать (рис. 2.37). Что будет чаще всего происходить с кнопкой такого, на что нам нужно будет реагировать? Оче-

видно — нажатие (причем не мышкой, а просто «нажатие»). На что чаще всего нужно реагировать в текстовом поле ввода? Очевидно — на изменение текста в этом поле. Поэтому мы говорим, что у кнопки «главное» событие — это `Click`, а у текстового поля — `TextChanged`. Если вы привяжете какую-то функциональность к наведению мыши на кнопку или к нажатию на текстовое поле ввода — пользователи будут очень недовольны. По крайней мере, я буду.

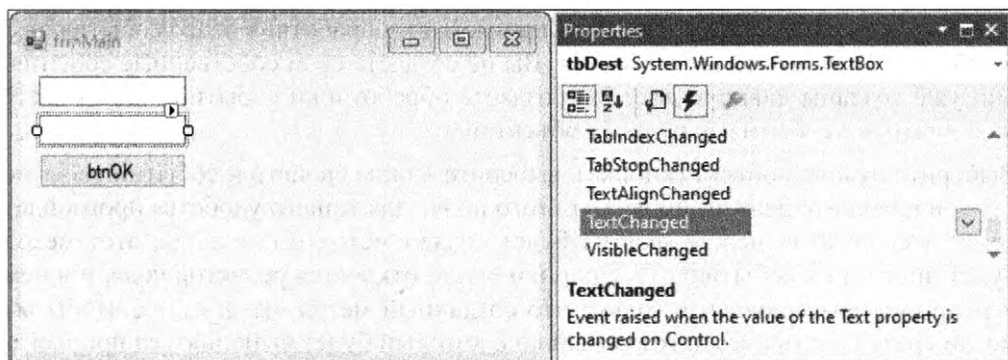


Рис. 2.36. Часть списка событий и главное событие объекта `TextBox`

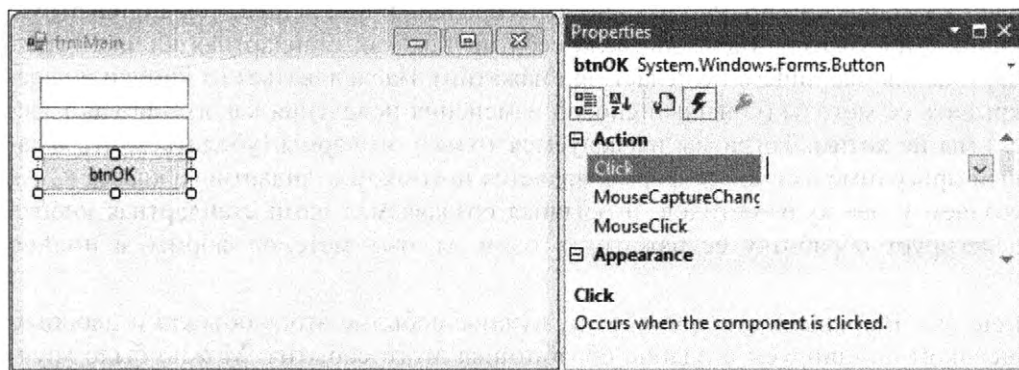


Рис. 2.37. Главное событие кнопки

Итак, если мы хотим, чтобы что-то в программе происходило (не обязательно с кнопкой) при нажатии конкретной кнопки? Происходить может только конкретный код. Поэтому переформулировать нашу хотелку можно следующим образом: как сделать так, чтобы при нажатии конкретной кнопки выполнялся конкретный код? Конкретный код должен быть оформлен в виде отдельной функции, и в случае используемого нами фреймворка эта функция должна быть методом какого-то класса.

Функция (метод), которая должна будет выполняться при возникновении определенного события, называется обработчиком события. Конкретный обработчик события «привязывается» к конкретному событию у конкретного объекта. У другого объекта может быть другой обработчик на другое событие, может

быть другой обработчик на то же самое событие или тот же самый обработчик на то же самое событие. Другими словами, если у вас есть много кнопок, то на часть из них вы можете «повесить» один обработчик, на часть — другой или на все кнопки — один и тот же обработчик или, чаще всего, на каждую кнопку — один и тот же обработчик.

Итак, еще раз. Обработчик события — это оформленный в виде метода кусок кода, который «привязан» к конкретному событию конкретного объекта. Как только это событие с этим объектом произойдет, обработчик этого события этого объекта автоматически вызовется. Вы не создаете свои собственные события, они уже созданы давно за вас. Вы создаете обработчики событий и связываете их с нужными событиями нужных объектов.

Выберите нужный объект (кнопка), выберите в окне свойств и событий событие `Click` и щелкните дважды справа от этого поля. Для вашего удобства произойдет сразу несколько вещей. У формы будет создан метод `btnOK_Click`, этот метод будет привязан к событию `Click` кнопки `btnOK`, откроется редактор кода, и в нем курсор будет установлен в только что созданный метод `btnOK_Click`, чтобы вы могли сразу начать писать тот самый код, который будет выполняться при нажатии кнопки.

Кстати, вспомните раздел про делегирование и убедитесь снова, что мы наблюдаем классический пример именно делегирования. Мы используем стандартную кнопку, но хотим, чтобы она вела себя не так, как стандартная кнопка (стандартная кнопка ничего не делает при нажатии). Наследоваться от кнопки и перекрывать ее методы (обычный способ изменения поведения какого-нибудь класса) мы не хотим. Тогда мы наследуемся только от формы (убедитесь, что в вашей программе ваш класс формы является потомком стандартного класса `Form`), создаем у нее кучу методов, и обычная создаваемая нами стандартная кнопка делегирует обработку ее нажатия в один из этих методов формы, а именно `btnOK_Click`.

Еще раз. Выбираем нужный объект, нужное событие этого объекта и двойным щелчком инициируем создание обработчика этого события. Можно было вручную у формы создать такой метод, и тогда нам осталось бы вместо двойного щелчка, в том же самом поле, просто в выпадающем списке выбрать нужный нам метод — видите там треугольник от выпадающего списка (рис. 2.38)?

Что мы можем сделать в созданном только что обработчике события? Ну, например, при нажатии кнопки `btnOK` мы будем стирать текст из текстового поля `tbDest`? Нет ничего проще: `tbDest.Text = ""`; (рис. 2.39).

Запустите программу и убедитесь, что теперь при нажатии кнопки текст в одном из текстовых полей ввода (если он там был) очищается. Нам ничего не мешает повесить этот же самый обработчик на то же самое событие у другого объекта или другое событие у другого объекта. Для этого достаточно выбрать сначала нужный объект (например, `tbDest`), а потом в выпадающем списке для нужного события (например, того же самого `Click`) выбрать нужный уже существующий обработчик (рис. 2.40).

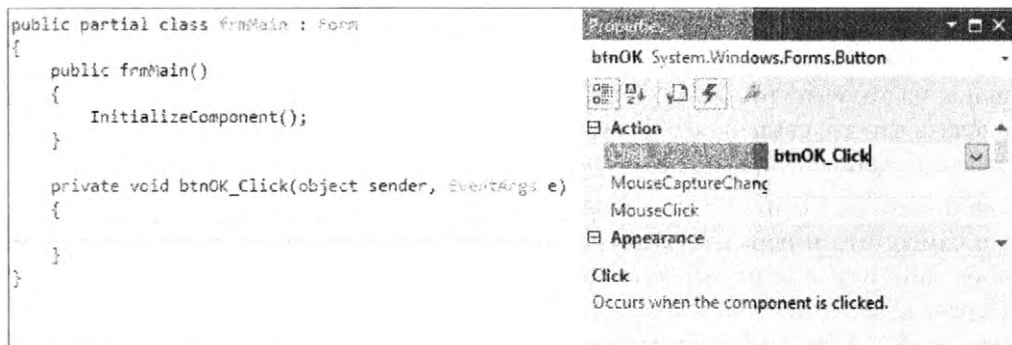


Рис. 2.38. Создание обработчика события

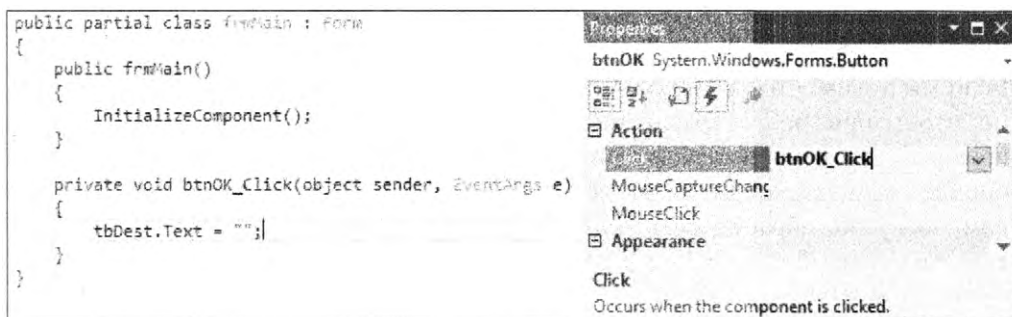


Рис. 2.39. Готовый обработчик события Click кнопки

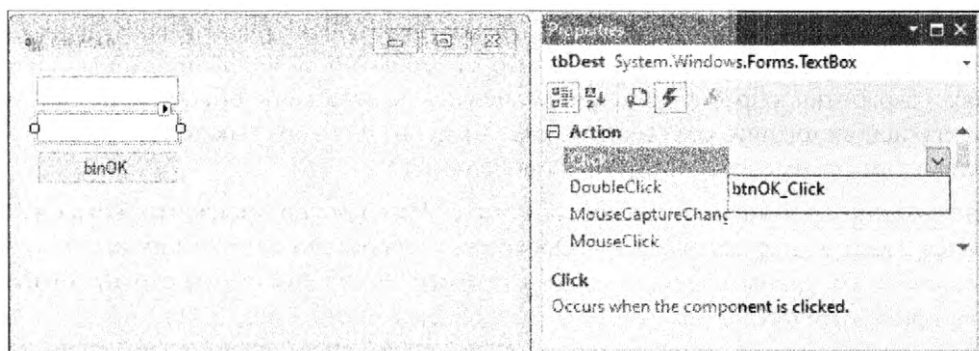


Рис. 2.40. Привязка к событию уже существующего обработчика

Запустите измененную программу и убедитесь, что очистка текста в `tbDest` происходит и при нажатии мышкой кнопки, и при нажатии мышкой на само текстовое поле `tbDest`. Да, можно было сделать два разных обработчика и просто вписать в них одинаковый код, но принцип DRY не позволяет нам так поступать.

Все GUI-программы, которые вы будете создавать, будут представлять собой набор взаимодействующих объектов. Поэтому очень важно освоить эту самую

главную технику: как привязывать функциональность к событиям. При этом важно понять, что происходит что-то может с одним объектом, а при этом реакция на это «что-то» может влиять на совсем другие объекты. Когда вам нужно сделать что-то, связанное с взаимодействием объектов, важно уметь переводить эти требования на правильный язык событий и обработчиков событий.

Например, вы хотите, чтобы второе текстовое поле ввода всегда отображало то же самое, что и первое текстовое поле. Как это требования перевести на язык событий? Как обеспечить выполнение этого требования в терминах событий? Переведите это требование на язык событий; при этом появятся два главных вопроса: «Когда?» и «Что делать?». Вы должны спросить себя: при каком событии у какого объекта что должно произойти с каким объектом? Вопрос «Когда?» означает всегда одно и то же: при каком событии у какого объекта? Вопрос «Что?», как правило, означает: у какого объекта какое свойство должно измениться или какой метод вызваться?

Если мы хотим, чтобы второе текстовое поле всегда отображало первое, то нужно, чтобы при любом изменении текста в первом или втором `TextBox` текст из первого окна переносился во второе. Очень часто люди делают примитивную ошибку: если мы хотим, чтобы во втором `TextBox` было то же самое, что и в первом, то нужно обрабатывать только событие второго `TextBox`. Чтобы не допустить такую ошибку, и нужно формулировать ответы на эти два главных вопроса: «Когда?» и «Что?».

Итак, создаем двойным щелчком для события `TextChanged` объекта `tbSource` обработчик события. Он автоматически получит имя `tbSource_TextChanged`, но привязан к событию `TextChanged` объекта `tbSource` он, конечно, будет не поэтому: этот метод можно переименовать и дать ему любое другое имя. (Справедливости ради нужно сказать, что в некоторых событийно-ориентированных системах разработки обработчики привязывались к событиям объектов именно по факту использования соответствующего имени, поэтому выясните, как обстоят дела в том фреймворке, который вы используете.)

Что должно произойти в этом обработчике? Что должно произойти, когда изменился текст в первом `TextBox`? Очевидно — свойство `Text` у второго `TextBox` должно стать равным свойству `Text` у первого `Textbox`. И тот же самый обработчик привяжите к событию `TextChanged` у объекта `tbDest` (рис. 2.41).

Запустите программу и убедитесь, что все работает так, как должно. В частности, убедитесь, что теперь у вас не получается изменять текст во втором `TextBox`, потому что при любой попытке такого изменения этот текст снова заменяется на текст из первого `TextBox`. Возможно, у вас возникает вопрос, почему мы не попадаем в бесконечную рекурсию, ведь при изменении текста в первом поле изменяется текст во втором, а это должно снова вызвать изменение текста в первом поле? Чаше всего о такой проблеме уже кто-то умный подумал за нас, и при изменении значения свойства на то же самое вызова события об изменении не происходит: это ограничивает «цунами» бесконечных обновлений.

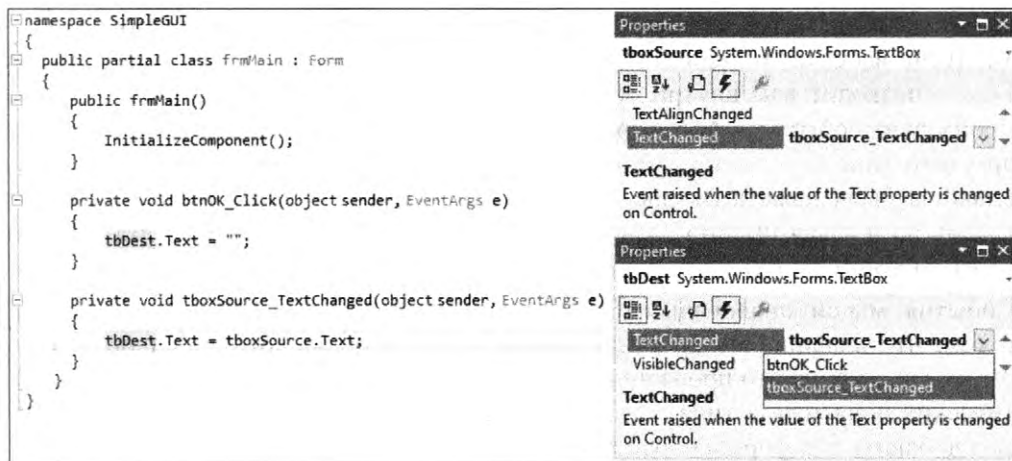


Рис. 2.41. Обработчик для переноса текста из первого TextBox во второй

Кстати, если текст в одном TextBox будет изменять не сам пользователь, а программный код, который будет изменять свойство `Text`, то обработчик события `TextChanged` все равно вызывается! Это значит, что обработчику все равно, инициировано событие (изменения текста) пользователем с помощью клавиатуры или мыши или же каким-то другим кодом путем программного изменения свойства `Text`.

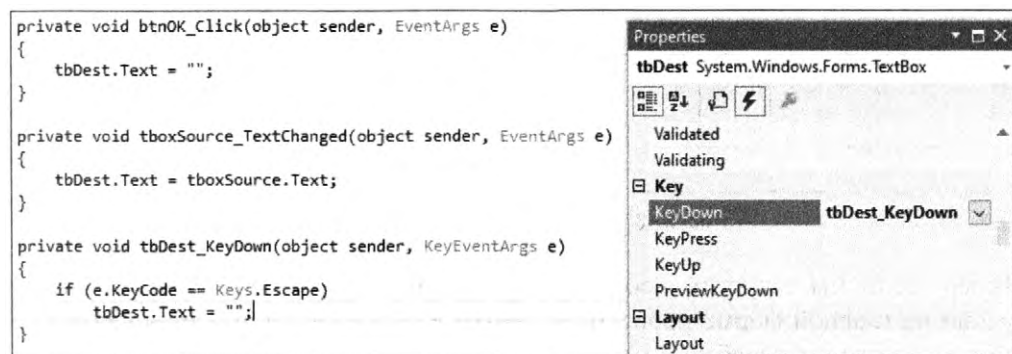


Рис. 2.42. Обработчик события может использовать дополнительную информацию о событии

Следующий интересный факт заключается в том, что кроме того, что каждое событие как бы несет с собой не только тот момент времени, когда оно произошло, но и какую-то дополнительную информацию (рис. 2.42). Например, если попробовать создать обработчик для события `KeyDown`, то выяснится, что в списке аргументов у этого обработчика (а обработчик, напоминаю, есть метод) есть объект `e` типа `KeyEventArgs`. То есть та внутренняя кухня, которая вам не видна, но которая и вызывает этот метод при возникновении соответствующего события, передает в этот метод два параметра. Параметр `sender` есть у любого

события, и в нем передается ссылка на тот объект, который инициировал событие (если вы свяжете один и тот же обработчик с разными объектами, параметр `sender` позволит вам внутри обработчика выяснить, с каким именно объектом произошло событие). А в обработчик события `KeyDown` передается еще один аргумент типа `EventArgs`, и он как раз несет информацию не только о том, что клавиша была нажата (и еще, кстати, не отпущена — для этого есть событие `KeyUp`), но и какая именно клавиша была нажата, была ли с ней зажата клавиша `<Ctrl>` и `<Shift>` и т. д.

События мыши аналогичным образом несут собой не только информацию о том, что клавиша мыши была нажата или курсор мыши был сдвинут, но и о том, где именно это произошло. Например, можно добавить на форму два статических поля текста типа `Label`, переименовать их в `lbCoordX` и `lbCoordY`, а потом добавить для формы обработчик события `MouseMove` (рис. 2.43).

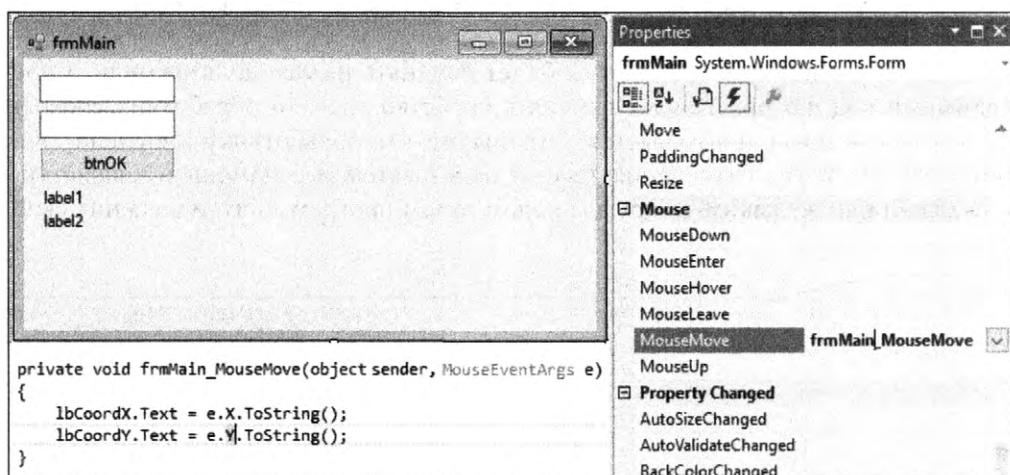


Рис. 2.43. Обработчик события `MouseMove`

Теперь если мы запустим программу, то убедимся, что при любом движении мыши на главной форме происходит событие `MouseMove`, вызывается связанный с этим событием обработчик `frmMain_MouseMove`, который вытаскивает из параметра `e` дополнительную информацию о том, где именно находился курсор мыши в момент происхождения события, и координаты курсора переносит в свойство `Text` объектов `lbCoordX` и `lbCoordY`. Таким образом, неправильно говорить о том, что программа как-то отслеживает положение курсора мыши. Программа ничего не отслеживает, она не обладает собственной активностью, она всего лишь реактивна, она реагирует на события. Любое передвижение мыши на форме вызывает событие, а программа на это событие реагирует, если определен соответствующий обработчик.

Кстати, обратите внимание, что при наведении курсора мыши на любой объект, находящийся на форме, координаты мыши «перестают обновляться». Это не

означает, что событие `MouseMove` не происходит, — оно, как и раньше, происходит, просто теперь не с формой, а тем объектом на форме, на который вы курсор навели. А раз с событием `MouseMove` у этого объекта не связан никакой обработчик, то никаких видимых проявлений этого события мы и не наблюдаем.

Самое важное, чему вы должны сейчас научиться, — это правильному использованию стандартных элементов управления с секций **Common Controls**, **Containers** и **Menus & Toolbars**. Это означает, что вы должны для каждого компонента выяснить, для чего он, вообще говоря, предназначен, какое у него главное свойство и какое у него главное событие. Например, очевидно, что если такой объект, как `Combobox`, предназначен для выбора в выпадающем списке одной из заготовленных заранее альтернатив, то его главным свойством будет коллекция этих альтернатив `Items`, а главным событием — выбор одной из альтернатив из списка по номеру (`SelectedIndexChanged`) или по значению (`TextChanged`).

Большинство событий возникают как реакция на действия пользователя непосредственно с объектами этого приложения. Но многие события являются результатом того, что в системе произошло нечто, требующее от приложения какой-то реакции. Например, приложение закрывается и нужна реакция пользователя на это закрытие (событие `FormClosing`), или форма изменила свой размер и требуется как-то перестроить положение объектов на форме (событие `SizeChanged`), или, например, какое-то другое окно было открыто поверх нашего окна, но теперь оно ушло и требуется перерисовка содержимого формы (событие `Paint`).

Кроме этого, различное поведение, связанное с задержками, тоже реализуется с помощью событий. В не-событийном приложении при необходимости организовать задержку можно сделать замер текущего времени, сохранить его, а потом просто в цикле запрашивать снова замеры текущего времени, пока разница не покажет нужный прошедший интервал. Понятно, что в таком случае приложение постоянно активно, потому что оно постоянно запрашивает у системы замер текущего времени и кроме этого не занимается больше ничем. В событийно-ориентированной системе того же самого поведения можно добиться с помощью таймера. Мы должны создать таймер, поставить ему в качестве интервала события нужный нам интервал времени, повесить на событие таймера нужную функцию-обработчик и активировать таймер. Приложение в процессе этого ожидания не будет делать ничего (вспомните: событийно-ориентированные приложения, пока не происходит никаких событий, находятся в режиме ожидания и почти не потребляют процессорного времени), но как только интервал времени истечет, система инициирует событие таймера, вызовется наш обработчик, который и сделает то, что должен был сделать по истечении заданного интервала.

Событийно-ориентированная система очень часто построена поверх внутренней кухни, основанной на обработке сообщений. Вам, скорее всего, никогда не понадобится работать напрямую с сообщениями, но понимать основы того, как устроена работа с сообщениями под капотом машины событий, все равно может быть полезным.

В чем разница между событиями и сообщениями? Событие (event) — это нечто происходящее в какой-то момент времени, наступления чего мы ждем и сразу после наступления чего мы должны выполнить какую-то реакцию. Событие — это изменение состояния системы. Сообщение (message) — это информационный пакет, кусочек данных, письмо с какой-то информацией о чем-то. Получение сообщения может стать причиной возникновения события, а может и не стать. Событийно-ориентированные системы очень часто основаны на системных процессах, обменивающихся сообщениями. Так, например, у каждого процесса или потока в операционной системе есть в ядре системы свой собственный «почтовый ящик», в который падают разнообразные сообщения (о том, что была нажата клавиша мыши или клавиатуры, о том, что сработал таймер, о том, что требуется перерисовка и т. д.). И вся работа такого приложения (процесса или потока) заключается в бесконечном цикле, в котором приложение запрашивает у системы очередное сообщение из своего почтового ящика, а потом выполняет какой-то код в зависимости от того, какое сообщение было получено. Если вдруг приложение решило вычислить число пи до квантильного знака после запятой, то оно в это время не сможет и не будет «проверять» свой почтовый ящик, и сколько бы вы не кликали в это приложение мышкой, не пытались нажимать клавиши на клавиатуре — никакой реакции не будет. Это происходит потому, что система добросовестно отправляет ваши сообщения в почтовый ящик приложения-получателя, но приложение-получатель слишком занято чем-то, чтобы свой почтовый ящик проверять. Чаще всего вы в этом случае увидите сообщение от системы о том, что приложение «не отвечает» — это значит, что приложение не обращается к системе за очередными сообщениями из своего почтового ящика. Хотите подождать или закрыть приложение? Как только приложение закончит считать свое число пи, оно снова начнет обрабатывать сообщения из своего почтового ящика, и вы увидите запоздалую реакцию на все те действия, которые вы выполняли уже давно, — появившийся текст, который вы как будто печатали в приложении минуту назад, срабатывание нажатых в приложении кнопок и т. д.

Внутри каждого приложения спрятанная от вас кухня получает сообщения из «почтового ящика» очереди сообщений и преобразует эти сообщения в события. Например, если пришло сообщение `WM_LBUTTONDOWN`, внутренняя кухня вашего же приложения разберется, на какую из кнопок в вашем окне приходятся эти координаты, и инициирует возникновение события `MouseDown` или `MouseClicked` или `Click` у одной из кнопок.

Это очень быстрое и очень грубое описание событийно-ориентированной технологии. В разных языках программирования и фреймворках детали могут отличаться при сохранении общей идеи. Если вам мало описанного здесь и хочется больше деталей, то я бы предложил вам ознакомиться с тем, как событийная механика реализована в библиотеке Qt. Там тоже есть аналог описанной выше технологии (только там события и обработчики называются сигналами и слотами), но кроме этого там есть и альтернативный (точнее, дополняющий) путь органи-

зации обработки событий-сообщений, знакомство с которым, безусловно, обогатит спектр знакомых вам технических решений.

2.31. Архитектура Model-View-Controller (MVC) и ее варианты. Самый простой способ ее нарушить и исправить

Вы наверняка слышали такую аббревиатуру, как MVC. Это тема непростая, и когда я в нее сам погружался первый раз много лет назад, я все время наталкивался на довольно абстрактные и обобщенные рассуждения, за которыми было сложно увидеть простую суть. Я попробую здесь на самом простом примере показать, какую проблему эта технология призвана решить и как именно.

Хочу подчеркнуть, что MVC — это именно технология организации кода, принцип организации взаимодействующих частей кода, ответственных за разные вещи. Иногда встречается мнение, что MVC обязательно привязана к какому-нибудь конкретному фреймворку или даже языку («MVC — это чаще всего Java»). На самом деле говорить так — это все равно, что утверждать, что «афоризмы — это обязательно латынь». Как афоризмы могут быть вообще не на латыни, так и на самой латыни может написан не афоризм, а цитероновский Lorem ipsum предыдущей эры. Так и парадигма MVC может быть реализована на многих языках, причем совсем не обязательно объектно-ориентированных или даже событийно-ориентированных. Да, MVC встроена в некоторые языки, она поддерживается многими фреймворками. Это «поддерживается фреймворками» означает, что часть кода, которую мы в этом разделе будем сейчас писать сами, уже была написана и является частью используемых графических компонент. Но сама идея от языка вообще не зависит. К концу этого раздела, когда вы поймете эту основную идею MVC, вы поймете, что я имею в виду.

Как и многие другие технологии, MVC предназначена для решения проблемы сложности, которая накапливается в вашей программе по мере того, как программа становится все больше и больше. Кроме этого, MVC чаще всего применяется для программ, имеющих визуальный интерфейс и взаимодействующих с пользователем. В такой программе у вас будет, условно, какая-то часть, отвечающая за внешний вид, визуальное оформление, представление информации; другая часть, ответственная за интерактивное взаимодействие с пользователем; и третья часть, ответственная за хранение, преобразование и поддержание логики в хранимой информации. Мы пока не даем этим частям имена и очень скоро поймем, что они собой представляют, но пока просто поймите, что технология MVC призвана снизить «лапшеобразность» вашего кода, убрать связи между тем, что не должно быть связано. Но довольно абстрактных рассуждений, давайте сразу простой пример!

Пусть вам нужна программа с формочкой, на которой нужно где-то хранить и отображать число. Мы берем подходящий компонент (у меня `NumericUpDown`, но

у вас может быть и какой-то другой, в соответствии с тем графическим фреймворком, который вы используете) и просто кидаем его на форму (рис. 2.44). Можно запустить приложение и убедиться, что все работает: значение отображается и редактируется. Казалось бы, зачем тут нужен MVC? Все же и так работает? Пока — да.

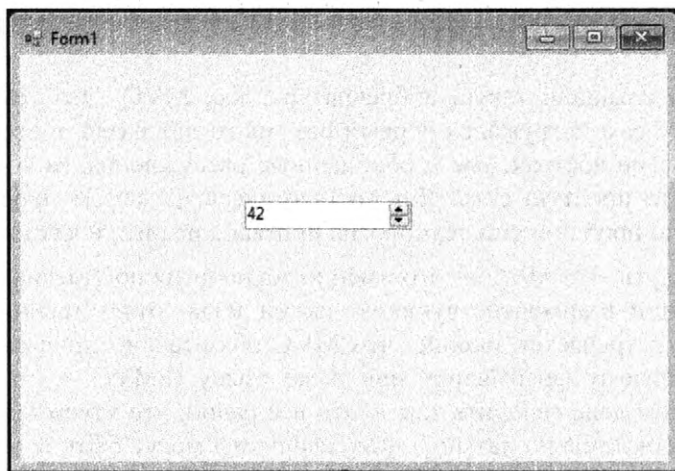


Рис. 2.44. Форма с одним компонентом

Следующий шаг: предположим, что нам обязательно нужно, чтобы значение в этом поле было обязательно четным. Как будто, тоже ничего сложного: у этого компонента есть событие `ValueChanged`, на которое просто нужно повесить обработчик (рис. 2.45).

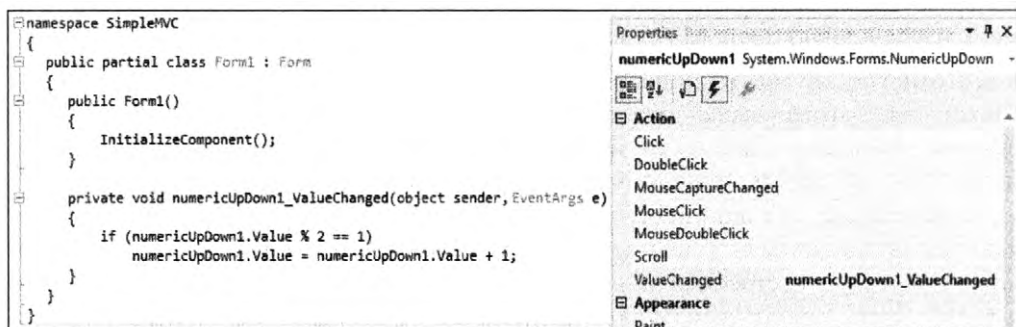


Рис. 2.45. Простой обработчик на изменение хранимого значения

Снова не понятно, зачем MVC? Предположим, что этим значением где-то еще в программе нам нужно воспользоваться. Например, при нажатии какой-то кнопки нам нужно отобразить это значение в сообщении. Просто кидаем на форму кнопку и в ее обработчике показываем сообщение, значение в который достаем из числового компонента (рис. 2.46).

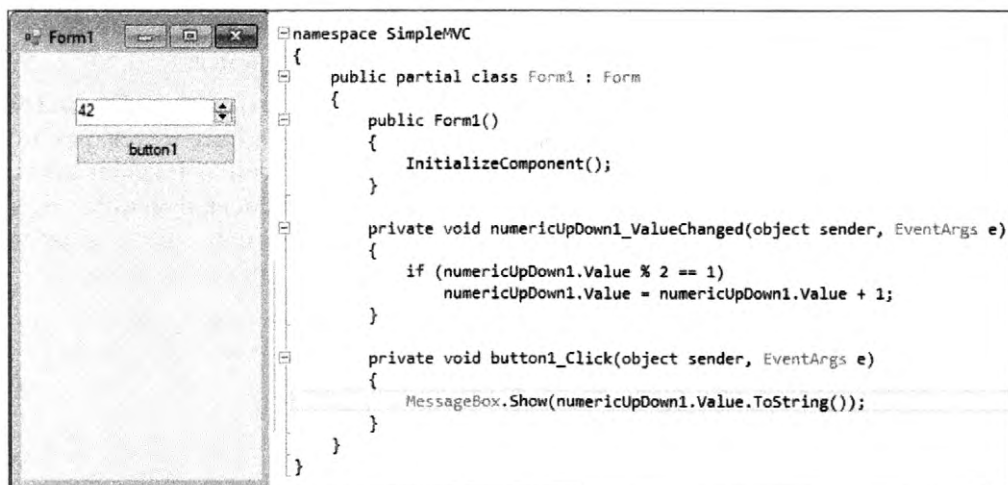
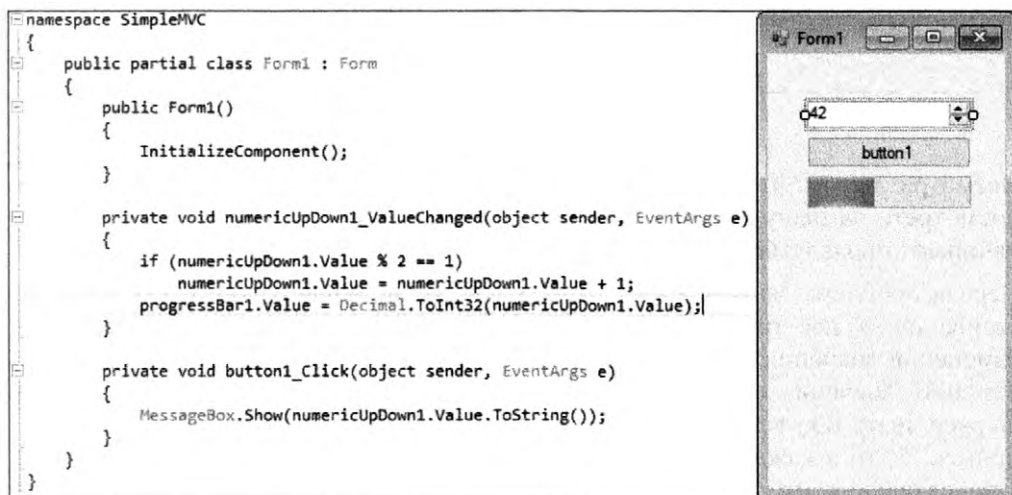


Рис. 2.46. Использование хранимого значения где-то еще

И здесь мы справились без MVC. И это правильно, потому что MVC проявит свою сущность и пользу позже, когда программа станет сложнее. Следующий шаг: то же самое значение должно отображаться в индикаторе `ProgressBar`. Такой индикатор не позволяет редактировать значение, а только отображает его. Добавим его на форму, а чтобы отображаемое им значение всегда было актуально, нам нужно его менять там и тогда, где оно меняется, — т. е. в обработчике для `NumericUpDown` (рис. 2.47).

Рис. 2.47. Добавлен `ProgressBar`

Запустите и убедитесь, что все работает правильно. За исключением одной мелочи — при запуске приложения `ProgressBar` показывает не то же самое, что `NumericUpDown`, а синхронизируется с ним только после первого обновления.

Мелочь? Казалось бы да, но из-за подобных мелочей на самом деле марсоходы и не приземляются успешно на Венере! Мы еще к этому вернемся.

Теперь представим себе, что это же значение должно храниться и отображаться еще в одном компоненте, пусть это будет `TextBox` (рис. 2.48). Разумеется, раз это то же самое значение, то оно должно быть всегда синхронизировано и с `NumericUpDown`, и с `ProgressBar`. Для этого нам придется и снова изменить обработчик для `NumericUpDown` и добавить обработчик для самого `TextBox` (повесьте этот обработчик на событие `KeyDown`, а не на событие `TextChanged`).

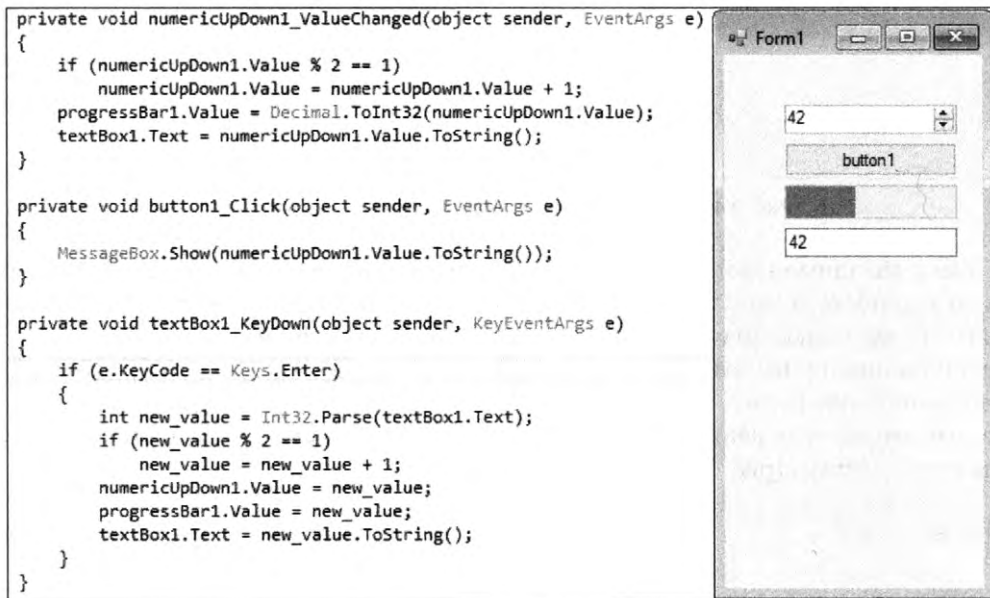


Рис. 2.48. Добавлен `TextBox`

Несмотря на то что всё вроде бы работает правильно, самое время уже посмотреть на нашу программу и подумать, какие проблемы в ней постепенно начинают проявляться и куда эти проблемы очень быстро будут расти.

Первая проблема заключается в том, что у нас начинают множиться переприсваивания. У нас три компонента отображают одно и то же значение, и при изменении значения в одном из них этот компонент вынужден самостоятельно изменять значение во всех других компонентах. Получается, каждый из них должен знать обо всех других? Да, это так, и если таких компонент будет, например, 5, то в каждом из них (при условии, что мы ничего не забыли) будет 4 присваивания, итого 20 присваиваний! Как только появляется еще одно место, где это значение может редактироваться, мы вынуждены вспомнить все те места, где оно раньше редактировалось, и вносить в них изменения. Очевидно, это очень плохо!

Вторая проблема заключается в том, что наше правило, которое мы должны поддерживать, про четность хранимого значения — тоже придется прописывать

столько раз, сколько компонент у нас будут позволять значение редактировать. Видите, оно уже написано два раза, и дальше будет хуже. «Не повторяй себя», принцип DRY, да и просто здравый смысл говорит, что это правило должно быть написано только в одном месте. В каком месте? В том, где само это значение хранится. А где оно у нас хранится?

Что стало причиной всех этих проблем? А то, что числовое значение, которое несет для нас какой-то смысл, хранится в виде нескольких копий в разных местах и, что того хуже, — в визуальных компонентах. Значение, несущее смысл, не должно храниться в визуальных компонентах. Логика обработки значения или, если по-другому, бизнес-правила, ограничения на хранимые и обрабатываемые значения, алгоритмы обработки этих значений, безусловно, должны существовать отдельно от визуальных компонент. Эти самые визуальные компоненты не должны хранить наше значение, они должны только отображать его и позволять редактировать. А храниться значение должно отдельно от них. Где?

Хранить его в простом целочисленном свойстве (рис. 2.49)? А где тогда должны быть записаны все правила о том, как его можно изменять, а как нельзя? Опять во всех обработчиках? А если просто взять и изменить значение этого свойства в коде? Тогда, получается, никакие проверки не будут выполнены? А где заполнить начальное значение этого свойства?

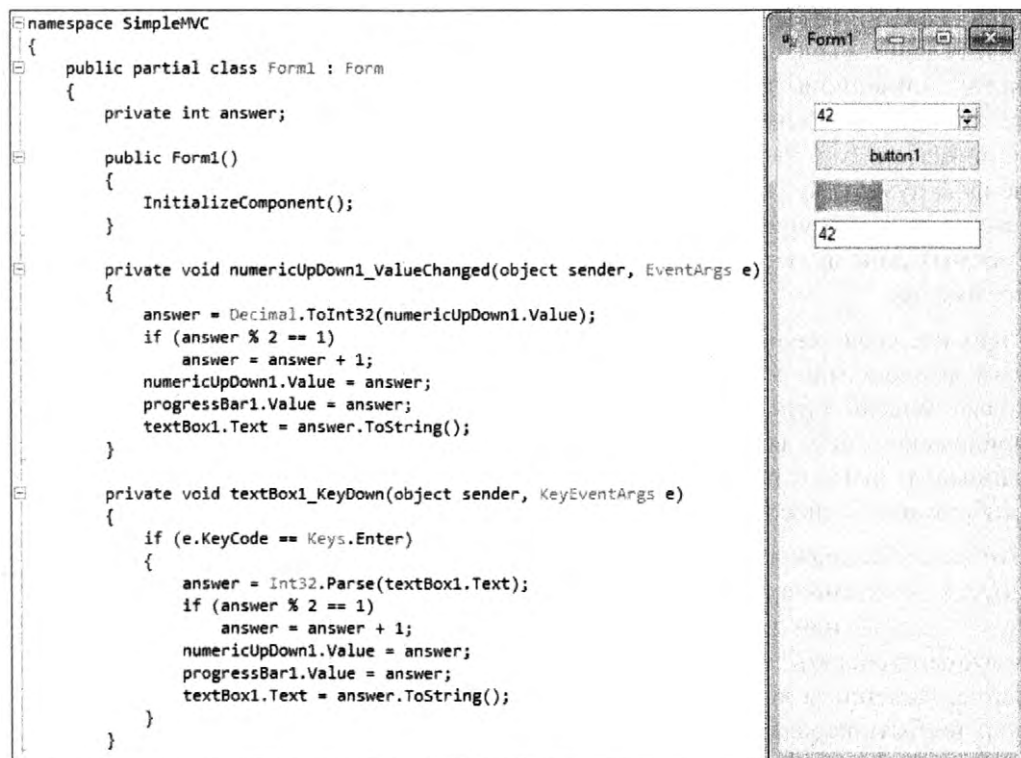


Рис. 2.49. Хранить значение в простой целочисленной переменной?

Я думаю, вы уже поняли, к чему я клоню: в полном соответствии с идеологией ООП это значение и все эти правила, которые позволяют это значение изменять одним образом и не позволяют изменять другим, должны быть вынесены в отдельный объект (рис. 2.50).

```
public class Model
{
    private int value;

    public void setValue(int value)
    {
        if (value % 2 == 1)
            this.value = value + 1;
        else
            this.value = value;
    }

    public int getValue()
    {
        return value;
    }
}
```

Рис. 2.50. Класс — модель для хранения нашего значения

Поле `value` у модели это как раз и есть то единственное место, где будет храниться наше значение. Оно приватное, изменить его можно только через setter-метод, и именно в этом методе в единственном экземпляре и будут описаны все те правила изменения нашего значения: какое значение принимать, какое не принимать и т. д. Рядом с приватным полем `value` могут быть и другие поля, — не надо думать, что модель обязательно хранит только одно число. Вы, я думаю, уже поняли, что модель должна хранить весь набор взаимосвязанных, взаимозависимых данных, которые подчиняются одним общим правилам и существуют все вместе.

Спросите себя: какое отношение этот класс модели имеет к форме, к визуальным компонентам на форме? Разумеется, никакого! Мы легко можем создать только модель и использовать ее методы, например, в каком-нибудь консольном приложении, безо всяких формочек. Это ключевой момент: модель описывает данные и правила предметной области, она никак не связана с правилами их отображения и способом редактирования.

Что же тогда должно быть в форме? У формы тогда должна быть ссылка на эту модель, и экземпляр класса модели должен создаваться в конструкторе. Когда `NumericUpDown` или в `TextBox` изменяется значение, они никакую четность не должны проверять, они должны просто требовать у модели изменить свое значение. Изменился компонент — визуальное значение, которое в нем изменено, должно быть передано в модель (рис. 2.51). А при нажатии кнопки мы должны откуда-то взять значение и отобразить его в сообщении, как и раньше. Теоретически нам не важно, откуда его брать, но правильнее всего — брать из модели!

```
public partial class Form1 : Form
{
    private Model model;

    public Form1()
    {
        InitializeComponent();
        model = new Model();
    }

    private void numericUpDown1_ValueChanged(object sender, EventArgs e)
    {
        model.setValue(Decimal.ToInt32(numericUpDown1.Value));
    }

    private void textBox1_KeyDown(object sender, KeyEventArgs e)
    {
        if (e.KeyCode == Keys.Enter)
        {
            model.setValue(Int32.Parse(textBox1.Text));
        }
    }

    private void button1_Click(object sender, EventArgs e)
    {
        MessageBox.Show(model.getValue().ToString());
    }
}
```

Рис. 2.51. Создание экземпляра модели

Итак, чего мы добились? Значение хранится в модели, при изменении этого значения в визуальных компонентах они не пытаются его как-то обрабатывать, они просто пересылают его в модель. Если где-то в программе это значение нужно, оно забирается из модели (рис. 2.52). Если вы запустите приложение, вы очень быстро обнаружите, что, изменяя значение в `NumericUpDown`, вы изменяете его внутри модели (кнопка покажет вам это), и правила при этом обрабатываются, но в `TextBox` значение при этом не изменяется. Все компоненты сейчас знают, что введенное пользователем значение они должны отправлять в модель, но если значение изменится внутри модели, они об этом не узнают. Они знают, откуда в модели значение нужно забирать, но не знают когда, потому что их никто не оповещает, что это значение внутри модели изменилось. Собственно, это и есть ответ: модель должна уведомлять о своих изменениях всех, кто хочет за этими изменениями следить. Учитывая, что список этих «всех» может меняться в процессе работы программы, мы должны реализовать механизм, подобный подписке. Это можно сделать разными способами: стандартными, но различными для тех графических фреймворков, которые вы используете. Мы здесь воспользуемся обычным механизмом подписки на события.

У модели объявим публичное событие — `DataChanged`, у формы объявим метод-обработчик для этого события — `Form1_updateFromModel`, и внутри этого метода все графические компоненты должны забрать себе значение из модели. Нам остается только в конструкторе формы, там, где создается модель, подписать


```

public partial class Form1 : Form
{
    private Model model;

    public Form1()
    {
        InitializeComponent();
        model = new Model();
        model.DataChanged += new System.EventHandler(this.Form1_updateFromModel);
        this.Form1_updateFromModel(this, null);
    }

    private void numericUpDown1_ValueChanged(object sender, EventArgs e)
    {
        model.SetValue(Decimal.ToInt32(numericUpDown1.Value));
    }

    private void textBox1_KeyDown(object sender, KeyEventArgs e)
    {
        if (e.KeyCode == Keys.Enter)
        {
            model.SetValue(Int32.Parse(textBox1.Text));
        }
    }

    private void button1_Click(object sender, EventArgs e)
    {
        MessageBox.Show(model.GetValue().ToString());
    }

    private void Form1_updateFromModel(object sender, EventArgs e)
    {
        numericUpDown1.Value = model.GetValue();
        progressBar1.Value = model.GetValue();
        textBox1.Text = model.GetValue().ToString();
    }
}

public class Model
{
    private int value;
    public System.EventHandler DataChanged;
    public void SetValue(int value)
    {
        if (value % 2 == 1)
            this.value = value + 1;
        else
            this.value = value;
        DataChanged.Invoke(this, null);
    }

    public int GetValue()
    {
        return value;
    }
}

```

Создание модели

Подписка формы на модель

Обновление формы из модели

Уведомление всех, кто хочет знать, что модель изменилась

Рис. 2.52. MVC, собранный целиком

форму на событие `DataChanged`, чтобы при возникновении такого события вызывался метод `Form1_updateFromModel`.

Ах да, в методе `setValue` у модели, где изменяется хранимое в модели значение, теперь нужно испускать событие `DataChanged`, чтобы уведомить всех, кто за моделью следит, о том, что модель изменилась. Заметьте еще, что сразу после того, как форма подписывается на событие `DataChanged`, метод-обработчик `Form1_`

`updateFromModel` принудительно вызывается для того, чтобы форма сразу при своем появлении уже отображала текущее значение из модели. Начальное значение `value` у модели мы никак пока не определили, а хорошо бы это сделать где-то в конструкторе самой модели (которого пока у нас нет).

Итак, вот вся последовательность происходящего. У нас есть форма и модель. Модель хранит значение и позволяет его «правильно» изменять, не допуская нарушения правил предметной области. Форма создает модель и подписывается на событие `DataChanged` у модели. Другими словами, форма как будто говорит: «Когда модель обновится, пожалуйста, уведомите меня». При любом изменении модели (не важно, инициированном самой формой или кем-то третьим со стороны!) испускается событие `DataChanged`, и обработчик этого события `Form1_updateFromModel` у формы забирает из модели изменившиеся значения и раскладывает их по графическим компонентам. Если пользователь изменяет эти значения в графических компонентах, то каждое из них отправляет изменения в модель. Важно, что графические компоненты на форме теперь ничего не знают друг про друга, но при этом отображают синхронизированные друг с другом и с моделью значения.

Да, как будто кода у нас стало больше. Но теперь появление каких-нибудь новых компонент, отображающих то же самое значение, не приводит к разбросанным по всей программе добавленным строкам кода! При усложнении взаимосвязей между графическими компонентами наступает тот критический момент, когда перенос всей логики в модель уменьшает количество кода и облегчает, а не усложняет понимание. Мы добились главного — уменьшения связи между компонентами на форме: они знают только про модель, но не про друг друга.

Если продолжать писать код в таком стиле, то вы увидите, что ваша программа будет разбиваться на три своеобразные части. Во-первых, будут разнообразные классы моделей, которые хранят данные и обеспечивают их непротиворечивость. Это, собственно, и есть *модель* — буква М в MVC. Кроме этого, у вас будет (как правило, но, вообще говоря, не всегда) часть кода, ответственная за визуальное представление этих данных. Это и есть *представление* (мне больше всего нравится именно такой перевод слова `view` в данном контексте) — буква V в MVC. И у вас будет код, который реагирует на действия пользователя и все остальное происходящее и отправляет изменения в модель. Эту часть кода принято называть *контроллер* — буква С в MVC.

Сделайте следующий шаг в сторону абстракции. Что такое модель? Классическую картинку ~~в трех шестеренках~~ про триаду Model-View-Controller вы найдете сами, но попробуйте соотнести ее части с только что написанной нами программой. Модель — это объект, хранящий данные, обеспечивающий их непротиворечивость и готовый уведомлять всех о том, что он изменился. Последнее, я считаю, очень важно, потому что первая часть этого определения относится, вообще говоря, к любому объекту. Любой наш `Point` или `Section` хранил данные и обеспечивал их непротиворечивость, но моделью мы готовы его назвать только тогда, когда на изменения в нем можно подписаться.

Что такое представление? Модель не имеет представления, модель — про данные. Но как-то ведь мы должны эти данные отображать, показывать или, другими словами, представлять. Этим и занимается в MVC представление. У одной и той же модели может быть много представлений: когда одни и те же данные одновременно, как у нас, отображаются в разных видах: и в виде таблицы, и в виде графика, например. Но вот представление — привязано к модели, сидит на попе ровно и ждет, пока модель ему просигнализирует, что она изменилась. Только тогда представление забирает из модели нужные ему данные и меняет свой внешний вид в соответствии с ними.

Контроллер — это код, сидящий между пользователем и моделью. Между представлением и пользователем ничего дополнительного не нужно, — там находятся органы чувств пользователя, которые отлично со своей задачей справляются. А вот напрямую с моделью пользователь работать не может (хотя если вы встроили в свою программу какой-нибудь IPython Notebook, то такую прямую работу пользователя с моделью вполне можно расценивать как работу без контроллера), для этого нужно действия пользователя преобразовывать в запросы к модели. В нашем случае это методы `numericUpDown1_ValueChanged` и `textBox1_KeyDown`, и именно их можно отнести к контроллеру.

Сейчас будет длинный абзац, во время которого вам может стать тяжело. Как только это произойдет, пропустите его и вернитесь когда-нибудь в другой раз: он не обязателен для чтения. Итак. Если внимательно посмотреть на только что созданный нами пример использования MVC, можно увидеть в нем однообразные типовые действия, которые будут чем дальше, тем больше повторяться. Мы вынуждены вешать обработчики на изменения в графических компонентах (те самые `numericUpDown1_ValueChanged` и `textBox1_KeyDown`), которые ничего особенного не делают, кроме того как просто передают свое значение в модель. Кроме этого, раз это стандартные `numericUpDown`, `progressBar` и `textBox`, то они ничего не знают ни про нашу модель, ни про какие вообще модели, а значит, мы вынуждены вручную помещать в них данные из модели, когда модель командует всем обновиться. Вот было бы здорово, если бы, например, `textBox` можно было просто сказать: «Вот тебе твоя модель, вот такое свойство из нее ты будешь отображать и в это же свойство будешь отправлять изменения, если пользователь что-то в тебе изменит». Если вы продолжите думать в этом направлении, то придете к тому, что называется Data-aware controls, и к вариации MVC, которая называется MVVM, Model-View-ViewModel (заметьте, контроллера здесь нет). В рамках этой технологии ViewModel — это такой объект, выступающий в виде прослойки между настоящей, «предметной» моделью со всеми бизнес-правилами и графическим компонентом. Например, это мог бы быть такой объект, который умеет организовывать «общение» между `TextBox` на форме (вообще-то отображающим строки) и целым числом в предметной модели. Он умел бы преобразовывать целое число в строку для отображения в `TextBox` и знал бы, когда и как правильно забирать текст из `TextBox` и преобразовывать его в число для отправки в модель. Если ваши данные в модели представимы и

в виде таблицы, и в виде графика, то вместо того, чтобы учить и таблицу, и график забирать из модели и готовить для себя данные в специфическом виде (понятно, что таблице данные нужны в одном виде, а графику — в другом), один такой специальный ViewModel может забирать данные из модели и представлять их в виде набора строк и столбцов (а такие данные умеет отображать любая, даже самая глупая и стандартная таблица), а другой специальный ViewModel будет забирать те же данные из предметной модели и представлять их в виде, например, массива пар (x,y), чтобы любой компонент для графиков с ними мог справиться.

Тема Model-View-Controller очень обширная, и в мои планы не входит вас детально ей научить. Надеюсь, я смог вам показать, откуда у нее «растут ноги», почему она логична и в чем ее основная идея. Для закрепления результата предлагаю вам теперь самостоятельно разработать простое приложение вот с такой формой (рис. 2.53).

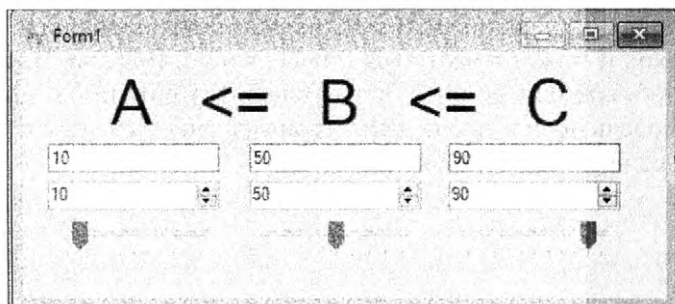


Рис. 2.53. Требуемый внешний вид приложения

Можете выбрать любой удобный вам язык, хоть Java, хоть C++, хоть Python, подтвердив тем самым, что MVC от языка не зависит. В приложении должно быть не одно, а три целых числа A, B и C со значением в пределах от 0 до 100. Каждое из чисел должно отображаться и редактироваться в трех разных компонентах: `textBox`, `numericUpDown` и `trackBar` (или аналогичных в других языках), при этом редактирование числа в одном поле должно приводить к изменению отображения этого числа во всех других полях. Второе число всегда должно быть не меньше первого и не больше третьего. Изменение A и C должно реализовывать разрешающее поведение (при нарушении ограничений порядка одним числом все остальные числа должны сами перестраиваться). Изменение B должно реализовывать запрещающее поведение (при введении пользователем «неправильного» числа нужно откатить назад все такие изменения) или ограничивающее поведение (при вводе «неправильного» числа оно должно быть скорректировано так, чтобы быть как можно ближе к тому, что ввел пользователь, но правила при этом не нарушать). Можете ради интереса сначала попробовать сделать такое приложение без использования подхода MVC и ощутить, так сказать, всю боль. Но, конечно же, подразумевается, что приложение должно быть реализовано в стиле MVC: хранение трех чисел должно быть организовано

в виде отдельного объекта-модели, ну и т. д. Если вы при этом еще и постараетесь, чтобы у вас при вводе пользователем любого значения не происходило так называемого каскадного ада обновлений, а событие об изменении модели происходило ровно один раз — будет совсем хорошо!

Напоследок хочу предостеречь вас от некоторых типовых ошибок, которые при создании такого приложения можно допустить. Во-первых, посмотрите, не сделали ли вы три модели вместо одной. Все три числа должны быть внутри одной модели, а не каждый внутри своей, — потому что они тесно взаимосвязаны друг с другом. Во-вторых, посмотрите внимательно, что вы делаете внутри модели при запросе на изменение числа А. Понятно, что при изменении числа А иногда нужно менять и число В, и число С. Убедитесь, что при этом вы НЕ используете публичные сеттеры В и С. Почему? Потому что тогда вы можете случайно уведомить всех подписавшихся на модель о произошедших изменениях в тот момент, когда А, В и С все еще находятся в процессе изменения и, например, В уже изменено, а А еще нет. В эту наносекунду на вашей форме может появиться комбинация значений А, В и С, нарушающая правила, и именно в этот момент марсоход и может брякнуться о поверхность Венеры! Внесение изменений в модель обязательно должно быть, говоря терминами баз данных, атомарным — пока модель полностью не перестроилась, она ни в коем случае не может никому сигнализировать об изменениях.

Использование: табл. П.1.

Таблица П.1

Действие	Значение
<code>int a = 5;</code>	Переменная <code>a</code> получает значение 5
<code>int *p = &a;</code> <code>// *p => 5</code>	Переменная-указатель <code>p</code> получает адрес переменной <code>a</code> (Вариант: переменная-указатель <code>p</code> теперь указывает на переменную <code>a</code>)
<code>int b = *p;</code> <code>// b => 5</code>	Переменная <code>b</code> получает значение переменной, на которую указывает <code>p</code>
<code>*p = 6;</code> <code>// *p => 6</code> <code>// a => 6</code> <code>// b => 5</code>	Переменная, на которую указывает <code>p</code> , получает значение 6

Если вы не понимаете, как это все работает, то вам придется читать полное описание.

П.3. Полное описание

Прежде всего, запомните: кое-что из того, что написано ниже, на самом деле неверно: некоторые низкоуровневые детали тут намеренно пропущены. Например, значение `int` занимает вовсе не два байта, а больше, и расположены они в памяти могут быть как «слева направо», так и «справа налево», — эти детали опущены для простоты картины, но будьте готовы потом с ними столкнуться. Или, например, игнорируется разница между инициализацией и присваиванием переменных.

П.3.1. Переменная

Что такое переменная? Переменная — это определенное место в памяти, имеющее имя, по которому мы его используем. Переменная имеет значение (текущее значение) — грубо говоря, это то, что записано сейчас по определенному месту в памяти. Переменная также имеет тип, описывающий, какого рода значения в ней хранятся. Переменные устроены таким образом, что мы можем работать с ними по их имени, не задумываясь, где на самом деле в памяти и как они хранятся, — это их главная идея. Прежде чем использовать переменную, ее надо объявить и определить ее тип. Одновременно с этим можно задать начальное значение, а можно установить значение переменной после в любой момент. (Примеры — в табл. П.2).

Во многих языках это было бы практически все, что можно делать с переменными, как с ними можно работать. Однако в C++, как в языке низкого уровня, с переменными можно работать и по-другому, а именно — косвенно.

Для того чтобы в этом разобраться, нужно для начала разобраться в том, что представляет собой память, в которой переменные хранятся. Ведь переменные

имеют различные типы, а память одна — как в одной и той же памяти хранятся и целые числа, и дробные числа, и символы и т. д.

Таблица П.2

Действие	Значение
float f;	Объявляем переменную f типа float
char s = 'w';	Объявляем переменную s типа char и присваиваем ей значение 'w'
f = -3.2;	Переменная f получает значение -3.2
int a = 5;	Объявляем переменную a типа int и присваиваем ей значение 5
int b = 6;	Объявляем переменную b типа int и присваиваем ей значение 6
int t;	Объявляем переменную t типа int
t = a;	Копируем значение переменной a в переменную t
a = b;	Копируем значение переменной b в переменную a
b = t;	Копируем значение переменной t в переменную b

П.3.2. Память

Память — это нумерованная последовательность числовых двоичных байтов. Именно нумерованная (т. е. каждый байт имеет номер), именно последовательность (т. е. каждый следующий байт имеет номер, отличающийся на единицу), именно двоичных (т. е. байт содержит 8 двоичных битов, содержащих, в свою очередь, 0 или 1), именно байтов (т. е. можно обратиться только к байту целиком, но не к отдельному биту) и именно числовых (т. е. каждый байт представляет собой число). Каждый байт этой последовательности имеет свой номер и текущее значение, которое, как несложно догадаться, в двоичном виде представляет собой число от 00000000 до 11111111, в шестнадцатеричном виде — число от 00 до FF и в десятичном виде — число от 0 до 255 (рис. П.1).

значение	0	0	109	101	100	118	101	100	0
номер	0	1	2	3	4	5	6	7	9

Рис. П.1. Содержимое и нумерация байтов в памяти

Как уже говорилось, каждая переменная занимает некоторое место в памяти. Разные типы требуют различного количества байтов. Еще раз: переменная — это определенное место в памяти, имеющее имя, по которому мы его используем. Теперь мы можем сказать точнее. Определенное место — это некоторая определенная непрерывная последовательность байтов в памяти (рис. П2).

Что мы здесь видим? Во-первых, все переменные занимают разное количество (порядк идущих!) байтов. Во-вторых, некоторые байты могут быть не заняты,

но, смотря только на значения этих байтов, не скажешь, заняты они или нет, и значениями каких типов заняты. Незанятые байты могут иметь произвольные значения, оставшиеся от каких-то переменных, которые там раньше размещались, но теперь не размещаются. Иногда мы такие байты будем помечать символами «?», но надо обязательно держать в уме, что там не вопросы находятся, а какие-то реальные значения. И, в-третьих и в самых главных, все переменные хранятся в той же самой памяти, которая состоит из числовых байтов. Внимание, вопрос. Как различные типы данных могут храниться в одной и той же памяти? Как в числовой памяти можно хранить символ 'w'? Как в двух ячейках, каждая из которых может содержать число от 0 до 255, можно хранить число 258? Для каждого типа данных известно, сколько байтов в памяти занимает переменная его типа: для `char` — один байт, для `int` — два, четыре или восемь байтов (на рисунках будем делать вид, что `int` — это два байта) и т. д. И самое главное — как любое значение этого типа представить в виде последовательности числовых байтов? Для каждого значения `char`, например, выделяется один байт, и в этом числовом байте хранится номер этого символа в специальной таблице ASCII (а может не один байт, а больше — в случае разных вариаций Unicode).

значение	0	119	5	0	2	1	101	100	0
номер	0	1	2	3	4	5	6	7	8
переменная	не занято	s	a		b		не занято		
тип		char	int		int				
значение		'w'	5		258				

Рис. П.2. Значения разных типов в памяти

Теперь, когда мы разобрались, что такое память и как там хранятся переменные, нужно понять, что такое адрес переменной.

П.3.3. Адрес переменной

Адрес переменной — это, грубо говоря, номер первого байта, занимаемого этой переменной в памяти. В вышеприведенном примере (см. рис. П.2) адрес переменной `s` есть 1, адрес переменной `a` есть 2, адрес переменной `b` есть 4. Зная адрес некоторой переменной и ее тип, можно точно узнать, с какого по какой байты в памяти эта переменная занимает. А именно байты с номерами s *адрес(переменной)* по $s + \text{размер(переменной)} - 1$. Для каждой переменной можно узнать ее адрес с помощью операции `&`. Адрес — это, грубо говоря, номер, т. е. число. Но не все операции над адресом допустимы и, что важнее, осмысленны. Приведу такую аналогию. Адрес — это, например, номер почтового ящика. Если рассмотреть ящик номер 5, то его адрес — это соответственно 5. К этому адресу я могу прибавить какое-то число, например 2, и получить новый адрес, а именно 7. Если рассмотреть ящик номер 5 и ящик номер 8, то я могу из адреса второго вычесть адрес первого и получить уже не адрес, но

число, равное количеству ящиков между ящиком номер 5 и ящиком номер 8. Но складывать два адреса нельзя, это бессмысленная операция. Итак, над адресами переменных возможны следующие операции:

- ☐ `address2 - address1 => value1` — количество байтов между байтами по адресам `address1` и `address2`;
- ☐ `address1 + value1 => address2` — адрес байта, отстоящего от байта по адресу `address1` на количество `value1` байтов.

Примеры см. в табл. П.3.

Таблица П.3

Действие	Значение
<code>int a = 5;</code>	Переменная <code>a</code>
<code>int b = 6;</code>	Переменная <code>b</code>
<code>if (&b - &a > 10)</code>	Далеко ли друг от друга эти переменные?
<code>if (&a + 1 == &b)</code>	Может быть, переменная <code>b</code> идет в памяти сразу за <code>a</code> ?

Важный вопрос: а если мы захотим получить адрес некоторой переменной и сохранить его временно где-то в памяти, какого размера область памяти нам надо выделить? Или подобный вопрос: в переменную какого типа можно записать адрес? Если адрес — это номер, то ответ зависит от того, как много может быть таких адресов? В общем-то адресов может быть столько, сколько ячеек в памяти, а это может быть и 10^9 , и даже больше. Допустим, адрес у нас всегда занимает ровно 4 байта. Можете сами оценить, что тогда адресовать с помощью 4 байтов можно $2^{4 \times 8} = 2^{32} = 4$ гигабайта.

Теперь, когда мы разобрались, что такое память и как там хранятся переменные, можно переходить непосредственно к указателям.

П.3.4. Переменные-указатели

Что такое указатель? Указатель — это «всего лишь» переменная, значением которой является адрес. То есть это именно такая переменная, в которой можно сохранить адрес какой-то другой переменной.

Представьте себе переменную как почтовый ящик с бумажкой внутри, на которой что-то написано. Адрес этой переменной — это номер почтового ящика. Так вот указатель — это почтовый ящик с бумажкой внутри, на которой написан номер какого-то другого почтового ящика.

Зачем это в принципе может быть нужно? Если продолжать аналогию с почтовыми ящиками, то можно привести такой пример. Пусть у меня есть два почтовых ящика, `ящик1` и `ящик2`. Я хочу, чтобы мои письма доставлялись в какие-то

другие почтовые ящики. Первый вариант этого добиться — поставить перед почтальоном задачу вида: «Доставь письмо из ящика1 в ящик5». Завтра мне понадобится доставить письмо в ящик6, и мне придется менять задание почтальону, и в конце концов он все перепутает (грубо говоря, нехорошо каждый раз менять задание, это требует дополнительной информации почтальону). Другой вариант этого добиться — поставить перед почтальоном постоянную задачу вида: «Доставь письмо из ящика1 в ящик, номер которого находится в ящике2». Да, это чуть более сложная задача для почтальона, но она постоянная, она не меняется, и он всегда может действовать по одному и тому же шаблону.

Второй вариант называется *косвенной адресацией*, и ящик2, как вы уже догадались, — это указатель, т. е. такая переменная, которая содержит адрес другой переменной.

Так же как переменные различаются типами, указатели тоже бывают разными. Так, при объявлении указателя обязательно задается, на переменную какого типа это указатель. То есть по сути — адрес переменной какого типа будет содержаться в этом указателе. Это может показаться лишним, потому что в любом случае указатель — это адрес первого байта в памяти, которую занимает переменная (вне зависимости от того, какого типа эта переменная). Но, так или иначе, этот тип нужно указывать (рис. П.3, табл. П.4).

Таблица П.4

Действие	Значение
int a = 5;	Переменная a типа int получает значение 5
int *ptr;	Переменная-указатель на другую переменную типа int

значение	?	5	0	?				?
иссер	0	1	2	3	4	5	6	7
переменная	не занято	a		ptr				не занято
тип		int		указатель на int				
значение		5		не определено				

Рис. П.3. Переменная и переменная-указатель

В данном случае мы объявляем новую переменную ptr, в которой будет храниться адрес какой-то переменной типа int. Другими словами, в переменной ptr будет храниться адрес первого байта области в памяти, в которой хранится переменная типа int. Пока мы только объявили эту переменную, и она пока не содержит никакого осмысленного значения (не содержит осмысленного адреса). Занесем теперь в ptr адрес, например, переменной a (рис. П.4).

```
ptr = &a;
```

Так же как с самой переменной a, мы могли значение указателя (значением указателя является адрес!) установить сразу при объявлении указателя:

```
int *ptr = &a;
int *ptr2 = &a;
int *ptr3 = ptr;
```

значение	?	5	0	номер 1				?
номер	0	1	2	3	4	5	6	7
переменная	не занято	a		ptr				не занято
тип		int		указатель на int				
значение		5		адрес перем. a				

Рис. П.4. Переменная и переменная-указатель инициализированы сразу

В приведенном примере создаются три указателя на переменную типа `int` (рис. П.5). В переменные-указатели `ptr` и `ptr2` сразу заносится адрес переменной `a`, а в переменную `ptr3` заносится значение указателя `ptr`, а раз значением указателя является адрес и в `ptr` к тому моменту находится адрес переменной `a`, в переменной `ptr3` тоже окажется адрес переменной `a`.

значение	?	5	0	номер 1			номер 1			номер 1		
номер	0	1	2	3	4	5	6	7	8	9	10	11
переменная	не занято	a		ptr			ptr2			ptr3		
тип		int		указатель на int			указатель на int			указатель на int		
значение		5		адрес перем. a			адрес перем. a			адрес перем. a		

Рис. П.5. Несколько указателей

Может возникнуть вопрос: каким образом адрес кодируется числовыми байтами, ведь не слова же «номер 1» записаны в памяти? Мы будем для удобства писать «номер 1», хотя на самом деле там просто число.

Итак, пусть имеется переменная `ptr`, которая является указателем на переменную типа `int`. Как работают переменные-указатели и для чего их используют?

```
int a = 5;
int *ptr = &a;
```

Здесь `int *ptr;` — это объявление `ptr` как новой переменной-указателя, т. е. `ptr` — это переменная-указатель, ее значением является адрес, а `*ptr` — значение по адресу, хранимому в переменной `ptr`. Самой главной операцией с указателями является `*`. При объявлении указателя символ `*` показывает, что это именно не переменная типа `int`, а указатель на переменную типа `int`. А при использовании указателя символ `*` перед именем указателя позволяет получить не сам адрес, а значение по этому адресу.

Рассмотрим такой пример (рис. П.6):

```
int a = 5;
int *ptr = &a;
int b = a;
// a => 5
// ptr => адрес переменной a
```

```
// b => 5
// *ptr => 5
```

Что теперь произойдет, если мы в переменную a запишем 6 (рис. П.7)?

a = 6;

значение	?	5	0	номер 1				5	0
номер	0	1	2	3	4	5	6	7	8
переменная	не занята	a		ptr				b	
тип		int		указатель на int				int	
значение		5		адрес перем. a				5	

Рис. П.6. Переменная a имеет значение 5

значение	?	6	0	номер 1				5	0
номер	0	1	2	3	4	5	6	7	8
переменная	не занята	a		ptr				b	
тип		int		указатель на int				int	
значение		6		адрес перем. a				5	

Рис. П.7. Переменная a получила значение 6

Как видим, значение ptr и значение b не изменились, но:

```
// a => 6
// ptr => адрес переменной a
// b => 5
// *ptr => 6
```

*ptr раньше было равно 5, а теперь стало равно 6.

П.3.5. Операции с указателями

Как уже говорилось, указатели допускают несколько видов операций над ними. В частности разность двух указателей есть число, а указатель плюс число есть опять указатель.

Разберемся поподробнее.

```
char c1 = 'h';
char c2 = 'e';
char c3 = 'l';
char c4 = 'l';
char c5 = 'o';
char c6 = 0;
char *ptr = &c1;
char *nptr;
// *ptr => 'h'
```

В результате, вполне возможно, в памяти будет следующая последовательность байтов (рис. П.8).

значение	?	104	101	108	108	111	0	номер 1				?			
номер	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
переменная		c1	c2	c3	c4	c5	c6	ptr				nptr			
тип	не занято	char	char	char	char	char	char	указатель на char				указатель на char			
значение		'h'	'e'	'l'	'l'	'o'	0	адрес перем. c1				не определено			

Рис. П.8. Строка однобайтовых символов в памяти с нулем в конце

Теперь прибавим к первому указателю 4 и запишем результат во второй указатель:

```
np_ptr = ptr+4;
// *np_ptr => 'o'
```

То есть второй указатель теперь указывает на последний из символов. Также справедливо следующее:

```
// np_ptr-ptr => 4;
// *(ptr+1) => 'e'
// *(ptr+2) => 'l'
// *(ptr+3) => 'l'
// *(ptr+4) => 'o'
// *(ptr+5) => 0
```

То есть указатель плюс число есть новый указатель, у которого потом с помощью символа * можно получить значение переменной по новому адресу. Тонкий момент, который здесь нужно учесть, заключается в следующем.

Если к указателю ptr прибавить число i, то в результате получится адрес, смещенный не на i байтов, а на $i \cdot \text{sizeof}(\text{type})$ байтов, где type — это тип переменной, на которую указывает указатель. То есть отсчет идет не в байтах, а как бы в единицах того типа, на который указывает указатель. Если ptr есть указатель на char, то, т. к. char занимает в памяти один байт, ptr+1 указывает на область памяти, отстоящую от ptr действительно на 1 байт. А если ptr есть указатель на int, то ptr+1 указывает на область памяти, отстоящую от ptr на два байта, потому что мы договорились, что переменная типа int занимает в памяти 2 байта. Аналогично разность ptr2-ptr1 даст не количество байтов между областями памяти, на которые указывают ptr1 и ptr2, а количество переменных, которые могут «уместиться» в памяти между ptr1 и ptr2. Причем переменных именно того типа, указателями на который ptr1 и ptr2 являются.

П.3.6. Массивы

Волей разработчиков языка массивы тесно связаны с указателями. Объявим, например, массив символов и запишем в него строчку (которая, как вы помните, обязательно неявно заканчивается символом с кодом 0):

```
char s[6] = "hello";
// s[0] => 'h'
// s[1] => 'e'
```

```
// s[2] => 'l'
// s[3] => 'l'
// s[4] => 'o'
// s[5] => 0
char *ptr;
ptr = s;
```

Волей разработчиков языка имя массива есть указатель на его первый элемент (рис. П.9).

значение	?	104	101	108	108	111	0	номер 1			
номер	0	1	2	3	4	5	6	7	8	9	10
переменная	не занято	s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	ptr			
тип		char	char	char	char	char	char	указатель на char			
значение		'h'	'e'	'l'	'l'	'o'	0	адрес перем. s[0]			

Рис. П.9. Массив однобайтовых символов в памяти

Поэтому к элементам массива можно обращаться как с помощью обычного синтаксиса `имя[индекс]`, так и используя возможности указателей:

```
s[0] => 'h'
// *s => 'h'
// *ptr => 'h'
s[1] => 'e'
// *(s+1) => 'e'
// *(ptr+1) => 'e'
```

Прибавив к любому из указателей, например, `l`, мы заставим этот указатель указывать на следующий символ массива:

```
// *ptr => 'h'
ptr++;
// *ptr => 'e'
```

Обратите внимание, что `*(s+1)` и `(*s)+1` — это совсем разные вещи. Почему так? В первом случае сначала к указателю `s` будет прибавлена единица, так что `s+1` — это указатель на следующий символ за тем, на который указывает `s`. Следовательно, `*(s+1)` — это сам следующий символ, т. е. `'e'`. Во втором случае сначала выполнится операция `*`, а стало быть, `*s` — это символ `'h'`. А после выполнения операции `*` к результату ее будет прибавлена единица. Следовательно, символ `'h'` будет расценен как число 104 и результатом `(*s)+1` будет число 105, или символ `'i'`:

```
char c;
c = *(s+1)
// c => 'e'
c = (*s)+1
// c => 'i'
```

При этом приоритет операций таков, что `*s+1` будет расценено как `(*s)+1`.

Еще раз напомним, что разница между двумя указателями считается не в байтах, а в единицах того, на что эти указатели указывают, и для массивов это тоже верно. Проверьте и убедитесь:

```
double a[2];
// &a[1] - &a[0] => 1
// (int)(&a[1]) - (int)(&a[0]) => 8
```

А уж когда в дело вмешивается так называемое выравнивание — все становится еще сложнее. Проверьте сами и удивитесь!

```
struct S{
    double a;
    bool b;
};

S a[2];
// sizeof(double) => 8
// sizeof(bool) => 1
// sizeof(S) => 16
// &a[1] - &a[0] => 1
// (int)(&a[1]) - (int)(&a[0]) => 16
```

П.3.7. Ссылки

Еще одно понятие, имеющееся в C++, — это ссылка: нечто внешне очень похожее на указатель, но принципиально другое по смыслу. Ссылка — это просто другое имя для той же самой переменной. Напоминаю: переменная — это именованный участок памяти, содержащий изменяющееся значение. Так вот с помощью ссылок у одного и того же участка памяти может быть несколько имен.

```
int a = 6;
int &r = a;
r = 7;
```

Здесь сначала создается переменная `a`, содержащая значение 6, потом создается ссылка `r`, как альтернативное имя для переменной `a`. Теперь у нас по сути одна переменная (одно место в памяти), но два имени: `a` и `r`, с помощью которых можно менять значение этой переменной. Третьим действием мы используя имя `r`, заносим в нашу единственную переменную значение 7. Можете убедиться, что и значением `a`, и значением `r` после этого является именно 7.

Того же самого результата можно было добиться с помощью указателей, поэтому кто-то может сказать, что ссылки — это те же указатели, только все время автоматически разыменовывающиеся:

```
int a = 6;
int *r = &a;
*r = 7;
```


Но есть принципиальная разница. Указатель — это переменная, содержащая адрес. Раз это отдельная переменная, то у нее есть свое собственное место в памяти. Ссылка — это не переменная, у нее своего собственного места в памяти нет, это просто другое имя для какой-то уже существующей переменной. Если переменная — это почтовый ящик, имя переменной — это имя, написанное на почтовом ящике, а указатель — это почтовый ящик, в котором хранится номер какого-то другого почтового ящика, то ссылка — это еще одно имя «Маша», написанное на почтовом ящике, на котором уже написано имя «Коля».

Ссылка — это имя, она появляется только тогда, когда его пишут на каком-то почтовом ящике. Поэтому ссылку нельзя создать, «не инициализировав», т. е. не указав, синонимом какой другой переменной она является. Вот это, например, будет ошибкой:

```
int &r2;
```

И еще одно ключевое отличие: ссылку нельзя «переназначить». Если вы написали имя «Маша» на каком-то почтовом ящике, то переписать это же имя на другой почтовый ящик не получится. Попробуйте предсказать, какой будет результат:

```
int a = 6;
int b = 7;
int &r = a;
r = b;
r = 7;
// a => ?
```

Когда мы пишем `r = b`, может показаться, что мы переназначаем ссылку `r`, делаем так, что теперь `r` является синонимом не `a`, но `b`. Но это не так, этим действием мы просто присваиваем переменной `r` значение переменной `b`. А раз `r` и `a` есть одна и та же переменная, то теперь и `a`, и `b` получают одно и то же значение 7.

Создатели C++ были жадные люди, они сэкономили грамматику языка и текстовые символы таблицы ASCII и не вводили лишние ключевые слова, используя уже существующие. Они жадничали, а страдать — вам, потому что теперь одни и те же символы означают очень разные вещи.

```
int *x = *y;
```

Вот здесь и слева (в объявлении) и справа (в выражении) — вроде бы одна и та же звездочка, но это совсем разные звездочки по смыслу. Если бы создатели C++ не жадничали, то было бы написано что-то вроде:

```
int pointer x = dereference(y);
```

Потому что звездочка справа — это конкретная операция, действие, смысл которого: «возьми переменную `y`, вытащи хранящийся в ней адрес, перейди по этому адресу и верни значение, которое по этому адресу хранится». Звездочка слева — это вообще не операция, это указание компилятору, что `x` — это пере-

менная, в которой будет храниться не `int`, а адрес какого-то другого места в памяти, где этот `int` будет храниться. Кстати, проверьте себя: как должна быть объявлена переменная `y`, чтобы строка `int *x = *y` скомпилировалась?

```
int n = 10;
int &m = n;
int *p = &m;
```

Аналогично вот здесь амперсанд слева и амперсанд справа — совсем разные амперсанды по смыслу. Если бы создатели C++ не жадничали, то было бы написано что-то вроде:

```
int n = 10;
int alias m = n;
int pointer p = address(m);
```

Амперсанд, который справа, — это конкретная операция, действие, смысл которой: «возьми переменную `m` и верни ее адрес в памяти» (адрес в памяти — это как раз то, что можно складывать в переменную-указатель `p`). Амперсанд, который слева, — это вообще не операция, это указание компилятору, что `m` — это не переменная, а ссылка, т. е. синоним какой-то другой переменной.

И наконец, в версии C++11 появился еще один тип «амперсанда, который слева», а именно `&&`. Это не оператор, это аналог слова `alias` выше, т. е. способ объявления специфической переменной или специфического параметра у функции.

```
int n = 10;
int &m = n;
int &&r = n + 1;
```

Вот он, этот двойной амперсанд, и если бы типы переменных объявляли словами человеческого языка, то было бы написано:

```
int n = 10;
int alias m = n;
int alias of expression r = n + 1;
```

Смысл `&&` заключается в том, что, как видно в приведенном выше примере, `r` есть так называемая «ссылка на временный объект, полученный в результате расчета выражения» (это я так написал, чтобы не произносить страшные слова `rvalue`). Если написать выражение `n+1` и подумать, как оно будет вычисляться, то понятно, что сначала мы имеем переменную `n`, но увеличивать на единицу ее мы не можем, потому что после этой строки сама переменная `n` остается неизменной. Поэтому можно считать, что создается временный объект, получает сначала значение `n`, потом увеличивается на единицу. Если бы было написано `n = n + 1`, то этот объект записывался бы обратно в переменную `n`. Если бы было написано `t = n + 1`, то этот объект записывался бы в другую переменную `t`. Если бы было написано просто `n + 1`, то этот временный объект никуда не записывался бы и просто был бы уничтожен после вычисления. А если мы пишем `int &&r = n + 1`,

то мы этому временному объекту даем имя. Обычно, конечно, делают не переменные такого типа, а аргументы в некоторых функциях (например, в конструкторе перемещения). Если какая-то функция `void foo(Object &&x)` ... получает значение такого типа, то ей как бы говорят: «Переданный тебе объект `x` больше никому не нужен, можешь разбирать его на запчасти как хочешь» (это только часть правды на самом деле, поэтому, если хотите `&&` использовать, разберитесь в шаблонах и `universal (forwarding) reference`).

П.4. Вопросы для самоконтроля

Не рассматривайте приведенные ниже вопросы как проверку ваших знаний. Бессмысленно подсматривать где-то ответы. Бессмысленно также пытаться сразу трассировать этот код — это приведет только и единственно к тому, что вы не сможете оценить, разбираетесь вы теперь в указателях или нет. Рекомендуется для каждого вопроса нарисовать, используя бумагу и ручку, содержимое всех переменных и его изменение, после этого предсказать ответ, и только после этого обязательно внести приводимый кусок кода в любую программу и посмотреть, что получается на самом деле. Несовпадение ваших ожиданий является показателем того, что вы не до конца разобрались в указателях.

```
☐ int a = 2;
    int b = 3;
    int t = a;
    a = b;
    b = t;
```

Внимание, вопрос: чему равны значения `a`, `b` и `t`?

```
☐ int a = 2;
    int *pa;
    pa = &a;
    int b = *pa;
    a = 3;
```

Внимание, вопрос: чему равны значения `a` и `b`?

```
☐ int a = 2;
    int b = 3;
    int *pa = &a;
    int *pb = &b;
    *pa = *pb;
```

Внимание, вопрос: чему равны значения `a`, `b`, `pa`, `pb`?

```
☐ int a = 2;
    int b = 3;
    int *pa = &a;
```

```
int *pb = &b;  
pa = pb;
```

Внимание, вопрос: чему равны значения a , b , pa , pb ?

```
❑ int a[3] = {6, 5, 4};  
int *b = a+1;
```

Внимание, вопрос: чему равны значения $*(a+2)$, $(*a)+2$, $*a+2$, $*b$?

```
❑ int a = 5;  
int &r = a;  
r = 6;
```

Внимание, вопрос: чему равны значения a , r , $\&a$, $\&r$?

СПИСОК ЛИТЕРАТУРЫ

1. Шилдт Г. Полный справочник по C++. — М.: Вильямс, 2016.
2. Страуструп Б. Язык программирования C++. — любое, любой.
3. Саттер Г., Александреску А. Стандарты программирования на C++. 101 правило и рекомендация. — М.: Вильямс, 2019.
4. Буч Г. и др. Объектно-ориентированный анализ и проектирование с примерами приложений. — М.: Вильямс, 2017.
5. Амини К. Экстремальный Си. Параллелизм, ООП и продвинутые возможности. — СПб.: Питер, 2021.
6. Bjarne Stroustrup и Herb Sutter. C++ Core Guidelines. — URL: <https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md>.
7. Гамма Э. и др. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер, 2019.
8. Herb Sutter. Smart Pointer Parameters. — URL: <https://herbsutter.com/2013/06/05/gotw-91-solution-smart-pointer-parameters/>.
9. Фримен Э., Робсон Э., Сьерра К. Head First. Паттерны проектирования. — СПб.: Питер, 2021.
10. Тепляков С. Паттерны проектирования на платформе .NET. — СПб.: Питер, 2015.
11. Ларман К. Применение UML 2.0 и шаблонов проектирования. — М.: Диалектика, 2019.
12. Мартин Р. Принципы, паттерны и методики гибкой разработки на языке C#. — СПб.: Символ-Плюс, 2011.
13. Макеев Г. О сложности в работе программиста. — URL: <https://habr.com/ru/post/560950/>.
14. Тепляков С. Цикл статей о SOLID принципах. — URL: <http://sergeyteplyakov.blogspot.com/search/label/SOLID>.

ПРЕДМЕТНЫЙ УКАЗАТЕЛЬ

A

ADT 152

C

C# 225

C++ 12, 18, 21, 28, 30, 32, 33, 36, 37, 44,
49, 52, 65, 67, 69, 70, 76, 80, 83, 88, 90,
91, 112, 113, 115, 116, 118, 119, 150, 152,
200, 210, 249

C++11 90, 156

cpp-файл 34

C-массив 153

◊ динамический 153

D

Drupal 225

DRY 231, 241

G

GRASP-паттерн 100

GRASP-принцип 100

◊ информационный эксперт 101

◊ контроллер 101

◊ полиморфизм 102

◊ принцип высокой связности 102

◊ принцип слабого зацепления 102

◊ слой абстракции 102

◊ создатель 101

◊ устойчивость к изменениям 102

◊ чистая выдумка 102

GUI-приложение 225

GUI-фреймворк 225

H

h-файл 34

M

Mixin 115, 171

Move-семантика 91

MVC 28, 101, 237–239, 245–247

MVVM 246

O

OMT 55, 57

P

Pimpl 199, 200

Q

Qt 236

R

RAII 88, 90

Reference-семантика 51, 52

RTTI 80, 222

S

SOLID-принцип 115, 160, 178, 185, 207, 211

◊ принцип единственной
ответственности 115, 212

◊ принцип инверсии зависимостей 216

◊ принцип открытости/закрытости 213

◊ принцип подстановки Лисков 213

◊ принцип разделения интерфейса 215

Static Initialization Order Fiasco 119
STL 149, 152, 153, 156

U

UB 50, 74, 76, 85, 86
UML 55, 57
◊ диаграммы классов 55, 56

V

Value-семантика 51, 52
ViewModel 246, 247
Visual Studio 225
Visual Studio 2013 86

W

Windows Forms 225

A

Агрегатор 54
Агрегация 53, 127
Ассоциация 54

Б

Бизнес-логика 27
Бизнес-правило 27, 28, 31
Бинарное дерево 96
Брейкпоинт 24, 30, 36

Д

Деструктор 35
Диаграмма классов 56, 58
Диаграмма последовательностей 58
Диаграмма состояний 58, 59

З

Зависимость 47, 52, 55

И

Инкапсуляция 17, 25, 26, 28
Интерфейс 28, 43, 63, 111, 113, 215, 217
Итератор 143

К

Класс 17
◊ абстрактный 71
Композиция 47, 53, 62
Конечный автомат 58

Конструктор 30, 34
◊ копирования 31, 32, 45, 83
◊ перемещения 33
◊ по умолчанию 31, 32, 45
◊ с параметрами 31, 32, 45
Контейнер 54, 132, 142, 153
Копирование
◊ глубокое 50
◊ поверхностное 50

М

Массив 136
◊ ассоциативный 154, 175, 178
Метод 23
◊ абстрактный 71
Миксин 115

Н

Наследование 40, 52, 62
◊ виртуальное 114
◊ множественное 113

О

Обработчик события 229, 230
Очередь 143, 154

П

Паттерн 99, 100
◊ GRASP 100
Подписка 167
Подсчет ссылок 89
Полиморфизм 66, 74, 102, 209

Примесь 115
Проблема ромба 114
Прототип 107

С

C++11 261
Сборщик мусора 88
Синглтон 119
Событие 59, 228, 236
Сообщение 236
Состояние 59, 60
Список
◊ двусвязный 153
◊ односвязный 153
Статический атрибут 116, 117
Стек 143, 154, 175
Стратегия 131, 132

У

Умные указатели 91
Утечка памяти 19, 72, 85–87, 96, 166

Ф

Фабрика 97, 184, 185, 188, 189
◊ абстрактная 188, 189

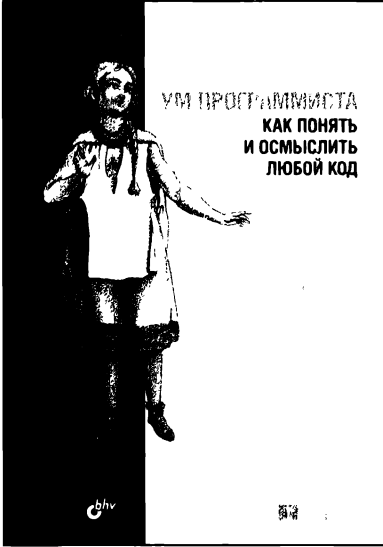
Х

Хелпер-функция 26
Хеш-таблица 154

Ум программиста. Как понять и осмыслить любой код

Отдел оптовых поставок:

e-mail: opt@bhv.ru



Прочитав эту книгу, вы

- узнаете, как именно ваш мозг видит код;
- научитесь бегло читать и быстро усваивать код;
- убедитесь, что есть простые приемы, позволяющие распутать самый сложный код;
- сможете навести порядок в любой, даже самой запущенной базе кода.

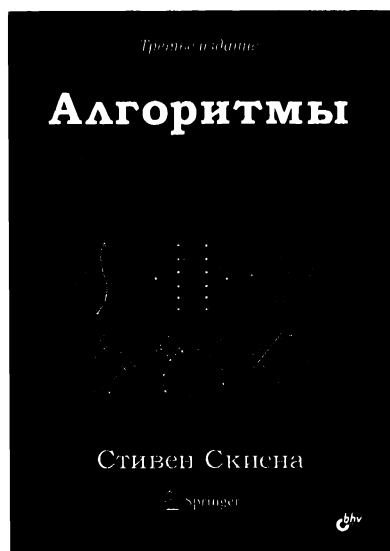
Книга освещает практические основы когнитивистики для программистов. Основные темы: осмысление и развитие чужого и собственного кода, изучение новых языков программирования, мнемонические приемы для программистов, поддержка кода в читаемом состоянии. Объяснено, как снижать когнитивную нагрузку при работе программиста, как делать код логичным и понятным для себя и коллег. Рассмотрены приемы именования функций, классов и переменных, подходы к ведению репозиторий, совместной разработке и доработке кода.

Доктор Фелин Херманс, приглашенный профессор в Лейденском университете в Нидерландах. Вот уже более 10 лет она исследует механизмы программирования, выясняя, как лучше учиться этому искусству и обучать других.

Алгоритмы. Руководство по разработке. 3-е изд.

Отдел оптовых поставок:

e-mail: opt@bhv.ru



Новое в третьем издании:

- Расширенный набор рандомизированных алгоритмов, хеширования, алгоритмов «разделяй и властвуй», аппроксимации и квантовых вычислений
- Онлайн-поддержка для преподавателей, включающая слайды и видеоуроки
- Полноцветные иллюстрации и код, наглядно разъясняющие сложные концепции
- Новые «истории из жизни», рассказывающие об опыте работы с реальными приложениями
- Более 100 новых задач, включая задачи по программированию от LeetCode и Hackerrank
- Актуальные ссылки к лучшим реализациям на языках C, C++ и Java

Книга является наиболее полным руководством по разработке эффективных алгоритмов. Первая часть книги содержит практические рекомендации по разработке алгоритмов: приводятся основные понятия, дается анализ алгоритмов, рассматриваются типы структур данных, основные алгоритмы сортировки, операции обхода графов и алгоритмы для работы со взвешенными графами, примеры использования комбинаторного поиска, эвристических методов и динамического программирования. Вторая часть книги содержит обширный список литературы и каталог из 75 наиболее распространенных алгоритмических задач, для которых перечислены существующие программные реализации.

В третьем издании расширен набор рандомизированных алгоритмов, алгоритмов хеширования, аппроксимации и квантовых вычислений. Добавлено более 100 новых задач, даны ссылки к реализациям на C, C++ и Java.

Книгу можно использовать в качестве справочника по алгоритмам для программистов, исследователей и в качестве учебного пособия для студентов соответствующих специальностей.

Стивен Скиена, профессор кафедры вычислительной техники Университета Стоуни — Брук, известный исследователь алгоритмов, лауреат премии Института IEEE, автор популярной книги «Programming Challenges: The Programming Contest Training Manual» («Олимпиадные задачи по программированию. Руководство по подготовке к соревнованиям»).



ИНТЕРНЕТ-МАГАЗИН

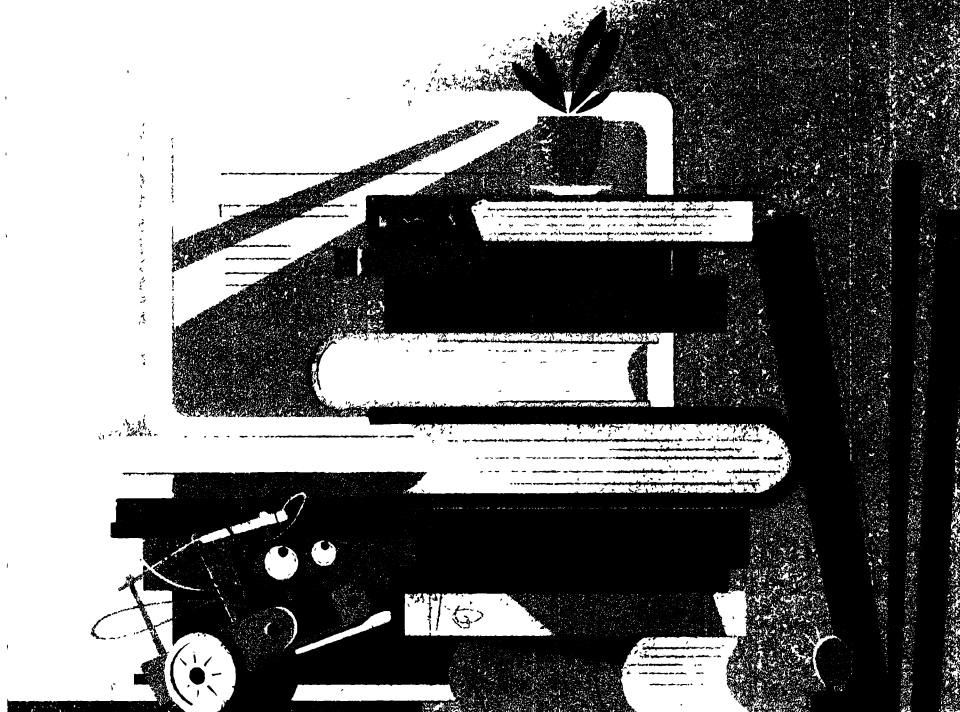
BHV.RU

КНИГИ, РОБОТЫ,
ЭЛЕКТРОНИКА

Интернет-магазин издательства «БХВ»

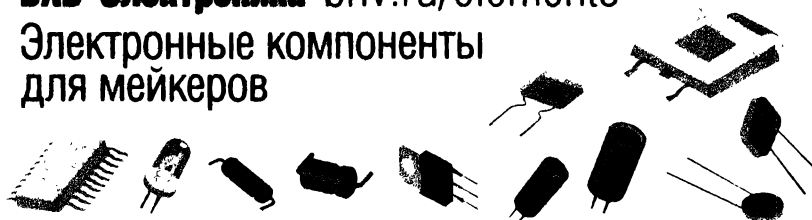
Более 30 лет на российском рынке

- Книги и наборы по электронике и робототехнике по издательским ценам
- Электронные архивы книг и компакт-дисков
- Ответы на вопросы читателей



БХВ-Электроника bhv.ru/elements

Электронные компоненты
для мейкеров



MVC и SOLID для уверенного понимания паттернов проектирования

Вот уже более 30 лет объектно-ориентированное программирование (ООП) остается основной парадигмой разработки программного обеспечения, доминирующей в академической информатике и в прикладном программировании на C++, Java, C#, Python и других языках, лидирующих в рейтинге TIOBE. При этом многие современные книги по ООП представляют собой огромные фолианты и не подходят для базового изучения этой темы.

Данная книга, написанная на основе многолетних наработок автора в преподавании и индустрии программного обеспечения, раскрывает тему ООП максимально доступно и просто. Кроме базовых основ ООП, книга описывает паттерны проектирования, принципы SOLID, архитектуру MVC (модель-представление-контроллер), а также затрагивает особенности ООП в языке C++.

Материал подходит в качестве вводной книги по ООП. Книга будет интересна студентам, осваивающим C++ и ООП, но и опытные специалисты тоже найдут в ней интересные детали об использовании паттернов проектирования.



Григорий Анатольевич Макеев, кандидат технических наук, практикующий инженер-программист, руководитель группы разработчиков, занимающейся разработкой наукоемкого инженерного ПО, преподаватель Уфимского университета науки и технологий. Обладает почти 20-летним профессиональным стажем. На протяжении преподавательской карьеры вел дисциплины, связанные с объектно-ориентированным и функциональным программированием, а также с базами данных.



191036, Санкт-Петербург,
Гончарная ул., 20
Тел.: (812) 717-10-50,
339-54-17, 339-54-28
E-mail: mail@bhv.ru
Internet: www.bhv.ru

ISBN 978-5-9775-1913-7



9 785977 519137